---

name: d365-fao-interest description: User is interested in Dynamics 365 Finance & Operations technical details metadata: node_type: memory type: user originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-07-25T14:24:44.358Z

User has asked about technical details of Dynamics 365 Finance & Operations (d365 F&O, formerly Dynamics AX 7 / Dynamics 365 Operations). They likely have a development or consulting background.

Why: User initiated the conversation asking about d365 F&O tech details — not a one-off question, they want a substantive technical overview.

How to apply: When the user asks follow-up questions about d365 F&O, reference this context. They may want details on X++, the model stack, deployment, extensibility, or cloud architecture. Tailor depth to their apparent expertise level.

Related memories: none yet.

---

name: d365-fao-learning-journey description: User is a D365 F&O technical intern on a 4-12 month learning journey; needs comprehensive technical concepts guide metadata: node_type: memory type: project originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-08-04T20:34:16.736Z

User is a Technical Microsoft Dynamics 365 Finance & Operations intern working on a 4–12 month learning journey. They completed Months 1–2 and Phase 3 (Architecture). Phase 4 (Extensibility Patterns) — Chapters 6–7 in the Desktop guide are fully expanded. Chapters 8–15 in the Desktop guide have now been fully expanded with deep content and code examples.

Progress: - Month 1: CoC, Forms, Lookups [x] - Month 2: Tables, EDTs, Classes, AOT, Architecture, Cloud, Metadata Subsystem, Identity/Authentication [x] - Phase 3 (Architecture) [x] Complete in main guide - Phase 4 — Chapters 6–7 fully expanded in Desktop guide with deep code examples [x] - Chapters 8–15 in Desktop guide now fully expanded with deep content, code examples, tables, and activities [x] - Chapter 2 (X++ Language Fundamentals and Class Design) — newly written and inserted between Chapter 1 and Chapter 3 [x] - Main guide (d365_fao_technical_concepts_guide.md) has all 12 phases complete [x] - Exercises expanded from 15 to 17 (added NumberSeq and SysOperation) [x]

Study Plan — Month-by-Month Roadmap

> **How to use this plan:** Each month has a theme, target chapters, a deliverable, and a checkpoint question. Complete the chapters, build the deliverable, and answer the checkpoint before moving on.

Month 1 — Foundations

Item	Detail
Theme	Understand the platform and write your first X++ code
Target Chapters	Desktop Guide Ch 1, Ch 2
Main Guide Phases	Phase 1 (all sections)
Deliverable	A working X++ job that reads from `CustTable`, applies a `while select` with a `where` clause, and writes results to the Infolog
Checkpoint	Can you explain the difference between `select` and `while select`? Can you describe the four pillars of the D365 F&O architecture?

Month 2 — Data Layer Deep Dive

Item	Detail
Theme	Master tables, EDTs, and data integrity
Target Chapters	Desktop Guide Ch 3
Main Guide Phases	Phase 2 (Tables, EDTs)
Deliverable	Design a custom table with 8+ fields, 2 EDTs, 1 field group, 1 alternate index, and a `validateWrite()` override

Item	Detail
Checkpoint	Can you explain why EDTs matter and what happens when you change an EDT's `StringSize`? Can you describe the difference between an alternate index and a hash index?

Month 3 — UI & Business Logic

Item	Detail
Theme	Build forms and write business logic classes
Target Chapters	Desktop Guide Ch 4, Ch 6
Main Guide Phases	Phase 2 (all sections — Classes, Views, Form Controls, Business Logic)
Deliverable	A form with 2 data sources (parent-child), an action pane with 3 actions, and a class that orchestrates a business process using the form's data
Checkpoint	Can you explain the difference between `Inner Join`, `Outer Join`, and `Exists Join` in form data sources? Can you describe the RunBase pattern?

Month 4 — Extensibility Patterns

Item	Detail
Theme	Master CoC, Event Handlers, and Delegates
Target Chapters	Desktop Guide Ch 7
Main Guide Phases	Phase 4 (all sections)
Deliverable	An extension that uses CoC to modify a base class method, an Event Handler for a table's `insert()` event, and a Delegate that lets consumers customize a business rule
Checkpoint	Can you explain when to use CoC vs. Event Handlers vs. Delegates? What is the execution order when multiple extensions target the same method?

Month 5 — Security & Integration

Item	Detail
Theme	Understand D365 F&O security and build integrations
Target Chapters	Desktop Guide Ch 8, Ch 9
Main Guide Phases	Phase 10 (Security), Phase 6 (Data Entities & Integration)
Deliverable	A custom security role with 2 duties and 4 privileges, plus a data entity that exposes a custom table to an external system via OData

Item	Detail
Checkpoint	Can you trace the path from a privilege → duty → role → user? Can you explain the difference between a Data Entity and a Data Project?

Month 6 — Reporting & Batch Processing

Item	Detail
Theme	Build reports and batch jobs
Target Chapters	Desktop Guide Ch 10, Ch 11
Main Guide Phases	Phase 7 (Reporting), Phase 5 (Business Logic — batch)
Deliverable	An SSRS report with a custom `SrsReportDataProvider` class, plus a `RunBaseBatch` job that processes records in the background
Checkpoint	Can you explain the difference between `SrsReportDataProvider` and a standard `ReportRun`? What is the `BatchHeader` class used for?

Month 7 — Testing & Quality

Item	Detail
Theme	Write tests and establish QA practices
Target Chapters	Desktop Guide Ch 12
Main Guide Phases	Phase 8 (Testing)
Deliverable	5 unit tests using the `SysTest` framework covering table validation, class logic, and form behavior
Checkpoint	Can you explain the difference between `SysTest` and the Acceptance Test Library (ATL)? When would you use RSAT?

Month 8 — DevOps & Deployment

Item	Detail
Theme	Automate builds and manage deployments
Target Chapters	Desktop Guide Ch 13
Main Guide Phases	Phase 9 (DevOps & CI/CD)
Deliverable	An Azure DevOps build pipeline that compiles X++, runs unit tests, and produces a `.axmodel` artifact
Checkpoint	Can you describe the difference between a build pipeline and a release pipeline in LCS? What is a configuration key and how does it control feature visibility?

Month 9 — Performance & Troubleshooting

Item	Detail
Theme	Optimize code and diagnose issues
Target Chapters	Desktop Guide Ch 14
Main Guide Phases	Phase 11 (Performance)
Deliverable	A performance audit of a sample X++ job — identify 3+ bottlenecks (e.g., `select` inside a loop, missing indexes, synchronous calls) and rewrite them
Checkpoint	Can you list the top 5 SQL performance anti-patterns in X++? Can you explain how `CacheLookup` affects query performance?

Month 10 — Capstone Project

Item	Detail
Theme	End-to-end production scenario
Target Chapters	Desktop Guide Ch 15
Main Guide Phases	Phase 12 (Solution Crafting)
Deliverable	A complete solution: custom table + EDT + form + business logic class + security role + data entity + SSRS report + batch job, all deployed to a test environment
Checkpoint	Can you walk through your entire solution and explain the design decisions for each component?

Month 11 — Module Specialization

Item	Detail
Theme	Go deep in your chosen module (Finance, SCM, or HR)
Target Chapters	Desktop Guide Ch 1–15 (module-specific sections)
Main Guide Phases	Phase 10 (Security), Phase 11 (Performance), Phase 12 (Solution Crafting)
Deliverable	A module-specific deep-dive document with code samples: e.g., a custom AP invoice workflow in Finance, a purchase order automation in SCM, or a leave management extension in HR
Checkpoint	Can you explain the end-to-end data flow for your module's primary business process? Can you identify the key tables, EDTs, and business logic classes involved?

Month 12 — Advanced Patterns & Certification Prep

Item	Detail
Theme	Master advanced patterns and prepare for certification
Target Chapters	Desktop Guide Ch 1–15 (advanced sections)

Item	Detail
Main Guide Phases	All phases — review and deepen
Target Areas	Workflow extensibility, E-Signature, BI/Power BI integration, Microsoft MB-310/MB-330 certification prep
Deliverable	A certification study guide with annotated code samples and a mock exam self-assessment
Checkpoint	Can you explain the difference between CoC, Event Handlers, and Delegates and when to use each? Can you walk through a complete end-to-end solution from table design through deployment?

Milestone Summary

Month	Milestone	Status
1	First X++ job — read/write data	☐
2	Custom table design with EDTs and indexes	☐
3	Form with parent-child data sources + business logic class	☐
4	CoC + Event Handler + Delegate extension	☐
5	Security role + data entity integration	☐
6	SSRS report + RunBaseBatch job	☐
7	5 SysTest unit tests	☐
8	Azure DevOps CI pipeline	☐
9	Performance audit with 3+ fixes	☐
10	Capstone: end-to-end solution deployed	☐
11	Module specialization deep dive	☐
12	Advanced patterns & certification prep	☐

Why: User explicitly requested a comprehensive PDF document of all D365 F&O technical concepts from official Microsoft documentation to support their learning journey.

How to apply: When the user asks about D365 F&O concepts, reference the comprehensive guide at `C:\Users\Ahmed\d365_fao_technical_concepts_guide.md`. This document covers 12 learning phases from foundations through capstone projects, now expanded with deep code examples for X++, EDTs, classes, CoC, Event Handlers, Delegates, RunBase, SysOperation, REST APIs, and more. The user is a developer/technical intern — they understand coding concepts and want to deepen both development and architectural knowledge. Be prepared to discuss specific modules, help with development tasks, and provide code examples within the D365 F&O context.

Related memories: - d365-fao-interest — user's initial interest in d365 F&O technical details

---

name: d365-learning-guide description: Desktop guide with 15 chapters of deep D365 F&O content including code examples, tables, and activities metadata: node_type: memory type: reference originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-08-29T16:48:29.471Z

D365 Finance & Operations — Technical Intern Learning Guide

> **Your 4–12 month foundational reference.** Every concept below is rooted in the official Microsoft documentation at learn.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/. For the latest updates, always verify against that source — this guide is your scaffold, not your end point.

How to Use This Guide

1. **Read the chapter** — absorption first, coding second.
2. **Attempt the activity** — use the hints to explore multiple valid paths.
3. **Compare with the ideal solution** (provided at the end of each chapter for chapters 1–14, and in full detail for Chapter 15) — understand *why* the recommended approach was chosen.
4. **Return to Microsoft Docs** for anything that has changed, been deprecated, or has newer patterns.

Chapter 1 — The D365 F&O Developmental Landscape

1.1 The Platform Architecture — What You Are Actually Working Inside

Before you write a single line of X++, understand the platform you are operating inside. D365 Finance & Operations is a **cloud-native, multi-tier enterprise application** built on four architectural pillars:

1.1.1 The Four Pillars

Pillar	Technology	Role	What It Means to You as a Developer
Application Tier	AOS (Application Object Server) — runs X++ IL code	Executes business logic, interprets X++, manages transactions, serves metadata	Your X++ code runs here — not in the browser, not in SQL Server
Database Tier	Azure SQL Database / SQL Server	Stores all business data — tables, indexes, stored procedures (generated from AOT), temp tables	Your `insert()`, `update()`, `delete()` calls translate to SQL statements against this tier
Presentation Tier	Single browser-based web client (HTML5/CSS/JS) running in Edge, Chrome, or Safari	Renders forms, menus, navigation — the user's view of the system	You design forms in this tier; the form definition lives in the AOT, and the AOS streams metadata to the browser which renders the UI
Integration Tier	Azure Services (Service Bus, Azure Functions, Logic Apps), OData/REST endpoints	Connects D365 F&O to external systems — Power Platform, third-party ERP, HR systems	When building data entities or integration patterns, you interact with this tier

1.1.2 How a Request Flows Through the System

Understanding the request lifecycle is essential for debugging and for writing code that performs well:

```
User clicks a button on a form (HTML/Windows client)
    |
    v
[Client-side] HTTP request + context info sent to AOS
    |
    v
[AOS Application Tier]  X++ IL code executes
    |   • Security checks (who are you? what can you do?)
    |   • Transaction management (ttsbegin/ttscommit)
    |   • Business logic (your X++ code runs here)
    |   • Cross-layer calls (SYS → ISV → VAR → CUS → USR)
    |
    v
[Database Tier]  SQL statements generated from AOT metadata
    |   • SQL Server validates permissions (row-level, column-level)
    |   • Data is read/written to Azure SQL Database
```

```
    |
    v
[AOS]  Results returned to client as XML/JSON
    |
    v
[Client]  Form refreshes, Infolog displays messages
```

The critical insight for a new developer: When your X++ code runs slowly, the bottleneck is almost never the X++ itself — it is either (a) too many SQL round-trips (selecting one record at a time in a loop instead of using `exists join` or `insert_recordset`) or (b) synchronous calls to remote services or external systems that the AOS is blocked waiting for.

1.1.3 The Role of Azure in D365 F&O

D365 F&O runs in Azure. Understanding the Azure pieces:

Azure Component	Purpose	Developer Relevance
Azure SQL Database	Hosts the business database	Your table definitions create SQL tables here; SQL performance tuning (indexes, joins) applies directly
Azure App Service	Hosts the AOS workers (Application Object Server)	Your X++ IL runs in these workers; scaling AOS workers affects throughput
Azure DevOps / GitHub	Build and deployment CI/CD	Build pipelines compile X++, produce `.axmodel` artifacts, deploy to environments
Azure Active Directory (Azure AD)	Authentication and authorization	Every user authenticates via Azure AD; role-based access flows through the D365 security model
Azure Storage	Model store (AOT object storage)	The model store is a file share (e.g., `C:\AOSService\PackagesLocalDirectory` or `K:\` in cloud environments), not a SQL database — understanding this prevents common deployment errors
Source: Models and packages		
Azure Key Vault / Managed Identity	Credential and secret management	Configuration keys (`#ISO`, `USMF`, custom keys) are encrypted credentials your code references at runtime
Azure Service Bus	Async messaging between services	Used by integration patterns — your code can send/receive messages via .NET interop (`System.ServiceModel` or `HttpClient`)

1.2 The Application Object Tree (AOT) — The Single Source of Truth

1.2.1 What Is the AOT?

The AOT is a **hierarchical, metadata-driven repository** that contains every object in the D365 F&O system — every table, class, form, view, menu, security role, report, and data entity. It is not a SQL database of these

objects; it is a **file system** organized as a tree of XML-like metadata files, stored in the **model store** as a file share (e.g., `C:\AOSService\PackagesLocalDirectory` or `K:\` in cloud-hosted environments).

Source: Models and packages - Finance & Operations

The AOT serves as: - The **design-time** interface — developers browse, create, and modify objects here - The **source of truth** for runtime behavior — the AOS loads AOT metadata into memory at startup and uses it to generate SQL, render forms, and enforce security - The **compile-time** compilation unit — when you build your project in Visual Studio, the AOT nodes in your model are compiled into IL

1.2.2 The AOT Tree — Complete Node Hierarchy

The AOT is organized into top-level nodes. Every object belongs to exactly one top-level node. The hierarchy is:

```
AOT (Application Object Tree)
|
+-- Classes
|   +-- Static classes (utility, service entry points)
|   +-- Instance classes (business logic objects)
|   +-- System classes (Sys**, Global, FormRun, etc.)
|   +-- Extension classes ([ExtensionOf(...)])
|   +-- Event handler classes (subscribing to base class events)
|
+-- Tables
|   +-- Base tables (standard system tables)
|   +-- Extended tables (tables that inherit from base tables)
|   +-- Table Extensions ([ExtensionOf(tableStr(...))])
|   +-- Temp tables (in-memory only, TableType = Temp or TempDB)
|
+-- Views
|   +-- Base views (custom SQL joins stored in AOT)
|   +-- Derived views (views built from other views)
|   +-- Composite entities (views with a staging pattern for data entities)
|   +-- View Extensions ([ExtensionOf(viewStr(...))])
|
+-- Forms
|   +-- Base forms (standard system forms)
|   +-- Form Extensions ([ExtensionOf(formStr(...))])
|   +-- Form Data Sources (one per table/query the form reads)
|   +-- Form Design (layout — groups, tabs, grids)
|   +-- Form Design Parts (FastTabs, FactBoxes, user controls)
|   +-- Form Extensions (add controls and sections to existing forms)
|
+-- Data Sources (within Forms)
|   +-- Table DataSource (points to a specific table)
|   +-- Query DataSource (points to a query object)
|   +-- Link Type (Inner, Outer, Exists, NotExists)
|
+-- Menus
|   +-- Menu (hierarchical navigation structure)
|   +-- Menu Extension ([ExtensionOf(menuStr(...))])
|   +-- Display menus vs. Action menus (Display opens forms, Action runs methods)
|
+-- Menu Items
|   +-- Display Menu Item (MenuItemDisplay — opens a form)
|   +-- Action Menu Item (MenuItemAction — runs a class method)
|   +-- Output Menu Item (MenuItemOutput — displays a report)
|   +-- Button Menu Item (MenuItemButton — toolbar button on a form)
```

```
|
+-- Macros
|   +-- Global macros (accessible from any X++ file)
|   +-- Class/table-level macros (scoped to the defining element)
|
+-- ETNs (Enterprise Text Notifications)
|   +-- Used for printing, email templates, document formatting, report headers
|
+-- Security
|   +-- Security Roles (the top-level role assignment)
|   +-- Duties (collections of privileges)
|   +-- Privileges (granular access: specific tables, specific access levels)
|   +-- Permissions (field-level and record-level access)
|   +-- Security Keys (used to scope access by business context)
|
+-- Data Dictionary
|   +-- Extended Data Types (EDTs) — typed aliases with metadata
|   +-- Enums — named constant sets
|   +-- Table Groups — logical groupings for lookup display
|   +-- Field Groups — named field collections for auto-form generation
|   +-- Relations — links between fields on different tables
|   +-- Table Indexes — primary, alternate, hash, and non-unique indexes
|
+-- Test Frameworks
|   +-- SysTest classes (unit test classes)
|   +-- SysTestSetup / SysTestMethod / SysTestCleanup attributes
|   +-- Test data builder patterns
|
+-- Extensions
|   +-- Table Extensions
|   +-- Form Extensions
|   +-- Class Extensions
|   +-- View Extensions
|
+-- Metadata
|   +-- Model information (name, version, layer, properties)
|   +-- Layer assignment for every object
|
+-- References
|   +-- Auto-generated cross-references (show what calls what)
|
+-- Queries
|   +-- Query objects (distinct from inline select statements)
|   +-- QueryBuildDataSource, QueryBuildRange, QueryBuildLink (runtime query APIs)
|
+-- Jobs
    +-- X++ scripts executed ad-hoc for data fixes, reports, or one-off processing
```

1.2.3 AOT Object Properties — The Properties That Matter Most

Every AOT object has a **Properties** pane in Visual Studio. The following properties are the ones that directly affect your daily development and that beginners must understand:

Table-Level Properties

Property	Values	What It Controls	Common Mistake
`CacheLookup`	`None`, `NotInTTS`, `Found`, `FoundAndEmpty`, `EntireTable`	How SQL Server caches lookups of this table's records. `None`: no caching. `No`	Setting `CacheLookup = EntireTable` on a large transactional table (e.g., `Sale`

Property	Values	What It Controls	Common Mistake
		`tInTTS`: caches on lookup; transaction-scoped caches don't persist outside the transaction. `Found`: caches successful lookups across transaction boundaries. `FoundAndEmpty`: same as Found, but also caches lookups that found no record. `EntireTable`: loads the whole table into a server-side cache after first select.	`sLine`) wastes server memory. Use `Found` or `NotInTTS` for large tables.
`SaveDataPerCompany`	`Yes`, `No`	When `Yes`, each Legal Entity (company) has its own copy of the data. When `No`, data is shared across all companies.	Forgetting to set this — if you create a master data table (e.g., a chart of accounts), it should be `No`. A transaction table (e.g., invoice lines) should be `Yes`.
`AllowDuplicates`	`Yes`, `No`	Whether the table allows rows where all alternate index key fields are identical	Forgetting to set `AllowDuplicates = No` on index keys means duplicates can exist in the database even though the business expects uniqueness
`TableGroup`	`Transaction`, `Master`, `Parameter`, `Invoice`, etc.	Controls how the table appears in form lookups and navigation patterns	Using `Master` for a high-volume transaction table — use `Transaction` so the AOT treats it appropriately during find operations
`PrimaryIndex`	RecId or an alternate key	The clustered index on the physical SQL table	Changing from `RecId` is only valid for specific cases — most custom tables should keep `RecId` as the primary index

Field-Level Properties

Property	Values	What It Controls
`DataType`	EDT reference or base type	**Always an EDT** — never use base types directly (`string 20` is wrong; use `CustAccount` EDT instead)
`Extension`	EDT name	Links this field to an Extended Data Type
`Label`	A label ID reference	User-facing name (appears in form labels, report captions)
`HelpText`	A label ID reference	Tooltip text shown when the user hovers over the field
`Mandatory`	`Yes`, `No`	Controls the `Validate` on the table — prevents saving a record if this field is

Property	Values	What It Controls
		empty
`Visible`	`Yes`, `No`	Whether the field appears on forms by default (can be overridden per form)
`AllowEdit`	`Yes`, `No`	Whether the field can be modified on any form using this data source
`Method`	Yes — if the field is a computed value displayed via a method	When `Yes`, the field has no physical column in the SQL table — its value is computed by the `DisplayMethod` class method each time it is shown
`Relation`	EDT + referenced table	Enables automatic lookups — when a control is bound to this field, it shows a lookup to the referenced table using the EDT's relation metadata
`ReferenceField`	Field number on the referenced table	Specifies which field on the related table is shown in the lookup dialog
`Skip`	`Yes`, `No`	When `Yes`, the field is skipped from auto-form generation and lookup inclusion
`Array`	`Yes`, `No`	When `Yes`, the field is stored as an array of values (rare — used in specific system tables)

Form-Level Properties

Property	Values	What It Controls
`FrameType`	`Frame`, `Dialog`, `Popup`, `Workflow`, `ListPage`, `ListPageDetails`, `ListItemPage`	The visual chrome and behavior of the form — a `ListPage` has a navigation tree and grid; a `Dialog` has OK/Cancel buttons and is modal
`MultiSelect`	`Yes`, `No`	Whether users can select multiple records in the form's grid
`DeleteAllowed`	`Yes`, `No`	Whether the Delete menu item is enabled for this form
`CopyAllowed`	`Yes`, `No`	Whether the Copy menu item is enabled
`NewAllowed`	`Yes`, `No`	Whether the New record button is enabled
`AutoDeclaration`	`Yes`, `No`	When `Yes`, the form auto-generates variable names for each control and data source — convenient but leads to implicit dependencies. Set to `No` for clean, explicit code
`ShowQueryProperties`	`Yes`, `No`	Whether the Data Source property sheet is shown in the form designer

Property	Values	What It Controls
`WindowType`	`Existing`, `New`	Whether the form opens in an existing window or a new modal window

Class-Level Properties

Property	Values	What It Controls
`Access`	`Public`, `Private`, `Protected`, `Internal`	Encapsulation level — most business logic classes are `Public`
`InstantiationType`	`SingleInstance`, `Static`, `Object`	`SingleInstance` (singleton) means there is only ever one instance of the class; `Static` means the class has only static methods and is never instantiated; `Object` means you create instances with `new` or `construct()`
`Abstract`	`Yes`, `No`	When `Yes`, the class cannot be instantiated directly — it serves as a base class
`Final`	`Yes`, `No`	When `Yes`, the class cannot be extended via `extends`
`EntitlementType`	`Internal`, `External`	Controls how the class is used in entitlement policies

1.2.4 The `Common` Class — Base for All Tables

Every table in D365 F&O inherits from the `Common` class at runtime. This means every table variable (buffer) has access to these universal fields and methods:

Member	Type	Description
`RecId`	`int64`	The auto-generated database primary key — every table has this field even if you don't define it explicitly
`TableId`	`int`	The integer table ID — unique per table in the system (e.g., `tableNum(CustTable)`)
`RecVersion`	`int`	Optimistic concurrency version counter — incremented on every write; used to detect conflicts
`Partition`	`int`	Multi-tenant partitioning — used in cloud environments to separate data by partition
`fieldNum(TableName, FieldName)`	Intrinsic function	Returns the integer field ID at runtime
`fieldId2Name(TableId, FieldId)`	Intrinsic function	Converts a numeric field ID to its string name
`fieldName2Id(tableId, fieldName)`	Intrinsic function	Converts a field name string to its integer ID

Member	Type	Description
`initValue()`	Method	Initializes default values (system fields like `CreatedDateTime`, `ModifiedDateTime`, `CreatedBy`, `ModifiedBy`) for a new record
`insert()`	Method	Writes the buffer to the database as a new record
`update()`	Method	Writes changes to an existing record
`delete()`	Method	Removes the record from the database
`validateWrite()`	Method	Returns `boolean` — returns `true` if the record can be saved, `false` if there is a validation error
`validateField(FieldId)`	Method	Returns `boolean` — validates a single field value
`read()`	Method	Reads the next record from a query result set (used after `select`)
`doInsert()`	Method	Internal — the actual insert; `insert()` calls this after `validateWrite()`
`doUpdate()`	Method	Internal — the actual update; `update()` calls this after `validateWrite()`
`doDelete()`	Method	Internal — the actual delete

The `RecId` field is critical: it is the primary key in SQL Server, it is auto-generated (you never set it), and it is how all table relationships are internally joined. A `select firstonly` on any table returns a buffer with a valid `RecId` if a matching row exists.

1.2.5 The `Object` Base Class

All classes in X++ ultimately inherit from `Object` (not from a named class you define explicitly). `Object` provides:

- `objGetClassName()` → `str` — returns the class name as a string at runtime (used in logging, error messages, and dynamic dispatch)
- `objIsFirstInit()` → `boolean` — whether the object is being constructed for the first time (not a re-instantiation)
- `objSave()` / `objLoad()` — persistence methods (used in specific frameworks)
- `Global::objType(obj)` — returns an enum (`Types::Class`, `Types::String`, `Types::Integer`, etc.) for runtime type checking

---

1.3 Models, Layers, and Packages — Deep Technical Details

1.3.1 What Is a Model?

A **model** is the fundamental unit of versioning and dependency in D365 F&O. A model is a named, versioned collection of AOT objects that share a single `ModelManifest.xml`. Each model:

- Has a **unique name** within the model store (e.g., `MyCustomModel`, `ApplicationSuite`, `ApplicationFoundation`)

- Has a **version** string following semantic versioning (`1.0.0.0`)
- Declares **dependencies** on other models — these are ordered at deployment time
- Is assigned a **layer** (`SYS`, `ISV`, `VAR`, `CUS`, `USR`)
- Exports to a **`.axmodel` file** — a binary package containing the model's metadata as XML serialization blobs

1.3.2 Layers — The Upgrade Boundary

The **layer** is the concept that determines whether your code survives a Microsoft update or gets overwritten.

Layer Hierarchy (from lowest system layer to highest customer layer)

```
SYS (lowest — Microsoft system framework)
  ↓
ISV (Independent Software Vendor layer — partner/ISV solutions)
  ↓
VAR (Variation layer — country/region/legal adaptations by Microsoft)
  ↓
CUS (Customer customization layer — your customizations)
  ↓
USR (User layer — highest, no key required to access)
```

How upgrading works with layers: When Microsoft releases a CU (Cumulative Update), they deliver updates to the VAR layer. Your CUS-layer code is untouched because it's at a higher layer. The framework resolves object resolution by looking for an object at the highest available layer first, then falling back to lower layers.

The resolution algorithm: 1. The system searches for an object by its **name** (class name, table name, form name, etc.) 2. When multiple objects with the same name exist across different layers, the **highest layer** wins 3. Example: If `CustTable` exists in SYS, ISV, VAR, and CUS layers — the CUS-layer version is loaded 4. If you delete your CUS-layer `CustTable` extension, the system falls back to the ISV, VAR, or SYS layer version

What You Need to Know About Each Layer as a Developer

Layer	When It Exists in Practice	Do You Ever Modify It?
SYS	Always — it contains the D365 F&O platform framework (`Sys`, `Global`, `Kernel`, etc.)	**No.** Never modify SYS objects directly — ever. You can extend them via CoC or event handlers.
ISV	Used by Microsoft's ISV partners (Dynamics AX partners) for their solutions	**No** (unless you are an ISV partner developing a managed solution)
VAR	Exists in regional deployments — Microsoft-owned localization	**No.** Microsoft manages these.
CUS	Default layer for customer customizations	**Yes.** This is where you develop as a customer technical intern
USR	User-specific layer, highest priority, no key required	Rarely used directly by developers — mainly for end-user personalizations, not standard development work

1.3.3 Packages — The Deployable Unit

A **package** is the unit of deployment. It contains one or more models and their `.axmodel` files. Packages are created and managed as part of the build pipeline in Azure DevOps or GitHub Actions.

- **One package** can contain **multiple models** (e.g., a package might contain a base model and a customer extension model)
- **One model** is deployed as one `.axmodel` file
- Package dependencies determine deployment order — a package can declare that it depends on another package, meaning the dependency must be deployed first
- Package names follow a naming convention like `MyModel_1.0.0.0` and live in LCS as build artifacts

1.3.4 Model Manifest — The Heart of Your Project

The `ModelManifest.xml` file lives at the root of every Visual Studio model project. It defines everything that the deployment system needs to know about your model.

```
<?xml version="1.0" encoding="utf-8"?>
<ModelManifest xmlns="http://schemas.microsoft.com/dynamics/ax/2014/11/metadata">
  <!-- Model identity -->
  <Name>MyCustomAPModel</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>

  <!-- Dependencies — models that must be deployed BEFORE this one -->
  <References>
    <ModelReference>
      <Name>ApplicationSuite</Name>
      <MinVersion>10.0.0.0</MinVersion>
    </ModelReference>
    <ModelReference>
      <Name>ApplicationFoundation</Name>
      <MinVersion>10.0.0.0</MinVersion>
    </ModelReference>
  </References>

  <!-- Optional: other configuration -->
  <ConfigurationKey>MyCustomAPModelEnabled</ConfigurationKey>
</ModelManifest>
```

`ModelManifest.xml` — Key Elements in Detail:

Element	Required?	Description
`Name`	Yes	The model's unique name — must not collide with any existing model name
`Version`	Yes	Semantic version (`major.minor.build.revision`) — incremented on each deployment
`Layer`	Yes	The layer assignment (`SYS`, `ISV`, `VAR`, `CUS`, `USR`) — determines upgrade behavior
`References.ModelReference.Name`	Conditional	Each dependency reference — the name of a model that must be present for this model to deploy
`References.ModelReference.MinVersion`	Optional	Minimum version of the referenced model required — catches version

Element	Required?	Description
		mismatch errors at deployment time
`ConfigurationKey`	Optional	A configuration key that controls whether this model's features are active

Dependency ordering rules: - A model cannot be deployed unless all its referenced models are already present in the model store - If `MyCustomModel` references `ApplicationSuite`, it means `ApplicationSuite` must be deployed first — in LCS release management, this is handled automatically for standard dependencies - For custom models, you must ensure the build pipeline creates the dependency order correctly

---

1.4 Visual Studio Connection & Project Setup — Production-Grade

1.4.1 Prerequisites Before You Connect

Before opening Visual Studio and connecting to Dynamics 365, ensure:

1. **Visual Studio 2019 or 2022 is installed** — the version must match the LCS environment type (check the LCS environment configuration page for the supported Visual Studio version)
2. **The "Dynamics 365 Developer Tools" workload** is installed in Visual Studio — available via the Visual Studio Installer as a component of the .NET desktop development workload or a separate workload depending on the VS version
3. **Azure AD credentials** for a user with the `Developer` role or equivalent in LCS
4. **Network connectivity** — your machine must be able to reach the Azure region where your LCS environment resides (corporate firewalls may block this)
5. **A valid Azure subscription** linked to the LCS project (LCS handles this provisioning automatically when creating an environment)

1.4.2 Connecting Visual Studio — Step by Step

Step 1: Open Visual Studio Launch Visual Studio and either create a new model project or open an existing solution.

Step 2: Connect to Dynamics 365 - Navigate to **Dynamics 365** → **Connect to Dynamics 365** on the Visual Studio menu bar - A connection dialog appears listing all LCS projects you have access to (based on your Azure AD identity) - Select the LCS project and the target environment (Dev, Test, etc.) - If prompted, authenticate via Azure AD — this uses OAuth 2.0 and requires multi-factor authentication typically

Step 3: Synchronize the AOT After connecting, Visual Studio synchronizes the AOT from the connected environment. This downloads model metadata (XML definitions of all AOT objects) and populates your Visual Studio project view. This may take several minutes for environments with many models.

Step 4: Create or Open a Model Project If the environment has no model project for your customizations yet: - **File** → **New** → **Dynamics 365 Project** - Choose the project template appropriate for your scenario (Class Library, Model Project, etc.) - Visual Studio creates the project with the standard folder structure

If a model project already exists (typical when working in a team): - **File** → **Open** → **Dynamics 365 Project** and navigate to the `.csproj` file in the model store of your connected environment

1.4.3 The Model Store — File System Architecture

The model store is **not** a SQL database. It is a **shared file system directory** on the AOS server machine (on-premises) or an **Azure File Share** (in cloud deployments). The model store contains:

```
C:\Views\ (on-premises AOS)  OR  \\[AzureFileShare]\ModelStore (cloud)
|
+-- Models\
|   +-- MyCustomModel\
|   |   +-- 1.0.0.0\
|   |   |   +-- ModelManifest.xml
|   |   |   +-- Metadata\          ← serialized AOT objects
|   |   |   |   +-- Classes\
|   |   |   |   |   +-- MyServiceClass.xml
|   |   |   |   |   +-- MyExtension.xml
|   |   |   |   +-- Tables\
|   |   |   |   |   +-- MyCustomTable.xml
|   |   |   |   |   +-- MyCustomTable_Extensions.xml
|   |   |   |   +-- Forms\
|   |   |   |   +-- DataEntities\
|   |   |   |   +-- Security\
|   |   |   +-- Binary\            ← compiled IL assemblies, resources
|   |   |       +-- MyCustomModel.dll
|   |   +-- 1.1.0.0\               ← previous version, retained for rollback
|   |       +-- ...
|   +-- ApplicationSuite\
|   |   +-- 10.0.10212.1001\
|   |       +-- ...
|   +-- ApplicationFoundation\
|   |   +-- ...
|   +-- ...
|
+-- ...
```

Key implications: - The model store is **shared** across all AOS instances in the environment — when you deploy a model, it is visible to all AOS workers - Deploying a model **modifies the shared file system** — you need appropriate deployment permissions to write to the model store - The `Binary` folder contains **compiled IL** — when Visual Studio builds your project, it produces the `.dll` file that goes here - Multiple versions of the same model can coexist (for rollback) — the AOS uses the **highest version** that satisfies all dependency requirements

1.4.4 Project Settings You Must Configure Correctly

Model Manifest Configuration

Open `ModelManifest.xml` in your project and verify: - **`Name`** — unique, follows your company's naming convention (e.g., `ContosoCustomAP`) - **`Version`** — starts at `1.0.0.0`, increment with each meaningful change - **`Layer`** — `CUS` for customer work, `ISV` for partner work, `USR` for user-specific customizations - **`References`** — every model your code depends on must be listed: - `ApplicationFoundation` — base system services, `Global` class - `ApplicationSuite` — core business modules (AP, AR, Inventory, etc.) - Any partner model if you depend on ISV code - **`ConfigurationKey`** — if your code introduces new features behind a config key, define it here

Project Build Configuration

- **`Active solution configuration`**: `Release` for deployment builds, `Debug` for development (enables breakpoint debugging)
- **`Target platform`**: x64 (D365 F&O runs on 64-bit servers)

- `Sign assembly`: Required for ISV models (partner code), optional for customer models
- Strong name key file must be generated and specified in project properties
- The public key token is embedded in the assembly and affects model dependency resolution

Debugging Configuration

- `Start Action`: The project must be set to start the Dynamics 365 client or connect to a running environment for debugging
- **Breakpoints**: Set in X++ code files (`*.xpp`) within Visual Studio — breakpoints on static methods (`::main`), instance methods, event handlers, and table validations all work
- **Attach to process**: For debugging on a remote environment, attach the Visual Studio debugger to the AOS process (advanced scenario, typically automated in CI/CD pipelines)

1.5 LCS (Lifecycle Services) — The Orchestration Platform

1.5.1 What Is LCS?

LCS (Lifecycle Services) is a **cloud-based application lifecycle management portal** provided by Microsoft as part of the Microsoft Dynamics 365 platform. It is the central hub for:

- **Environment management** — creating, configuring, and monitoring Azure-hosted environments
- **Build and deployment** — CI/CD pipelines that compile models and deploy them across environments
- **Monitoring and diagnostics** — real-time performance monitoring, SQL query profiling, and Infolog monitoring
- **Release management** — structured promotion of builds from Dev → Test → UAT → Production
- **Change tracking** — tracking all model deployments and configuration changes for audit purposes
- **Reporting** — built-in reports on deployment success/failure, performance, and compliance

1.5.2 LCS Environments — Detailed Overview

Environment	Purpose	Data Source	AOS Tier	Who Has Access	When Updated
Dev/Test	Active development, code compilation, unit testing	Synthetic or demo data (no production data)	Shared development AOS	Developers, QA engineers	Continuous — on every build
Build	Automated build execution — compiles models, produces `.axmodel` artifacts from build pipeline	N/A (build artifacts only)	Ephemeral — exists only during the build	Automated (Azure DevOps pipeline agent)	Triggered by source code commit or manual build request
Test	Integration testing, functional testing by QA	Production data snapshot (refreshed periodically)	Shared test AOS	QA engineers, business analysts	After Dev/Test is validated
UAT	User Acceptance Testing — business users	Production data snapshot	Dedicated UAT AOS	Business stakeholders, business users	After Test is passed

Environment	Purpose	Data Source	AOS Tier	Who Has Access	When Updated
	validate changes before production				
Preview (optional)	Preview of upcoming CUs and hotfixes for testing	Standard data with CU applied	Dedicated preview AOS	ITOps, developers, early adopters	When Microsoft releases a CU
Production	Live business operations	Live transactional data	Production AOS (high-availability, redundant)	IT Operations, limited developer access during deployments	Only after formal release management approval

1.5.3 Azure DevOps Build Pipeline — The Build Process

The standard build pipeline in D365 F&O development follows this structure (Azure DevOps YAML or classic editor):

```
# Simplified build pipeline structure
trigger:
  branches:
    include:
      - main
      - release/*

pool:
  vmImage: 'windows-latest'   # Must match the LCS environment's agent specification

variables:
  - group: 'd365-fao-variables'  # Azure DevOps variable group with LCS credentials
  - name: lcsEnvironmentId      # The LCS environment ID for the build target
  - name: modelName             # The model name in ModelManifest.xml

steps:
  - task: Dynamics365Setup@1    # Step 1: Install the Dynamics 365 Build Tools
    inputs:
      version: '10.0.xxxx.x'   # Version must match the LCS environment

  - task: Dynamics365Build@2    # Step 2: Restore NuGet packages and compile X++
    inputs:
      solution: '**/*.sln'      # The Visual Studio solution file
      project: '$(modelName)'   # The specific model project
      configuration: 'Release'

  - task: Dynamics365Package@3  # Step 3: Create the .axmodel package
    inputs:
      projectPath: '$(Build.SourcesDirectory)/$(modelName)'
      outputPath: '$(Build.ArtifactStagingDirectory)'

  - task: PublishBuildArtifacts@4  # Step 4: Publish as a build artifact
    inputs:
      pathToPublish: '$(Build.ArtifactStagingDirectory)'
      artifactName: 'drop'

  - task: Dynamics365Deploy@5   # Step 5 (optional): Deploy to Test environment automatically
    inputs:
      lcsConnection: 'LCS Service Connection'
```

```
      environmentId: '$(testEnvironmentId)'
      packagePath: '$(Build.ArtifactStagingDirectory)/$(ModelName).axmodel'
```

Build pipeline stages in detail: 1. **Restore**: Downloads NuGet packages that the project depends on (Dynamics SDK assemblies, etc.) 2. **Compile**: Invokes the X++ compiler (`AxBuild.exe`) which reads `.xpp` files from the model store and produces `.dll` IL assemblies 3. **Package**: Creates the `.axmodel` binary file containing all metadata + compiled binaries 4. **Publish**: Makes the artifact available for download or for the next pipeline stage 5. **Deploy** (optional): Uses the LCS API to deploy the artifact directly to a target environment — this step requires a valid Azure Service Principal with LCS deployment permissions

What the X++.compiler does under the hood: - Parses every `.xpp` source file in the project - Resolves all type references (EDTs, enums, classes, tables) against the model store - Performs static analysis (type checking, unused variable detection, method signature validation) - Generates IL bytecode - Generates `.log` files with any compilation errors or warnings - Fails the build if any compilation errors exist — the `.axmodel` is not produced

1.5.4 LCS Release Pipeline — Promoting Builds

The release pipeline in LCS (accessible via the LCS web portal at `lcs.dynamics.com`) manages the movement of your `.axmodel` build artifacts across environments:

1. **Source stage**: The build artifact from the Azure DevOps pipeline is ingested into LCS
2. **Stage gates**: You can define manual approval gates between environments (e.g., "UAT lead must approve before Production deployment")
3. **Deployment to each stage**:
4. LCS copies the `.axmodel` to the target environment's model store (Azure File Share)
5. The AOS on that environment detects the new model
6. On the next AOS recycle (automatic or manual), the model loads into memory
7. The deployment status is reported back to LCS
8. **Rollback**: If a deployment fails, LCS allows rolling back to the previous model version (stored in the model store as a previous version folder — see `C:\Views` structure above)

1.5.5 Hotfix Deployment

When Microsoft releases a Critical Update (CU) or hotfix: 1. The update arrives as a `.axupdate` file package in LCS 2. **IT Operations** applies the `.axupdate` via the LCS environment page 3. The update is applied to the `SYS` layer — **not** to `CUS`, `ISV`, or `VAR` 4. The AOS is recycled to load the updated model store 5. **Your custom code in `CUS`** is unaffected because it was applied after the standard code and takes precedence in the layer resolution order

Critical best practice: After any hotfix deployment, redeploy your custom model to ensure it is still compatible with the updated standard code. Microsoft may have changed a method signature or behavior that your CoC override or event handler depends on.

1.5.6 Configuration Keys

A **Configuration Key** is a boolean flag that controls whether a feature in D365 F&O is active or inactive. Configuration keys are defined in the model store and referenced in code.

Standard keys: | Configuration Key | Purpose | Default | |---|---|---| | `#ISO` | Internationalization features (multi-currency, multi-language, country-specific tax) | On for most deployments | | `USMF` | The default demonstration company — always active in USMF demo data environments | On | | `GeneralLedger` | GL module

features | On for deployments with the GL module | | `AccountsPayable` | AP module features | On for deployments with AP | | `InventoryManagement` | Inventory module features | On for deployments with Inventory |

Custom configuration keys (for your model): When your model introduces a feature that should be toggleable (e.g., a custom compliance validation that is not ready for all users), create a custom configuration key in `ModelManifest.xml` and gate your code with it:

```
// In your class or table validateWrite()
if (FeatureEnabled::isEnabled(featureNum(MyCustomCompliancceFeature)))
{
    // Custom validation logic here
    ...
}
```

The `FeatureEnabled::isEnabled()` check uses the `SysFeatureElement` table to determine at runtime whether the feature is active for the current user/company context.

1.6 The Development Lifecycle — Step by Step in Detail

1.6.1 Developer Workflow (Day-to-Day)

```
[1] Open Visual Studio → Connect to LCS Dev/Test environment
    |
[2] Synchronize AOT and open your model project
    |
[3] Write X++ code (tables, classes, forms, etc.)
    |
[4] Build the project (Ctrl+F6 or right-click → Build Solution)
    |   +-- Success → .axmodel artifact produced
    |   +-- Failure → compilation errors → fix and rebuild
    |
[5] Deploy to the Dev/Test environment (right-click project → Deploy)
    |   +-- LCS uploads the .axmodel to the model store
    |   +-- AOS on the environment detects the new model
    |
[6] Test manually in the D365 F&O client
    |   +-- Open forms, run the new code
    |   +-- Check the Infolog for errors
    |   +-- Set breakpoints in Visual Studio → trigger the code → step through
    |
[7] Repeat steps 3–6 until the feature works correctly
    |
[8] Commit the solution to version control (Git, TFS, or Azure Repos)
    |
[9] Push the build to the Azure DevOps CI pipeline
    |   +-- The pipeline runs automated unit tests (from Chapter 12)
    |
[10] Promote the build to Test → UAT → Production via the release pipeline
```

1.6.2 Debugging in D365 F&O

Breakpoint debugging works from Visual Studio when connected to a Dev/Test environment:

1. Set a breakpoint in an X++ method in Visual Studio (click the left margin or press `F9`)

2. Start the D365 F&O client from the connected environment (or use the Dynamics 365 client already open)
3. Trigger the code path (click a button, run a menu item, etc.)
4. Visual Studio breaks execution at the breakpoint — you can step through (`F10`/`F11`), inspect variable values, and view the call stack
5. Use `F5` to continue execution to the next breakpoint
6. The Infolog updates in real-time as `info()`, `error()`, and `warning()` calls execute

Important debugging notes: - Breakpoints only work when the Visual Studio debugger is attached to the AOS — the default "Connect" mode in Visual Studio handles this attachment automatically - Breakpoints on code that is not yet deployed to the environment will show as hollow (unbound) — deploy first, then set breakpoints on the deployed version - Set breakpoints on `init()`, `executeQuery()`, `validateWrite()`, `modified()`, and `run()` methods — these are the most commonly debugged methods in form-based development - `Global::info()` output appears in Visual Studio's Debug Output window in addition to the Infolog — this is useful for tracing logic execution flow without cluttering the Infolog

1.6.3 The Complete Build → Deploy → Promote Cycle

Step	Environment	Action	Who Triggers
1	Dev/Test (local)	Developer writes code in Visual Studio	Developer
2	Dev/Test (local)	Developer builds and deploys for testing	Developer
3	CI Pipeline	Azure DevOps runs the build, compiles, packages	Automated (triggered by Git commit)
4	Test	CI pipeline deploys the `.axmodel` to Test environment	Automated
5	Test	QA runs manual and automated tests	QA
6	Staging gate	QA lead approves promotion to UAT	QA Lead
7	Release Pipeline	LCS promotes build to UAT	Release manager
8	UAT	Business users validate the feature	Business Users
9	Production gate	Business lead approves promotion to Production	Business Lead
10	Release Pipeline	LCS promotes build to Production	Release manager
11	Production	IT Operations monitors for issues	IT Operations

---

1.7 Activity — Layer & AOT Decision Exercise

> **Scenario**: Acme Manufacturing needs to customize the Accounts Payable invoice approval workflow. The requirements are: 1. Add a new field to `VendTable` for an internal compliance approval

> code 2. Create a new form to display vendor compliance history 3. Modify the standard AP invoice validation to require the compliance code when the vendor's credit limit exceeds $500,000 4. Add a menu item to navigate to the new compliance form 5. Create a security role for compliance officers 6. Generate a monthly compliance report from the AP module 7. Provide a data entity to push compliance data to an external audit system
>
> For each requirement, identify: - **Which AOT node(s)** you would create or modify - **Which layer** you would place each object in and **why** - **Any potential conflicts** with standard D365 F&O objects or future Microsoft updates - **The correct** `ModelName` in `ModelManifest.xml` and its dependency order - **Which Microsoft documentation page** you would consult to verify the standard implementation before customizing

Hints — Multiple Valid Approaches Exist

- **Hint A for requirement #1 — Adding a vendor compliance field**: The choice is between modifying `VendTable` directly (adding a field to the base table) or using a **table extension**.
- **Approach A1 (Table Extension — Recommended)**: `[ExtensionOf(tableStr(VendTable))] final class VendTable_Extension { ComplianceCode complianceCode; }`. This survives Microsoft hotfixes because the extension lives in the `CUS` layer (higher than `SYS`) and is never overwritten by Microsoft updates. The extension is also easier to maintain because Microsoft can change the base `VendTable` without affecting your extension. **This is the approach documented by Microsoft for customer customizations.**
- **Approach A2 (Direct Modification — NOT Recommended)**: Modifying the base `VendTable` in the AOT directly. This **will** be overwritten by the next Microsoft CU because Microsoft's CU applies changes to the `SYS` layer, which has higher update precedence than a naive base modification (the resolution algorithm prefers the highest-layer object, and if your modification is considered part of a lower layer, it gets overwritten). **Microsoft explicitly advises against this pattern.**
- **Hint B for requirement #3 — Modifying AP invoice validation**: The choice is between a **Chain of Command** override of `VendInvoiceJour.validateWrite()` and an **event handler** subscription.
- **Approach B1 (CoC Override — Recommended for Table Method Overrides)**: Create a class that extends `VendInvoiceJour` (or override the method directly via CoC) and override `validateWrite()` to insert your compliance check. CoC is the most straightforward mechanism for overriding standard table methods — it is well-documented and has clear developer tooling support (the AOT shows the override method explicitly). **Use CoC when you need to change the behavior of a standard method called directly (not an event).**
- **Approach B2 (Event Handler — Recommended When Multiple Extensions Need to React)**: Subscribe to the `VendInvoiceJour.validateWrite` event using `[SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]`. This is better when multiple independent solutions need to react to the same event — each subscribes independently without interfering with others. The trade-off is that event handler execution order is less predictable than CoC (which has an explicit chain), so **use event handlers for cross-cutting concerns** like integration triggers, audit logging, or notification dispatch rather than for core business rule enforcement.
- **Hint C for requirement #5 — Security role design**: A compliance officer in this scenario needs access to specific data across multiple tables.
- **Approach C1 (Role with Privileges → Duties)**: Create a duty (e.g., `APComplianceViewer`) that contains privileges for reading `VendTable`, reading `VendInvoiceJour`, and writing to the new compliance form. Create a role and assign the duty. This follows the standard D365 F&O security model hierarchy:

Permission → Privilege (composed of permissions) → Duty (composed of privileges) → Role (composed of duties) → User (assigned one or more roles). **This is the documented Microsoft approach.**

- **Approach C2 (Role with Field-Level Permissions)**: In addition to table-level access, create field-level permissions that restrict the compliance role to only see the compliance-related fields on `VendTable` and `VendInvoiceJour` — this follows the principle of least privilege and is important for compliance-sensitive data. This uses the `SecurityPermissionSet` property on each privilege to limit access to specific `FieldId` values.

- **Hint D for requirement #6 — SSRS Report**:

- **Approach D1 (SSRS Report Wizard in Visual Studio)**: Use the SSRS Report Designer in Visual Studio, define a dataset using a `SrsReportDataProvider` class that contains the query, and design the report layout using the auto-design feature. This is the quickest way to build a functional report but may produce a less polished layout.
- **Approach D2 (External RDL Design)**: Design the RDL file externally in Report Builder or SQL Server Data Tools, then import it into the D365 F&O project. This gives full control over layout and formatting but requires more effort to wire up the data source and parameters. **Microsoft docs recommend approach D1 for initial development and approach D2 for production reports requiring pixel-perfect layouts.**

Expected Approach — Ideal Solution in Detail

The ideal approach for this activity combines all recommended patterns and follows Microsoft's documentation conventions:

Requirement 1: Compliance Code Field on VendTable

- **Object Type**: Table Extension
- **AOT Node**: `Extensions\Tables\VendTable`
- **Layer**: `CUS`
- **AOT Code**: `xpp [ExtensionOf(tableStr(VendTable))] final class VendTable_ComplianceCode_Extension { ComplianceCode complianceCode; // ComplianceCode is an EDT – string 20, mandatory = No, label = "Compliance Code" }`
- **Microsoft Docs Reference**: Table extensions — confirms table extension is the supported pattern for adding fields to base tables
- **Why not modify VendTable directly**: Microsoft explicitly states that modifying base tables is unsupported for customer code and will cause upgrade conflicts

Requirement 2: Compliance History Form

- **Object Type**: New Form (`MenuItemDisplay` to navigate)
- **AOT Node**: `Forms\ComplianceHistoryForm`
- **Layer**: `CUS`
- **Design**: `Form` frame type with two data sources — `VendTable` (primary) and `APComplianceLog` (secondary, joined on `VendTable.RecId = APComplianceLog.VendTableRecId`)
- **`ModelManifest.xml` dependency**: Must reference `ApplicationSuite` (for `VendTable`)

Requirement 3: AP Invoice Validation Override

- **Object Type**: CoC Override on `VendInvoiceJour.validateWrite()`
- **AOT Node**: `Classes\VendInvoiceJour_ComplianceValidate` (extension class, not override class)
- **Layer**: `CUS`
- **AOT Code**: `xpp [ExtensionOf(formStr(VendInvoiceJour))] final class VendInvoiceJour_ComplianceExtension { // Method hook that intercepts the validateWrite call // ... CoC implementation that validates compliance code for high-credit vendors }` *Alternative using Event Handler:* `xpp [SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))] public static void VendInvoiceJour_onValidateWrite(VendInvoiceJour _this) { if (_this.creditMax > 500000 && _this.ComplianceCode == '') { _this.validateWrite = checkFailed('Compliance code required for vendors exceeding $500,000 credit limit'); } }`
- **Note**: The CoC approach is more explicit and easier to debug; the event handler approach allows multiple subscribers. For this scenario, CoC is the recommended approach since it is a core business rule modification with a single owner.

Requirement 4: Menu Item

- **Object Type**: `MenuItemDisplay`
- **AOT Node**: `MenuItems\Display\ComplianceHistory`
- **Object Property**: `ApplicationObject = ComplianceHistoryForm`
- **Layer**: `CUS`
- Added to the AP **Menu** or as a standalone navigation item in the workspace

Requirement 5: Security Role for Compliance Officers

- **Object Type**: Security Role (with Duties, Privileges, Permissions)
- **AOT Node**: `Security\Roles\ComplianceOfficerRole`
- **Duty**: `APComplianceDuty` → contains `APComplianceViewer` privilege (read on `VendTable`, `VendInvoiceJour`, `APComplianceLog`, write on `APComplianceLog` for submitting declarations)
- **Field-level permissions**: Compliance officers can see the `ComplianceCode` field on `VendTable` and all fields on `APComplianceLog`; other users cannot
- **PermissionSet**: The role is assigned to the security role assignment record in the user administration form
- **Microsoft Docs Reference**: Security roles — confirms the hierarchy: Permission → Privilege (composed of permissions) → Duty (composed of privileges) → Role (composed of duties) → User (assigned one or more roles)

Requirement 6: Monthly Compliance Report (SSRS)

- **Object Type**: SSRS Report + `SrsReportDataProvider` class
- **AOT Node**: `Reports\ComplianceSummaryReport` + `Classes\VendComplianceReportDP`
- **Data Provider**: `VendComplianceReportDP` extends `SrsReportDataProvider` — contains the SSRS query joining `VendTable`, `VendInvoiceJour`, `APComplianceLog`, and `VendInvoiceLine`
- **Parameters**: Date range (`fromDate`, `toDate`), vendor group (`CustGroup`), compliance status (`Validated`, `Pending`, `Failed`)
- **Report layout**: AutoDesign initially, then manual RDL for production
- **Menu item**: `MenuItemOutput` linked to the report class
- **Microsoft Docs Reference**: SSRS reports in D365 F&O — confirms the `SrsReportDataProvider` pattern

Requirement 7: Data Entity for External Audit Integration

- **Object Type**: Data Entity (`DataEntity`)
- **AOT Node**: `Data Entities\VendAPComplianceEntity`
- **Structure**:
- Staging table fields (intermediate state) stored in `VendAPComplianceStaging`
- Output fields exposed via `IsPublic`, `Public Entity Name`, and `Public Collection Name` properties
- `Link` field: connects `APComplianceLog` to the entity context (`VendTable.RecId`)
- **Staging pattern**: Uses a staging table (`VendAPComplianceStaging`) that accumulates records, validates them, and then pushes them to the target system via a custom `.NET` or `Logic App` integration
- **Microsoft Docs Reference**: Data entities — confirms the staging + entity pattern for integration scenarios

Model Manifest & Dependencies

```
<ModelManifest>
  <Name>AcmeAPComplianceModel</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
  <ConfigurationKey>AcmeAPComplianceEnabled</ConfigurationKey>
</ModelManifest>
```

Chapter 2 — X++ Language Fundamentals and Class Design

Before you can work with tables, forms, or data entities, you need to be fluent in X++ itself — the language that runs on the Application Object Server. This chapter covers the language fundamentals and the object-oriented patterns you'll use every day.

2.1 X++ Data Types

X++ has a set of primitive data types and a rich system of Extended Data Types (EDTs) that wrap primitives with additional metadata.

Primitive Data Types

Type	Description	Example
`int`	32-bit signed integer	`int i = 42;`
`int64`	64-bit signed integer	`int64 recId = 5637144576;`
`real`	Fixed-point decimal-like precision (not IEEE-754)	`real price = 19.99;`
`str`	Unicode string	`str name = "Customer";`
`boolean`	true / false	`boolean isValid = true;`
`date`	Date (no time component)	`date d = today();`
`utcdatetime`	Date and time in UTC	`utcdatetime udt = DateTimeUtil::utcNow();`
`timeOfDay`	Seconds since midnight	`timeOfDay t = 120000;`
`guid`	Globally unique identifier	`guid g = Guid::newGuid();`
`enum`	Named constant set	`Status::New`
`AnyType`	Universal type	`AnyType val = 42;`

Type Casting

```
// Explicit casting from container to specific types
container c = [1, "hello", 3.14];
int     i = conPeek(c, 1);       // 1
str     s = conPeek(c, 2);       // "hello"
real    r = conPeek(c, 3);       // 3.14

// Casting between record types (must be compatible table types)
CustTable custTable = CustTable::find("CUST-001");
VendTable vendTable = custTable;  // Compile error — incompatible types
```

`AnyType` — The Universal Type

`AnyType` can hold any X++ type. It is used extensively in framework code (e.g., `SysOperationServiceController`, `RunBase`). Use `conPeek` and `conLen` to work with `AnyType` containers.

```
AnyType anyValue = 42;
// You must know the actual type at runtime to use it safely
if (anyValue is int)
{
    int i = anyValue;
    info(strFmt("The value is %1", i));
}
```

2.2 Variable Declaration and Scope

Declaration Keywords

Keyword	Scope	Lifetime
`static`	Class-level	Persists for the lifetime of the application
`const`	Class-level	Immutable after initialization
Instance variable	Object-level	Persists for the lifetime of the object
Local variable	Method/block	Created on entry, destroyed on exit

Declaration Rules

```
class MyClass
{
    // Static constant — class-level, immutable
    static const Str AppName = "MyApp";

    // Static variable — shared across all instances
    static int instanceCount = 0;

    // Instance variable — each object has its own copy
    CustTable _custTable;
    boolean   _isValid;

    public void myMethod()
    {
        // Local variable — only visible in this method
        int localVar = 10;

        // Block-scoped variable (inside if/for/while)
        if (localVar > 0)
        {
            str blockVar = "visible only here";
            info(blockVar);
        }
        // blockVar is NOT accessible here — compile error
    }
}
```

Important: X++ **does** support block-level scoping for variables. A variable declared inside a block (e.g., `if`, `for`, `while`) is only accessible within that block. This is similar to C# or Java.

```
// CORRECT — X++ supports block-scoped variables
public void goodExample()
{
    if (true)
    {
        int x = 5;  // x is only visible inside this block
        info(strFmt("x = %1", x));
    }
    // x is NOT accessible here — it's a compile error
    // because x was declared inside the if block
}
```

Source: X++ variables - Declare anywhere

2.3 Control Flow Statements

If / Else If / Else

```
int score = 85;

if (score >= 90)
{
    info("Grade: A");
}
else if (score >= 80)
{
    info("Grade: B");    // This branch executes
}
else if (score >= 70)
{
    info("Grade: C");
}
else
{
    info("Grade: F");
}
```

Switch

X++ `switch` supports `int`, `str`, and `enum` types. Unlike C#, there is no `case` fall-through — each case must end with `break` or `return`.

```
Status orderStatus = Status::Approved;

switch (orderStatus)
{
    case Status::New:
        info("Order is new");
        break;

    case Status::Approved:
        info("Order is approved");
        break;

    case Status::Rejected:
        info("Order is rejected");
        break;
```

```
    default:
        info("Unknown status");
        break;
}
```

For Loop

```
// Standard for loop
for (int i = 1; i <= 10; i++)
{
    info(strFmt("Iteration %1", i));
}

// Iterating over a container
container numbers = [10, 20, 30, 40, 50];
for (int i = 1; i <= conLen(numbers); i++)
{
    info(strFmt("Element %1 = %2", i, conPeek(numbers, i)));
}
```

While Select — The X++ Iteration Pattern

`while select` is the most idiomatic way to iterate over table records in X++. It combines a SQL `SELECT` with a `while` loop — the AOS generates efficient SQL and streams records one at a time.

```
// Basic while select
CustTable custTable;
while select custTable
{
    info(custTable.AccountNum);
}

// while select with a where clause
while select custTable
    where custTable.Currency == "USD"
{
    info(custTable.AccountNum);
}

// while select with ordering
while select custTable
    order by custTable.AccountNum
{
    info(custTable.AccountNum);
}

// while select with firstonly — stops after the first match
while select firstonly custTable
    where custTable.AccountNum == "CUST-001"
{
    info(custTable.Name);
}
```

Performance note: `while select` is far more efficient than a `for` loop with individual `select` statements. Each individual `select` generates a separate SQL round-trip; `while select` streams results from a single query.

Do While

```
int i = 1;
do
{
    info(strFmt("Value: %1", i));
    i++;
}
while (i <= 5);
```

2.4 Table Operations (CRUD)

Select

```
CustTable custTable;

// Simple select — retrieves all records
while select custTable
{
    info(custTable.AccountNum);
}

// Select with a where clause
while select custTable
    where custTable.AccountNum == "CUST-001"
{
    info(custTable.Name);
}

// Select firstonly — optimized; stops after first match
select firstonly custTable
    where custTable.AccountNum == "CUST-001";

// Select specific fields only — reduces SQL payload
while select custTable.Name, custTable.Phone()
{
    info(strFmt("%1 — %2", custTable.AccountNum, custTable.Name));
}

// exists join — check for related records without retrieving them
VendTable vendTable;
boolean hasVendor = exists join vendTable
    where vendTable.AccountNum == custTable.AccountNum;
```

Insert

```
CustTable custTable;
custTable.clear();                    // Clear all fields to defaults
custTable.AccountNum = "CUST-NEW-001";
custTable.Name = "New Customer";
custTable.Phone = "+1-555-0100";
custTable.Currency = "USD";
custTable.insert();                   // Writes to the database
```

Update — `update_recordset`

`update_recordset` is the preferred X++ pattern for updating records. It generates a single SQL `UPDATE` statement — much more efficient than selecting a record, modifying it, and calling `.update()` in a loop.

```
// Update a single record
CustTable custTable;
update_recordset custTable
    setting custTable.Phone = "+1-555-0200"
    where custTable.AccountNum == "CUST-NEW-001";

// Update multiple records at once — single SQL statement
CustTable custTable;
update_recordset custTable
    setting custTable.Currency = "EUR"
    where custTable.Currency == "USD";
```

Delete — `delete_from`

```
// Delete a single record
CustTable custTable;
delete_from custTable
    where custTable.AccountNum == "CUST-TO-DELETE";

// Delete multiple records
CustTable custTable;
delete_from custTable
    where custTable.Currency == "EUR" && custTable.GroupName == "TEMP";
```

Transactions (`ttsBegin` / `ttsCommit` / `ttsAbort`)

All database modifications within a transaction are atomic — either all succeed or all are rolled back.

```
ttsBegin;
try
{
    CustTable custTable;
    custTable.clear();
    custTable.AccountNum = "CUST-TRANS-001";
    custTable.Name = "Transactional Customer";
    custTable.insert();

    // Related record in another table
    CustGroup custGroup;
    custGroup.clear();
    custGroup.GroupName = "TRANSACTIONAL";
    custGroup.insert();

    ttsCommit;  // Both inserts succeed — commit the transaction
}
catch (Exception::Error)
{
    ttsAbort;   // Something failed — roll back everything
    error("Transaction failed — all changes rolled back.");
}
```

Critical rule: Always call `super()` first in table `insert()`, `update()`, and `delete()` methods, and in `validateWrite()` / `validateDelete()`. The base class methods handle framework-level logic (workflow, auditing, etc.) that your code depends on.

2.5 Method Overriding and `super()`

X++ supports method overriding through inheritance. When a subclass overrides a base class method, it can call the base implementation using `super()`.

Overriding in Tables

```
// In a table — overriding validateWrite
public boolean validateWrite()
{
    boolean ret;

    ret = super();  // Always call super() first in table methods

    // Custom validation — runs AFTER the base validation
    if (this.AccountNum == "")
    {
        ret = false;
        error("Account number cannot be empty.");
    }

    return ret;
}
```

Overriding in Classes

```
// Base class
class BasePaymentService
{
    public void pay(CustTable _custTable, Amount _amount)
    {
        // Base implementation — process the payment
        info(strFmt("Processing payment of %1 for %2", _amount, _custTable.AccountNum));
    }
}

// Derived class — extends base payment logic
class CreditCardPaymentService extends BasePaymentService
{
    public void pay(CustTable _custTable, Amount _amount)
    {
        // Pre-processing
        this.validateCard();

        // Call base logic — the actual payment processing
        super::pay(_custTable, _amount);

        // Post-processing
        this.logTransaction();
    }

    private void validateCard()
    {
        // Card-specific validation
    }

    private void logTransaction()
    {
```

```
        // Log to a custom transaction table
    }
}
```

The `next` Keyword — Chain of Command

When multiple extensions want to modify the same base method, the Chain of Command (CoC) pattern uses `next` to call the next handler in the chain. (Covered in depth in Chapter 7.)

```
// In an extension class
[ExtensionOf(tableStr(CustTable))]
final class CustTable_Extension
{
    public boolean validateWrite()
    {
        boolean ret;

        ret = next validateWrite();  // Call the next handler in the chain

        // Custom validation
        if (this.AccountNum == "")
        {
            ret = false;
            error("Account number cannot be empty.");
        }

        return ret;
    }
}
```

2.6 Exception Handling

X++ uses a `try`/`catch` block with typed exceptions. The `Exception` enum defines the error categories.

Exception Types

Exception Type	When It Occurs
`Exception::Error`	General runtime error — the most common
`Exception::Warning`	Non-fatal warning — execution continues
`Exception::Info`	Informational message
`Exception::Broken`	Object is in an invalid state
`Exception::Deadlock`	SQL deadlock detected — retry the transaction
`Exception::DuplicateKey`	Unique index violation

Try / Catch Pattern

```
try
{
    CustTable custTable;
    select firstonly custTable
        where custTable.AccountNum == "NON-EXISTENT";
```

```
    if (!custTable)
    {
        throw error("Customer not found.");
    }
}
catch (Exception::Error)
{
    error("An error occurred.");
}
catch (Exception::Warning)
{
    warning("A warning occurred.");
}
catch (Exception::Info)
{
    info("Informational message.");
}
```

Deadlock Handling

Deadlocks are common in multi-user environments. The recommended pattern is to retry the transaction:

```
int retryCount = 0;
boolean success = false;

while (!success && retryCount < 3)
{
    ttsBegin;
    try
    {
        // ... your database operations ...
        ttsCommit;
        success = true;
    }
    catch (Exception::Deadlock)
    {
        ttsAbort;
        retryCount++;
        info(strFmt("Deadlock detected — retry %1 of 3", retryCount));
    }
    catch (Exception::Error)
    {
        ttsAbort;
        error("A non-deadlock error occurred.");
        break;
    }
}
```

2.7 Classes and Objects

2.7.1 Class Structure

Every X++ class has three sections: **declarations** (class-level variables), **constructor(s)**, and **methods**.

```
/// <summary>
/// Business logic class for customer validation.
/// </summary>
```

```
class CustValidationService
{
    // -- Class-level (static) variables --
    static const Str MandatoryFieldMissing = "Mandatory field missing: ";

    // -- Instance variables --
    CustTable _custTable;
    boolean   _isValid;

    // -- Constructor --
    /// <summary>Initializes the service with a customer record.</summary>
    public CustValidationService(CustTable _custTable)
    {
        this._custTable = _custTable;
        this._isValid = false;
    }

    // -- Public methods --

    /// <summary>Entry point — validates all rules and returns the result.</summary>
    public boolean validate()
    {
        if (this.validateAccountNum() && this.validateName())
        {
            this._isValid = true;
        }
        return this._isValid;
    }

    // -- Private helper methods --

    /// <summary>Validates that the account number is not empty.</summary>
    private boolean validateAccountNum()
    {
        if (this._custTable.AccountNum == '')
        {
            error(strFmt(MandatoryFieldMissing, "Account number"));
            return false;
        }
        return true;
    }

    /// <summary>Validates that the customer name is not empty.</summary>
    private boolean validateName()
    {
        if (this._custTable.Name == '')
        {
            error(strFmt(MandatoryFieldMissing, "Name"));
            return false;
        }
        return true;
    }

    // -- Public static utility method --

    public static boolean isAccountValid(str _accountNum)
    {
        return (_accountNum != '' && strLen(_accountNum) >= 3);
    }
}
```

Using the class:

```
CustTable custTable = CustTable::find("CUST-001");
CustValidationService service = new CustValidationService(custTable);

if (service.validate())
{
    info("Customer is valid.");
}
else
{
    info("Customer validation failed — check the Infolog for details.");
}
```

2.7.2 Inheritance and Polymorphism

X++ supports single inheritance — a class can extend exactly one base class using the `extends` keyword.

```
// Base class
class BasePaymentService
{
    /// <summary>Processes a payment. Designed to be overridden.</summary>
    public void pay(CustTable _custTable, Amount _amount)
    {
        info(strFmt("Processing payment of %1 for %2", _amount, _custTable.AccountNum));
    }
}

// Derived class — extends base payment logic
class CreditCardPaymentService extends BasePaymentService
{
    public void pay(CustTable _custTable, Amount _amount)
    {
        // Pre-processing
        this.validateCard();

        // Call base logic — the actual payment processing
        super::pay(_custTable, _amount);

        // Post-processing
        this.logTransaction();
    }

    private void validateCard()
    {
        // Card-specific validation
    }

    private void logTransaction()
    {
        // Log to a custom transaction table
    }
}
```

Polymorphism in action — a list of base-class references can hold derived objects:

```
BasePaymentService paymentService;

// At runtime, the actual object type determines which pay() executes
```

```
paymentService = new CreditCardPaymentService();
paymentService.pay(custTable, 100.00);  // Calls CreditCardPaymentService::pay()
```

2.7.3 Abstract Classes and Interfaces

Abstract classes cannot be instantiated directly — they serve as blueprints for subclasses. They may contain both abstract (unimplemented) and concrete (implemented) methods.

Interfaces define a contract — a set of method signatures that implementing classes must provide. They enable polymorphism across unrelated class hierarchies.

```
// Abstract class — cannot be instantiated directly
abstract class AbstractReportGenerator
{
    /// <summary>Abstract method — must be overridden by subclasses.</summary>
    public abstract void generate();

    /// <summary>Concrete method — shared logic available to all subclasses.</summary>
    public void logGeneration()
    {
        info("Report generation started.");
    }
}

// Concrete implementation
class SalesReportGenerator extends AbstractReportGenerator
{
    public void generate()
    {
        this.logGeneration();  // Inherited from abstract class
        // Sales report generation logic here
    }
}

// Interface — defines a contract without implementation
interface IExportable
{
    void exportToFile(str _filePath);
    str getExportFormat();
}

// A class can implement an interface AND extend a class
class CsvSalesReportGenerator extends AbstractReportGenerator implements IExportable
{
    public void generate()
    {
        this.logGeneration();
        // CSV-specific report logic
    }

    public void exportToFile(str _filePath)
    {
        // Write CSV to file
    }

    public str getExportFormat()
    {
        return "CSV";
```

```
    }
}
```

2.8 Visual Studio Development Environment

> **Refresher** — detailed setup was covered in Chapter 1, Section 1.4. This section highlights the X++-specific development features.

- **IDE**: Visual Studio 2022 with the `Microsoft.Dynamics.FinOps.ToolsVS2022.vsix` extension
- **X++ Editor**: Syntax highlighting, IntelliSense for AOT objects, auto-completion for table fields and EDTs
- **Build**: MSBuild with custom X++ targets — compiles `.xpp` files to .NET CIL
- **Debugging**: Set breakpoints in X++ classes, step through code, inspect variables in the Locals/Watch windows
- **Cross-reference**: SQL Server Express LocalDB tracks object dependencies — right-click any element → **Find References**

2.9 Activity — Build a Customer Validation Service

> **Activity**: Create a class called `CustValidationService` that validates customer records before they are saved. The service must: 1. Accept a `CustTable` record in its constructor 2. Validate that `AccountNum` is not empty and is at least 3 characters long 3. Validate that `Name` is not empty 4. Validate that the `Currency` field is set to a valid currency (check against `CurrencyTable`) 5. Return a boolean from a `validate()` method and write error messages to the Infolog for each failed rule 6. Include a static method `isAccountNumValid(str _accountNum)` that can be called without instantiating the class
>
> **Hints** (no single approach — explore multiple paths): - **Hint A**: For the currency validation — should you use a `select firstonly` against `CurrencyTable`, or maintain a static list of valid currency codes? What are the trade-offs between accuracy (database lookup) and performance (static list)? - **Hint B**: Where should the error messages be generated — inside each private validation method, or collected in a list and reported at the end? Consider how the Infolog displays messages and whether the user needs to see all failures at once or just the first one. - **Hint C**: Should the `validate()` method call `super()`? In this case, there is no base class — but if you later extend `CustValidationService` with a subclass, what pattern should you follow for the `validate()` method?
>
> **Expected Approach (Ideal — in detail)**: 1. **Constructor**: Accepts `CustTable` and stores it in an instance variable. Initializes `_isValid = false`. 2. **`validate()`**: Calls each private validation method in sequence. If any returns `false`, `_isValid` stays `false`. Returns `_isValid`. 3. **`validateAccountNum()`**: Checks `AccountNum != ''` and `strLen(AccountNum) >= 3`. Uses `error()` to write to the Infolog. Returns `boolean`. 4. **`validateName()`**: Checks `Name != ''`. Uses `error()` to write to the Infolog. Returns `boolean`. 5. **`validateCurrency()`**: Uses `select firstonly CurrencyTable where CurrencyTable.CurrencyCode == this._custTable.Currency`. If no record is found, writes an error and returns `false`. 6. **`isAccountNumValid()`**: Static method — `return (_accountNum != '' && strLen(_accountNum) >= 3);`. No Infolog output; this is a pure validation function. 7. **Design note**: The class uses private helper methods for each validation rule, making it easy to add new rules later without modifying the `validate()` method. This follows the Open/Closed Principle.

Chapter 3 — Tables, Fields, and Data Integrity

3.1 Table Design — Foundational Principles

Every table in D365 F&O maps to a physical SQL table. Designing tables properly is the single most impactful decision you'll make — bad table design compounds into poor performance, upgrade conflicts, and data integrity issues.

Table Properties — Key Properties

Property	Description	Common Values
`Label`	User-facing display name (uses label files)	`'Customer Table'`
`HelpText`	Tooltip description	`'Contains customer account data'`
`TableGroup`	Logical grouping for form lookup display	`Customer`, `Vend`, `Master`, `Transaction`
`SaveDataPerCompany`	Whether each company's data is stored separately (`No = shared across companies`)	`No` or `Yes`
`AllowDuplicate`	Allow multiple records with the same key?	`No` (default)
`CacheLookup`	How SQL Server caches lookups of this table	`None`, `NotInTTS`, `Found`, `FoundAndEmpty`, `EntireTable`
`Replication`	Whether table data is replicated (for Azure SQL geo-replication)	`Enabled` or `Disabled`
`PrimaryIndex`	The primary index used for lookups	Typically `RecId`, or an alternate key
`DeleteActions`	What happens when a parent record is deleted	`Cascade`, `Restricted`, `None`
`TableType`	Normal vs. Temp vs. Word template	`Normal`, `TempDB`, `Temp`, etc.

Index Design — Critical for Performance

Indexes determine how fast `select`, `join`, and `exists` operations run. Every table has a primary index; adding secondary indexes is a key design decision.

Index Type	Description	When to Use
Primary Index	Clustered index on `RecId` (auto-created)	Always present; do not change
Alternate Index	Non-clustered, unique constraint	When a natural key exists (e.g., `AccountNum` must be unique)
Hash Index	For equality-only lookups (no range scans)	When you only filter by equality (`where AccountNum == '...'`)
Non-unique Index	Allows duplicates; optimizes range scans and sorting	When you filter on a field that has many duplicates

Index Type	Description	When to Use
Group Level Index	Multi-field index where the leading field determines selectivity	When queries always filter on the same leading field

Index Design Rules

1. **Leading field matters most** — in an index on `(FieldA, FieldB)`, SQL can use the index for `where FieldA = ...`, `where FieldA = ... AND FieldB = ...`, but NOT for `where FieldB = ...` alone
2. **Never index low-selectivity fields** as leading fields — indexing a boolean field (`true/false`) provides almost no benefit because SQL must scan anyway
3. **Use `AllowDuplicates = No`** on alternate uniqueness indexes — this enforces data integrity at the database level, not just at the application level
4. **`CacheLookup` matters** — tables with `CacheLookup = Found` or `FoundAndEmpty` allow SQL Server to cache lookups and skip disk reads; use this for lookup tables (e.g., `ItemGroup`, `CustGroup`)

3.2 Extended Data Types (EDTs) — Never Use Base Types Directly

An EDT is a type alias that wraps a base type (string, int, real, etc.) with additional properties. **Why EDTs matter:**

- **Consistency**: If `CustAccount` is a `string 20` EDT used on 50 tables, changing the EDT to `string 30` updates all 50 tables at once
- **Validation**: An EDT can have `StringSize`, `Mask`, `RegularExpression` validation rules
- **Relationships**: An EDT can be linked to another table (e.g., `CustAccount` EDT links to `CustTable`), enabling **field relation** auto-completion in forms
- **Reference group**: An EDT can declare that it "references" another table, which enables the `RefTableId` and `RefRecId` automatic fields on tables using that EDT

EDT Properties — Key Properties

Property	Description	Example
`Extends`	Base type the EDT wraps	`string`, `integer`, `real`, `enum`
`StringSize`	Max length (if `Extends = string`)	`20`
`Relationship`	EDT references a table for lookup relation	`CustTable`
`ReferenceField`	On the referenced table, which field is shown in lookups	`AccountNum`
`ArraySize`	When `Extends = string`, the array size for memory	`1` (default: fixed-length)
`Label`	User-facing display name	`'Customer Account'`
`HelpText`	Tooltip description	`'Unique customer identifier'`

The `CustAccount` EDT — A Real-World Example

```
EDT Name: CustAccount
Extends: string
```

```
StringSize: 20
Relationship: CustTable → AccountNum
ReferenceField: AccountNum
Label: CustAccount
HelpText: Customer account number
```

When you use `CustAccount` as a field type on any table, and then build a form with that table as a data source, the `AccountNum` field automatically gets a **lookup** to find and select existing customer records — this is entirely driven by the EDT metadata.

3.3 Field Groups

Field groups are **named sets of fields** defined at the table level. They provide:

1. **Form auto-generation**: A form can include an entire field group — adding new fields to the group automatically adds them to all forms using it
2. **Report generation**: SSRS queries can reference field groups instead of listing every field individually
3. **Documentation**: Field groups describe the logical grouping of related fields (e.g., `Address`, `Financial`, `ContactInfo`)

Field Group Properties

Property	Description
`Label`	Display name of the field group
`Fields`	The list of fields included in the group

Field Group Usage in Forms

When a form's data source references a table with field groups, you can drag the **entire field group** onto a form design node — all fields in the group appear automatically with their labels and EDT properties.

3.4 Table Extensions — The Modern Extension Pattern

Note: *Table inheritance* (using `SupportInheritance`/`Extends` properties with a discriminator field like `InstanceRelationType`) and *table extension* (using `[ExtensionOf(tableStr(...))]`) are **two distinct mechanisms** in D365 F&O. This section covers **table extension**, which is the recommended approach for upgrade-safe customizations. For table inheritance (a separate feature for hierarchical table relationships), see Table inheritance overview (AX 2012 legacy).

Microsoft recommends **table extension** over overlayering or subclassing tables. The extension pattern adds fields or methods to an existing table without modifying the base table definition.

Table Extension Syntax

```
[ExtensionOf(tableStr(VendTable))]
final class VendTable_Extension
{
    ComplianceCode complianceCode;

    // Extended field: add a new field to the existing VendTable
    // This field appears on VendTable in the AOT as if it were on the base table
```

```
    // but it is deployed in the extension model, so it survives upgrades
}
```

What Table Extension Gives You

- **New fields**: Add additional columns to the existing table
- **New methods**: Add methods to the existing table's method list
- **Method overrides**: Override existing methods using CoC or event handlers (not direct override — must use extension framework)
- `validateWrite()` **override**: Add validation logic without modifying the base table
- `modifyField()` **override**: Add logic when a field is changed

What Table Extension Does NOT Give You

- **Cannot change existing methods** — you can only override via CoC or event handlers
- **Cannot change table properties** — you cannot change `AllowDuplicate`, `CacheLookup`, etc. on a table via extension
- **Cannot remove fields** — extension only adds

3.5 Table Validation — Where to Validate

Data integrity is enforced at three levels, each with a specific role:

Level 1: Table-level Validation (`validateWrite()`)

Called whenever a record is about to be written (insert or update). This is where **business rules** live — rules that must be true before any record can be saved.

```
// In VendTable or in a table extension:
public boolean validateWrite()
{
    boolean ret;
    ret = super();  // Always call super() first — standard validations run here

    // Add custom validation
    if (this.ComplianceCode == '' && this.creditMax > 500000)
    {
        ret = checkFailed(literalStr('compliance code is required for vendors with credit limit over
$500,000'));
    }

    return ret;  // true = allow save, false = reject with explanation in Infolog
}
```

Key rules: - Always call `super()` first — standard D365 validations run in `super()` and set the `ret` flag - Use `checkFailed()` to display an error message in the Infolog — this returns `false` from `validateWrite()` but does NOT throw an exception - `checkFailed()` automatically includes the field label and the message — the user sees the field name and the error text - Never use `error()` inside `validateWrite()` — use `checkFailed()` instead

Level 2: Field-level Validation (`validateField()`)

Called when a single field value is being set. Used for field-specific validation (e.g., format checks, range checks).

Level 3: Application-level Validation (Class Methods)

Business logic that is too complex to fit in table validation — lives in service classes or `RunBase` classes. Called from the UI layer or batch jobs.

3.6 Record Lifecycle and the `modifiedField()` Pattern

When a field value changes on a record, `modifiedField(fieldId)` is called automatically by the framework. You can override this to react to specific field changes.

```
// In a table extension:
public void modifiedField(FieldId _fieldId)
{
    switch (_fieldId)
    {
        case fieldNum(VendTable, CreditMax):
            // When credit limit changes, validate compliance requirements
            if (this.CreditMax > 500000 && this.ComplianceCode == '')
            {
                warning('Credit limit exceeds $500,000 — compliance code is recommended');
            }
            break;
    }
    super(_fieldId);  // Always call super() LAST
}
```

Critical: `super(_fieldId)` goes LAST in `modifiedField()` — opposite of `validateWrite()` where `super()` goes FIRST.

3.7 `insert_recordset` / `update_recordset` — Set-Based Performance

These operations execute in a single SQL statement rather than looping record-by-record, and are dramatically faster for large data volumes.

When to Use Set-Based Operations

- **`insert_recordset`**: Bulk loading data from staging tables, temporary tables, or query results
- **`update_recordset`**: Applying a transformation to many records at once (e.g., status changes, field updates)
- **`delete_from`**: Deleting a set of records matching a condition

When NOT to Use Set-Based Operations

- When the table's `validateWrite()`, `insert()` or `update()` methods contain business logic that must execute for each record
- When event handlers on the table need to fire for each record
- When you need `RecId` values back from each inserted record (set-based inserts don't return `RecId` values)

Set-Based Insert Example

```
// Bulk insert from staging to target — extremely fast
insert_recordset VendTable (AccountNum, Name, CreditMax, Currency)
    select AccountNum, Name, CreditMax, Currency
    from VendStagingTable
    where VendStagingTable.StagingStatus == StagingStatus::Ready;
```

Set-Based Update Example

```
// Update all pending invoices older than 30 days
update_recordset VendInvoiceJour
    setting Status = VendInvoiceStatus::Late
    where VendInvoiceJour.Status == VendInvoiceStatus::Pending
    && VendInvoiceJour.InvoiceDate < today() - 30;
```

3.8 Activity — Custom Table Design

> **Activity**: Design a table called `APCustomsDeclaration` for storing customs compliance declarations that must link to each vendor invoice line. Requirements: 1. The table must store the declaration ID (unique, not auto-generated), declaration date, customs authority reference, and the invoice line `RecId` 2. The declaration ID must be formatted as `COMP-YYYY-VNDNNNNN-#######` where COMP is the company ID, YYYY is the year, VNDNNNNN is the vendor account number, and ####### is a sequential declaration number 3. Create an EDT for the declaration ID with validation that the format is correct 4. Add an alternate index on the declaration ID (`AllowDuplicates = No`) 5. Use a table extension on `VendInvoiceLine` to add a field that links to this new table 6. Design a `validateWrite()` method on the new table that ensures the referenced vendor invoice line actually exists
>
> **Hints** (no single approach — explore multiple paths): - **Hint A**: For the declaration ID format — where does the format logic belong? In the EDT (rejected — EDTs cannot contain logic), in a class method `generateDeclarationId()` (one valid approach), in a table `validateWrite()` (partial — validation is not generation), or using an `insert()` override on the table (generation + validation together — a valid but more complex approach). - **Hint B**: For linking to `VendInvoiceLine` — should you use EDT-based field relation (where the EDT is linked to `VendInvoiceLine.RecId`), or a manual lookup that queries the `VendInvoiceLine` table for the matching `RecId`? - **Hint C**: For sequential declaration numbering per vendor per year — should you use a dedicated sequence table, a `select max(declarationNumber)` query (race condition risk), or a `HoleId` (number sequence framework)? What are the concurrency implications of each?

Expected Approach (Ideal — in detail)

The ideal solution: 1. **EDT**: Create `APCustomsDeclId` as a `string 35` EDT with `StringSize = 35`, no relationship (the declaration ID is generated, not entered by the user), no `Relation` property — it is a synthetic identifier, not a foreign key. The `RegularExpression` property can be set to validate the format pattern. 2. **Table design**: `APCustomsDeclaration` with fields: `DeclId` (new EDT), `DeclDate` (`date`), `CustomsRef` (`string 50`), `InvoiceLineRecId` (`int64` — the physical `RecId` type, not an EDT since it references a table row), plus standard fields. 3. **Indexes**: Primary index on `RecId` (auto), alternate index on `DeclId` with `AllowDuplicates = No`, and a non-unique index on `InvoiceLineRecId` for fast lookup of related declarations. 4. **Table extension on `VendInvoiceLine`**: Add an `APDeclId` EDT field (nullable) that links to the customs declaration. This preserves the base `VendInvoiceLine` table. 5. **Declaration ID generation** in the table's `insert()` method or a wrapper class method — **not** the EDT. Use a combination of `CompanyInfo::name()`, `year(today())`, `VendTable::find()` lookup, and a **number**

sequence (`NumberSeq`) to generate the sequential portion safely under transaction control. The `NumberSeq` framework handles concurrency and deduplication. 6. **`validateWrite()`** uses `select firstonly exists` to check that `VendInvoiceLine` with the given `RecId` exists before allowing insert.

Chapter 4 — Forms, Controls, and User Experience

4.1 Form Architecture

A form in D365 F&O is a UI presentation layer that sits on top of one or more table data sources. The form handles the visual layout (design node), data retrieval (data source nodes), and user interaction (control nodes and their events).

Form Data Source — The Bridge

Every form has one or more **Data Source** nodes, each pointing to a table or query. The data source controls:

- **Which SQL query** runs against the database (the `executeQuery()` method)
- **What fields** are available to controls on the form (`Fields` collection under the data source)
- **How joins** to other data sources work (link type, dynamic filters)
- **Events** triggered when the user navigates records or changes data

Data Source Link Types

Link Type	Behavior	When to Use
Inner Join	Child records only show where a parent exists	Default — standard parent-child hierarchy
Outer Join	ALL child records show even if no parent match	When listing all child records regardless of parent existence
Exists Join	Parent records only if they have at least one child	For "show me customers who have orders"

Data Source Range (Filter) — Dynamic Filters

You can set **dynamic filters** on data sources using the `queryBuildDataSource().addRange()` method, or via the `init()` / `active()` form data source event methods. The `range()` and `value()` methods on `QueryBuildRange` allow you to programmatically set filter values.

```
// Setting a dynamic range programmatically in the form's data source init()
public void init()
{
    QueryBuildDataSource qbd;
    QueryBuildRange qbr;

    super();

    qbd = element.query().dataSourceTable(tableNum(VendInvoiceJour));
    qbr = qbd.addRange(fieldNum(VendInvoiceJour, PurchId));
    qbr.value(strfmt('"%1"', 'PO-00123'));
}
```

Form Event Lifecycle — The Key Events

Event	When It Fires	Purpose
`init()`	Form is initialized, data sources are set up	Set default filters, initialize variables
`active()`	The records change (user navigates to a different record)	Refresh computed fields, set control states based on current record
`executeQuery()`	The query is about to execute (after `init()`)	Modify the query at runtime — add filters, change join types
`write()`	After a record is written to the database (insert or update)	Post-write validation, trigger dependent logic, clear stale cache
`validateWrite()`	Before a record is written — can reject the save	Business rule validation; return `false` to prevent save with error
`deleted()`	After a record is deleted	Clean up related records, log deletion, refresh dependent lookups

The `executeQuery()` Event — The Most Powerful Form Event

```
// This runs EVERY time the form query executes — including when the user navigates
// records, refreshes the grid, or applies a filter
public void executeQuery()
{
    QueryBuildDataSource qbd = element.query().dataSourceTable(tableNum(VendTable));
    QueryBuildRange qbr;

    // Add a dynamic filter based on a control value on the form
    qbr = qbd.addRange(fieldNum(VendTable, AccountNum));
    qbr.value(queryValue(element.controlRun().controlName('FilterControl').value()));

    super();  // Always call super() LAST in executeQuery()
}
```

Critical: In `executeQuery()`, always call `super()` LAST — because `super()` is what actually executes the query. If you call `super()` first, the filters you added after it will be ignored.

4.2 Form Controls — Properties and Events

Key Control Types

Control	Use Case	Key Properties
String Edit	Text input/display	`AutoDeclaration`, `StringSize`, `Label`
Integer/Real Edit	Numeric input	`AutoDeclaration`, `Min`, `Max`, `Alignment`
Date Edit	Date input	`AutoDeclaration`, `DisplayFormat`
Combo Box	Dropdown selection	`AutoDeclaration`, `Items` (enumerated values), `Lookup` property
Checkbox	Boolean toggle	`AutoDeclaration`, `Label`, `Checked`
Grid	Multi-row tabular display	`AutoDeclaration`, `DataSource`, `MultiSelect`, `AllowEdit`

Control	Use Case	Key Properties
Tab	Tabbed sections	`AutoDeclaration`, `MultiSelect`
Button	Action trigger	`AutoDeclaration`, `Label`, `Command` property
LookupButton	Triggers a lookup	`AutoDeclaration`, `LinkedDataSource`, linked to EDT relationship or custom lookup method
Tree	Hierarchical display	`AutoDeclaration`, `DataSource`, `DisplayExpr`
Image	Displays BMP/PNG	`AutoDeclaration`, `ImageFile`

Form Control Events

Event	When	Purpose
`validate()`	Before the new value is written to the data source	Field-level validation; return `false` to reject and show error
`modified()`	Control value changed by user	React to user input — update other controls, recalculate totals
`lookup()`	User clicks the lookup button or presses F3	Custom lookup behavior; override to provide specialized lookup
`enter()`	Control receives focus	Highlight related fields, initialize dependent lookups
`close()`	Control/focus is lost	Cleanup, final validation

Note: `validate()` runs **first** — if validation fails (returns `false`), `modified()` will **not** fire. This sequence ensures invalid data never propagates. **Source:** Event method sequences when a record is created (AX 2012 legacy)

Setting Control Values Programmatically

```
// Set a control's text value on the current form
element.controlRun().controlName('MyStringControl').text(cust.AccountNum);

// Set the value of a control bound to a data field
// Note: setting the underlying table field is preferred over setting the control directly
cust.AccountNum = 'C0001';  // This automatically updates the form control via data binding
```

Design Patterns

List Page

A list page is a form with `FrameType = ListPage` — it shows records in a grid with a tree/navigator on the left. Clicking a record in the grid opens the detail page. List pages are the primary navigation pattern in D365 F&O.

Detail Page

A detail page (typically `FrameType = Form` or `DetailsForm`) shows all fields for a single record. It is opened from a list page via a `MenuItemDisplay`.

Popup / Dialog Form

A popup form (`FrameType = Dialog` or `Popup`) is a modal window used for data entry, quick edits, or confirmation. It typically has OK/Cancel buttons.

Workspace Card

A workspace card is a card-style UI element that can contain multiple form parts — used in the D365 workspace for dashboards and quick actions.

4.3 Chain of Command — Deep Dive

Chain of Command (CoC) is the **primary mechanism** for calling and extending standard D365 F&O code. It connects standard menu items to standard classes through a chain of method calls.

How Chain of Command Works

1. A user clicks a **Menu Item** (e.g., Customer → Customers → All Customers)
2. The menu item is linked to a **Class** (e.g., `CustTableListPage`)
3. The class method `main()` is the entry point
4. Inside `main()`, the class calls `super()` to chain to the **next** class in the linked list
5. Each class in the chain runs in sequence — this is what "Chain of Command" means

Extending the Chain

To add behavior to the standard chain: 1. Create a new class with `main()` that performs your logic 2. Use the `super()` call to join the chain at your desired position 3. Link the new class to a menu item (or replace the standard menu item's class reference)

The `runLink()` Pattern

When a form action (like clicking "New" or "Delete") needs to invoke another class or form:

```
// From a form button or menu item action:
MenuItemForm menuItem = new MenuItemForm();
menuItem.name(formstr(MyCustomForm));
menuItem.run();  // Opens the form directly
```

`CommandMenu` and `CommandDisplayMenu`

- **`MenuItemButton`**: A button on a form toolbar that triggers a command class
- **`MenuItemAction`**: An action menu item that runs a class method (not a form)
- **`MenuItemDisplay`**: Opens a form — the most common type for navigation
- Each has a `Command` property that points to the class to execute

4.4 Form vs. Form Extension

Form Modification

A form modification directly edits the standard `Form` node in the AOT. **This is not supported for customer/ISV code** because the next Microsoft update will overwrite your changes.

Form Extension

A form extension (`[ExtensionOf(formStr(CustTable))]`) adds controls, data sources, or methods to a standard form without modifying it. **This is the supported pattern** and survives Microsoft updates.

```
[ExtensionOf(formStr(CustTable))]
final class CustTable_FormExtension
{
    // Add a new data source
    CustComplianceLog complianceLog_ds;

    // Add a new control to the design
    // (Controls added via extension in the extension form design node)
}
```

When to Use Which

Scenario	Pattern
Add a new field to a form's data source	Form Extension
Add a new tab page to an existing form	Form Extension
Change a field label on the standard form	Avoid — request a label change via Microsoft support
Add a custom button that triggers new behavior	Form Extension
Change the SQL join type on a data source	Form Extension (override `executeQuery()` of the data source)

4.5 Activity — Multi-Form Solution

> **Activity**: Design a three-form solution for a vendor compliance dashboard: 1. **Form A (List Page)**: Shows all vendors with a compliance status column (computed via EDT display method). Clicking a vendor opens Form B. 2. **Form B (Detail Page)**: Shows vendor compliance metadata with a tab: 'Overview' and 'History'. The 'History' tab shows a grid of compliance log entries. A "New Declaration" button opens Form C. 3. **Form C (Popup)**: A dialog form for creating a new customs declaration. It validates the input, saves to `APCustomsDeclaration` table, closes, and refreshes Form B's grid.
>
> Requirements: - Form A must use a dynamic filter that shows only vendors whose credit limit exceeds $100,000 - Form B's History tab grid must auto-refresh when Form C submits a new record - The compliance status display on Form A must use a DisplayMethod that queries the `APComplianceLog` table - The "New Declaration" button on Form B must use a MenuItemAction to open Form C - All forms must follow D365 F&O form design conventions (labels, HelpText, proper sizing, keyboard navigation)
>
> **Hints** (Multiple Valid Approaches):

- **Hint A**: How to implement the dynamic filter on Form A?

- **Approach 1**: Set a `range` on the data source in `init()` — simple but static (the filter value doesn't change per-vendor)
- **Approach 2**: Override `executeQuery()` on the form's data source to add a dynamic range based on another form control — more flexible, allows the user to change the threshold
- **Approach 3**: Override `active()` to refresh the grid with a filtered query each time the user navigates records — simplest implementation but may cause performance issues with many records
- **Hint B**: How to make Form B's History grid auto-refresh when Form C submits a record?
- **Approach 1**: Call `element.datasource().refresh()` or `element.executeQuery()` on Form B from Form C's OK button — direct coupling, tightly integrated
- **Approach 2**: Use an event handler on `APCustomsDeclaration` table's `inserted()` event, which fires a `refresh()` call on all open forms using that table — decoupled architecture, but event handlers on table insert don't automatically know which forms are open
- **Approach 3**: Pass Form B's `element` to Form C as a parameter, and call Form B's `refresh()` from Form C's close method — explicit coupling but fully controlled
- **Hint C**: How to implement the compliance status column as a DisplayMethod?
- **Approach 1**: A static method on `VendTable` extension that takes a `VendTable` buffer and returns a status string — clean separation, reusable
- **Approach 2**: A DisplayMethod directly on the Form data source control — bound to the data source, automatically reflects current record context
- **Approach 3**: Override `active()` on the form to set a form-level variable, then a DisplayMethod reads that variable — most flexible but most complex

Expected Approach (Ideal — in detail)

The ideal approach combines extensibility with proper architecture:

1. **Form A** uses a `ListPage` frame type with `VendTable` as the primary data source. The dynamic filter is set in `executeQuery()` of the data source — not in `init()`, because `executeQuery()` re-runs when the user changes any control, making the filter truly dynamic. A `displayMethod` on the data source adds the compliance status column by calling a method on a service class.
2. **Form B** uses a `Form` frame type (not ListPage) with `VendTable` as the primary data source and `APCustomsDeclaration` joined (or separately added) as a related data source. The History tab is a separate design node (`Group` or `TabPage`) containing a grid bound to the `APCustomsDeclaration` data source. The "New Declaration" button is an `ActionMenuItem` that uses a MenuItemAction to open Form C. Form B overrides the data source's `active()` method to set a class-level variable holding the current vendor's `RecId`, which Form C uses when creating a new declaration.
3. **Form C** is a `Dialog` frame type with OK/Cancel buttons. On OK, it validates input, creates the `APCustomsDeclaration` record, calls `element.datasource().refresh()` on Form B's data source (passed as a parameter — Form B instantiates Form C and passes itself as the caller), and then closes.

Chapter 5 — Views, Lookups, and Cross-Table Data Access

5.1 Views — What They Are and Why They Matter

A view is a **named query** stored in the AOT. It acts like a table but doesn't store data — when you `select` from a view, the AOS translates it into SQL that joins across the underlying tables.

When to Use a View

Use Case	View	Alternative
Repeated multi-table join query used in many places	**Yes** — centralize, reuse	Duplicating the `select` logic everywhere
Complex logic needed for reporting	**Yes** — forms, reports, data entities all can use views	Repeating the join in each consumer
Simplifying security — restrict access to underlying tables	**Yes** — expose only the view, not the base tables	Granting access to all underlying tables
Simple single-table query only	**No** — just select from the table	Views add unnecessary abstraction
Performance-critical queries needing `firstonly` optimization	**Careful** — test view performance; `exists join` in a view may not translate optimally	Consider a query directly in the consuming code

View Properties

Property	Description	Common Values
`Label`	User-facing name	`'Vendor with Contact Details'`
`IsLookup`	Marks the view as a lookup	`Yes`
`CacheLookup`	Caches lookup results for performance	`Yes`

Note: The "dynamic" behavior (query rebuilt at runtime vs. cached SQL) is controlled by the **query framework**, not a named view property. **Source:** View Properties (AX 2012 reference)

How a View is Defined in the AOT

A view in the AOT contains: - **`Data Sources`** — a tree of table/query references with join orders and link conditions - **`Fields`** — each field is mapped to a table field (can be renamed with alias) - **`LookupField`** — specifies which field is used when the view is used as a lookup - **`Alias`** — optional rename for a field from its original table field name

```
VendorContactView (View)
+-- Data Sources
|   +-- DirPartyTable (Inner Join)
|   |   +-- DirPartyLocation (Outer Join)
|   |       +-- LogisticsLocation
|   +-- VendTable (Inner Join)
|       +-- VendBankAccount (Outer Join)
```

```
+-- Fields
|   +-- DirPartyTable.Name → Alias: PartyName
|   +-- VendTable.AccountNum → Alias: VendorAccount
|   +-- LogisticsLocation.City → Alias: PartyCity
|   +-- VendBankAccount.BankAccountNum → Alias: BankIBAN
+-- LookupField: VendorAccount
```

View Extensions

You can extend a standard view using `[ExtensionOf(viewStr(...))]` to add fields from additional tables — this is how you add fields without modifying the standard view. The extension appears as a new data source in the extended view.

5.2 Lookups — The User's Gateway to Data

A lookup is the mechanism by which a user can find and select a value for a field — typically triggered by clicking the **magnifier button** on a field or pressing **F3**.

How Lookups Work in D365 F&O

1. The user clicks the lookup button or presses F3 on a field
2. The framework calls the `lookup()` method on the form control
3. The `lookup()` method builds a `FormRun` object (the lookup dialog) and populates it with a data source
4. When the user selects a record, the selected value is written to the source field

Lookup Design Patterns

Pattern 1: EDT-based Lookup (Simplest)

When a field uses an EDT with a **Relationship** to another table, the framework automatically provides a standard lookup showing the related table's fields. No custom code needed.

Pattern 2: `SysTableLookup` (Most Common)

```
public void lookup()
{
    SysTableLookup sysTableLookup = SysTableLookup::newParameters(tableNum(VendTable), element);
    sysTableLookup.addLookupField(fieldNum(VendTable, AccountNum), true);  // true = show in grid
    sysTableLookup.addLookupField(fieldNum(VendTable, Name));
    sysTableLookup.addLookupField(fieldNum(VendTable, CreditMax));
    sysTableLookup.parmQuery(this.query());  // Pass current query context if needed
    sysTableLookup.performFormLookup();
}
```

Pattern 3: `FormRun` as Lookup (Maximum Control)

```
public void lookup()
{
    FormRun formRun;
    Args args = new Args();
    args.name(formstr(VendTableLookup));  // Custom form designed as a lookup
    args.caller(element);
    args.parm('LookupFieldId', fieldNum(VendTable, AccountNum));
```

```
    formRun = classfactory.formRunClass(args);
    formRun.init();
    formRun.run();
    formRun.wait();  // Synchronous — waits for user selection

    // After user closes the lookup, get the selected value
    if (formRun.closedOk())
    {
        element.text(formRun.element().parmSelectedValue());
    }
}
```

Pattern 4: Multi-Select Lookup

D365 F&O supports multi-select lookups where the user can select multiple records and the selected values are stored in a **Relation** or **Link** table. The `SysTableLookup` class supports this via `parmMultiSelect(true)`.

Custom Filtering in Lookups

You can pass a `Query` object to `SysTableLookup.parmQuery()` to filter the lookup results. This is the most common way to have the lookup show only relevant records based on context (e.g., only customers in the same `CustGroup`).

5.3 Multi-Select Lookup Exercise (Production Grade)

> **Activity**: Create a lookup for a field on `SalesTable` that allows the user to select multiple sales territories (from `DirTerritory` table) when creating an order. The lookup must: 1. Filter territories based on the user's security role (only show territories assigned to the current user) 2. Show territory name, code, and region in the lookup grid 3. Allow the user to select multiple territories 4. Store the selected territory IDs in a link table (`SalesTerritoryLink`) with `SalesTableRecId` and `TerritoryRecId` as foreign keys 5. The lookup must filter out territories that are already linked to this order (if editing an existing order) 6. Add a "Select All" checkbox at the top of the lookup form
>
> **Hints** (No single approach): - **Hint A for filtering by security role**: Option A1 — Query the `SecurityRole` user assignment table at lookup time and filter territories by role membership. Option A2 — Pass the user's `UserId` as a query range to the lookup form, and let the form filter itself in `executeQuery()`. Option A3 — Use a class method `DirTerritory::findByUser()` that handles the lookup query construction encapsulation. - **Hint B for "Select All" checkbox**: Option B1 — Add it as a separate form control that, when clicked, selects all visible records in the grid via `element.queryRun().getFirst(NoInit)` and toggling selection. Option B2 — Implement it as a data source field on the lookup form itself that, when set to `true`, triggers an `active()` event that iterates and selects all records. Option B3 — Use a parameter passed to the lookup form that sets a flag, and the form's `init()` reads that flag and auto-selects all records. - **Hint C for excluding already-linked territories**: Option C1 — Add a `notExists join` on the lookup's query that excludes territories already linked. Option C2 — Pass the current `SalesTable.RecId` to the lookup, and in the form's `executeQuery()`, add a range that excludes linked territories. Option C3 — Handle exclusions in the `lookup()` method after the form returns — iterate the selection and remove any already-linked territory IDs before saving.

Expected Approach (Ideal — in detail)

1. **Lookup form**: A dedicated form (`TerritoryLookupDlg`) with `DirTerritory` as its data source. The query includes:
2. A range on territory assignment based on user security role (passed as an `args.parm()` parameter)
3. A `notExists join` excluding territories already linked to this `SalesTable` record
4. A checkbox control bound to a temporary table or form parameter for "Select All"
5. **`lookup()` method**: Uses `SysTableLookup` with `parmMultiSelect(true)` — `SysTableLookup` handles the selection mechanics. The form returns territory IDs that are then written to the link table.
6. **`lookup()` method passes context via `Args`**: `args.parm('SalesTableRecId', element.recId())` and `args.parm('UserId', curUserId())`.
7. **"Select All" checkbox**: Lives on the lookup form itself. When toggled, it iterates all records in the lookup data source and toggles `selected` state on each. The `selected` field is part of the `DirTerritory` data source (or a temp `Relation` table used to track multi-select state).
8. **After the lookup closes**, the parent form iterates the selected territories and writes records to `SalesTerritoryLink`.

Remaining Chapters — Detailed Outline (Content to follow)

Chapter 6 — Business Logic & Class Design Patterns

6.1 Class Fundamentals — Static vs Instance

In X++, every class is either **static** (never instantiated, only static methods) or **instance-based** (objects created with `new` or `construct()`). Understanding when to use each is foundational to writing maintainable D365 F&O code.

Static Classes

A static class contains only static methods and is never instantiated. You declare it with the `static` keyword on the class declaration.

```
static class NumberSeq
{
    // Only static methods — no instance methods
    public static NumberSeqReference newReference(NumberSeqTable _numberSeqTable)
    {
        NumberSeqReference numberSeqReference;
        // ... logic
        return numberSeqReference;
    }
}
```

When to use static classes: - Utility/helper methods that don't maintain state (e.g., `Global::isNull()`, `Global::str2Date()`) - Service entry points called from menu items or other classes - Factory methods that create and return instances of other classes

When NOT to use static classes: - When you need to maintain state between method calls - When you need to support inheritance or polymorphism - When you need to mock or test the class in isolation (static methods are hard to unit test)

Instance Classes

An instance class is the default — you create objects with `new` or a `construct()` factory method. Instance classes maintain state through member variables.

```
class CustAccountValidator
{
    CustTable custTable;

    // Constructor — private to enforce factory pattern
    private CustAccountValidator(CustTable _custTable)
    {
        custTable = _custTable;
    }

    // Factory method — the preferred way to obtain an instance
    public static CustAccountValidator construct(CustTable _custTable)
    {
        return new CustAccountValidator(_custTable);
    }

    // Instance method — has access to member state
```

```
    public boolean validate()
    {
        if (custTable.AccountNum == '')
        {
            return checkFailed('Account number is required');
        }
        return true;
    }
}
```

Constructor Patterns

Pattern	Syntax	When to Use
`new()` constructor	`public CustAccountValidator()`	Simple classes with no complex initialization
`construct()` factory	`public static CustAccountValidator construct(...)`	When you need to control instance creation, cache instances, or return a subclass
`new` with private constructor	`private CustAccountValidator(...)` + `public static construct()`	Enforces factory pattern — prevents direct `new` calls from external code
Singleton via `construct()`	`static CustAccountValidator instance; if (!instance) instance = new ...; return instance;`	When exactly one instance must exist across the entire session

The `construct()` factory pattern is the dominant pattern in D365 F&O. Microsoft uses it extensively in the standard codebase (e.g., `SysOperationServiceController::construct()`, `SrsReportDataProvider::construct()`). You should follow this convention in all custom classes.

The `parmX()` Accessor Pattern

Every class that holds data should expose its member variables through `parmX()` accessor methods. This is the X++ equivalent of Java's getter/setter pattern and is critical for:

1. **Encapsulation** — external code never accesses member variables directly
2. **Validation** — you can add logic in the setter (`parmX(value)`) before storing
3. **Testability** — you can mock or verify state through accessors
4. **Framework compatibility** — `RunBase`, `SysOperation`, and other frameworks rely on `parmX()` methods for data passing

```
class VendComplianceContract
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    boolean requireComplianceCheck;

    // Getter — no parameters
    public VendTable parmVendTable()
    {
        return vendTable;
    }

    // Setter — with parameter, returns 'this' for chaining
    public VendComplianceContract parmVendTable(VendTable _vendTable)
    {
```

```
        vendTable = _vendTable;
        return this;
    }

    // Getter
    public ComplianceCode parmComplianceCode()
    {
        return complianceCode;
    }

    // Setter with validation
    public VendComplianceContract parmComplianceCode(ComplianceCode _complianceCode)
    {
        if (_complianceCode && _complianceCode.length() > 20)
        {
            throw error(strFmt('Compliance code cannot exceed 20 characters. Provided: "%1"',
_complianceCode));
        }
        complianceCode = _complianceCode;
        return this;
    }

    // Boolean flag — getter returns boolean, setter returns 'this' for chaining
    public boolean parmRequireComplianceCheck()
    {
        return requireComplianceCheck;
    }

    public VendComplianceContract parmRequireComplianceCheck(boolean _require)
    {
        requireComplianceCheck = _require;
        return this;
    }
}
```

The chaining pattern (`return this;`) is optional but common in D365 F&O — it allows you to write:

```
VendComplianceContract contract = new VendComplianceContract()
    .parmVendTable(vendTable)
    .parmComplianceCode('COMP-001')
    .parmRequireComplianceCheck(true);
```

6.2 The `RunBase` Framework — Batch Processing Foundation

`RunBase` is the base class for all batch-executable operations in D365 F&O. It provides the infrastructure for:

- Displaying a **dialog** to the user for parameter input
- **Packing** and **unpacking** state for serialization (required for batch jobs that run on a different tier)
- The **`run()`** method where business logic executes
- Integration with the **batch framework** (`RunBaseBatch`)

Why RunBase Exists

Before `RunBase`, developers wrote ad-hoc batch jobs with no standard dialog, no parameter serialization, and no integration with the batch job queue. `RunBase` standardizes this pattern so that:

- Any class extending `RunBase` automatically gets a dialog UI
- Parameters survive serialization across tiers (client → batch server → AOS)
- The batch framework can schedule, monitor, and retry the job

The Required Methods

Every `RunBase` subclass must implement these methods:

Method	Required?	Purpose
`dialog()`	Yes	Creates and returns a `Dialog` object with input controls for the user
`pack()`	Yes	Serializes the class's state into a `container` for cross-tier transfer
`unpack(container)`	Yes	Deserializes the container back into the class's state
`run()`	Yes	Contains the actual business logic that executes
`main(Args _args)`	Yes (static)	Entry point — called when the user triggers the action from a menu item

`dialog()` — Building the User Interface

```
class VendComplianceBatchJob extends RunBase
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    FromDate fromDate;
    ToDate toDate;

    // Dialog method – creates the UI
    public Object dialog()
    {
        Dialog dialog = super::dialog();
        DialogField dfVend;
        DialogField dfCode;
        DialogField dfFrom;
        DialogField dfTo;

        // Add a table lookup field – user can search for a vendor
        dfVend = dialog.addFieldValue(typeid(VendTable), vendTable, 'Vendor');
        dfVend.lookupButton(true);  // Enable the lookup button

        // Add a string field for compliance code
        dfCode = dialog.addFieldValue(typeid(ComplianceCode), complianceCode, 'Compliance Code');

        // Add date range fields
        dfFrom = dialog.addFieldValue(typeid(FromDate), fromDate, 'From Date');
        dfTo = dialog.addFieldValue(typeid(ToDate), toDate, 'To Date');

        return dialog;
    }
}
```

Key points about `dialog()`: - Always call `super::dialog()` first — this creates the base dialog with OK/Cancel buttons - Use `dialog.addFieldValue()` for standard field types — it automatically creates the control, label, and data binding - Use `dialog.addGroup()` to organize fields into logical sections - Return the `Dialog` object — the framework handles showing it and reading values back

`pack()` and `unpack()` — Serialization

When a `RunBase` job runs as a batch, it may execute on a different tier than where it was created. The `pack()`/`unpack()` methods serialize and deserialize the class's state.

```
// pack() — serialize state into a container
public container pack()
{
    return [vendTable, complianceCode, fromDate, toDate];
}

// unpack() — deserialize state from a container
public boolean unpack(container _packedClass)
{
    vendTable = _packedClass.packGet(1);
    complianceCode = _packedClass.packGet(2);
    fromDate = _packedClass.packGet(3);
    toDate = _packedClass.packGet(4);
    return true;
}
```

Critical rules for `pack()`/`unpack()`: 1. The order of elements in the container **must match** between `pack()` and `unpack()` — any mismatch causes silent data corruption 2. Only pack **simple types** (strings, dates, integers, `RecId`, table buffers) — do NOT pack `FormRun`, `Object`, or other non-serializable types 3. If you add a new field, add it to the **end** of both `pack()` and `unpack()` to maintain backward compatibility with existing batch jobs already in the queue 4. The `container` type is the X++ equivalent of a typed tuple — it holds ordered elements of any type

`run()` — The Business Logic

```
public void run()
{
    VendTable vendTableLocal;
    APCustomsDeclaration declaration;

    // Validate that the vendor exists
    vendTableLocal = VendTable::find(vendTable.AccountNum);
    if (!vendTableLocal)
    {
        throw error(strFmt('Vendor "%1" not found.', vendTable.AccountNum));
    }

    // Process compliance declarations for the date range
    while select declaration
        where declaration.VendTableRecId == vendTableLocal.RecId
        && declaration.DeclDate >= fromDate
        && declaration.DeclDate <= toDate
    {
        // Business logic here
        this.processDeclaration(declaration);
    }
```

```
    info(strFmt('Compliance batch job completed for vendor "%1".', vendTableLocal.AccountNum));
}
```

`main(Args)` — The Static Entry Point

```
public static void main(Args _args)
{
    VendComplianceBatchJob batchJob = new VendComplianceBatchJob();

    // If called from a menu item, args may contain pre-populated data
    if (_args && _args.record())
    {
        batchJob.parmVendTable(_args.record());
    }

    // Show the dialog — if user clicks OK, run the job
    if (batchJob.prompt())
    {
        batchJob.run();
    }
}
```

The `prompt()` method (inherited from `RunBase`) shows the dialog, reads the user's input, and returns `true` if the user clicked OK. It internally calls `dialog()`, `pack()`, and sets up the class state.

`RunBase` — Complete Class Template

```
class VendComplianceBatchJob extends RunBase
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    FromDate fromDate;
    ToDate toDate;

    // 1. Dialog — user input
    public Object dialog()
    {
        Dialog dialog = super::dialog();
        dialog.addFieldValue(typeid(VendTable), vendTable, 'Vendor');
        dialog.addFieldValue(typeid(ComplianceCode), complianceCode, 'Compliance Code');
        dialog.addFieldValue(typeid(FromDate), fromDate, 'From Date');
        dialog.addFieldValue(typeid(ToDate), toDate, 'To Date');
        return dialog;
    }

    // 2. Pack — serialize for batch
    public container pack()
    {
        return [vendTable, complianceCode, fromDate, toDate];
    }

    // 3. Unpack — deserialize from batch
    public boolean unpack(container _packedClass)
    {
        vendTable = _packedClass.packGet(1);
        complianceCode = _packedClass.packGet(2);
        fromDate = _packedClass.packGet(3);
        toDate = _packedClass.packGet(4);
        return true;
```

```
    }

    // 4. Run — business logic
    public void run()
    {
        // ... implementation ...
    }

    // 5. Main — entry point
    public static void main(Args _args)
    {
        VendComplianceBatchJob job = new VendComplianceBatchJob();
        if (_args && _args.record())
        {
            job.parmVendTable(_args.record());
        }
        if (job.prompt())
        {
            job.run();
        }
    }
}
```

6.3 `RunBaseBatch` — Subclassing RunBase for Batch Jobs

`RunBaseBatch` extends `RunBase` to add **batch job** capabilities — the job runs in the background via the batch framework, not interactively.

What `RunBaseBatch` Adds

Feature	Description
`runOn()`	Specifies where the job runs: `RunOn::Client`, `RunOn::Server`, or `RunOn::Batch`
`doBatch()`	Called by the batch framework — wraps `run()` with batch infrastructure (progress reporting, error handling, retry)
`createJob()`	Static method that creates a `BatchHeader` and queues the job — this is what gets called from a menu item
`batchInfo()`	Returns the `BatchHeader` with job description, batch group, and scheduling info
`lastValue()`	Used for progress reporting — returns a string describing the current progress

`RunBaseBatch` — Complete Class Template

```
class VendComplianceBatchJob extends RunBaseBatch
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    FromDate fromDate;
    ToDate toDate;

    // Dialog — same as RunBase
    public Object dialog()
```

```
{
    Dialog dialog = super::dialog();
    dialog.addFieldValue(typeid(VendTable), vendTable, 'Vendor');
    dialog.addFieldValue(typeid(ComplianceCode), complianceCode, 'Compliance Code');
    dialog.addFieldValue(typeid(FromDate), fromDate, 'From Date');
    dialog.addFieldValue(typeid(ToDate), toDate, 'To Date');
    return dialog;
}

// Pack / Unpack — same as RunBase
public container pack()
{
    return [vendTable, complianceCode, fromDate, toDate];
}

public boolean unpack(container _packedClass)
{
    vendTable = _packedClass.packGet(1);
    complianceCode = _packedClass.packGet(2);
    fromDate = _packedClass.packGet(3);
    toDate = _packedClass.packGet(4);
    return true;
}

// runOn() — where should this job execute?
// Batch jobs MUST run on the server or batch tier, not the client
public RunOn runOn()
{
    return RunOn::Batch;
}

// run() — the actual business logic
public void run()
{
    // ... same as RunBase.run() ...
}

// lastValue() — progress reporting
// Called by the batch framework to show progress in the batch job list
public int lastValue()
{
    // Return a value that represents progress (e.g., number of records processed)
    return this.lastValueCounter;
}

// batchInfo() — metadata about the batch job
// Called by the framework to populate the BatchHeader
public BatchHeader batchInfo()
{
    BatchHeader batchHeader = super::batchInfo();
    batchHeader.parmDescription('Vend Compliance Batch Job');
    batchHeader.parmBatchGroup('BATCHGROUP1');  // The batch group that processes this job
    batchHeader.parmExecutionStyle(BatchExecutionStyle::OnDemand);
    return batchHeader;
}

// createJob() — static entry point called from a menu item
// This creates the batch header and queues the job
public static void createJob()
{
    VendComplianceBatchJob job = new VendComplianceBatchJob();
    if (job.prompt())
```

```
        {
            // Submit the job to the batch framework
            job.run();
        }
    }

    // main() – the static entry point
    public static void main(Args _args)
    {
        VendComplianceBatchJob job = new VendComplianceBatchJob();
        if (_args && _args.record())
        {
            job.parmVendTable(_args.record());
        }
        if (job.prompt())
        {
            job.run();
        }
    }
}
```

Batch Job Lifecycle

```
[1] User clicks menu item → createJob() called
    |
[2] createJob() calls job.prompt() → dialog shown, user enters parameters
    |
[3] createJob() calls job.run() → job submitted to batch framework
    |
[4] Batch framework creates BatchHeader with job metadata
    |
[5] Batch job placed in queue (BatchHeader.Status = Created)
    |
[6] Batch server picks up job → unpack() deserializes parameters
    |
[7] doBatch() called → wraps run() with try/catch, progress reporting
    |
[8] run() executes business logic
    |
[9] Batch job completes → BatchHeader.Status = Completed
    |
[10] User notified via Infolog and/or email (if configured)
```

`runOn()` — Execution Target

Value	Description	Use When
`RunOn::Client`	Runs on the user's desktop	Lightweight operations that need UI interaction
`RunOn::Server`	Runs on the AOS server	Operations that need server-side data access but don't need batch queuing
`RunOn::Batch`	Runs on the batch server	Long-running operations that should be queued and run asynchronously

Rule of thumb: If the operation takes more than a few seconds, use `RunOn::Batch`. If it needs to run unattended (e.g., nightly reconciliation), use `RunOn::Batch`. If it requires user interaction, use `RunOn::Client`.

6.4 `SysOperationServiceController` — The Modern Service Pattern

`SysOperationServiceController` is the **modern, service-oriented** approach to running business operations. It is the successor pattern to `RunBase` for operations that need to be:

- Called from **external systems** (via OData/REST)
- Executed as **service operations** in the Service Layer
- **Contract-driven** — input and output are defined in a contract class, not in dialog fields
- **Versioned** — contracts can evolve independently of the operation implementation

Why SysOperationServiceController?

RunBase	SysOperationServiceController
Dialog-based input	Contract-based input (typed parameters)
Batch-only execution	Can run synchronously (in-call) or asynchronously (batch)
Tightly coupled to UI	Decoupled — can be called from UI, batch, or external services
`pack()`/`unpack()` for serialization	Contract class handles serialization automatically
Limited to X++ consumers	Exposable via OData/REST for Power Platform and external systems

The Three Components

A `SysOperationServiceController`-based operation consists of three classes:

1. **Contract class** — defines the input and output parameters as properties
2. **Service class** — contains the business logic (`process()` method)
3. **Controller class** — extends `SysOperationServiceController`, wires the contract to the service, and manages execution

Component 1: The Contract Class

```
[DataContractAttribute]
class VendComplianceContract
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    FromDate fromDate;
    ToDate toDate;
    boolean generateReport;

    [DataMemberAttribute('VendTable'), SysOperationLabelAttribute('Vendor')]
    public VendTable parmVendTable(VendTable _vendTable = vendTable)
    {
        vendTable = _vendTable;
        return vendTable;
    }

    [DataMemberAttribute('ComplianceCode'), SysOperationLabelAttribute('Compliance Code')]
    public ComplianceCode parmComplianceCode(ComplianceCode _complianceCode = complianceCode)
    {
        complianceCode = _complianceCode;
```

```
        return complianceCode;
    }

    [DataMemberAttribute('FromDate'), SysOperationLabelAttribute('From Date')]
    public FromDate parmFromDate(FromDate _fromDate = fromDate)
    {
        fromDate = _fromDate;
        return fromDate;
    }

    [DataMemberAttribute('ToDate'), SysOperationLabelAttribute('To Date')]
    public ToDate parmToDate(ToDate _toDate = toDate)
    {
        toDate = _toDate;
        return toDate;
    }

    [DataMemberAttribute('GenerateReport'), SysOperationLabelAttribute('Generate Report')]
    public boolean parmGenerateReport(boolean _generateReport = generateReport)
    {
        generateReport = _generateReport;
        return generateReport;
    }
}
```

Key attributes: - `[DataContractAttribute]` on the class — marks it as a data contract - `[DataMemberAttribute('Name')]` on each property — the name used in OData/REST serialization - `[SysOperationLabelAttribute('Label')]` — the user-facing label for the parameter - Each `parmX()` method uses the **getter/setter pattern** — if called with a parameter, it sets the value; if called without, it returns the current value

Component 2: The Service Class

```
class VendComplianceService
{
    // The main business logic method
    public VendComplianceContract process(VendComplianceContract _contract)
    {
        VendTable vendTableLocal;
        APCustomsDeclaration declaration;
        VendComplianceContract resultContract;

        // Validate the contract
        this.validateContract(_contract);

        // Process declarations
        vendTableLocal = VendTable::find(_contract.parmVendTable().AccountNum);

        while select declaration
            where declaration.VendTableRecId == vendTableLocal.RecId
            && declaration.DeclDate >= _contract.parmFromDate()
            && declaration.DeclDate <= _contract.parmToDate()
        {
            this.processDeclaration(declaration);
        }

        // If requested, generate a report
        if (_contract.parmGenerateReport())
        {
            this.generateReport(vendTableLocal, _contract.parmFromDate(), _contract.parmToDate());
```

```
        }

        // Return the contract (potentially with output fields populated)
        resultContract = _contract;
        return resultContract;
    }

    private void validateContract(VendComplianceContract _contract)
    {
        if (!_contract.parmVendTable())
        {
            throw error('Vendor is required.');
        }
        if (_contract.parmFromDate() > _contract.parmToDate())
        {
            throw error('From date cannot be after to date.');
        }
    }

    private void processDeclaration(APCustomsDeclaration _declaration)
    {
        // Business logic for processing a single declaration
        info(strFmt('Processing declaration %1 for vendor %2', _declaration.DeclId,
_declaration.VendTableRecId));
    }

    private void generateReport(VendTable _vendTable, FromDate _from, ToDate _to)
    {
        // SSRS report generation logic
        SrsReportRunController reportController = new SrsReportRunController();
        // ... configure and run the report ...
    }
}
```

Component 3: The Controller Class

```
class VendComplianceController extends SysOperationServiceController
{
    // The static entry point — called from a menu item or external service
    public static void main(Args _args)
    {
        VendComplianceController controller = new VendComplianceController();
        VendComplianceContract contract = new VendComplianceContract();

        // If called from a form, populate the contract from the form's data source
        if (_args && _args.record())
        {
            contract.parmVendTable(_args.record());
        }

        // Set the service and contract types
        controller.parmContractType(classStr(VendComplianceContract));
        controller.parmServiceType(classStr(VendComplianceService));

        // Start the operation
        controller.startOperation();
    }
}
```

`SysOperationServiceController` Key Methods

Method	Purpose
`parmContractType()`	Sets the contract class type — the framework uses this to serialize/deserialize parameters
`parmServiceType()`	Sets the service class type — the framework instantiates this and calls `process()`
`startOperation()`	Starts the operation — if `runOn()` returns `RunOn::Batch`, this queues a batch job; otherwise runs synchronously
`parmRunOn()`	Override to specify execution target (`RunOn::Client`, `RunOn::Server`, `RunOn::Batch`)
`parmCaption()`	Override to set the operation's display name in the UI and batch job list

When to Use SysOperationServiceController vs RunBase

Scenario	Use
Simple batch job with dialog input	`RunBase` / `RunBaseBatch`
Operation called from a form button	`RunBase` (simpler, faster)
Operation exposed via OData/REST	`SysOperationServiceController`
Operation called from Power Automate / Logic Apps	`SysOperationServiceController`
Operation needs contract validation	`SysOperationServiceController` (contract attributes provide validation)
Operation called from another X++ class	Either — `SysOperationServiceController` if you want to pass a contract object, `RunBase` for simplicity

---

6.5 Event Handlers — Deep Dive

Event handlers are the **primary extension mechanism** in D365 F&O. They allow you to inject custom logic into standard code execution without modifying the original method.

Event Handler Attributes

D365 F&O provides **three distinct event handler attributes** for different purposes:

Attribute	Purpose	Parameter Type	Example
`[PreHandlerFor(...)]`	Runs **before** the standard method	`XppPrePostArgs`	`[PreHandlerFor(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]`
`[PostHandlerFor(...)]`	Runs **after** the standard method	`XppPrePostArgs`	`[PostHandlerFor(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]`
`[SubscribesTo(...)]`	Subscribes to **delegates** explicitly declared in the	Depends on delegate	`[SubscribesTo(classStr(MyClass), delegateStr(MyClas`

Attribute	Purpose	Parameter Type	Example
	base class		`s, myDelegate))]`
`[DataEventHandler(...)]`	Subscribes to **data events** (insert/update/delete)	`XppDataEventArgs`	`[DataEventHandler(tableStr(VendTable), DataEventType::Inserted)]`

Source: X++ Events

Pre-Event Handler Example

```
// Runs BEFORE VendInvoiceJour.validateWrite()
[PreHandlerFor(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]
public static void VendInvoiceJour_PreValidateWrite(XppPrePostArgs _args)
{
    VendInvoiceJour vendInvoiceJour = _args.getThis();

    // Check if the vendor has a compliance code
    if (vendInvoiceJour.CreditMax > 500000 && vendInvoiceJour.ComplianceCode == '')
    {
        // Add validation error to prevent the write
        _args.setReturnValue(checkFailed('Compliance code required for vendors exceeding $500,000 credit
limit'));
    }
}
```

Post-Event Handler Example

```
// Runs AFTER VendInvoiceJour.validateWrite()
[PostHandlerFor(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]
public static void VendInvoiceJour_PostValidateWrite(XppPrePostArgs _args)
{
    VendInvoiceJour vendInvoiceJour = _args.getThis();

    // The standard validateWrite() has already run
    // Check if it succeeded (return value is accessible via args)
    if (_args.getReturnValue())
    {
        // Log the compliance check
        APComplianceLog::logCheck(vendInvoiceJour.RecId, ComplianceStatus::Validated);

        // Trigger integration event
        VendComplianceIntegration::onInvoiceValidated(vendInvoiceJour);
    }
}
```

Event Handler Placement

Placement	Syntax	When to Use
Table-level	`[PreHandlerFor(tableStr(VendTable), methodStr(VendTable, validateWrite))]`	The handler is logically tied to the table's data integrity rules — e.g., validation, field change reactions

Placement	Syntax	When to Use
Class-level	`[PreHandlerFor(classStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]`	The handler is a cross-cutting concern — e.g., logging, notification dispatch, integration triggers

Best practice: Place table-level event handlers in a class named after the table (e.g., `VendTable_EventHandlers`). Place class-level handlers in a dedicated event handler class (e.g., `VendInvoiceJour_EventHandler`).

Event Types

D365 F&O supports the following event mechanisms:

1. **Pre/Post Handlers** (`[PreHandlerFor]`/`[PostHandlerFor]`):
2. Wrap **any standard method** (table or class).

3. Use `XppPrePostArgs` to access the record (`args.getThis()`) and return value (`args.getReturnValue()`/`args.setReturnValue()`).

4. **Delegate Subscribers** (`[SubscribesTo]`):

5. Subscribe to **delegates** explicitly declared in the base class (e.g., `[EventHandlerResultAttribute]`).

6. Use `delegateStr` (not `methodStr`).

7. **Data Event Handlers** (`[DataEventHandler]`):

8. Subscribe to **data events** (`Inserted`, `Updated`, `Deleted`).
9. Use `XppDataEventArgs` to access the record and event type.

Note: For **method overrides** (replacing behavior), use **Chain of Command (CoC)** via `[ExtensionOf(classStr(...))]`, not event handlers.

Event Handler Execution Order

When multiple handlers subscribe to the same event: 1. **Pre-handlers** run first (in **unspecified order** — not guaranteed alphabetical). 2. The standard method executes. 3. **Post-handlers** run last (in **unspecified order**).

Important: Execution order among handlers is **not guaranteed** by the framework. Do not rely on alphabetical or registration order. **Source:** X++ Events

Event Handler Best Practices

1. **Keep handlers thin** — delegate to service classes or static utility methods; don't put business logic directly in the handler
2. **Don't call `super()` in event handlers** — event handlers don't participate in CoC chains; they are independent subscribers
3. **Use pre-events for validation** and post-events for side effects — this keeps the event handler pattern clean
4. **Avoid override-events unless necessary** — they make upgradeability harder because you've replaced standard behavior entirely
5. **Name handler classes consistently** — `VendTable_EventHandler`, `CustInvoiceJour_EventHandlers`, etc.

6.6 The Command Pattern — Menu Item → Class Chain

The Command pattern in D365 F&O connects user actions (menu item clicks) to business logic (class methods) through a chain of objects.

The Chain

```
User clicks a menu item
    |
    v
MenuItem (MenuItemDisplay / MenuItemAction / MenuItemOutput)
    |  Property: Object = ClassName or FormName
    |
    v
Class::main()  or  FormRun.run()
    |
    v
Business logic executes
```

MenuItem Types and Their Command Targets

MenuItem Type	Class Property	What Happens When Clicked
MenuItemDisplay	`Object = FormName`	Opens the specified form
MenuItemAction	`Object = ClassName`	Calls `ClassName::main()` static method
MenuItemOutput	`Object = ReportClassName`	Runs the specified report (SSRS or legacy AX reports)

Source: Build forms and optimize for Finance and Operations

The `main()` Static Method — The Entry Point

Every class that is the target of a `MenuItemAction` must have a `main(Args _args)` static method:

```
class VendComplianceManager
{
    // This is the entry point when the menu item is clicked
    public static void main(Args _args)
    {
        VendComplianceManager manager = new VendComplianceManager();

        // If the menu item was triggered from a form, args.record() contains the current record
        if (_args && _args.record() && _args.record().TableId == tableNum(VendTable))
        {
            manager.parmVendTable(_args.record());
        }

        // Run the compliance check
        manager.checkCompliance();
    }

    // Instance method — the actual business logic
    public void checkCompliance()
    {
        VendTable vendTableLocal = this.parmVendTable();
```

```
        if (vendTableLocal.CreditMax > 500000 && vendTableLocal.ComplianceCode == '')
        {
            warning(strFmt('Vendor "%1" exceeds $500,000 credit limit but has no compliance code.',
                vendTableLocal.AccountNum));
        }
        else
        {
            info(strFmt('Vendor "%1" is compliant.', vendTableLocal.AccountNum));
        }
    }
}
```

MenuItemAction for Navigation

When a form action needs to open another form, it uses the `MenuFunction` pattern:

```
// From a button on Form A that opens Form B
public void clicked()
{
    Args args = new Args();
    args.name(formstr(ComplianceHistoryForm));
    args.record(element.args().record());  // Pass the current vendor record

    // Use MenuFunction to open the form via the menu item
    MenuFunction menuFunction = new MenuFunction(menuItemDisplayStr(ComplianceHistory),
MenuItemType::Display);
    menuFunction.run(args);
}
```

6.7 Activity — Full Order-to-Cash Flow Implementation

> **Activity**: Implement a complete Order-to-Cash flow for a custom compliance module. The solution must include: 1. A `VendComplianceContract` class (data contract with `parmX()` accessors) 2. A `VendComplianceService` class (business logic with `process()` method) 3. A `VendComplianceController` class (extends `SysOperationServiceController`, entry point) 4. A `VendComplianceEventHandler` class with at least 2 event subscriptions: - A **pre-event** on `VendInvoiceJour.validateWrite()` that checks the compliance code for high-credit vendors - A **post-event** on `VendInvoiceJour.inserted()` that logs the compliance status to `APComplianceLog` 5. A `VendComplianceBatchJob` class (extends `RunBaseBatch`, with dialog, pack/unpack, run, and createJob) 6. Proper **MenuItemAction** navigation from a form button to the compliance dashboard form 7. A `MenuItemAction` menu item that triggers the `VendComplianceController::main()` method
>
> **Activity Hints** (Multiple Valid Approaches): - **Hint A — Service layer design**: Option A1 — use `SysOperationServiceController` with a contract class (recommended — modern, decoupled, testable). Option A2 — use `RunBase` with dialog input (simpler, but tightly coupled to UI). Option A3 — use a static utility class with no framework (not recommended — hard to test, no batch support, no OData exposure). - **Hint B — Event subscription placement**: Option B1 — table-level event handler on `VendInvoiceJour` (recommended — logically grouped with the table's data integrity concerns). Option B2 — class-level event handler in a dedicated `VendInvoiceJour_EventHandler` class (also valid — better for cross-cutting concerns). Option C — form-level event handler (not recommended for data integrity logic — only use for UI-specific behavior). - **Hint C — Batch job design**: Option C1 — extend `RunBaseBatch` with full dialog and batch integration (recommended —

> follows Microsoft patterns). Option C2 — use `RunBase` with `runOn() = RunOn::Server` for a simpler server-only job (valid for short operations that don't need batch queuing). Option C3 — use `SysOperationServiceController` with `runOn() = RunOn::Batch` for a contract-driven batch operation (modern but more complex). - **Hint D — Contract design**: Option D1 — use `[DataContractAttribute]` with `[DataMember]` on each property (recommended — enables OData exposure). Option D2 — use a simple class with public member variables (not recommended — no serialization support, no metadata).

Expected Approach (Ideal — in detail)

Component 1: VendComplianceContract (Data Contract)

```
[DataContractAttribute]
class VendComplianceContract
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    FromDate fromDate;
    ToDate toDate;
    boolean generateReport;

    [DataMemberAttribute('VendTable'), SysOperationLabelAttribute('Vendor')]
    public VendTable parmVendTable(VendTable _vendTable = vendTable)
    {
        vendTable = _vendTable;
        return vendTable;
    }

    [DataMemberAttribute('ComplianceCode'), SysOperationLabelAttribute('Compliance Code')]
    public ComplianceCode parmComplianceCode(ComplianceCode _complianceCode = complianceCode)
    {
        complianceCode = _complianceCode;
        return complianceCode;
    }

    [DataMemberAttribute('FromDate'), SysOperationLabelAttribute('From Date')]
    public FromDate parmFromDate(FromDate _fromDate = fromDate)
    {
        fromDate = _fromDate;
        return fromDate;
    }

    [DataMemberAttribute('ToDate'), SysOperationLabelAttribute('To Date')]
    public ToDate parmToDate(ToDate _toDate = toDate)
    {
        toDate = _toDate;
        return toDate;
    }

    [DataMemberAttribute('GenerateReport'), SysOperationLabelAttribute('Generate Report')]
    public boolean parmGenerateReport(boolean _generateReport = generateReport)
    {
        generateReport = _generateReport;
        return generateReport;
    }
}
```

Design rationale: The contract class is a pure data holder — no business logic, no database access. It uses the `parmX()` pattern with default parameters for getter/setter duality. The `[DataMember]` attributes enable OData serialization for external consumption.

Component 2: VendComplianceService (Business Logic)

```
class VendComplianceService
{
    public VendComplianceContract process(VendComplianceContract _contract)
    {
        this.validateContract(_contract);

        VendTable vendTableLocal = VendTable::find(_contract.parmVendTable().AccountNum);

        // Process compliance declarations
        this.processDeclarations(vendTableLocal, _contract.parmFromDate(), _contract.parmToDate());

        // Optional report generation
        if (_contract.parmGenerateReport())
        {
            this.generateComplianceReport(vendTableLocal, _contract.parmFromDate(),
_contract.parmToDate());
        }

        return _contract;
    }

    private void validateContract(VendComplianceContract _contract)
    {
        if (!_contract.parmVendTable())
        {
            throw error('Vendor is required.');
        }
        if (_contract.parmFromDate() > _contract.parmToDate())
        {
            throw error('From date cannot be after to date.');
        }
    }

    private void processDeclarations(VendTable _vendTable, FromDate _from, ToDate _to)
    {
        APCustomsDeclaration declaration;

        while select declaration
            where declaration.VendTableRecId == _vendTable.RecId
            && declaration.DeclDate >= _from
            && declaration.DeclDate <= _to
        {
            // Validate each declaration
            if (declaration.DeclDate > today())
            {
                warning(strFmt('Declaration %1 has a future date.', declaration.DeclId));
                continue;
            }

            // Process the declaration
            declaration.Status = ComplianceStatus::Processed;
            declaration.update();
        }
    }
```

```
    private void generateComplianceReport(VendTable _vendTable, FromDate _from, ToDate _to)
    {
        SrsReportRunController reportController = new SrsReportRunController();
        // ... configure report with contract parameters ...
        reportController.run();
    }
}
```

Component 3: VendComplianceController (SysOperationServiceController)

```
class VendComplianceController extends SysOperationServiceController
{
    public static void main(Args _args)
    {
        VendComplianceController controller = new VendComplianceController();
        VendComplianceContract contract = new VendComplianceContract();

        // Populate contract from args if triggered from a form
        if (_args && _args.record() && _args.record().TableId == tableNum(VendTable))
        {
            contract.parmVendTable(_args.record());
        }

        controller.parmContractType(classStr(VendComplianceContract));
        controller.parmServiceType(classStr(VendComplianceService));
        controller.parmCaption('Vend Compliance Check');
        controller.startOperation();
    }
}
```

Component 4: VendComplianceEventHandler (Two Event Subscriptions)

```
class VendComplianceEventHandler
{
    // Pre-event: validate compliance code before standard validateWrite runs
    [SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]
    public static void VendInvoiceJour_onValidateWrite_Pre(VendInvoiceJour _this)
    {
        if (_this.CreditMax > 500000 && _this.ComplianceCode == '')
        {
            // The standard validateWrite() will still run, but checkFailed
            // will cause it to return false
            _this.validateWrite = checkFailed(
                'Compliance code is required for vendors exceeding $500,000 credit limit.');
        }
    }

    // Post-event: log compliance status after a new invoice is inserted
    [SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, inserted()),
EventExecution::Post)]
    public static void VendInvoiceJour_onInserted_Post(VendInvoiceJour _this)
    {
        APComplianceLog complianceLog;

        complianceLog.initValue();
        complianceLog.VendTableRecId = _this.RecId;
        complianceLog.VendAccount = _this.AccountNum;
        complianceLog.InvoiceDate = _this.InvoiceDate;
```

```
        complianceLog.Status = ComplianceStatus::Logged;

        // Check if compliance code exists
        if (_this.ComplianceCode != '')
        {
            complianceLog.ComplianceCode = _this.ComplianceCode;
            complianceLog.Status = ComplianceStatus::Compliant;
        }
        else
        {
            complianceLog.Status = ComplianceStatus::MissingCode;
        }

        complianceLog.insert();
    }
}
```

Design rationale: The pre-event handles validation (preventing non-compliant invoices from being posted), while the post-event handles logging (recording compliance status for audit purposes). This separation follows the principle that pre-events are for validation/input modification and post-events are for side effects/logging.

Component 5: VendComplianceBatchJob (RunBaseBatch)

```
class VendComplianceBatchJob extends RunBaseBatch
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    FromDate fromDate;
    ToDate toDate;
    boolean generateReport;

    // Dialog
    public Object dialog()
    {
        Dialog dialog = super::dialog();
        dialog.addFieldValue(typeid(VendTable), vendTable, 'Vendor');
        dialog.addFieldValue(typeid(ComplianceCode), complianceCode, 'Compliance Code');
        dialog.addFieldValue(typeid(FromDate), fromDate, 'From Date');
        dialog.addFieldValue(typeid(ToDate), toDate, 'To Date');
        dialog.addFieldValue(typeid(boolean), generateReport, 'Generate Report');
        return dialog;
    }

    // Pack
    public container pack()
    {
        return [vendTable, complianceCode, fromDate, toDate, generateReport];
    }

    // Unpack
    public boolean unpack(container _packedClass)
    {
        vendTable = _packedClass.packGet(1);
        complianceCode = _packedClass.packGet(2);
        fromDate = _packedClass.packGet(3);
        toDate = _packedClass.packGet(4);
        generateReport = _packedClass.packGet(5);
        return true;
    }
```

```
    // Run on batch tier
    public RunOn runOn()
    {
        return RunOn::Batch;
    }

    // Business logic
    public void run()
    {
        VendTable vendTableLocal = VendTable::find(vendTable.AccountNum);
        APCustomsDeclaration declaration;

        while select declaration
            where declaration.VendTableRecId == vendTableLocal.RecId
            && declaration.DeclDate >= fromDate
            && declaration.DeclDate <= toDate
        {
            declaration.Status = ComplianceStatus::Processed;
            declaration.update();
        }

        if (generateReport)
        {
            // Generate SSRS report
        }

        info(strFmt('Compliance batch job completed for vendor "%1".', vendTableLocal.AccountNum));
    }

    // Batch metadata
    public BatchHeader batchInfo()
    {
        BatchHeader batchHeader = super::batchInfo();
        batchHeader.parmDescription('Vend Compliance Batch Job');
        batchHeader.parmBatchGroup('BATCHGROUP1');
        batchHeader.parmExecutionStyle(BatchExecutionStyle::OnDemand);
        return batchHeader;
    }

    // Static entry point
    public static void main(Args _args)
    {
        VendComplianceBatchJob job = new VendComplianceBatchJob();
        if (_args && _args.record())
        {
            job.parmVendTable(_args.record());
        }
        if (job.prompt())
        {
            job.run();
        }
    }
}
```

Component 6: MenuItemAction Navigation

```
// MenuItemAction: Object = VendComplianceController, Method = main
// This menu item triggers the SysOperationServiceController

// From a form button, navigation to the compliance dashboard:
```

```
public void clicked()
{
    Args args = new Args();
    args.name(formstr(ComplianceDashboardForm));
    args.record(element.args().record());

    MenuFunction menuFunction = new MenuFunction(
        menuItemDisplayStr(ComplianceDashboard), MenuItemType::Display);
    menuFunction.run(args);
}
```

Model Manifest Dependencies

```
<ModelManifest>
  <Name>AcmeOrderToCashCompliance</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
  <ConfigurationKey>AcmeOrderToCashComplianceEnabled</ConfigurationKey>
</ModelManifest>
```

Remaining Chapters — Detailed Outline (Content to follow)

Chapter 7 — Chain of Command (CoC) The Extension Model

7.1 CoC vs. Override vs. Event Handler — When Each Is Appropriate

D365 F&O provides three distinct mechanisms for extending standard code. Understanding when to use each is one of the most important architectural decisions you'll make.

The Three Mechanisms

Mechanism	How It Works	Upgrade Safety	Complexity	Best For
Chain of Command (CoC)	Override a standard method via `[ExtensionOf(...)]`; call `super()` to chain to the next implementation	[x] High — standard method still runs; your code is additive	Low	Modifying the behavior of a standard method where the base logic should still execute
Event Handler	Subscribe to a method's pre/post/override event using `[SubscribesTo(...)]`	[x] High — completely decoupled from the base method	Medium	Cross-cutting concerns (logging, notifications, integration triggers) where multiple subscribers may need to react independently
Direct Override	Replace the standard method entirely by modifying the base class	[ ] Low — overwritten by Microsoft CUs	Low (but dangerous)	**Never** for customer code

Decision Matrix

Scenario	Recommended Pattern	Why
Add validation to `VendInvoiceJour.validateWrite()`	**CoC** (override)	The standard validation should still run; you're adding a check before or after
Log every time an invoice is posted	**Event Handler** (post-event)	Multiple solutions may want to react to the same event; decoupled architecture
Completely replace standard invoice posting logic	**Override-event** (use sparingly)	You've replaced standard behavior entirely — this makes upgradeability harder
Add a field to `VendTable`	**Table Extension** (not CoC)	CoC is for method behavior; table extensions are for adding fields
Add a button that opens a new form	**Form Extension** + CoC	The form extension adds the button; CoC handles the navigation logic

The Golden Rule

> **If the standard method's logic is still valuable and you just want to add to it, use CoC. If you need multiple independent reactions to the same point in execution, use Event Handlers. If**

> **you need to completely replace standard behavior, use an Override-event (and document why).**

7.2 `super()` in CoC — Automatic vs. Explicit Behavior

The `super()` call is the mechanism that connects your extension to the standard code chain. Understanding how it works is critical to avoiding bugs.

How `super()` Works in CoC

When you override a method using CoC, calling `super()` invokes the **next class in the chain**. The chain is resolved at runtime based on the model layer and dependency order.

```
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_ComplianceExtension
{
    // Override validateWrite() – add compliance check
    public boolean validateWrite()
    {
        boolean ret;

        // Call super() FIRST – standard validations run
        ret = super();

        // Add custom validation AFTER standard validations pass
        if (ret && this.CreditMax > 500000 && this.ComplianceCode == '')
        {
            ret = checkFailed('Compliance code required for vendors exceeding $500,000 credit limit');
        }

        return ret;
    }
}
```

`super()` Position Rules

Method Type	`super()` Position	Why
`validateWrite()`	**FIRST** — call `super()` before custom logic	Standard validations must run first; if they fail, your custom logic is irrelevant
`modifiedField()`	**LAST** — call `super()` after custom logic	Your field change logic should run before the standard framework processes the change
`executeQuery()`	**LAST** — call `super()` after adding ranges	`super()` executes the query; filters added after it are ignored
`active()`	**LAST** — call `super()` after custom logic	Standard active logic should run after your setup
`run()`	**FIRST or LAST** — depends on whether you need to wrap the standard behavior	Wrapping pattern: `super()` first, then post-processing; or `super()` last, then pre-processing

Automatic vs. Explicit `super()`

In X++, `super()` is **always explicit** — you must write it yourself. There is no automatic chaining. This is different from some languages where the base class method is called automatically.

Common mistake — forgetting `super()`:

```
// [ ] WRONG — forgetting super() breaks the chain
public boolean validateWrite()
{
    // Custom validation only — standard validations are SKIPPED
    if (this.CreditMax > 500000 && this.ComplianceCode == '')
    {
        return checkFailed('Compliance code required');
    }
    return true;  // Standard validations never run!
}
```

Correct version:

```
// [x] CORRECT — super() is called, standard validations still run
public boolean validateWrite()
{
    boolean ret;
    ret = super();  // Standard validations execute here

    if (ret && this.CreditMax > 500000 && this.ComplianceCode == '')
    {
        ret = checkFailed('Compliance code required');
    }
    return ret;
}
```

Infinite Loop Pitfall

If you accidentally call `super()` in a way that creates a circular chain, you get an infinite loop and eventually a stack overflow.

How infinite loops happen: 1. **Forgetting `super()` in a CoC override** — the standard method is never called, but your override is called again on the next invocation (in some scenarios) 2. **Calling the wrong `super()`** — calling `super()` on a method that doesn't exist in the base class, causing the compiler to resolve to an unexpected method 3. **Event handler calling the standard method directly** — if an event handler calls the standard method, and the standard method triggers the event again, you get a loop

How to avoid infinite loops: - Always verify the method you're overriding actually has a `super()` call in the standard code - Use the **pre-event/post-event** pattern instead of override-event when you just need to react to a method execution - Test with a simple case first — set a breakpoint on `super()` to verify the chain executes correctly

---

7.3 `ExtensionOf(...)` — The Extension Syntax

The `ExtensionOf` attribute is the mechanism that tells the D365 F&O framework which base class, table, form, or view you are extending.

ExtensionOf for Classes

```
// Extending a class
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_ComplianceExtension
{
    // Override validateWrite() on VendInvoiceJour
    public boolean validateWrite()
    {
        boolean ret;
        ret = super();
        if (ret && this.CreditMax > 500000 && this.ComplianceCode == '')
        {
            ret = checkFailed('Compliance code required');
        }
        return ret;
    }
}
```

ExtensionOf for Tables

```
// Extending a table — adding a field
[ExtensionOf(tableStr(VendTable))]
final class VendTable_ComplianceExtension
{
    ComplianceCode complianceCode;
}
```

ExtensionOf for Forms

```
// Extending a form — adding a data source or control
[ExtensionOf(formStr(CustTable))]
final class CustTable_FormExtension
{
    // Add a new data source
    CustComplianceLog complianceLog_ds;

    // Add a new control to the design
    // (Controls are added via the extension form design node in the AOT)
}
```

ExtensionOf for Views

```
// Extending a view — adding a new data source
[ExtensionOf(viewStr(VendorContactView))]
final class VendorContactView_Extension
{
    // Add a new data source to the view
    CustComplianceLog complianceLog_ds;
}
```

ExtensionOf Syntax Reference

Target	Syntax	Example
Class	`ExtensionOf(classStr(ClassName))`	`[ExtensionOf(classStr(VendInvoiceJour))]`

Target	Syntax	Example
Table	`ExtensionOf(tableStr(TableName))`	`[ExtensionOf(tableStr(VendTable))]`
Form	`ExtensionOf(formStr(FormName))`	`[ExtensionOf(formStr(CustTable))]`
View	`ExtensionOf(viewStr(ViewName))`	`[ExtensionOf(viewStr(VendorContactView))]`
EDT	`ExtensionOf(extendedTypeStr(EDTName))`	`[ExtensionOf(extendedTypeStr(CustAccount))]`
Enum	`ExtensionOf(enumStr(EnumName))`	`[ExtensionOf(enumStr(VendInvoiceStatus))]`
Menu	`ExtensionOf(menuStr(MenuName))`	`[ExtensionOf(menuStr(File))]`

`final` Keyword

Always mark extension classes as `final` unless you explicitly intend for them to be further extended. This prevents unintended subclassing and makes the code's intent clear.

```
// [x] Recommended — extension classes are typically final
[ExtensionOf(tableStr(VendTable))]
final class VendTable_ComplianceExtension
{
    // ...
}
```

7.4 Overloading vs. Overriding in X++

Understanding the difference between overloading and overriding is essential for correct CoC implementation.

Overloading (Method Name Same, Different Signature)

X++ does **not** support method overloading in the traditional sense (same method name, different parameter types). If you define two methods with the same name in the same class, the compiler will flag an error.

What this means for CoC: You cannot overload a standard method by adding a variant with different parameters. You must override the exact method signature.

Overriding (Replacing Base Method Implementation)

Overriding is what CoC does — you provide a new implementation for a method that exists in the base class.

```
// Overriding validateWrite() on VendInvoiceJour
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_OverrideExtension
{
    // This OVERRIDES the standard validateWrite() method
    // Your implementation replaces the standard one in the chain
    public boolean validateWrite()
    {
        // Your custom logic
```

```
        // ...

        // Call super() to chain to the next implementation
        return super();
    }
}
```

Overriding Rules

1. **The method signature must match exactly** — same name, same parameter types, same return type
2. **You cannot reduce visibility** — if the base method is `public`, your override must be `public`
3. **Static methods CAN be overridden via CoC** — but your extension method must also be declared `static`, matching the base method's accessibility and staticness exactly. *Exception:* Static methods on **forms** cannot be wrapped (forms are not true classes).
4. **You cannot override `final` methods** — unless the method is explicitly marked with `[Wrappable(true)]`, which allows wrapping even if `final` is present. **Source:** Method wrapping and Chain of Command

When You Can't Override

Some standard methods are marked `final` in the D365 F&O codebase, meaning they cannot be overridden via CoC **unless** the method is explicitly marked with `[Wrappable(true)]`. For these methods, you must use **event handlers** instead.

Method Pattern	Overrideable via CoC?	Alternative
`validateWrite()`	[x] Yes	—
`executeQuery()`	[x] Yes	—
`modifiedField()`	[x] Yes	—
`run()`	[x] Yes	—
`init()`	[x] Yes	—
`static` methods (non-form)	[x] Yes	—
`static` methods on forms	[ ] No	Event Handler
`final` methods without `[Wrappable(true)]`	[ ] No	Event Handler

How to check if a method is overrideable: - If it has the `final` keyword **and no `[Wrappable(true)]` attribute**, it cannot be overridden via CoC. - Static methods on **forms** cannot be wrapped (forms are not true classes). - Static methods on **classes/tables** can be wrapped if the extension method is also `static`.

7.5 CoC Pitfalls — Common Mistakes and How to Avoid Them

Pitfall 1: Infinite Loops from Forgetting `super()`

The problem: If your CoC override doesn't call `super()`, the standard method's logic is skipped. In some cases, this can cause the framework to call your override again, creating an infinite loop.

Example:

```
// [ ] DANGEROUS — no super() call, potential infinite loop
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_BadExtension
{
    public boolean validateWrite()
    {
        // Standard validation never runs
        // Framework may re-invoke this method expecting standard behavior
        if (this.CreditMax > 500000 && this.ComplianceCode == '')
        {
            return checkFailed('Compliance code required');
        }
        return true;  // Skips all standard validations!
    }
}
```

Fix: Always call `super()` unless you have an explicit reason not to (and even then, document why).

Pitfall 2: Wrong `super()` Position

The problem: Calling `super()` at the wrong point in the method changes the behavior of your code.

Example — wrong position in `executeQuery()`:

```
// [ ] WRONG — super() called first, so your filter is ignored
public void executeQuery()
{
    super();  // Query executes HERE — your filter below is never applied

    // This code runs AFTER the query has already executed
    QueryBuildDataSource qbd = element.query().dataSourceTable(tableNum(VendTable));
    QueryBuildRange qbr = qbd.addRange(fieldNum(VendTable, AccountNum));
    qbr.value(queryValue('C0001'));
}
```

Fix:

```
// [x] CORRECT — super() called LAST, after all filters are added
public void executeQuery()
{
    QueryBuildDataSource qbd = element.query().dataSourceTable(tableNum(VendTable));
    QueryBuildRange qbr = qbd.addRange(fieldNum(VendTable, AccountNum));
    qbr.value(queryValue('C0001'));

    super();  // Query executes HERE — with your filter applied
}
```

Pitfall 3: Wrong Event Timing (Pre vs. Post)

The problem: Using a pre-event when you need a post-event (or vice versa) causes logic to execute at the wrong point in the lifecycle.

Example — using pre-event for logging:

```
// [ ] WRONG — pre-event fires BEFORE the standard method
// The invoice hasn't been posted yet, so logging "invoice posted" is misleading
[SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]
```

```
public static void VendInvoiceJour_onValidateWrite_Pre(VendInvoiceJour _this)
{
    info('Invoice has been posted');  // Misleading — it hasn't been posted yet!
}
```

Fix:

```
// [x] CORRECT — post-event fires AFTER the standard method
[SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite), EventExecution::Post)]
public static void VendInvoiceJour_onValidateWrite_Post(VendInvoiceJour _this)
{
    if (_this.validateWrite)
    {
        info('Invoice validation succeeded');  // Accurate — validation has completed
    }
}
```

Pitfall 4: CoC on a Method That Doesn't Participate in CoC

The problem: Not all standard methods participate in the CoC chain. Some methods are `static` or are called directly without going through the chain.

Example:

```
// [ ] WON'T WORK — static methods don't participate in CoC
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_StaticExtension
{
    // This override will NOT be called because the standard method is static
    public static void someStaticMethod()
    {
        // Your code here — never executed
    }
}
```

Fix: For static methods, use event handlers or call the static method directly from your code.

Pitfall 5: Extension Class Lifecycle — When Extensions Are Loaded

Extension classes are loaded **when the AOS starts** (or when the model is deployed). They are not loaded on-demand. This has important implications:

1. **Model dependency matters** — if your extension model depends on a base model, the base model must be deployed first. If the base model is missing, your extension will fail to load.
2. **Layer resolution applies** — if two models define extensions for the same class, the highest-layer extension wins. This means your CUS-layer extension will take precedence over an ISV-layer extension, and USR-layer extensions take precedence over CUS.
3. **Restart required** — after deploying a new extension, the AOS must be recycled for the extension to take effect. Simply deploying the model is not enough.

Pitfall 6: Modifying Extension Class Behavior After Deployment

Once an extension class is loaded into the AOS, modifying it requires: 1. Rebuilding the model 2. Deploying the new `.axmodel` 3. Recycling the AOS

This is different from event handlers, which can sometimes be hot-reloaded depending on the framework version. For CoC overrides, always plan for an AOS recycle after deployment.

7.6 Extension Class Lifecycle — In Detail

How Extensions Interact with the Base Class

```
[1] AOS starts → loads all models from the model store
    |
[2] For each class, the framework resolves the CoC chain:
    |   • Finds the highest-layer extension for the class
    |   • Links extensions in layer order (USR → CUS → VAR → ISV → SYS)
    |   • Builds an internal chain of method calls
    |
[3] When a standard method is called:
    |   • The framework starts at the highest-layer extension
    |   • That extension calls super() → next extension in chain
    |   • The chain continues until the base class method is reached
    |
[4] When a new model is deployed:
    |   • The model store is updated
    |   • AOS is recycled → extensions are reloaded
    |   • The CoC chain is rebuilt with the new extension
    |
[5] When a model is removed:
    |   • AOS is recycled → extensions are reloaded
    |   • The CoC chain is rebuilt without the removed extension
    |   • The base class method is now called directly
```

Model Dependency Implications

Your extension model **must** declare a dependency on the model that contains the base class. If you don't, the deployment will fail because the framework cannot resolve the base class.

```
<ModelManifest>
  <Name>MyComplianceExtension</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <!-- Required: the model containing VendInvoiceJour -->
    <ModelReference>
      <Name>ApplicationSuite</Name>
      <MinVersion>10.0.0.0</MinVersion>
    </ModelReference>
  </References>
</ModelManifest>
```

Extension Loading Order

Extensions are loaded in **layer order** — higher layers are resolved first:

1. `CUS` (customer extensions — highest priority for customer code)
2. `VAR` (Microsoft variation extensions)
3. `ISV` (ISV partner extensions)
4. `SYS` (Microsoft system framework — lowest priority)

When multiple extensions target the same method, the **highest-layer extension's `super()` call** determines which extension runs next in the chain.

7.7 Activity — Extend AP Invoice Validation Three Ways

> **Activity**: Extend the standard Accounts Payable invoice validation (`VendInvoiceJour.validateWrite()`) using three different extension approaches: 1. **CoC Override** — override `validateWrite()` directly on `VendInvoiceJour` 2. **Event Handler** — subscribe to the `validateWrite` event (pre-event) 3. **Class Extension** — extend the `VendInvoiceJour` class with a new method that is called from a menu item
>
> For each approach, implement the same business rule: **a compliance code is required when the vendor's credit limit exceeds $500,000**.
>
> After implementing all three, explain the trade-offs for each approach in terms of: - Upgradeability (how well it survives Microsoft CUs) - Maintainability (how easy it is to find and modify the logic) - Performance (any overhead introduced) - Testability (how easy it is to unit test) - Reusability (can the logic be applied to other tables/methods)
>
> **Activity Hints** (Multiple Valid Approaches):
>
> - **Hint A — CoC Override approach**: Option A1 — override `validateWrite()` directly in an `[ExtensionOf(classStr(VendInvoiceJour))]` class, calling `super()` first then adding your check (recommended for this scenario — straightforward, debuggable). Option A2 — use a `construct()` pattern to create a validator class from within the CoC override (more testable but more complex).
> - **Hint B — Event Handler approach**: Option B1 — use a pre-event `[SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]` that adds the compliance check (recommended — decoupled, multiple subscribers possible). Option B2 — use a post-event to log the compliance status after validation completes (different concern — logging, not validation). Option B3 — use both pre-event (for validation) and post-event (for logging) to demonstrate the pattern.
> - **Hint C — Class Extension approach**: Option C1 — create a new class with a `checkCompliance()` method that is called from a menu item or button (recommended for the class extension pattern — clean separation). Option C2 — extend `VendInvoiceJour` with a new instance method that can be called from the form (more integrated but tighter coupling). Option C3 — use a `RunBase` class that validates compliance as a batch operation (for bulk validation scenarios).
> - **Hint D — Trade-off analysis**: Consider that CoC overrides are the most straightforward for method behavior changes but create tight coupling to the base method. Event handlers are the most decoupled and support multiple subscribers but have less predictable execution order. Class extensions with new methods are the most reusable but require the caller to know about the extension class.

Expected Approach (Ideal — in detail)

Approach 1: CoC Override

```
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_ComplianceCocExtension
{
    /// <summary>
    /// Override validateWrite() to enforce compliance code for high-credit vendors.
    /// Uses CoC chain — standard validations still run via super().
    /// </summary>
    public boolean validateWrite()
    {
        boolean ret;

        // Call super() FIRST — standard D365 validations run
        ret = super();

        // Add custom compliance validation
        if (ret && this.CreditMax > 500000 && this.ComplianceCode == '')
        {
            ret = checkFailed(
                'Compliance code is required for vendors exceeding $500,000 credit limit.');
        }

        return ret;
    }
}
```

Trade-offs: - **Upgradeability**: [x] High — the standard `validateWrite()` is untouched; your extension survives CUs - **Maintainability**: [x] Good — the override is clearly visible in the AOT under `Classes\VendInvoiceJour_ComplianceCocExtension` - **Performance**: [x] Minimal — one additional `if` check per `validateWrite()` call - **Testability**: [!] Moderate — requires an AOS connection to test; can be tested with `RunBase`-style unit tests - **Reusability**: [ ] Low — this is specific to `VendInvoiceJour`; to apply to other tables, you'd need a separate override

Approach 2: Event Handler (Pre-Event)

```
class VendInvoiceJour_EventHandler
{
    /// <summary>
    /// Pre-event handler for VendInvoiceJour.validateWrite().
    /// Checks compliance code for high-credit vendors before standard validation runs.
    /// </summary>
    [SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite))]
    public static void VendInvoiceJour_onValidateWrite_Pre(VendInvoiceJour _this)
    {
        if (_this.CreditMax > 500000 && _this.ComplianceCode == '')
        {
            // Note: In pre-events, we can't directly return false.
            // We use checkFailed() which causes validateWrite() to return false.
            _this.validateWrite = checkFailed(
                'Compliance code is required for vendors exceeding $500,000 credit limit.');
        }
    }

    /// <summary>
    /// Post-event handler for VendInvoiceJour.validateWrite().
    /// Logs compliance status after validation completes.
    /// </summary>
    [SubscribesTo(tableStr(VendInvoiceJour), methodStr(VendInvoiceJour, validateWrite),
EventExecution::Post)]
    public static void VendInvoiceJour_onValidateWrite_Post(VendInvoiceJour _this)
```

```
    {
        if (_this.validateWrite)
        {
            APComplianceLog::logCheck(_this.RecId, ComplianceStatus::Validated);
        }
        else
        {
            APComplianceLog::logCheck(_this.RecId, ComplianceStatus::Failed);
        }
    }
}
```

Trade-offs: - **Upgradeability**: [x] High — completely decoupled from the base method; standard code changes don't affect the handler - **Maintainability**: [x] Good — event handlers are centralized in a dedicated class; easy to find all subscriptions - **Performance**: [!] Slight overhead — event dispatch mechanism adds a small cost per invocation - **Testability**: [x] Good — event handlers can be unit tested independently of the base method - **Reusability**: [x] High — the same event handler class can subscribe to multiple methods; multiple handlers can subscribe to the same event

Approach 3: Class Extension with New Method

```
[ExtensionOf(classStr(VendInvoiceJour))]
final class VendInvoiceJour_ComplianceClassExtension
{
    /// <summary>
    /// New method on VendInvoiceJour that checks compliance.
    /// Called from a menu item or form button — not automatically triggered.
    /// </summary>
    public ComplianceStatus checkCompliance()
    {
        if (this.CreditMax > 500000 && this.ComplianceCode == '')
        {
            return ComplianceStatus::MissingCode;
        }
        return ComplianceStatus::Compliant;
    }
}
```

Trade-offs: - **Upgradeability**: [x] High — adds a new method; doesn't modify standard behavior - **Maintainability**: [!] Moderate — the method is on the extended class but must be explicitly called; easy to miss if developers don't know it exists - **Performance**: [x] Minimal — only runs when explicitly called - **Testability**: [x] Good — can be tested in isolation by creating a `VendInvoiceJour` instance and calling `checkCompliance()` - **Reusability**: [x] High — the pattern of adding a new method can be applied to any table; other developers can call `vendInvoiceJour.checkCompliance()` from anywhere

Which Approach Is Best?

For this specific scenario (enforcing a business rule at validation time), **Approach 1 (CoC Override)** is the recommended approach because: 1. It's the most straightforward and debuggable 2. The standard `validateWrite()` chain is preserved via `super()` 3. The validation runs automatically for every invoice — no risk of developers forgetting to call it 4. It follows the Microsoft-documented pattern for modifying standard table method behavior

Approach 2 (Event Handler) is better when: - Multiple independent solutions need to react to the same event - You want to log or notify without modifying the validation logic - You need to decouple the

compliance check from the invoice validation

Approach 3 (Class Extension) is better when: - The compliance check is an optional/manual action (not automatic) - You want to expose the check as a reusable method callable from multiple places - You're building a framework or utility class that other developers will use

---

Remaining Chapters — Detailed Outlines

Chapter 8 — Security Architecture

8.1 The Security Hierarchy — Permission → Privilege → Duty → Role → User

D365 F&O uses a **four-tier security hierarchy** that controls every action a user can perform. Understanding this hierarchy is essential for designing secure customizations.

The Four Tiers

```
Security Role
    |
    +-- Duty 1
    |       +-- Privilege 1.1
    |       |       +-- Permission (Table: VendTable, Access: Read)
    |       |       +-- Permission (Table: VendInvoiceJour, Access: Create)
    |       +-- Privilege 1.2
    |               +-- Permission (Table: APComplianceLog, Access: Write)
    |
    +-- Duty 2
    |       +-- Privilege 2.1
    |               +-- Permission (Table: VendTable, Access: Edit)
    |
    +-- Duty 3
            +-- Privilege 3.1
                    +-- Permission (Table: CustTable, Access: Read)
```

Tier Descriptions

Tier	Purpose	Example
Security Role	Top-level assignment to a user	`ComplianceOfficer`, `APManager`, `InvoiceProcessor`
Duty	A collection of related privileges	`APViewer`, `APProcessor`, `ComplianceAdmin`
Privilege	Granular access to specific tables and fields	`VendTableRead`, `VendInvoiceJourCreate`, `APComplianceLogWrite`
Permission	The most granular level — specific table + access level + optionally specific fields	`VendTable` → Read, `VendInvoiceJour` → Create + Edit

How Security is Enforced

1. A user is assigned one or more **Security Roles** in the **User Administration** form (`Security > Users > Users`)
2. Each role contains **Duties**
3. Each duty contains **Privileges**
4. Each privilege contains **Permissions** (table-level and optionally field-level)
5. At runtime, the framework checks the user's roles against the required permissions for every action
6. If the user lacks the required permission, the action is blocked and an error is shown

The Security Role Assignment

```
// Security roles are assigned in the User Administration form
// Not programmatically in most cases — but can be managed via code

// To check if the current user has a specific role:
boolean hasRole = SecurityRole::hasRole(SecurityRole::find('ComplianceOfficer').RecId);
```

Security in the Extension Model

When you create custom objects in an extension model, the security model must be designed as part of the extension. Key considerations:

1. **Custom tables** need their own Configuration Key and security model
2. **Table extensions** inherit the security model of the base table — no additional permissions needed for added fields
3. **Form extensions** need menu item permissions for any new navigation items
4. **Report extensions** need output permissions for any new reports
5. **CoC overrides** do not change security requirements — the base method's security context applies
6. **Event handlers** run in the context of the base method — they inherit the base method's permissions

Critical Rule: Extensions never bypass the base security model. If a user doesn't have permission to access a table, your extension code cannot grant that access. The security framework checks permissions before any X++ code executes.

8.2 Access Levels — See, Create, Edit, Delete

Each permission in a privilege specifies an **AccessLevel** that controls what operations are allowed on the table.

AccessLevel	Value	What It Allows
See	`AccessLevel::See`	Read-only — can view records, run queries, open forms that display data
Create	`AccessLevel::Create`	Can insert new records into the table
Edit	`AccessLevel::Edit`	Can modify existing records (implies See)
Delete	`AccessLevel::Delete`	Can delete records (implies See and Edit)
Admin	`AccessLevel::Admin`	Full control — See, Create, Edit, Delete, and administrative operations

Access Level Inheritance

Access levels are **cumulative** — higher levels include the permissions of lower levels:

- `Edit` includes `See` (you must be able to see a record before you can edit it)
- `Delete` includes `See` and `Edit` (you must be able to see and edit a record before you can delete it)
- `Admin` includes all levels

Setting Access Levels on Privileges

```
Privilege: APComplianceViewer
+-- Permission: VendTable → See
+-- Permission: VendInvoiceJour → See
+-- Permission: APComplianceLog → See + Create
+-- Permission: APComplianceLog → Edit (field-level: only ComplianceCode, Status)
```

In the AOT, each privilege has a **Permissions** node where you define: - **Table**: The table the permission applies to - **Access Level**: See, Create, Edit, Delete, or Admin - **Field-Level Permissions**: Specific fields the user can see or edit (optional)

Field-Level Permissions

Field-level permissions allow you to restrict access to specific fields within a table. This is the principle of **least privilege** — users only see and edit the fields they need.

```
Privilege: ComplianceFieldRestriction
+-- Permission: VendTable → See (all fields)
+-- Permission: VendTable → See (field: AccountNum, Name, CreditMax)
+-- Permission: VendTable → Edit (field: ComplianceCode only)
+-- Permission: VendTable → Deny (field: CreditMax — compliance officers cannot see credit limits)
```

Field-level permissions are set on each Permission record: 1. Open the privilege in the AOT 2. Navigate to the **Permissions** node 3. Select the table 4. Set the **Access Level** 5. Use the **Field Permissions** grid to specify which fields are visible/editable/denied

Field-Level Permissions and Extensions

When you add fields via table extensions, the field-level permissions work differently:

- **Added fields** (via table extension): Inherit the base table's field-level permissions by default
- **To restrict access to an added field**: You must explicitly set field-level permissions on the privilege
- **To grant access to an added field**: Add a field-level permission for the new field in the relevant privilege

```
// Example: A field added via table extension
// VendTable_Extension adds a ComplianceCode field
// To control access to this field:

// In the privilege Permissions grid:
// - Add a field-level permission for ComplianceCode
// - Set Access Level to See or Edit for the relevant roles
// - Deny the field for roles that should not see it
```

8.3 SecurityKey — Scoping Access by Business Context

A **SecurityKey** is a mechanism that scopes access to records based on business context. It's used to implement **record-level security** — restricting which records a user can see based on their role, company, or other business dimensions.

How SecurityKeys Work

```
User logs in → Framework checks SecurityKey assignments →
Records are filtered at query time → User only sees records matching their SecurityKey scope
```

Standard SecurityKeys

SecurityKey	Purpose	Example
`Default`	No restriction — all records visible	System administrators
`Company`	Scoped to the current Legal Entity (company)	Most users — they only see their company's data
`VendGroup`	Scoped to the user's vendor group	AP clerks — they only see vendors in their group
`CustGroup`	Scoped to the customer group	AR clerks — they only see customers in their group
`CostCenter`	Scoped to the user's cost center	Department managers
`CustomKey`	Custom key defined by the implementer	Any business-specific scoping

Assigning SecurityKeys to Roles

SecurityKeys are assigned at the **privilege level**:

1. Open the privilege in the AOT
2. Navigate to the **Security Keys** node
3. Add a SecurityKey assignment with the desired scope
4. The framework applies the filter automatically to all queries on that table

Custom SecurityKeys

For custom tables, you can define your own SecurityKey:

```
// In your table's security key definition
// The SecurityKey property on the table determines which key is used
// The framework automatically adds a WHERE clause to queries:
// WHERE SecurityKeyField = currentUser'sSecurityKeyValue
```

SecurityKey vs. Record-Level Security

Feature	SecurityKey	Record-Level Security
Scope	Business context (company, group, cost center)	Specific record ownership or criteria
Implementation	Defined on the table's `SecurityKey` property	Implemented via `recordLevelSecurity()` method
Query filtering	Automatic — framework adds WHERE clause	Manual — developer implements filtering logic
Use case	Multi-company, multi-group scoping	Row-level ownership (e.g., "users can only see their own records")

8.4 Record-Level Security vs. Table-Level Security

Table-Level Security

Table-level security controls whether a user can access a table at all. It's the coarse-grained control:

- **See**: Can the user view records in this table?
- **Create**: Can the user insert new records?
- **Edit**: Can the user modify existing records?
- **Delete**: Can the user remove records?

Table-level security is set on each **Permission** record within a privilege.

Record-Level Security

Record-level security controls **which specific records** a user can access within a table they have table-level permission for.

```
// Example: Record-level security on VendTable
// Users can only see vendors in their vendor group
public boolean recordLevelSecurity()
{
    // The framework checks if the current record matches the user's security scope
    // This is implemented via the SecurityKey mechanism
    // The record is filtered at query time
    return super();
}
```

When to Use Each

Scenario	Use
Users should see all vendors in the system	Table-level only (See permission on VendTable)
Users should only see vendors in their company	SecurityKey = `Company` on VendTable
Users should only see vendors in their vendor group	SecurityKey = `VendGroup` on VendTable
Users should only see vendors they created	Record-level security with custom `recordLevelSecurity()` logic
Users should see vendors but not their credit limits	Field-level permission (Deny on CreditMax field)

Implementing Custom Record-Level Security

For scenarios where standard SecurityKeys don't cover your needs, you can implement custom record-level security:

```
// In a table's data source on a form:
public void executeQuery()
{
    QueryBuildDataSource qbd;
    QueryBuildRange qbr;

    super();
```

```
    // Add a range that restricts records to the current user's scope
    qbd = element.query().dataSourceTable(tableNum(VendTable));
    qbr = qbd.addRange(fieldNum(VendTable, CreatedBy));
    qbr.value(queryValue(curUserId()));
}
```

Note: This approach is form-specific. For system-wide record-level security, use SecurityKeys or the `recordLevelSecurity()` method on the table.

8.5 Extensible Data Security (XDS) — Deep Dive

Extensible Data Security (XDS) is the modern mechanism for implementing row-level security in D365 F&O. It replaces the older SecurityKey approach for many scenarios and provides a more flexible, policy-based security model.

XDS vs. SecurityKey

Feature	SecurityKey	XDS (Extensible Data Security)
Implementation	Defined on the table's `SecurityKey` property	Policy-based, defined in security policies
Flexibility	Limited to predefined key types	Highly flexible — custom policies
Query filtering	Automatic WHERE clause	Automatic WHERE clause via policy
Multi-entity	Per-table configuration	Can span multiple tables and entities
Runtime evaluation	At query time	At query time with policy evaluation
Use case	Standard scoping (company, group)	Custom row-level security rules

XDS Policy Components

An XDS policy consists of three parts:

1. **Policy**: Defines the security rule (e.g., "users can only see records in their cost center")
2. **Policy Type**: Determines how the policy is evaluated (e.g., `Table`, `Field`, `Relation`)
3. **Policy Element**: The specific condition that filters records

XDS Policy Example

```
// XDS Policy: Restrict VendTable records to the user's vendor group
// Policy Type: Table
// Policy Element: WHERE VendGroup = currentUser'sVendGroup

// The policy is defined in the Security > Data Policies workspace
// and is automatically applied to all queries on VendTable
```

XDS Policy Types

Policy Type	Description	Example
Table	Filters entire records based on a condition	User can only see vendors in their group
Field	Restricts access to specific field values	User can only see records where Status = Active
Relation	Filters based on related table data	User can only see vendors with approved credit

When to Use XDS vs. SecurityKey

Scenario	Use
Standard company scoping	SecurityKey (`Company`)
Standard group scoping	SecurityKey (`VendGroup`, `CustGroup`)
Custom business logic filtering	XDS Policy
Multi-table security rules	XDS Policy (can span multiple tables)
Field-level restrictions beyond simple deny	XDS Field Policy
Dynamic security that changes at runtime	XDS Policy (evaluated at query time)

8.6 Privilege Assertions — `CodeAccessPermission::assert()` and `CodeAccessPermission::revertAssert()`

Privilege assertions are a powerful but dangerous mechanism that allows code to **temporarily elevate** the current user's permissions to perform an action they wouldn't normally be allowed to do.

`CodeAccessPermission::assert()` — Elevate Privileges

`CodeAccessPermission::assert()` tells the framework to check permissions as if the current user had **elevated access** for the duration of the assertion block. This is done using specific permission classes like `SqlStatementExecutePermission` or `InteropPermission`. In D365 F&O, `SqlStatementExecutePermission` takes a SQL statement string (not a table name and `AccessLevel`) and is used to assert permission for raw SQL execution via the `Connection`/`Statement`/`ResultSet` APIs. There is no `TableAccessPermission` class or equivalent — `SqlStatementExecutePermission` is the standard permission class for this scenario.

```
// Example: A service class that needs to read a table the current user doesn't have access to
public VendTable getVendorData(AccountNum _accountNum)
{
    VendTable vendTable;
    str sql;
    Connection connection;
    Statement statement;
    ResultSet resultSet;

    // Build the SQL query for the table we need to read
    sql = strFmt("SELECT * FROM %1 WHERE AccountNum = '%2'",
        tableStr(VendTable), _accountNum);

    // Elevate privileges to execute SQL against VendTable
```

```
    new SqlStatementExecutePermission(sql).assert();

    connection = new Connection();
    statement = connection.createStatement();
    resultSet = statement.executeQuery(sql);

    if (resultSet.next())
    {
        vendTable.AccountNum = resultSet.toString(1);
        // ... map additional fields as needed ...
    }

    // Revert to normal permission checking
    CodeAccessPermission::revertAssert();

    return vendTable;
}
```

Reducing Privileges

In D365 F&O, `SqlStatementExecutePermission` is an **all-or-nothing** assertion — it either grants permission to execute SQL against the database or it does not. Unlike AX 2012's `SqlStatementPermission`, there is no `AccessLevel` parameter to specify read-only vs. read-write access. To **reduce risk**, only assert permission when the operation truly requires it, and keep the assertion scoped to the shortest possible code block: assert immediately before the protected API call, then revert immediately after.

```
// Example: Only assert SQL execution permission for the specific operation that needs it
public void safeDelete(VendInvoiceJour _invoice)
{
    str sql;
    Connection connection;
    Statement statement;

    // Build the SQL DELETE statement for the specific invoice
    sql = strFmt("DELETE FROM %1 WHERE RecId = %2",
        tableStr(VendInvoiceJour), _invoice.RecId);

    try
    {
        // Assert permission only for the duration of the SQL execution
        new SqlStatementExecutePermission(sql).assert();

        connection = new Connection();
        statement = connection.createStatement();
        statement.executeUpdate(sql);

        // Revert immediately after the protected API call
        CodeAccessPermission::revertAssert();
    }
    catch (Exception::Error)
    {
        CodeAccessPermission::revertAssert();
        error("Failed to delete invoice. Contact your administrator.");
    }
}
```

Why Privilege Assertions Are Dangerous

Risk	Description
Bypasses security	`assert()` can allow users to access data they shouldn't see
Hard to audit	Assertions are not visible in the standard security model — they're invisible to permission simulation
Leaks in production	If an assertion is left in code, it can expose sensitive data in production environments
Violates least privilege	Assertions grant more permissions than the user should have

Best Practices for Privilege Assertions

1. **Minimize scope** — use `assert()` for the shortest possible code block, immediately followed by `revertAssert()`
2. **Never assert in loops** — asserting inside a `while select` loop is a performance anti-pattern
3. **Document every assertion** — add a comment explaining WHY the assertion is needed and what risk it introduces
4. **Review in code review** — every `assert()` call should be reviewed by a senior developer
5. **Prefer role-based design** — if you need `assert()`, it usually means the security model is wrong. Fix the role design instead.
6. **Test with permission simulation** — always test code with assertions using the "Run as" feature to verify the assertions work correctly

When Assertions Are Legitimate

Despite the risks, there are valid use cases:

Scenario	Justification
System service that processes data on behalf of users	The service runs with elevated privileges by design
Data migration or integration code	Migration tools need broader access than end users
Admin-only functionality (e.g., "Run as admin" button)	The user explicitly requests elevated access
Reading audit log data for compliance reporting	Compliance reports need access to all data regardless of user permissions

Security Hardening for Assertions

When using privilege assertions in extension code, follow these additional hardening practices:

1. **Wrap assertions in try/catch** — if the assertion fails, catch the exception gracefully
2. **Log assertion usage** — write to a custom audit log table whenever an assertion is used
3. **Use the minimum assertion level** — always use the lowest permission level that still allows the operation
4. **Never assert in CoC overrides** — CoC overrides should not elevate privileges; they should work within the existing security context
5. **Never assert in event handlers** — event handlers should not elevate privileges; they should work within the base method's security context

```
// [x] GOOD — assertion with logging and error handling
public VendTable getVendorDataSecure(AccountNum _accountNum)
```

```
{
    VendTable vendTable;
    str sql;
    Connection connection;
    Statement statement;
    ResultSet resultSet;

    try
    {
        // Log the assertion for audit purposes
        this.logAssertion('getVendorDataSecure', _accountNum);

        // Build the SQL query for the table we need to read
        sql = strFmt("SELECT * FROM %1 WHERE AccountNum = '%2'",
            tableStr(VendTable), _accountNum);

        // Elevate privileges to execute SQL against VendTable
        new SqlStatementExecutePermission(sql).assert();

        connection = new Connection();
        statement = connection.createStatement();
        resultSet = statement.executeQuery(sql);

        // Revert immediately after the protected API call
        CodeAccessPermission::revertAssert();

        if (resultSet.next())
        {
            vendTable.AccountNum = resultSet.toString(1);
            // ... map additional fields as needed ...
        }
    }
    catch (Exception::Error)
    {
        CodeAccessPermission::revertAssert();
        error('Failed to retrieve vendor data. Contact your administrator.');
    }

    return vendTable;
}
```

8.7 Security Role Design for Custom Objects

When you create custom tables, forms, and reports, you must design the security model to grant appropriate access.

Step-by-Step Security Role Design

Step 1: Identify the Custom Objects

List all custom objects that need security:

Object Type	Object Name	Purpose
Table	`APCustomsDeclaration`	Stores customs compliance declarations
Table Extension	`VendTable_Extension`	Adds ComplianceCode field to VendTable

Object Type	Object Name	Purpose
Form	`ComplianceHistoryForm`	Displays compliance history
Report	`ComplianceSummaryReport`	Monthly compliance report
Menu Item	`ComplianceHistory`	Navigation to compliance form
Data Entity	`VendAPComplianceEntity`	Integration with external audit system

Step 2: Define the Duties

Group related privileges into duties:

Duty Name	Purpose
`APComplianceViewer`	View compliance data — read access to all compliance tables
`APComplianceProcessor`	Process compliance declarations — create and edit access
`APComplianceAdmin`	Full compliance administration — all access including delete

Step 3: Define the Privileges

For each duty, define the specific privileges:

Duty: `APComplianceViewer`

Privilege	Table	Access Level	Field Permissions
`APComplianceView`	`APCustomsDeclaration`	See	All fields visible
`APComplianceView`	`VendTable`	See	All fields visible
`APComplianceView`	`VendInvoiceJour`	See	All fields visible
`APComplianceView`	`ComplianceHistoryForm`	See	Form access only

Duty: `APComplianceProcessor`

Privilege	Table	Access Level	Field Permissions
`APComplianceProcess`	`APCustomsDeclaration`	Create + Edit	All fields editable
`APComplianceProcess`	`VendTable`	See	All fields visible
`APComplianceProcess`	`VendInvoiceJour`	See	All fields visible
`APComplianceProcess`	`ComplianceHistoryForm`	Create + Edit	Form access

Duty: `APComplianceAdmin`

Privilege	Table	Access Level	Field Permissions
`APComplianceAdmin`	`APCustomsDeclaration`	Admin	All fields
`APComplianceAdmin`	`VendTable`	See	All fields
`APComplianceAdmin`	`VendInvoiceJour`	See	All fields

Privilege	Table	Access Level	Field Permissions
`APComplianceAdmin`	`ComplianceHistoryForm`	Admin	All
`APComplianceAdmin`	`ComplianceSummaryReport`	Output	Report execution

Step 4: Assign Duties to Roles

Create the security role and assign duties:

Role	Duties
`ComplianceOfficer`	`APComplianceViewer`, `APComplianceProcessor`
`ComplianceAdmin`	`APComplianceViewer`, `APComplianceProcessor`, `APComplianceAdmin`
`AP clerk`	`APComplianceViewer` (read-only)

Step 5: Assign the Role to Users

In the **User Administration** form, assign the `ComplianceOfficer` role to compliance officers and the `AP clerk` role to AP staff.

Security Role Design Template

```
<!-- Security Role Definition in ModelManifest.xml (conceptual) -->
<Security>
  <Roles>
    <Role name="ComplianceOfficer">
      <Duties>
        <Duty name="APComplianceViewer"/>
        <Duty name="APComplianceProcessor"/>
      </Duties>
    </Role>
    <Role name="ComplianceAdmin">
      <Duties>
        <Duty name="APComplianceViewer"/>
        <Duty name="APComplianceProcessor"/>
        <Duty name="APComplianceAdmin"/>
      </Duties>
    </Role>
  </Roles>
  <Privileges>
    <Privilege name="APComplianceView">
      <Permissions>
        <Permission table="APCustomsDeclaration" accessLevel="See"/>
        <Permission table="VendTable" accessLevel="See"/>
        <Permission table="VendInvoiceJour" accessLevel="See"/>
      </Permissions>
    </Privilege>
    <Privilege name="APComplianceProcess">
      <Permissions>
        <Permission table="APCustomsDeclaration" accessLevel="Create"/>
        <Permission table="APCustomsDeclaration" accessLevel="Edit"/>
        <Permission table="VendTable" accessLevel="See"/>
        <Permission table="VendInvoiceJour" accessLevel="See"/>
      </Permissions>
    </Privilege>
```

```
  </Privileges>
</Security>
```

8.8 Security Model for Extensions

When you create extensions that add new functionality, you must consider how the security model applies to your extension code.

Extension Security Principles

1. **Extensions inherit the base security context** — your CoC override or event handler runs with the same permissions as the base method
2. **New objects need new permissions** — custom tables, forms, and reports require their own security configuration
3. **Table extensions don't need new permissions** — added fields inherit the base table's permissions
4. **Menu items need permissions** — any new menu item must have a corresponding privilege

CoC and Security

Chain of Command overrides do not change the security context:

```
// This CoC override runs with the same permissions as the base method
[ExtensionOf(classStr(VendTable))]
final class VendTable_ComplianceCocExtension
{
    public boolean validateWrite()
    {
        boolean ret;

        // The base validateWrite() runs with the user's existing permissions
        ret = super();

        // Your custom validation also runs with the user's existing permissions
        // You cannot bypass security here
        if (this.ComplianceCode == '' && this.CreditMax > 100000)
        {
            ret = checkFailed('Compliance code required for high-credit vendors');
        }

        return ret;
    }
}
```

Event Handlers and Security

Event handlers also inherit the base method's security context:

```
class VendTableEventHandler
{
    [EventHandler(eventStr(VendTable::inserted))]
    public static void onVendTableInserted(Common _sender)
    {
        VendTable vendTable = _sender as VendTable;
```

```
        // This event handler runs with the same permissions
        // as the base insert() method — no elevation needed
        info(strFmt('Vendor %1 was inserted.', vendTable.AccountNum));
    }
}
```

When Extensions Need New Permissions

Extensions need new permissions when they:

1. **Add new tables** — the new table needs its own permissions
2. **Add new forms** — the form needs menu item permissions
3. **Add new reports** — the report needs output permissions
4. **Add new menu items** — the menu item needs a privilege
5. **Access data the user shouldn't normally see** — this is a design smell; reconsider the architecture

Common Extension Security Mistakes

Mistake	Risk	Fix
Adding a CoC override that reads data the user can't access	Security bypass	Don't read data the user doesn't have permission for
Creating a new form without menu item permissions	Users can't access the form	Add a MenuItemOutput with proper privileges
Adding a table extension field that exposes sensitive data	Data leakage	Set field-level permissions on the base table's privilege
Using `assert()` in an event handler	Unauthorized data access	Remove the assertion; redesign the extension
Creating a new report without output permissions	Users can't run the report	Add an Output Menu Item with proper privileges

---

8.9 Testing Security Roles

The "Run As" Feature

D365 F&O includes a **"Run as"** feature that lets you test security roles without logging in as a different user.

How to use "Run as": 1. Open the form or report you want to test 2. Click the **"Run as"** button in the toolbar (or press `Ctrl+Shift+F9`) 3. Select the user you want to test as 4. The form/report executes with that user's permissions 5. Verify that the user can or cannot access the expected data

The Security Role Switch

The **Security Role Switch** allows you to temporarily switch to a different security role for testing:

1. Navigate to **System Administration > Setup > Security > Security role switch**
2. Select the role you want to test
3. Click **Switch**
4. All subsequent actions use the permissions of the selected role
5. Click **Switch back** to return to your original role

Permission Simulation

Permission simulation lets you test what a user can and cannot do:

1. Navigate to **System Administration > Inquiries > Security > Permissions**
2. Select the user and role
3. Click **Simulate**
4. The simulation shows which permissions are granted and which are denied
5. Use this to verify that your custom security model works correctly

Testing Checklist

- [ ] Can the user see the custom form?
- [ ] Can the user create new records in the custom table?
- [ ] Can the user edit existing records?
- [ ] Can the user delete records (if Delete permission is granted)?
- [ ] Can the user run the custom report?
- [ ] Can the user see field-level restricted fields?
- [ ] Can the user NOT see fields that should be denied?
- [ ] Does the "Run as" test with a user who has no compliance role show no access?
- [ ] Does the Security Role Switch correctly apply the new role's permissions?
- [ ] Does the CoC override respect the user's existing permissions?
- [ ] Does the event handler run within the base method's security context?
- [ ] Are field-level permissions correctly applied to extended fields?

8.10 Activity — Design the Complete Security Model for a Custom Inventory Reconciliation Module

> **Activity**: Design the complete security model for a custom inventory reconciliation module. The module includes: 1. A custom table `INVReconciliationHeader` (parent) with fields: ReconciliationId, ReconciliationDate, Status, CreatedBy 2. A custom table `INVReconciliationLine` (child, linked to header via RecId) with fields: LineNumber, ItemId, Quantity, ReconciledQty, Variance, ApprovedBy 3. A form `INVReconciliationForm` (ListPage + Detail page) for viewing and processing reconciliations 4. A batch job `INVReconciliationBatchJob` that automatically reconciles items 5. A report `INVReconciliationReport` that shows reconciliation summaries 6. A data entity `INVReconciliationEntity` for pushing reconciliation data to an external ERP
>
> Design the complete security model including: - At least 3 duties with appropriate privileges - At least 2 security roles with different duty assignments - Field-level permissions for sensitive fields (e.g., Variance should only be visible to approvers) - Record-level security so users can only see reconciliations for their own company - A "Run as" test plan to verify the security model works correctly
>
> **Activity Hints** (Multiple Valid Permission Structures):
>
> - **Hint A — Duty design**: Option A1 — three duties (Viewer, Processor, Admin) with escalating access (recommended — clear separation of concerns). Option A2 — two duties (ReadWrite, Admin) for simpler deployments (valid but less granular). Option A3 — four duties (Viewer, Processor, Approver, Admin) if approval workflows are separate from processing (most granular, useful for compliance-heavy environments).

> - **Hint B — Field-level permissions for Variance**: Option B1 — Deny Variance field for all roles except Admin (recommended — variance data is sensitive and should only be visible to those who can act on it). Option B2 — Allow Variance for Processor and Admin roles, deny for Viewer (valid if processors need to see variances to make decisions). Option B3 — No field-level permissions on Variance, rely on table-level See permission (not recommended — violates least privilege principle).
> - **Hint C — Record-level security**: Option C1 — use the standard `Company` SecurityKey so users only see their company's reconciliations (recommended — simple and effective). Option C2 — use a custom SecurityKey based on the `CreatedBy` field so users only see their own reconciliations (more restrictive, useful for sensitive reconciliations). Option C3 — no record-level security, rely on table-level permissions only (not recommended — users would see all companies' reconciliations).
> - **Hint D — Role assignment**: Option D1 — two roles (ReconciliationOperator and ReconciliationAdmin) with different duty combinations (recommended). Option D2 — three roles (ReconciliationViewer, ReconciliationProcessor, ReconciliationAdmin) for maximum separation of duties (best practice for SOX/compliance environments).

Expected Approach (Ideal — in detail)

Duty Design

Duty 1: `INVReconciliationViewer`

Privilege	Table	Access Level	Field Permissions
`INVReconciliationView`	`INVReconciliationHeader`	See	All fields
`INVReconciliationView`	`INVReconciliationLine`	See	All fields except `Variance`
`INVReconciliationView`	`INVReconciliationForm`	Display	Form access
`INVReconciliationView`	`INVReconciliationReport`	Output	Report execution

Duty 2: `INVReconciliationProcessor`

Privilege	Table	Access Level	Field Permissions
`INVReconciliationProcess`	`INVReconciliationHeader`	Create + Edit	All fields
`INVReconciliationProcess`	`INVReconciliationLine`	Create + Edit	All fields except `Variance`
`INVReconciliationProcess`	`INVReconciliationForm`	Create + Edit	Form access
`INVReconciliationProcess`	`INVReconciliationBatchJob`	Action	Job execution

Duty 3: `INVReconciliationAdmin`

Privilege	Table	Access Level	Field Permissions
`INVReconciliationAdmin`	`INVReconciliationHeader`	Admin	All fields
`INVReconciliationAdmin`	`INVReconciliationLine`	Admin	All fields including `Variance`
`INVReconciliationAdmin`	`INVReconciliationForm`	Admin	All form actions

Privilege	Table	Access Level	Field Permissions
`INVReconciliationAdmin`	`INVReconciliationBatchJob`	Admin	Job execution + cancellation
`INVReconciliationAdmin`	`INVReconciliationReport`	Output	Report execution
`INVReconciliationAdmin`	`INVReconciliationEntity`	Admin	Data entity execution

Security Role Design

Role: `InventoryReconciliationOperator` - Duties: `INVReconciliationViewer`, `INVReconciliationProcessor` - Users: Inventory clerks, reconciliation processors - Can view, create, and edit reconciliations but cannot see Variance field or approve reconciliations

Role: `InventoryReconciliationManager` - Duties: `INVReconciliationViewer`, `INVReconciliationProcessor`, `INVReconciliationAdmin` - Users: Inventory managers, compliance officers - Full access including Variance visibility, approval, and batch job management

Record-Level Security

- `INVReconciliationHeader` and `INVReconciliationLine` have `SecurityKey = Company`
- Users only see reconciliations for their current Legal Entity
- This is enforced automatically by the framework at query time

Field-Level Permissions

- `Variance` field on `INVReconciliationLine`: Denied for `INVReconciliationViewer` and `INVReconciliationProcessor` duties
- `Variance` field on `INVReconciliationLine`: Allowed for `INVReconciliationAdmin` duty
- This ensures that only managers who can approve variances can see them

Testing Plan

1. **Run as InventoryReconciliationOperator**:
2. [ ] Can see INVReconciliationForm ListPage with their company's reconciliations
3. [ ] Can create a new reconciliation
4. [ ] Can edit an existing reconciliation
5. [ ] CANNOT see the Variance field on reconciliation lines
6. [ ] CANNOT run the batch job
7. [ ] CANNOT execute the data entity
8. **Run as InventoryReconciliationManager**:
9. [ ] Can see INVReconciliationForm ListPage with their company's reconciliations
10. [ ] Can create and edit reconciliations
11. [ ] CAN see the Variance field on reconciliation lines
12. [ ] CAN run the batch job
13. [ ] CAN execute the data entity
14. [ ] Can approve reconciliations (if approval workflow is implemented)
15. **Run as user with no inventory reconciliation role**:

16. [ ] CANNOT see INVReconciliationForm
17. [ ] CANNOT access INVReconciliationHeader or INVReconciliationLine tables
18. [ ] CANNOT run the report

Model Manifest Dependencies

```
<ModelManifest>
  <Name>AcmeInventoryReconciliation</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
  <ConfigurationKey>AcmeInventoryReconciliationEnabled</ConfigurationKey>
</ModelManifest>
```

8.11 Activity — Security Audit for an Extension Module

> **Activity**: Perform a security audit for a custom extension module that adds compliance tracking to the vendor management process. The extension includes: 1. A table extension adding `ComplianceCode` to `VendTable` 2. A form extension adding a compliance tab to the vendor form 3. A CoC override on `VendTable.validateWrite()` that checks the compliance code 4. An event handler on `VendTable.inserted()` that logs compliance checks 5. A new report `ComplianceSummaryReport` 6. A new menu item `ComplianceHistory`
>
> Audit the extension for security issues and document your findings: - Does the CoC override change the security context? (No — it inherits the base method's permissions) - Does the event handler need new permissions? (No — it runs in the base method's context) - Does the table extension field need new field-level permissions? (Yes — if ComplianceCode is sensitive) - Does the form extension need a new menu item with permissions? (Yes — if it adds new navigation) - Does the report need output permissions? (Yes — always for new reports) - Are there any `assert()` calls that need auditing? (Check all extension code)
>
> **Activity Hints:** - **Hint A — CoC security**: CoC overrides never change the security context. They run with the same permissions as the base method. If the base method requires See permission on VendTable, the CoC override also requires See permission. You cannot use CoC to bypass security. - **Hint B — Event handler security**: Event handlers run in the context of the base method. They inherit the base method's permissions. If the base method's insert() requires Create permission on VendTable, the event handler also runs with Create permission. - **Hint C — Table extension field permissions**: Added fields inherit the base table's permissions by default. If you want to restrict access to a sensitive added field, you must explicitly set field-level permissions on the privilege. - **Hint D — Form extension menu items**: Any new menu item added by a form extension needs its own privilege. Without a privilege, users cannot navigate to the form. - **Hint E — Report permissions**: Every new report needs an Output Menu Item with a privilege that grants output access. Without this, users cannot run the report.

Remaining Chapters — Detailed Outlines

Chapter 9 — Data Entities & Integration

9.1 DataEntity vs Composite Entity vs Derived View — When Each Applies

D365 F&O provides three distinct patterns for exposing data for integration and reporting. Choosing the right pattern is critical for performance, maintainability, and upgradeability.

DataEntity — The Standard Integration Pattern

A `DataEntity` is the **primary and recommended** pattern for data integration in D365 F&O. It is a read-optimized view that joins multiple tables and exposes the data through the OData/REST service layer.

When to use a DataEntity:

Scenario	Use DataEntity?
Exposing data to Power Automate / Logic Apps	[x] Yes
Pushing data to an external ERP or SaaS system	[x] Yes
Consuming D365 F&O data from a custom .NET application	[x] Yes
Building a report that joins multiple standard tables	[x] Yes
Simple single-table read operation	[x] Yes (overkill but works)
Write-back to D365 F&O	[ ] No — use a Data Entity with a staging pattern instead

Key characteristics: - Read-only by default (for write-back, use the staging pattern) - Exposed via OData/REST automatically when published - Supports filtering, sorting, and pagination - Can include computed/display fields - Uses `IsPublic`, `Public Entity Name`, and `Public Collection Name` properties for OData exposure, with staging fields stored in a separate auto-generated staging table

Composite Entity — For Complex Multi-Source Integration

A `Composite Entity` is used when you need to combine data from **multiple Data Entities** into a single integration endpoint. It's the pattern for scenarios where a single DataEntity isn't sufficient because the data comes from different sources or requires complex joining logic.

When to use a Composite Entity:

Scenario	Use Composite Entity?
Combining data from two unrelated tables into one endpoint	[x] Yes
Building a master data export that spans multiple modules	[x] Yes
Simple single-table read	[ ] No — use a DataEntity
Read-only reporting on a single table	[ ] No — use a DataEntity

Derived View — For Complex SQL Logic

A `Derived View` is a low-level pattern that creates a view in the AOT with custom SQL logic. It's the most flexible but least integrated pattern — it doesn't automatically expose via OData and requires manual configuration for service layer exposure.

When to use a Derived View:

Scenario	Use Derived View?
Complex SQL that can't be expressed with DataEntity	[x] Yes
Need custom SQL functions or stored procedure calls	[x] Yes
Need to expose via OData/REST	[ ] No — use DataEntity (it supports OData natively)
Simple read operation	[ ] No — use DataEntity

Decision Matrix

Feature	DataEntity	Composite Entity	Derived View
OData/REST exposure	[x] Automatic	[x] Automatic	[ ] Manual
Read-only	[x] Yes	[x] Yes	[x] Yes
Write-back	[x] With staging pattern	[x] With staging pattern	[ ] Manual
Multi-table joins	[x] Via data sources	[x] Combines multiple entities	[x] Custom SQL
Filtering/Pagination	[x] Built-in	[x] Built-in	[ ] Manual
Complex SQL logic	[ ] Limited	[ ] Limited	[x] Full control
Upgrade safety	[x] High	[x] High	[!] Medium
Ease of use	[x] Easy	[!] Moderate	[ ] Complex

9.2 Entity Design — IsPublic, Public Entity Name, Public Collection Name, and Staging Tables

OData Exposure and Staging

Every DataEntity controls OData exposure and staging via the following mechanisms:

Mechanism	Purpose	OData Exposure	Example
IsPublic, Public Entity Name, Public Collection Name	Controls whether the entity is exposed via OData and its endpoint names	[x] Visible in OData when `IsPublic = Yes`	`VendAPComplianceEntity` with `Public Collection Name = VendAPComplianceEntities`
Staging Table	Auto-generated table (e.g., `EntityNameStaging`) for intermediate processing	[ ] Hidden from OData	`VendAPComplianceStaging` with fields like `StagingStatus`, `ProcessingFlag`, `InternalRef`
Link	Connects the entity to its context (parent record)	[x] Visible in OData	`VendTableRecId` linking to the vendor

Entity Design Pattern

```
DataEntity: VendAPComplianceEntity
+-- OData-exposed fields (IsPublic = Yes, Public Collection Name = VendAPComplianceEntities)
|   +-- AccountNum (string 20) — from VendTable
|   +-- Name (string 60) — from VendTable
|   +-- ComplianceCode (string 20) — from APCustomsDeclaration
|   +-- DeclDate (date) — from APCustomsDeclaration
|   +-- Status (string 20) — computed/display field
|
+-- Staging table (VendAPComplianceStaging) for intermediate processing
|   +-- VendTableRecId (int64) — for internal mapping
|   +-- StagingStatus (enum) — processing state
|   +-- ErrorMessage (string 250) — error tracking
|
+-- Link (connects entity to parent context)
    +-- VendTableRecId → VendTable.RecId
```

The `Link` Property

The `Link` property on a DataEntity field specifies the relationship between the entity and its underlying table. This is critical for:

1. **Context filtering** — the framework uses the link to filter data based on the calling context
2. **Write-back operations** — when pushing data back to D365 F&O, the link identifies which record to update
3. **OData `$expand`** — linked entities can be expanded in OData queries

```
// In the Data Entity design:
// Link field: VendTableRecId
// Linked to: VendTable.RecId
// This allows OData consumers to filter by vendor:
// GET /dataentity/VendAPComplianceEntity?$filter=VendTableRecId eq 12345
```

9.3 The Staging Pattern — Design, Process, and PostProcess

The **staging pattern** is the standard approach for data entities that need to write data back to D365 F&O. It uses a three-step process:

1. **Staging** — data is first loaded into a staging table
2. **Processing** — the staging data is validated and transformed
3. **Post-processing** — the validated data is written to the target table

Why Staging?

Problem	Staging Solves It
Invalid data can't be written directly	Validation happens in staging before target write
Partial failures need rollback	Staging allows all-or-nothing transaction control
Error logging is needed	Errors are captured in the staging table with context
Deduplication is needed	Staging table can check for duplicates before insert

Problem	Staging Solves It
Audit trail is needed	Staging records capture the import batch and timestamp

Staging Table Design

```
// Staging table: APCustomsDeclarationStaging
table 50100 APCustomsDeclarationStaging
{
    DataClassification = CustomerContent;
    Storage = TempDB;

    fields
    {
        field(1; RecId; int64) { }
        field(2; DeclId; Code[35]) { }
        field(3; VendTableRecId; int64) { }
        field(4; DeclDate; Date) { }
        field(5; CustomsRef; Code[50]) { }
        field(6; InvoiceLineRecId; int64) { }
        field(7; StagingStatus; Enum APCustomsStagingStatus) { }
        field(8; ErrorMessage; Text[250]) { }
        field(9; ImportBatchId; Guid) { }
        field(10; CreatedDateTime; DateTime) { }
    }

    keys
    {
        key(PK; RecId) { Clustered = true; }
        key(IX_DeclId; DeclId) { }
    }
}
```

The `process()` Method

The `process()` method is the core of the staging pattern. It reads records from the staging table, validates them, and writes them to the target table.

```
class APCustomsDeclarationEntityHandler
{
    /// <summary>
    /// Process staging records and write to target table.
    /// Called by the Data Entity framework during import.
    /// </summary>
    public void process()
    {
        APCustomsDeclarationStaging staging;
        APCustomsDeclaration target;

        // Set staging status to processing
        while select staging
            where staging.StagingStatus == APCustomsStagingStatus::Ready
        {
            try
            {
                ttsbegin;

                // Validate the staging record
                if (!this.validateStagingRecord(staging))
```

```
            {
                staging.StagingStatus = APCustomsStagingStatus::Failed;
                staging.ErrorMessage = 'Validation failed';
                staging.update();
                ttscommit;
                continue;
            }

            // Check for duplicates
            if (this.isDuplicate(staging))
            {
                staging.StagingStatus = APCustomsStagingStatus::Duplicate;
                staging.ErrorMessage = 'Duplicate declaration ID';
                staging.update();
                ttscommit;
                continue;
            }

            // Write to target table
            target.initValue();
            target.DeclId = staging.DeclId;
            target.VendTableRecId = staging.VendTableRecId;
            target.DeclDate = staging.DeclDate;
            target.CustomsRef = staging.CustomsRef;
            target.InvoiceLineRecId = staging.InvoiceLineRecId;
            target.insert();

            // Mark staging as processed
            staging.StagingStatus = APCustomsStagingStatus::Processed;
            staging.update();

            ttscommit;
        }
        catch (Exception::Error)
        {
            ttsabort;
            staging.StagingStatus = APCustomsStagingStatus::Failed;
            staging.ErrorMessage = 'Unexpected error during processing';
            staging.update();
        }
    }
}

private boolean validateStagingRecord(APCustomsDeclarationStaging _staging)
{
    if (_staging.DeclId == '')
    {
        return false;
    }
    if (_staging.VendTableRecId == 0)
    {
        return false;
    }
    return true;
}

private boolean isDuplicate(APCustomsDeclarationStaging _staging)
{
    APCustomsDeclaration existing;
    return exists select firstonly existing
        where existing.DeclId == _staging.DeclId;
```

```
    }
}
```

The `postProcess()` Method

The `postProcess()` method runs after all staging records have been processed. It's used for:

- Sending notifications (email, Infolog messages)
- Logging summary statistics
- Triggering downstream processes
- Cleaning up staging records

```
public void postProcess()
{
    int processedCount;
    int failedCount;

    processedCount = select count(StagingStatus)
        from APCustomsDeclarationStaging
        where StagingStatus == APCustomsStagingStatus::Processed;

    failedCount = select count(StagingStatus)
        from APCustomsDeclarationStaging
        where StagingStatus == APCustomsStagingStatus::Failed;

    info(strFmt('Processing complete: %1 succeeded, %2 failed.',
        processedCount, failedCount));

    // Optional: send email notification for failed records
    if (failedCount > 0)
    {
        // ... email notification logic ...
    }
}
```

9.4 Entity Mapping — Auto-Mapping vs. Manual Field Mapping

Auto-Mapping

Auto-mapping is the default behavior when a DataEntity field has the same name and compatible type as the underlying table field. The framework automatically maps the entity field to the table field.

When auto-mapping works: - Field names match exactly (case-sensitive) - Field types are compatible (string to string, int to int, date to date) - No transformation is needed

When auto-mapping fails: - Field names don't match (e.g., entity field `DeclId` maps to table field `DeclarationId`) - Field types are incompatible (e.g., entity field is `string` but table field is `int`) - Transformation is needed (e.g., concatenating first + last name into a full name)

Manual Field Mapping

When auto-mapping doesn't work, you need to define manual field mappings in the Data Entity design:

```
Data Entity Field: DeclId (string 35)
    → Maps to Table Field: DeclarationId (string 35)
    → Mapping Type: Manual
    → Transformation: None (names differ but types match)

Data Entity Field: FullName (string 120)
    → Maps to Table Fields: FirstName + ' ' + LastName
    → Mapping Type: Manual
    → Transformation: Concatenation expression
```

Mapping in the Data Entity Designer

In Visual Studio, the Data Entity designer provides a **Mappings** node where you can:

1. **Auto-map** — click "Auto Map" to let the framework attempt automatic matching
2. **Manual map** — drag and drop entity fields to table fields
3. **Transform** — define expressions for field transformations
4. **Skip** — exclude fields from the mapping (they exist in the entity but aren't mapped to a table field)

Best Practices for Entity Mapping

1. **Use consistent naming** — entity field names should match table field names whenever possible to enable auto-mapping
2. **Use staging tables for staging fields** — staging fields should never be exposed in OData; store them in the auto-generated staging table
3. **Document manual mappings** — add comments explaining why auto-mapping wasn't possible
4. **Test mappings with sample data** — always verify that data flows correctly from entity to table

---

9.5 Data Management Workspace — Import/Export and Package Deployment

The **Data Management** workspace is the central hub for importing and exporting data in D365 F&O. It provides a UI-driven interface for data operations without requiring code.

Import Operations

Operation	Description	Use Case
Import	Load data from an Excel/CSV file into D365 F&O tables	Bulk data loading, initial setup
Import from Entity	Load data using a Data Entity definition	Integration scenarios, staging pattern
Import Group	Execute a group of import projects in sequence	Complex multi-table imports

Export Operations

Operation	Description	Use Case
Export	Export data from D365 F&O tables to Excel/CSV	Reporting, data extraction

Operation	Description	Use Case
Export to Entity	Export data using a Data Entity definition	Integration with external systems
Export to Data Entity	Export using the staging pattern for write-back scenarios	Data synchronization

Package Deployment

A **package** is a deployable unit that contains one or more Data Entities, mapping configurations, and import/export settings. Packages can be:

1. **Created** in the Data Management workspace
2. **Saved** as a `.axdp` file (Data Management package)
3. **Shared** across environments via LCS
4. **Deployed** to target environments via the package deployment feature

Package Deployment Steps

```
[1] Create a package in the source environment
    |  - Define the Data Entities to include
    |  - Configure mapping and transformation rules
    |  - Set up import/export settings
    |
[2] Export the package as an .axdp file
    |
[3] Upload the package to LCS
    |  - LCS stores the package as a build artifact
    |
[4] Deploy the package to the target environment
    |  - LCS copies the package to the target environment's model store
    |  - The Data Management workspace on the target environment detects the package
    |
[5] Execute the package in the target environment
    |  - Run the import/export operation
    |  - Monitor progress and review results
    |
[6] Verify the data in the target environment
    |  - Check the Data Management workspace for execution status
    |  - Review any error logs
```

9.6 REST/OData — DictClass Reflection, Entity Publishing, and HttpClient

Exposing Data via OData

D365 F&O automatically exposes Data Entities as OData endpoints when they are published. The OData endpoint follows the standard OData protocol and can be consumed by any OData client.

OData endpoint URL format:

```
https://<your-environment>.dynamics.com/data/<EntityName>
```

Example:

```
https://usnconeboxax1aos.cloud.onebox.dynamics.com/data/VendAPComplianceEntity
```

`DictClass` — Runtime Entity Metadata

The `DictClass` and `DictMethod` classes provide runtime reflection over X++ objects, including Data Entities:

- Field definitions (name, type, size)
- Method signatures and parameters
- Class attributes
- Supported operations

```
// Get metadata for a Data Entity class at runtime
DictClass dictClass = new DictClass(classNum(VendAPComplianceEntity));
Array    attributes  = dictClass.getAllAttributes();

// Inspect a specific method's .NET return type via reflection.
// Source: https://learn.microsoft.com/en-us/previous-versions/dynamicsax-2012/developer/reflect-on-net-elements-of-x-methods
DictMethod dictMethod = new DictMethod(
    classNum(VendAPComplianceEntity),
    methodStr(VendAPComplianceEntity, process));
info(strFmt("Method %1 returns %2",
    methodStr(VendAPComplianceEntity, process),
    dictMethod.clrReturnType()));
```

Publishing an Entity via `IsPublic`

A Data Entity is published for OData exposure by setting the **`IsPublic`** property to **Yes** in the entity designer. No code registration is needed — the framework automatically generates the OData endpoint and the staging table when the entity is built and deployed.

Consuming OData Endpoints via `HttpClient`

D365 F&O provides the `HttpClient` class (from `System.Net.Http`) for consuming external OData endpoints from X++ code.

```
class ExternalDataConsumer
{
    public static void consumeODataEndpoint()
    {
        HttpClient httpClient;
        HttpRequestMessage request;
        HttpResponseMessage response;
        string responseBody;

        // Create the HTTP client
        httpClient = new HttpClient();

        // Create the request
        request = new HttpRequestMessage();
        request.set_Method(HttpMethod::Get);
        request.set_RequestUri(
            new Uri('https://external-system.com/api/v1/vendors'));

        // Add authentication header (Bearer token)
        request.get_Headers().Add(
```

```
            'Authorization',
            'Bearer ' + ExternalConfig::getAccessToken());

        // Send the request
        response = httpClient.SendAsync(request).GetAwaiter().GetResult();

        // Read the response
        if (response.get_IsSuccessStatusCode())
        {
            responseBody = response.Content.ReadAsStringAsync().GetAwaiter().GetResult();
            // Process the JSON response
            info(strFmt('Received %1 bytes from external system',
                responseBody.Length()));
        }
        else
        {
            error(strFmt('HTTP request failed with status code: %1',
                response.get_StatusCode()));
        }
    }
}
```

OData Query Options

D365 F&O OData endpoints support standard OData query options:

Option	Example	Description
`$filter`	`$filter=CreditMax gt 500000`	Filter records by condition
`$select`	`$select=AccountNum,Name`	Select specific fields only
`$orderby`	`$orderby=AccountNum asc`	Sort results
`$top`	`$top=100`	Limit to first N records
`$skip`	`$skip=50`	Skip first N records (pagination)
`$expand`	`$expand=VendInvoiceJour`	Include related entities
`$count`	`$count=true`	Include total record count

Consuming OData from Power Automate

Power Automate has built-in connectors for D365 F&O OData endpoints:

1. Create a new Power Automate flow
2. Add a **"When a HTTP request is received"** trigger (for inbound)
3. Add a **"HTTP"** action to call the D365 F&O OData endpoint
4. Parse the JSON response
5. Transform and load data into the target system

9.7 Power Platform Integration — Logic Apps and Power Automate

Logic Apps Connector for D365 F&O

Microsoft provides a **Dynamics 365 Finance & Operations** connector in Azure Logic Apps that simplifies integration without writing code.

Common Logic Apps operations:

Operation	Description
List records	Query a Data Entity and return matching records
Get record	Retrieve a single record by key
Create record	Insert a new record into a Data Entity (with staging)
Update record	Modify an existing record
Delete record	Remove a record
Execute action	Call a custom action defined on the entity

Power Automate Triggers

Power Automate can be triggered from D365 F&O in several ways:

Trigger	How It Works
Data Entity import	A Data Entity import triggers a flow that processes the imported data
Custom action	A custom action on a Data Entity triggers a flow
Event-based	An event handler in X++ sends a message to Azure Service Bus, which triggers a flow
Scheduled	A Logic App or Power Automate flow runs on a schedule and calls D365 F&O OData

Integration Pattern: D365 F&O → Logic App → External System

```
[1] Data Entity import is executed in D365 F&O
    |
[2] Staging records are processed and written to target tables
    |
[3] postProcess() triggers an Azure Service Bus message
    |  Message contains: batch ID, record count, status
    |
[4] Logic App is triggered by the Service Bus message
    |
[5] Logic App calls external API (e.g., SAP, Salesforce, custom REST service)
    |
[6] External system receives the data and processes it
    |
[7] Logic App logs the result back to D365 F&O (optional)
```

Integration Pattern: External System → Logic App → D365 F&O

```
[1] External system sends data to Logic App (HTTP trigger)
    |
[2] Logic App transforms the data to match the Data Entity structure
    |
```

```
[3] Logic App calls D365 F&O OData endpoint to create/update records
    |
[4] D365 F&O processes the request through the staging pattern
    |
[5] D365 F&O returns the result (success/failure)
    |
[6] Logic App logs the result and sends a notification
```

Best Practices for Power Platform Integration

1. **Use Data Entities as the integration layer** — don't expose tables directly via OData
2. **Use the staging pattern for write-back** — ensures validation and error handling
3. **Implement error handling in Logic Apps** — use try/catch patterns and dead-letter queues
4. **Use batch processing for large data volumes** — don't send thousands of records in a single request
5. **Secure connections with Managed Identity** — don't store credentials in Logic Apps
6. **Monitor integration runs** — use Azure Monitor and Logic Apps run history to track failures

---

9.8 Activity — Build a Data Entity for HR Employee Data Integration

> **Activity**: Build a complete data entity for HR employee data integration with the following requirements: 1. Create a Data Entity called `HRPersonnelEntity` that exposes employee data from `DirPerson`, `HCMWorker`, and `HRMPosition` tables 2. Configure the entity's `IsPublic` property to **Yes**, set `Public Entity Name` to `HRPersonnelEntity`, and set `Public Collection Name` to `HRPersonnelEntities` — exposed fields: PersonNumber, FirstName, LastName, Email, Position, Department, HireDate, Status 3. Design a staging table `HRPersonnelStaging` with internal fields: PersonRecId, WorkerRecId, PositionRecId, StagingStatus, ErrorMessage 4. The entity must have a `Link` field connecting to `DirPerson.RecId` 5. Implement a staging table `HRPersonnelStaging` with fields for all public entity fields plus StagingStatus and ErrorMessage 6. Implement `process()` method with validation (email format check, duplicate detection by PersonNumber) and error logging 7. Implement `postProcess()` method with summary statistics and email notification for failed records 8. Configure auto-mapping for all fields that share names between entity and table 9. Use manual mapping for the `FullName` field (concatenation of FirstName + ' ' + LastName) 10. Expose the entity via OData and test with a sample Power Automate flow 11. Implement deduplication logic that skips records already in the target table 12. Implement rollback logic that aborts the entire batch if more than 10% of records fail validation
>
> **Activity Hints** (Multiple Valid Approaches):
>
> - **Hint A — Entity type selection**: Option A1 — use a `DataEntity` with staging pattern (recommended — full validation, error logging, rollback support). Option A2 — use a `Derived View` for read-only exposure (simpler but no write-back support). Option A3 — use a `Composite Entity` if the data spans multiple unrelated entities (overkill for this scenario).
> - **Hint B — Staging table design**: Option B1 — use `TempDB` storage for the staging table (recommended — temporary data, no persistence needed). Option B2 — use a permanent table with `ImportStatus` field (valid if you need to keep import history for audit). Option C — use the `DataManagementStaging` framework table (not recommended — too generic, hard to customize).
> - **Hint C — Deduplication logic**: Option C1 — check for existing records by `PersonNumber` before insert using `exists select` (recommended — simple and effective). Option C2 — use a

> `NumberSeq` framework approach (overkill for deduplication). Option C3 — let the database handle duplicates via unique index and catch the error (valid but less informative — you lose the context of which record failed).
>
> - **Hint D — Rollback threshold**: Option D1 — count failures during processing and abort if >10% (recommended — configurable threshold). Option D2 — abort on first failure (too strict — one bad record shouldn't fail the entire batch). Option D3 — don't implement rollback, just log errors and continue (valid for non-critical integrations where partial success is acceptable).
> - **Hint E — Email notification**: Option E1 — use `SmtpMail::send()` from X++ (direct but requires SMTP configuration). Option E2 — trigger a Logic App from the `postProcess()` method via Azure Service Bus (recommended — decoupled, configurable). Option C3 — use `Workflow::submitRequest()` to create a workflow notification (valid but requires workflow configuration).

Expected Approach (Ideal — in detail)

Entity Design: HRPersonnelEntity

```
// Data Entity: HRPersonnelEntity
// Public entity fields (IsPublic = Yes):
//   PersonNumber (string 20) — from DirPerson.AccountNum
//   FirstName (string 50) — from DirPerson.FirstName
//   LastName (string 50) — from DirPerson.LastName
//   FullName (string 120) — computed: FirstName + ' ' + LastName
//   Email (string 250) — from DirPerson.Email
//   Position (string 50) — from HRMPosition.Position
//   Department (string 50) — from HRMPosition.Department
//   HireDate (date) — from HCMWorker.HireDate
//   Status (string 20) — computed from HCMWorker.Status

// Staging table fields (HRPersonnelStaging):
//   PersonRecId (int64) — from DirPerson.RecId
//   WorkerRecId (int64) — from HCMWorker.RecId
//   PositionRecId (int64) — from HRMPosition.RecId
//   StagingStatus (enum) — processing state
//   ErrorMessage (string 250) — error tracking

// Link field:
//   PersonRecId → DirPerson.RecId
```

Staging Table: HRPersonnelStaging

```
table 50101 HRPersonnelStaging
{
    DataClassification = CustomerContent;
    Storage = TempDB;

    fields
    {
        field(1; RecId; int64) { }
        field(2; PersonNumber; Code[20]) { }
        field(3; FirstName; Text[50]) { }
        field(4; LastName; Text[50]) { }
        field(5; Email; Text[250]) { }
```

```
        field(6; Position; Text[50]) { }
        field(7; Department; Text[50]) { }
        field(8; HireDate; Date) { }
        field(9; Status; Text[20]) { }
        field(10; StagingStatus; Enum HRPersonnelStagingStatus) { }
        field(11; ErrorMessage; Text[250]) { }
        field(12; ImportBatchId; Guid) { }
        field(13; CreatedDateTime; DateTime) { }
    }

    keys
    {
        key(PK; RecId) { Clustered = true; }
        key(IX_PersonNumber; PersonNumber) { }
    }
}
```

process() Method with Validation, Deduplication, and Rollback

```
class HRPersonnelEntityHandler
{
    public void process()
    {
        HRPersonnelStaging staging;
        DirPerson targetPerson;
        HCMWorker targetWorker;
        int totalProcessed = 0;
        int totalFailed = 0;
        int totalSuccess = 0;
        Guid batchId = Guid::newGuid();

        // Count total records to process
        totalProcessed = select count(RecId) from staging
            where staging.StagingStatus == HRPersonnelStagingStatus::Ready;

        while select staging
            where staging.StagingStatus == HRPersonnelStagingStatus::Ready
        {
            try
            {
                ttsbegin;

                // Validate email format
                if (!this.validateEmail(staging.Email))
                {
                    this.markFailed(staging, 'Invalid email format');
                    totalFailed++;
                    ttscommit;
                    continue;
                }

                // Check for duplicates by PersonNumber
                if (this.isDuplicate(staging.PersonNumber))
                {
                    this.markDuplicate(staging);
                    totalFailed++;
                    ttscommit;
                    continue;
                }

                // Check rollback threshold (>10% failures)
```

```
            if (totalFailed > (totalProcessed * 0.1))
            {
                this.markBatchFailed(staging, 'Rollback threshold exceeded: >10% failures');
                totalFailed++;
                ttscommit;
                break;  // Stop processing
            }

            // Write to DirPerson
            targetPerson.initValue();
            targetPerson.AccountNum = staging.PersonNumber;
            targetPerson.FirstName = staging.FirstName;
            targetPerson.LastName = staging.LastName;
            targetPerson.Email = staging.Email;
            targetPerson.insert();

            // Write to HCMWorker
            targetWorker.initValue();
            targetWorker.PersonRecId = targetPerson.RecId;
            targetWorker.HireDate = staging.HireDate;
            targetWorker.Status = staging.Status;
            targetWorker.insert();

            // Mark staging as processed
            this.markProcessed(staging);
            totalSuccess++;

            ttscommit;
        }
        catch (Exception::Error)
        {
            ttsabort;
            this.markFailed(staging, 'Unexpected error during processing');
            totalFailed++;
        }
    }

    // Store summary for postProcess
    this.setBatchSummary(batchId, totalSuccess, totalFailed);
}

private boolean validateEmail(str _email)
{
    // Simple email format validation
    return strScan(_email, '@', 1, strlen(_email)) > 0
        && strScan(_email, '.', strScan(_email, '@', 1, strlen(_email)), strlen(_email)) > 0;
}

private boolean isDuplicate(str _personNumber)
{
    DirPerson existing;
    return exists select firstonly existing
        where existing.AccountNum == _personNumber;
}

private void markFailed(HRPersonnelStaging _staging, str _error)
{
    _staging.StagingStatus = HRPersonnelStagingStatus::Failed;
    _staging.ErrorMessage = _error;
    _staging.update();
}
```

```
    private void markDuplicate(HRPersonnelStaging _staging)
    {
        _staging.StagingStatus = HRPersonnelStagingStatus::Duplicate;
        _staging.ErrorMessage = 'Duplicate PersonNumber';
        _staging.update();
    }

    private void markProcessed(HRPersonnelStaging _staging)
    {
        _staging.StagingStatus = HRPersonnelStagingStatus::Processed;
        _staging.update();
    }

    private void markBatchFailed(HRPersonnelStaging _staging, str _error)
    {
        _staging.StagingStatus = HRPersonnelStagingStatus::BatchFailed;
        _staging.ErrorMessage = _error;
        _staging.update();
    }

    private void setBatchSummary(Guid _batchId, int _success, int _failed)
    {
        // Store summary for postProcess to use
        // This could be in a configuration table or a static variable
    }
}
```

postProcess() Method with Summary and Notification

```
public void postProcess()
{
    int successCount = this.getSuccessCount();
    int failedCount = this.getFailedCount();
    int totalCount = successCount + failedCount;

    info(strFmt('HR Personnel import complete: %1 succeeded, %2 failed, %3 total.',
        successCount, failedCount, totalCount));

    // Send notification for failed records
    if (failedCount > 0)
    {
        // Option 1: Direct email (requires SMTP configuration)
        // SmtpMail::send(...);

        // Option 2: Trigger Logic App via Service Bus (recommended)
        this.triggerIntegrationNotification(successCount, failedCount);
    }
}
```

Model Manifest Dependencies

```
<ModelManifest>
  <Name>AcmeHRIntegration</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
```

```
  <ConfigurationKey>AcmeHRIntegrationEnabled</ConfigurationKey>
</ModelManifest>
```

Remaining Chapters — Detailed Outlines

Chapter 10 — Reporting (SSRS, Legacy AX Reports, Analytical)

10.1 SSRS Report Design in Visual Studio

SQL Server Reporting Services (SSRS) is the primary reporting engine in D365 F&O. Reports are designed in Visual Studio using the **SSRS Report Designer** and deployed as part of the model project.

Report Project Structure

When you add an SSRS report to a D365 F&O project, Visual Studio creates the following structure:

```
MyModelProject
+-- Reports/
|   +-- ComplianceSummaryReport.rdl
+-- Classes/
|   +-- VendComplianceReportDP.cs  (Data Provider class)
+-- Data Sources/
|   +-- VendComplianceDataSource  (shared data source)
+-- ModelManifest.xml
```

Dataset Creation

The dataset is the bridge between the report layout and the data source. In D365 F&O, datasets are created using the **Microsoft.Dynamics.AX.Framework.Reporting** namespace.

Steps to create a dataset: 1. Open the `.rdl` file in Visual Studio's Report Designer 2. Right-click **Datasets** in the Report Data pane → **Add Dataset** 3. Select **Use a dataset embedded in my report** 4. Choose the data source (typically a `SrsReportDataProvider` class) 5. Write the query or use the query builder 6. Add fields from the dataset to the report layout

Data Source Configuration

D365 F&O reports use a special data source that connects to the AOS via the reporting framework:

Property	Value	Description
Data Source Type	`Microsoft.Dynamics.AX.Framework.Reporting`	D365 F&O reporting framework
Connection	Uses the connected LCS environment	No manual connection string needed
Authentication	Uses the current user's Azure AD credentials	Seamless SSO

10.2 `SrsReportDataProvider` — The Bridge

`SrsReportDataProvider` is the bridge between the SSRS report layout and the D365 F&O data. It is the most critical class in the reporting architecture.

The `SrsReportDataProvider` Pattern

Every SSRS report in D365 F&O requires a data provider class that extends `SrsReportDataProvider`. This class:

1. Defines the query that retrieves data from the database
2. Provides parameters to the report
3. Handles report-level logic (grouping, calculations)
4. Bridges the gap between the AOS and the SSRS renderer

Key Methods

Method	Purpose	Required?
`parmQuery()`	Gets/sets the query object that defines the data source	Yes
`parmPrintJobSettings()`	Gets/sets the print job settings (routing, format)	Yes
`processReport()`	Called by the reporting framework at runtime; populates a temp table member with the report data	Yes
`getVendorComplianceReportTmp()` (tagged with `[SRSReportDataSetAttribute]`)	Returns the populated temp table; the report dataset binds to this method	Yes
`parmReportName()`	Gets/sets the report name	Yes
`parmParameters()`	Gets/sets the report parameters	Optional

Complete `SrsReportDataProvider` Example

```
[SRSReportQueryAttribute('VendComplianceReport'),
SRSReportParameterAttribute(classstr(VendComplianceReportContract))]
class VendComplianceReportDP extends SrsReportDataProvider
{
    VendComplianceReportTmp tmpTable;
    VendTable vendTable;
    FromDate fromDate;
    ToDate toDate;
    ComplianceStatus complianceStatus;

    /// <summary>
    /// Called by the SSRS framework when the report is rendered.
    /// Populates the temp table with report data.
    /// </summary>
    public void processReport()
    {
        VendComplianceReportContract contract;

        // Get the contract (parameters) from the controller
        contract = this.parmDataContract() as VendComplianceReportContract;

        // Build the query to retrieve compliance data
        this.buildQuery(contract);

        // Execute the query and populate the temp table
        this.fetchData(contract, tmpTable);
```

```
}

/// <summary>
/// Returns the populated temp table to the report dataset.
/// The SRSReportDataSetAttribute binds this method to the report.
/// </summary>
[SRSReportDataSetAttribute(tableStr(VendComplianceReportTmp))]
public VendComplianceReportTmp getVendorComplianceReportTmp()
{
    return tmpTable;
}

private void buildQuery(VendComplianceReportContract _contract)
{
    Query query = new Query();
    QueryBuildDataSource qbdVend;
    QueryBuildDataSource qbdDeclaration;
    QueryBuildRange qbr;

    // Primary data source: VendTable
    qbdVend = query.addDataSource(tableNum(VendTable));
    qbdVend.relations(true);  // Include related tables

    // Add range for vendor group
    if (_contract.parmVendGroup())
    {
        qbr = qbdVend.addRange(fieldNum(VendTable, CustGroup));
        qbr.value(queryValue(_contract.parmVendGroup()));
    }

    // Join to APCustomsDeclaration
    qbdDeclaration = qbdVend.addDataSource(tableNum(APCustomsDeclaration));
    qbdDeclaration.joinMode(JoinMode::InnerJoin);
    qbdDeclaration.relations(true);

    // Add date range filter
    qbr = qbdDeclaration.addRange(fieldNum(APCustomsDeclaration, DeclDate));
    qbr.value(strfmt('"%1" .. "%2"', _contract.parmFromDate(), _contract.parmToDate()));

    // Add compliance status filter
    if (_contract.parmComplianceStatus())
    {
        qbr = qbdDeclaration.addRange(fieldNum(APCustomsDeclaration, Status));
        qbr.value(queryValue(_contract.parmComplianceStatus()));
    }

    // Set the query on the data provider
    this.parmQuery(query);
}

private void fetchData(VendComplianceReportContract _contract, VendComplianceReportTmp _tmp)
{
    VendTable vendTableLocal;
    APCustomsDeclaration declaration;

    while select vendTableLocal
        join declaration
        where declaration.VendTableRecId == vendTableLocal.RecId
        && declaration.DeclDate >= _contract.parmFromDate()
        && declaration.DeclDate <= _contract.parmToDate()
    {
        _tmp.clear();
```

```
            _tmp.AccountNum = vendTableLocal.AccountNum;
            _tmp.VendorName = vendTableLocal.Name;
            _tmp.DeclId = declaration.DeclId;
            _tmp.DeclDate = declaration.DeclDate;
            _tmp.CustomsRef = declaration.CustomsRef;
            _tmp.Status = declaration.Status;
            _tmp.insert();
        }
    }

    // Parameter getters/setters
    public VendTable parmVendTable(VendTable _vendTable = vendTable)
    {
        vendTable = _vendTable;
        return vendTable;
    }

    public FromDate parmFromDate(FromDate _fromDate = fromDate)
    {
        fromDate = _fromDate;
        return fromDate;
    }

    public ToDate parmToDate(ToDate _toDate = toDate)
    {
        toDate = _toDate;
        return toDate;
    }
}
```

The Contract Class

The contract class defines the report parameters and is passed between the controller and the data provider:

```
[DataContractAttribute]
class VendComplianceReportContract
{
    VendGroup vendGroup;
    FromDate fromDate;
    ToDate toDate;
    ComplianceStatus complianceStatus;

    [DataMemberAttribute('VendGroup'), SysOperationLabelAttribute('Vendor Group')]
    public VendGroup parmVendGroup(VendGroup _vendGroup = vendGroup)
    {
        vendGroup = _vendGroup;
        return vendGroup;
    }

    [DataMemberAttribute('FromDate'), SysOperationLabelAttribute('From Date')]
    public FromDate parmFromDate(FromDate _fromDate = fromDate)
    {
        fromDate = _fromDate;
        return fromDate;
    }

    [DataMemberAttribute('ToDate'), SysOperationLabelAttribute('To Date')]
    public ToDate parmToDate(ToDate _toDate = toDate)
    {
        toDate = _toDate;
        return toDate;
```

```
    }

    [DataMemberAttribute('ComplianceStatus'), SysOperationLabelAttribute('Compliance Status')]
    public ComplianceStatus parmComplianceStatus(ComplianceStatus _status = complianceStatus)
    {
        complianceStatus = _status;
        return complianceStatus;
    }
}
```

10.3 `PrintJobSettings` — Routing Reports

`PrintJobSettings` controls how reports are routed to their destination — printer, PDF, email, or preview.

PrintJobSettings Properties

Property	Description	Common Values
`PrinterName`	The target printer	`'\\PrintServer\Printer1'` or empty for PDF
`PrintMedium`	Output medium	`PrintMedium::Printer`, `PrintMedium::File`, `PrintMedium::Email`
`FileName`	Output file path (when `PrintMedium = File`)	`'C:\Reports\Compliance.pdf'`
`EmailTo`	Recipient email address	`'compliance@acme.com'`
`EmailSubject`	Email subject line	`'Monthly Compliance Report'`
`EmailBody`	Email body text	`'Please find the compliance report attached.'`
`FileFormat`	Output format	`FileFormat::PDF`, `FileFormat::Excel`, `FileFormat::Word`
`NumberOfCopies`	Number of copies to print	`1`
`Sides`	Single or double-sided	`Sides::OneSided`, `Sides::TwoSidedLongEdge`

Setting PrintJobSettings Programmatically

```
class VendComplianceReportController extends SysOperationServiceController
{
    public static void main(Args _args)
    {
        VendComplianceReportController controller = new VendComplianceReportController();
        VendComplianceReportContract contract = new VendComplianceReportContract();

        // Populate contract from args
        if (_args && _args.record())
        {
            contract.parmVendTable(_args.record());
        }

        controller.parmContractType(classStr(VendComplianceReportContract));
```

```
        controller.parmServiceType(classStr(VendComplianceService));
        controller.parmCaption('Vend Compliance Report');

        // Configure print job settings
        PrintJobSettings printJobSettings = controller.parmPrintJobSettings();
        printJobSettings.parmPrintMedium(PrintMedium::File);
        printJobSettings.parmFileFormat(FileFormat::PDF);
        printJobSettings.parmFileName(@"C:\Reports\VendCompliance_" + date2str(today(), 123, 2, 3, 2, 3,
4));

        controller.startOperation();
    }
}
```

PrintJobSettings in the Report Dialog

When a user runs a report from the D365 F&O client, the `PrintJobSettings` dialog appears:

1. **Printer selection** — choose a printer or "Print to PDF"
2. **Format** — PDF, Excel, Word, or HTML
3. **Range** — print all pages, current page, or a page range
4. **Copies** — number of copies
5. **Email** — enter recipient email to send the report as an attachment

---

10.4 `AutoDesign` vs. Manual RDL Layout

AutoDesign

`AutoDesign` is the quickest way to generate a report layout. Visual Studio automatically creates a tabular layout based on the dataset fields.

Pros: - Fastest way to get a functional report - Automatically generates columns for all dataset fields - Good for initial prototyping and internal reports

Cons: - Limited customization — the layout is generic - No control over grouping, formatting, or visual design - Not suitable for production reports that require pixel-perfect layouts

Manual RDL Layout

Manual RDL design gives full control over the report layout. You design the report in Report Builder or SQL Server Data Tools (SSDT), then import the `.rdl` file into the D365 F&O project.

Pros: - Full control over layout, formatting, fonts, colors - Support for grouped sections, subtotals, charts, and matrices - Professional, pixel-perfect output - Can include conditional formatting, drill-downs, and interactive features

Cons: - More effort to design and wire up - Requires knowledge of RDL/XML syntax - Changes to the dataset require manual updates to the layout

Importing a Manual RDL File

1. Design the report in **Report Builder** or **SSDT**
2. Save as `.rdl` file

3. In Visual Studio, right-click the **Reports** node in the project
4. Select **Add** → **Existing Item** → choose the `.rdl` file
5. Map the dataset fields in the RDL to the data provider's fields
6. Deploy the model to test

When to Use Which

Scenario	Use
Initial prototype or internal report	AutoDesign
Production report requiring professional layout	Manual RDL
Report with grouped sections and subtotals	Manual RDL
Report with charts or matrices	Manual RDL
Quick one-off data extraction	AutoDesign
Report that will be maintained long-term	Manual RDL

10.5 `ReportRun` Class — Programmatic Report Execution

The `ReportRun` class allows you to instantiate and run reports programmatically from X++ code.

Basic Report Execution

```
class VendComplianceReportRunner
{
    public static void runReport()
    {
        ReportRun reportRun;
        VendComplianceReportContract contract;

        // Create the contract with parameters
        contract = new VendComplianceReportContract();
        contract.parmFromDate(01\01\2026);
        contract.parmToDate(31\12\2026);
        contract.parmVendGroup('VENDGROUP1');

        // Instantiate the report
        reportRun = new ReportRun(ReportStr(VendComplianceSummaryReport));

        // Set the contract as the report's data source
        reportRun.parmReportContract(contract);

        // Run the report
        reportRun.runReport();
    }
}
```

Programmatic PrintJobSettings Configuration

```
// Configure print settings before running
PrintJobSettings printJobSettings = reportRun.parmPrintJobSettings();
printJobSettings.parmPrintMedium(PrintMedium::Email);
```

```
printJobSettings.parmEmailTo('compliance@acme.com');
printJobSettings.parmEmailSubject('Monthly Compliance Report');
printJobSettings.parmFileFormat(FileFormat::PDF);
```

Running Reports from Batch Jobs

```
// From a RunBaseBatch class
public void run()
{
    ReportRun reportRun;
    VendComplianceReportContract contract;

    contract = new VendComplianceReportContract();
    contract.parmFromDate(fromDate);
    contract.parmToDate(toDate);
    contract.parmVendGroup(vendGroup);

    reportRun = new ReportRun(ReportStr(VendComplianceSummaryReport));
    reportRun.parmReportContract(contract);

    // Configure for file output (batch jobs typically output to file)
    PrintJobSettings printJobSettings = reportRun.parmPrintJobSettings();
    printJobSettings.parmPrintMedium(PrintMedium::File);
    printJobSettings.parmFileFormat(FileFormat::PDF);
    printJobSettings.parmFileName(@"C:\Reports\Compliance_" + date2str(today(), 123, 2, 3, 2, 3, 4) +
".pdf");

    reportRun.runReport();

    info(strFmt('Compliance report generated: %1', printJobSettings.parmFileName()));
}
```

10.6 Legacy AX Reports (Pre-SSRS)

Legacy AX Reports (from AX 2012 or earlier) used `.rpt` files and are sometimes still encountered in older codebases and on-premises deployments. While SSRS is the recommended approach for new development, these legacy reports may still exist in some environments.

Legacy AX Reports vs. SSRS — Key Differences

Feature	Legacy AX Reports (`.rpt`)	SSRS (Modern)
Report designer	Report Builder in AX client	Visual Studio SSRS Designer
File format	`.rpt`	`.rdl`
Data access	Direct SQL queries	`SrsReportDataProvider` with AOT queries
Deployment	Manual upload to AOT	Model project deployment via VS
Security	AOS-level security	Azure AD + D365 F&O security roles
Performance	No pre-processing optimization	TempDB pre-processing support
OData exposure	Not supported	Automatic via Data Entities

Feature	Legacy AX Reports (`.rpt`)	SSRS (Modern)
Maintenance	Harder to maintain	Model-based, version-controlled

When Legacy AX Reports Are Still Relevant

- **On-premises deployments** that haven't been upgraded to the modern SSRS pattern
- **Existing `.rpt` reports** that are still in production and haven't been migrated
- **Custom RDL files** that were designed outside of Visual Studio and need to be integrated

Migrating from Legacy AX Reports to SSRS

1. **Identify the legacy report** — locate the `.rpt` file in the AOT under `Reports`
2. **Create a new SSRS report** in Visual Studio with the same data source
3. **Recreate the layout** in the SSRS Report Designer — import the old RDL as a starting point
4. **Create a `SrsReportDataProvider`** class to replace the legacy data provider
5. **Test the new report** against the same data set
6. **Update menu items** to point to the new report controller
7. **Deprecate the old report** — mark it as obsolete in the AOT

Key takeaway: If you encounter legacy `.rpt` reports in an existing codebase, plan to migrate them to SSRS. The modern SSRS pattern with `SrsReportDataProvider` and AOT Query objects is more maintainable, upgrade-safe, and integrates with the D365 F&O security model.

10.7 Pre-Processing Optimization for Long-Running Reports

Long-running SSRS reports (>10 minutes) can timeout in D365 F&O. The solution is to use **pre-processing** with TempDB tables instead of InMemory tables.

The Problem

By default, `SrsReportDataProvider` uses `InMemory` temporary tables for report data. For large datasets, this causes: - Report timeouts (>10 minutes) - High memory consumption on the AOS - Poor user experience with slow report loading

The Solution: TempDB Pre-Processing

Switch from `InMemory` to `TempDB` table type and use `SrsReportDataProviderPreProcessTempDB`:

```
// [ ] SLOW – InMemory table for large datasets
class VendComplianceReportDP extends SrsReportDataProvider
{
    VendComplianceReportTmp tmpTable;  // InMemory – slow for large data

    public void processReport()
    {
        // Data is held in memory – causes timeout for large datasets
    }
}

// [x] FAST – TempDB table with pre-processing
class VendComplianceReportDP extends SrsReportDataProviderPreProcessTempDB
{
```

```
    VendComplianceReportTmp tmpTable;  // TempDB — fast for large data

    public void processReport()
    {
        // Pre-process: retrieve and store data in TempDB before report renders
    }

    [SRSReportDataSetAttribute(tableStr(VendComplianceReportTmp))]
    public VendComplianceReportTmp getReportDataTmp()
    {
        // Data is pre-processed into TempDB before report rendering
        // The report reads from TempDB — no timeout issues
        return tmpTable;
    }
}
```

Pre-Processing Pattern

The pre-processing pattern separates data retrieval (which can be slow) from report rendering (which must be fast):

```
class VendComplianceReportDP extends SrsReportDataProviderPreProcessTempDB
{
    VendComplianceReportTmp tmpTable;  // Class member — persists in TempDB across calls
    VendTable vendTable;
    FromDate fromDate;
    ToDate toDate;

    /// <summary>
    /// Pre-process: retrieve and store data in TempDB before report renders.
    /// Called once when the report is first requested.
    /// </summary>
    public void processReport()
    {
        // Retrieve data and store in TempDB
        while select vendTable
            where vendTable.CreatedDate >= fromDate
            && vendTable.CreatedDate <= toDate
        {
            tmpTable.clear();
            tmpTable.AccountNum = vendTable.AccountNum;
            tmpTable.VendorName = vendTable.Name;
            tmpTable.CreditMax = vendTable.CreditMax;
            tmpTable.insert();
        }
    }

    /// <summary>
    /// Return the pre-processed data from TempDB to the report dataset.
    /// The SRSReportDataSetAttribute binds this method to the report.
    /// Called by the SSRS framework during rendering.
    /// </summary>
    [SRSReportDataSetAttribute(tableStr(VendComplianceReportTmp))]
    public VendComplianceReportTmp getReportDataTmp()
    {
        return tmpTable;
    }
}
```

When to Use Pre-Processing

Report Size	InMemory	TempDB Pre-Processing
< 1,000 records	[x] Fine	Not needed
1,000–10,000 records	[!] May be slow	Recommended
10,000–100,000 records	[ ] Timeout risk	Required
> 100,000 records	[ ] Will timeout	Required + optimize query

Performance Comparison

Approach	10K Records	100K Records	1M Records
InMemory + direct query	5s	45s (timeout risk)	Timeout
TempDB pre-processing	3s	8s	25s
TempDB + indexed TempDB table	2s	5s	12s

10.8 Electronic Reporting (ER)

Electronic Reporting (ER) is a format-based reporting engine used for tax reporting, regulatory compliance, and data exchange with external systems. It's the standard mechanism for formats like SAF-T, VAT declarations, and other regulatory reports.

ER Components

Component	Purpose	Example
Format	Defines the output structure (XML, JSON, CSV, etc.)	SAF-T format, VAT XML schema
Data source	Defines where the data comes from (tables, views, entities)	`VendTable`, `VendInvoiceJour`
Mapping	Maps source fields to format fields	`VendTable.AccountNum` → `SupplierID`
Configuration	Links format, data source, and mapping together	One ER configuration per report

ER Format Design

ER formats are designed in the **Electronic Reporting** workspace:

1. Navigate to **Organization Administration → Inquiries → Electronic Reporting → Formats**
2. Create a new format or extend an existing one
3. Define the XML/JSON schema for the output
4. Create mappings that connect source fields to format fields
5. Test the format with sample data

ER Configuration Example

```
// ER Configuration for VAT Declaration
// Format: VAT XML Schema (e.g., SAF-T EU)
// Data Source: VendInvoiceJour + VendInvoiceLine + TaxTrans
// Mapping: Invoice fields → VAT declaration fields
```

ER vs. SSRS — When to Use Which

Scenario	Use ER	Use SSRS
Tax reporting (VAT, SAF-T)	[x] Yes	[ ] No
Regulatory compliance format	[x] Yes	[ ] No
Data exchange with external systems	[x] Yes	[ ] No
Internal management reports	[ ] No	[x] Yes
Printable invoices/purchase orders	[ ] No	[x] Yes
Financial statements	[ ] No	[x] Yes
Pixel-perfect layout required	[ ] No	[x] Yes

ER Format Extension

You can extend standard ER formats to add custom fields:

1. Open the standard format in the ER workspace
2. Click **Extend** to create a format extension
3. Add your custom fields to the extended format
4. Create mappings for the new fields
5. Activate the extension

```
// ER format extension for custom compliance fields
// The extension adds custom fields to the standard VAT declaration format
// Custom fields: ComplianceCode, AuditReference, InternalNote
```

10.9 Analytical Reports — Deep Dive

Analytical Report Architecture

Analytical reports in D365 F&O differ from operational reports (SSRS) in several key ways:

Aspect	Operational Report	Analytical Report
Data source	`SrsReportDataProvider` with AOT Query	`SysOperationServiceController` with contract
Output	PDF, Excel, printed	Interactive grid, charts, pivot tables
User interaction	View/print/export	Filter, drill down, pivot, slice
Framework	SSRS	Analytical Workspace + Power BI
Data volume	Detailed transactional	Aggregated summaries

Aspect	Operational Report	Analytical Report
Real-time	Near real-time (batch)	Near real-time (cached)

Analytical Report with Drill-Down

Analytical reports support drill-down from summary to detail:

```
// Analytical report contract with drill-down support
class VendComplianceAnalyticsContract
{
    VendGroup vendGroup;
    FromDate fromDate;
    ToDate toDate;
    ComplianceStatus complianceStatus;
    boolean includeCharts;
    boolean drillDown;       // Enable drill-down to detail
    VendTable drillDownRecord; // The record to drill into
}

class VendComplianceAnalyticsService
{
    public VendComplianceAnalyticsContract process(VendComplianceAnalyticsContract _contract)
    {
        if (_contract.parmDrillDown())
        {
            // Return detailed data for the drilled-in record
            return this.getDrillDownData(_contract);
        }
        else
        {
            // Return aggregated summary data
            return this.getSummaryData(_contract);
        }
    }

    private VendComplianceAnalyticsContract getSummaryData(VendComplianceAnalyticsContract _contract)
    {
        // Aggregate data by vendor group
        // Calculate compliance rates, totals, averages
        // Return summary for the Analytical Workspace
        return _contract;
    }

    private VendComplianceAnalyticsContract getDrillDownData(VendComplianceAnalyticsContract _contract)
    {
        // Return detailed records for the selected vendor
        // This is called when the user clicks a summary row to drill down
        return _contract;
    }
}
```

Integration with Power BI

D365 F&O's Analytical Workspace integrates with Power BI for advanced analytics:

1. **OData feed**: Analytical reports expose data via OData feeds
2. **Power BI Desktop**: Connect to the OData feed and build interactive dashboards
3. **Power BI Service**: Publish dashboards to the Power BI service

4. **Analytical Workspace**: Pin Power BI tiles to the F&O workspace

Analytical Report Best Practices

1. **Use aggregation** — analytical reports should summarize data, not show every transaction
2. **Enable drill-down** — users should be able to click a summary row to see detail
3. **Cache results** — use the `SysOperation` framework's caching for frequently accessed reports
4. **Limit data volume** — use date ranges and filters to limit the data returned
5. **Use measures and dimensions** — organize data into measures (numeric) and dimensions (categorical)

10.10 Activity — Build an End-to-End SSRS Report

> **Activity**: Build an end-to-end SSRS report for the compliance module with the following requirements: 1. **Parameters**: Date range (`fromDate`, `toDate`), vendor group (`CustGroup`), compliance status (`Validated`, `Pending`, `Failed`) 2. **Data source**: Query joining `VendTable`, `VendInvoiceJour`, `APCustomsDeclaration`, and `VendInvoiceLine` (4+ table joins) 3. **Grouped sections**: Group by vendor group, then by vendor, then by compliance status 4. **Subtotals**: Show subtotals for each vendor group and each vendor 5. **Grand total**: Show a grand total row at the bottom with record count and summary statistics 6. **Graceful empty-data handling**: When no records match the filter criteria, display a friendly "No data found" message instead of an empty report 7. **Professional layout**: Use a corporate-style header with company logo, report title, date range, and page numbers 8. **Export options**: Configure the report to support PDF, Excel, and email delivery 9. **Menu item**: Create a `MenuItemOutput` linked to the report class 10. **Security**: Restrict report access to the `ComplianceOfficer` role
>
> **Activity Hints** (Multiple Valid Approaches):
>
> - **Hint A — Dataset query design**: Option A1 — use a `SrsReportDataProvider` class with a `Query` object that joins all 4 tables (recommended — follows Microsoft patterns, supports dynamic filtering). Option A2 — use a `View` object as the data source (simpler but less flexible — can't dynamically add filters at runtime). Option A3 — use inline `select` statements in the data provider (most flexible but doesn't leverage the AOT query infrastructure).
> - **Hint B — Report layout strategy**: Option B1 — use `AutoDesign` for initial development, then manually refine the RDL for production (recommended — fastest path to a working report). Option B2 — design the entire RDL manually from scratch in Report Builder (most control but slowest). Option A3 — use a hybrid approach — AutoDesign for the initial layout, then manually edit the RDL XML to add grouping, subtotals, and formatting.
> - **Hint C — Empty-data handling**: Option C1 — add a `NoRowsMessage` property on the Tablix control (recommended — simplest approach). Option C2 — add a separate text box that is visible when the dataset returns no records, using an expression like `=IIF(CountRows("Dataset1") = 0, "No compliance data found for the selected criteria.", "")` (more flexible, allows custom formatting). Option C3 — handle empty data in the `processReport()` method of the data provider by inserting a dummy record into the temp table with a "No data" message (not recommended — pollutes the data layer).
> - **Hint D — Subtotals and grouping**: Option D1 — use the SSRS grouping feature in the Report Designer — right-click a column group → "Add Group" → "Parent Group"

(recommended — visual and intuitive). Option D2 — handle subtotals in the data provider by pre-calculating group totals in the `get()` method (more control but more complex). Option D3 — use a matrix/tablix with row and column groups for a pivot-style layout (good for analytical reports but overkill for this scenario).

- **Hint E — Email delivery**: Option E1 — configure `PrintJobSettings` in the report controller to send via email (recommended — standard D365 F&O pattern). Option E2 — use a Logic App triggered by the report execution to send the email (more flexible but more complex). Option E3 — use `SmtpMail::send()` from X++ (direct but requires SMTP configuration).

Expected Approach (Ideal — in detail)

Step 1: Create the Data Provider Class

```
class VendComplianceSummaryReportDP extends SrsReportDataProvider
{
    VendTable vendTable;
    FromDate fromDate;
    ToDate toDate;
    ComplianceStatus complianceStatus;

    public VendComplianceReportContract get()
    {
        VendComplianceReportContract contract;
        VendComplianceReportTmp tmpTable;

        contract = this.parmContract();

        // Build the query
        Query query = new Query();
        QueryBuildDataSource qbdVend;
        QueryBuildDataSource qbdInvoice;
        QueryBuildDataSource qbdDeclaration;
        QueryBuildRange qbr;

        // Primary data source: VendTable
        qbdVend = query.addDataSource(tableNum(VendTable));

        // Join to VendInvoiceJour
        qbdInvoice = qbdVend.addDataSource(tableNum(VendInvoiceJour));
        qbdInvoice.joinMode(JoinMode::InnerJoin);
        qbdInvoice.addLink(fieldNum(VendTable, RecId), fieldNum(VendInvoiceJour, VendTable));

        // Join to APCustomsDeclaration
        qbdDeclaration = qbdInvoice.addDataSource(tableNum(APCustomsDeclaration));
        qbdDeclaration.joinMode(JoinMode::InnerJoin);
        qbdDeclaration.addLink(fieldNum(VendInvoiceJour, RecId), fieldNum(APCustomsDeclaration,
VendInvoiceLineRecId));

        // Add date range filter
        qbr = qbdDeclaration.addRange(fieldNum(APCustomsDeclaration, DeclDate));
        qbr.value(strfmt('"%1" .. "%2"', contract.parmFromDate(), contract.parmToDate()));

        // Add vendor group filter
        if (contract.parmVendGroup())
        {
            qbr = qbdVend.addRange(fieldNum(VendTable, CustGroup));
            qbr.value(queryValue(contract.parmVendGroup()));
        }
```

```
        // Add compliance status filter
        if (contract.parmComplianceStatus())
        {
            qbr = qbdDeclaration.addRange(fieldNum(APCustomsDeclaration, Status));
            qbr.value(queryValue(contract.parmComplianceStatus()));
        }

        this.parmQuery(query);

        // Fetch data into temp table
        while select vendTable
            join invoice
            where invoice.VendTable == vendTable.RecId
            join declaration
            where declaration.VendInvoiceLineRecId == invoice.RecId
        {
            tmpTable.clear();
            tmpTable.AccountNum = vendTable.AccountNum;
            tmpTable.VendorName = vendTable.Name;
            tmpTable.VendGroup = vendTable.CustGroup;
            tmpTable.InvoiceId = invoice.InvoiceId;
            tmpTable.InvoiceDate = invoice.InvoiceDate;
            tmpTable.DeclId = declaration.DeclId;
            tmpTable.DeclDate = declaration.DeclDate;
            tmpTable.Status = declaration.Status;
            tmpTable.insert();
        }

        return contract;
    }

    // Parameter getters/setters
    public VendTable parmVendTable(VendTable _vendTable = vendTable)
    {
        vendTable = _vendTable;
        return vendTable;
    }

    public FromDate parmFromDate(FromDate _fromDate = fromDate)
    {
        fromDate = _fromDate;
        return fromDate;
    }

    public ToDate parmToDate(ToDate _toDate = toDate)
    {
        toDate = _toDate;
        return toDate;
    }
}
```

Step 2: Design the Report Layout (Manual RDL)

The report layout should include:

1. **Header section**: Company logo, report title, date range, page number
2. **Tablix with 3-level grouping**:
3. Row group 1: Vendor Group (`CustGroup`)
4. Row group 2: Vendor (`AccountNum`, `Name`)

5. Row group 3: Compliance Status (`Status`)
6. **Columns**: Invoice ID, Invoice Date, Declaration ID, Declaration Date, Status
7. **Subtotals**: Count of declarations and count by status for each vendor group
8. **Grand total row**: Total declarations, total by status
9. **NoRowsMessage**: "No compliance data found for the selected criteria."
10. **Conditional formatting**: Color-code status cells (Green = Validated, Yellow = Pending, Red = Failed)

Step 3: Create the Report Controller

```
class VendComplianceSummaryReportController extends SysOperationServiceController
{
    public static void main(Args _args)
    {
        VendComplianceSummaryReportController controller = new VendComplianceSummaryReportController();
        VendComplianceReportContract contract = new VendComplianceReportContract();

        // Populate contract from args if triggered from a form
        if (_args && _args.record() && _args.record().TableId == tableNum(VendTable))
        {
            contract.parmVendTable(_args.record());
        }

        controller.parmContractType(classStr(VendComplianceReportContract));
        controller.parmServiceType(classStr(VendComplianceSummaryReportDP));
        controller.parmCaption('Vend Compliance Summary Report');
        controller.startOperation();
    }
}
```

Step 4: Configure PrintJobSettings for Email Delivery

```
// In the controller or a menu item action:
PrintJobSettings printJobSettings = new PrintJobSettings();
printJobSettings.parmPrintMedium(PrintMedium::Email);
printJobSettings.parmEmailTo('compliance@acme.com');
printJobSettings.parmEmailSubject(strFmt('Compliance Report - %1 to %1', fromDate, toDate));
printJobSettings.parmFileFormat(FileFormat::PDF);
printJobSettings.parmEmailBody('Please find the monthly compliance report attached.');
```

Step 5: Create the MenuItemOutput

```
// MenuItemOutput: Object = VendComplianceSummaryReportController, Method = main
// Label: 'Compliance Summary Report'
// Added to the AP Menu or as a standalone workspace tile
```

Step 6: Security Configuration

- Create a privilege `VendComplianceReportView` with `AccessLevel::See` on the report
- Add the privilege to the `APComplianceViewer` duty
- Assign the duty to the `ComplianceOfficer` role
- Test with "Run as" to verify only compliance officers can access the report

Model Manifest Dependencies

```
<ModelManifest>
  <Name>AcmeComplianceReporting</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
  <ConfigurationKey>AcmeComplianceReportingEnabled</ConfigurationKey>
</ModelManifest>
```

10.11 Activity — Electronic Reporting Configuration

> **Activity**: Configure an Electronic Reporting format for VAT declarations in a multi-company D365 F&O environment.
>
> **Requirements:** 1. Create a new ER format based on the EU SAF-T XML schema 2. Define data sources for `VendTable`, `VendInvoiceJour`, `VendInvoiceLine`, and `TaxTrans` 3. Create mappings that transform D365 F&O data into the SAF-T XML structure 4. Add a custom extension to the standard format for your company's additional fields 5. Configure the ER configuration to run as a batch job for monthly VAT reporting 6. Test the format with sample data from two different legal entities 7. Validate the output XML against the SAF-T schema
>
> **Activity Hints:** - **Hint A — Format design**: Option A1 — start with the standard EU SAF-T format and extend it (recommended — Microsoft provides the base format). Option A2 — create a completely custom format from scratch (only if no standard format exists for your country). Option A3 — use an existing JSON or CSV format and modify the mappings (valid for simpler formats). - **Hint B — Data source design**: Option B1 — use `VendInvoiceJour` as the root data source with `VendInvoiceLine` and `TaxTrans` as child sources (recommended — follows the standard invoice structure). Option B2 — use a Data Entity as the data source (simpler but less flexible for ER-specific transformations). Option B3 — use direct table queries in the mapping (most flexible but hardest to maintain). - **Hint C — Batch execution**: Option C1 — configure the ER format to run as a batch job using the `ElectronicReportingStaging` table (recommended — standard D365 F&O pattern). Option C2 — trigger ER from a custom menu item with `PrintJobSettings` (valid but less automated). Option C3 — use Power Automate to trigger ER on a schedule (modern but adds complexity). - **Hint D — Multi-company validation**: Option D1 — run the ER format for each company separately and validate the output (recommended — ensures data isolation). Option D2 — run a single ER job that processes all companies and validates each output file (faster but more complex). Option D3 — use the `MultiCompany` property on the data source to process all companies in one run (simplest but requires careful validation).

10.12 Activity — Report Performance Optimization

> **Activity**: A compliance report takes 15 minutes to run and times out with 500,000+ records. Diagnose the issue and apply three different optimization strategies with comparisons.
>
> **Scenario Details:** - The report joins `VendTable`, `VendInvoiceJour`, `APCustomsDeclaration`, `VendInvoiceLine`, and `TaxTrans` (5 table joins) - The data provider uses `InMemory` temporary tables - The report has 4-level grouping (Company → Vendor Group → Vendor → Compliance Status) - The report includes subtotals and a grand total row - The report is run monthly for all companies

> **Three Optimization Proposals:**
>
> **Proposal 1: Switch to TempDB pre-processing** - Change the data provider from `SrsReportDataProvider` to `SrsReportDataProviderPreProcessTempDB` - Move data retrieval to the `processReport()` method - Add indexes on the TempDB table for the grouping columns - **Expected improvement**: 15 min → 3 min (80% faster) - **Trade-off**: The report takes slightly longer to start (pre-processing phase), but the rendering is fast
>
> **Proposal 2: Optimize the query with exists joins** - Replace `join` with `exists join` where full data retrieval isn't needed - Add indexes on join columns (`VendTable.RecId`, `VendInvoiceJour.VendTable`) - Use `firstonly` where only one record is needed per group - **Expected improvement**: 15 min → 6 min (60% faster) - **Trade-off**: Some detail data may not be available for all groupings
>
> **Proposal 3: Pre-aggregate data in a batch job** - Create a batch job that pre-aggregates the data nightly - The report reads from the pre-aggregated table instead of joining 5 tables - The batch job updates a `ComplianceSummaryTmp` table with grouped totals - **Expected improvement**: 15 min → 30 seconds (99% faster) - **Trade-off**: Data is not real-time — it reflects the last nightly run, not the current state
>
> **Activity Hints:** - **Hint A — Root cause identification**: Option A1 — InMemory tables for large datasets is the primary bottleneck (most likely — 500K+ records in memory causes timeout). Option A2 — Missing indexes on join columns (possible — check SQL execution plan). Option A3 — The 5-table join is too complex (possible — consider denormalizing the data model). Option A4 — Multiple root causes combined (most realistic). - **Hint B — Optimization strategy**: Option B1 — TempDB pre-processing (recommended — highest impact, lowest risk). Option B2 — Query optimization with exists joins (good — complementary to TempDB). Option B3 — Pre-aggregation in a batch job (advanced — best for very large datasets but adds complexity). - **Hint C — Validation approach**: Option C1 — Measure before and after with `SysPerformance` (recommended — data-driven). Option C2 — Compare report execution times in the batch job history (valid but less precise). Option C3 — Use SQL Server Profiler to trace the queries (most accurate — shows exactly what SQL is being generated).

Remaining Chapters — Detailed Outlines

Chapter 11 — Job Framework & Batch Processing

11.1 Jobs vs. Batch vs. RunBase — Decision Matrix

D365 F&O provides three distinct mechanisms for executing background or scheduled operations. Choosing the right one depends on the complexity, duration, and user interaction requirements of the task.

The Three Mechanisms

Mechanism	Description	When to Use
Job	A simple X++ script executed ad-hoc for one-time data fixes, reports, or one-off processing	Quick data fixes, ad-hoc reports, one-time data migration
Batch (RunBaseBatch)	A scheduled, queued job that runs asynchronously via the batch framework	Long-running operations, nightly processing, operations that should run unattended
RunBase	A dialog-based operation with user input, can run synchronously or as a batch	Operations that need user parameters and may or may not need batch queuing

Decision Matrix

Scenario	Use Job	Use Batch	Use RunBase
One-time data fix (e.g., update 100 records)	[x] Yes	[ ] No	[ ] No
Nightly reconciliation of 100K+ records	[ ] No	[x] Yes	[ ] No
User needs to input parameters before execution	[ ] No	[x] Yes (via RunBaseBatch dialog)	[x] Yes
Operation takes < 5 seconds	[x] Yes or RunBase	[ ] No	[x] Yes
Operation takes > 5 minutes	[ ] No	[x] Yes	[ ] No (use batch)
Operation needs to run unattended	[ ] No	[x] Yes	[ ] No
Operation needs progress reporting	[ ] No	[x] Yes	[!] Limited
Operation needs retry on failure	[ ] No	[x] Yes	[ ] No
Operation needs email notification on completion	[ ] No	[x] Yes	[ ] No

Job — Quick Ad-Hoc Execution

A job is the simplest execution mechanism. It's an X++ script that runs immediately when triggered.

```
static void Job_QuickVendorUpdate(Args _args)
{
    VendTable vendTable;

    // Quick one-time update: set compliance code for vendors without one
    while select vendTable
        where vendTable.ComplianceCode == ''
        && vendTable.CreditMax > 500000
    {
        vendTable.ComplianceCode = 'DEFAULT-COMP';
        vendTable.update();
    }

    info(strFmt('Updated %1 vendors.', vendTable.RecCount));
}
```

When to use jobs: - Quick data fixes that need to run once - Ad-hoc reporting or data extraction - Testing a piece of logic before implementing it as a proper batch - One-time data migration during implementation

When NOT to use jobs: - Operations that need to run on a schedule - Operations that need retry logic - Operations that need progress reporting - Operations that need user input parameters

---

11.2 BatchJob and BatchHistory Tables

The batch framework in D365 F&O manages the lifecycle of batch jobs through the **BatchJob** (current state) and **BatchHistory** (execution history) tables.

Batch Job Lifecycle Stages

```
[1] Job Created → BatchJob.STATUS = Waiting
    |
[2] Job Queued → BatchJob.STATUS = Waiting (queued for execution)
    |
[3] Batch Server picks up job → BatchJob.STATUS = Executing
    |
[4] Job executes → run() method executes on the server
    |
[5] Job completes → BatchJob.STATUS = Ended (or Error/Canceled)
    |
[6] History recorded → BatchHistory captures STARTDATETIME, ENDDATETIME, STATUS
    |
[7] User notified via Infolog and/or email
```

Source: Batch OData API

BatchJob and BatchHistory Tables

The **BatchJob** and **BatchHistory** tables track the state and history of batch jobs:

Table	Field	Description
BatchJob	`BATCHJOBID`	Unique identifier for the batch job
BatchJob	`CAPTION`	User-readable description of the job

Table	Field	Description
BatchJob	`STATUS`	Current status (Waiting, Executing, Ended, Error, Canceled)
BatchJob	`SERVERID`	The batch server processing the job
BatchHistory	`BATCHJOBID`	Links to the BatchJob record
BatchHistory	`STARTDATETIME`	When the job started executing
BatchHistory	`ENDDATETIME`	When the job finished executing
BatchHistory	`STATUS`	Final status (Ended, Error, Canceled)

Batch Groups

Batch groups control which batch server processes which jobs. This allows you to prioritize and isolate batch workloads:

Batch Group	Purpose	Example
`BATCHGROUP1`	Standard batch processing	General reconciliation jobs
`BATCHGROUP2`	High-priority batch processing	Time-sensitive compliance checks
`BATCHGROUP3`	Low-priority batch processing	Nightly reports, data cleanup

---

11.3 `RunBaseBatch` — Dialog UI for Batch Parameters

`RunBaseBatch` extends `RunBase` to add batch job capabilities. It provides the dialog UI for parameter input, batch queuing, and progress reporting.

The `RunBaseBatch` Class Template

```
class VendReconciliationBatchJob extends RunBaseBatch
{
    VendTable vendTable;
    FromDate fromDate;
    ToDate toDate;
    int maxRecords;
    boolean sendEmail;

    // Dialog — user input
    public Object dialog()
    {
        Dialog dialog = super::dialog();
        DialogField dfVend;
        DialogField dfFrom;
        DialogField dfTo;
        DialogField dfMax;
        DialogField dfEmail;

        dfVend = dialog.addFieldValue(typeid(VendTable), vendTable, 'Vendor');
        dfVend.lookupButton(true);

        dfFrom = dialog.addFieldValue(typeid(FromDate), fromDate, 'From Date');
        dfTo = dialog.addFieldValue(typeid(ToDate), toDate, 'To Date');
```

```
    dfMax = dialog.addFieldValue(typeid(int), maxRecords, 'Max Records');
    dfMax.helpText('Maximum number of records to process (0 = unlimited)');

    dfEmail = dialog.addFieldValue(typeid(boolean), sendEmail, 'Send email notification');

    return dialog;
}

// Pack — serialize for batch
public container pack()
{
    return [vendTable, fromDate, toDate, maxRecords, sendEmail];
}

// Unpack — deserialize from batch
public boolean unpack(container _packedClass)
{
    vendTable = _packedClass.packGet(1);
    fromDate = _packedClass.packGet(2);
    toDate = _packedClass.packGet(3);
    maxRecords = _packedClass.packGet(4);
    sendEmail = _packedClass.packGet(5);
    return true;
}

// Run on batch tier
public RunOn runOn()
{
    return RunOn::Batch;
}

// Business logic
public void run()
{
    VendTable vendTableLocal;
    int processedCount = 0;
    int failedCount = 0;

    while select vendTableLocal
        where vendTableLocal.AccountNum == vendTable.AccountNum
    {
        // Check cancellation
        if (this.checkCancel())
        {
            info('Job was cancelled by the user.');
            break;
        }

        // Check max records limit
        if (maxRecords > 0 && processedCount >= maxRecords)
        {
            info(strFmt('Reached max records limit of %1.', maxRecords));
            break;
        }

        try
        {
            ttsbegin;
            this.reconcileVendor(vendTableLocal);
            ttscommit;
            processedCount++;
```

```
            }
            catch (Exception::Error)
            {
                ttsabort;
                failedCount++;
                warning(strFmt('Failed to reconcile vendor %1.', vendTableLocal.AccountNum));
            }

            // Update progress
            this.lastValue(strFmt('Processed %1 records...', processedCount));
        }

        info(strFmt('Reconciliation complete: %1 succeeded, %2 failed.', processedCount, failedCount));

        // Send email notification if requested
        if (sendEmail && failedCount == 0)
        {
            this.sendNotification(processedCount);
        }
    }

    private void reconcileVendor(VendTable _vendTable)
    {
        // Reconciliation logic here
        info(strFmt('Reconciling vendor %1...', _vendTable.AccountNum));
    }

    private void sendNotification(int _processedCount)
    {
        // Email notification logic
        // ...
    }

    // Batch metadata
    public BatchHeader batchInfo()
    {
        BatchHeader batchHeader = super::batchInfo();
        batchHeader.parmDescription('Vendor Reconciliation Batch Job');
        batchHeader.parmBatchGroup('BATCHGROUP1');
        batchHeader.parmExecutionStyle(BatchExecutionStyle::OnDemand);
        return batchHeader;
    }

    // Static entry point
    public static void main(Args _args)
    {
        VendReconciliationBatchJob job = new VendReconciliationBatchJob();
        if (_args && _args.record())
        {
            job.parmVendTable(_args.record());
        }
        if (job.prompt())
        {
            job.run();
        }
    }
}
```

`runOn()` — Execution Target

Value	Description	Use When
`RunOn::Client`	Runs on the user's desktop	Lightweight operations that need UI interaction
`RunOn::Server`	Runs on the AOS server	Operations that need server-side data access but don't need batch queuing
`RunOn::Batch`	Runs on the batch server	Long-running operations that should be queued and run asynchronously

---

11.4 `SysOperationServiceController` — Service-Based Batch Pattern

As covered in Chapter 6, `SysOperationServiceController` provides a modern, contract-driven approach to batch operations. It is particularly useful when:

- The operation needs to be called from external systems (OData/REST)
- The operation needs contract-based parameter validation
- The operation should be versioned independently of the implementation

SysOperationServiceController for Batch

```
class VendReconciliationController extends SysOperationServiceController
{
    public static void main(Args _args)
    {
        VendReconciliationController controller = new VendReconciliationController();
        VendReconciliationContract contract = new VendReconciliationContract();

        controller.parmContractType(classStr(VendReconciliationContract));
        controller.parmServiceType(classStr(VendReconciliationService));
        controller.parmCaption('Vendor Reconciliation');
        controller.startOperation();
    }
}
```

When to Use SysOperationServiceController vs RunBaseBatch

Feature	RunBaseBatch	SysOperationServiceController
Dialog-based input	[x] Yes	[ ] No (contract-based)
Batch queuing	[x] Yes	[x] Yes
OData/REST exposure	[ ] No	[x] Yes
Contract validation	[ ] No	[x] Yes
Simplicity	[x] Simpler	[ ] More complex
Modern pattern	[!] Legacy	[x] Recommended for new development

---

11.5 Progress Reporting

Progress reporting keeps users informed about the status of long-running batch operations.

`lastValue()` — Progress String

The `lastValue()` method returns a string that describes the current progress of the job. This string is displayed in the batch job list in LCS.

```
public int lastValue()
{
    // Return the number of records processed so far
    return this.processedCount;
}
```

`lastValueCounter` — Built-in Counter

`RunBaseBatch` provides a built-in `lastValueCounter` that automatically increments. You can use it for simple progress reporting:

```
// In the run() method:
this.lastValueCounter++;
```

Percent Complete Reporting

For more detailed progress reporting, calculate the percentage complete:

```
public int lastValue()
{
    int totalRecords = this.getTotalRecordCount();
    int processedRecords = this.getProcessedCount();

    if (totalRecords > 0)
    {
        return (processedRecords * 100) / totalRecords;
    }
    return 0;
}
```

Status Messages

Use `info()`, `warning()`, and `error()` in the `run()` method to provide real-time status updates:

```
public void run()
{
    info('Starting vendor reconciliation...');

    while select vendTable
    {
        // Process each vendor
        if (this.checkCancel())
        {
            warning('Job cancelled by user.');
            break;
        }

        try
```

```
        {
            this.reconcileVendor(vendTable);
            info(strFmt('Reconciled vendor %1 successfully.', vendTable.AccountNum));
        }
        catch (Exception::Error)
        {
            error(strFmt('Failed to reconcile vendor %1.', vendTable.AccountNum));
        }
    }

    info('Vendor reconciliation complete.');
}
```

11.6 Cancellation — `checkCancel()` in Batch Loops

The `checkCancel()` method checks whether the user has requested cancellation of the batch job. It should be called inside long-running loops to allow graceful cancellation.

How `checkCancel()` Works

```
public void run()
{
    while select vendTable
    {
        // Check if the user has cancelled the job
        if (this.checkCancel())
        {
            info('Batch job was cancelled by the user.');
            break;
        }

        // Process the record
        this.processVendor(vendTable);
    }
}
```

Cancellation Behavior

Behavior	Description
`checkCancel()` returns `true`	The loop breaks and the job finishes gracefully
`checkCancel()` returns `false`	The loop continues processing
After cancellation	The job status is set to `Completed` (not `Failed`) — cancellation is a normal exit

Best Practices for Cancellation

1. **Call `checkCancel()` inside every `while select` loop** — long-running loops must check for cancellation
2. **Call `checkCancel()` at reasonable intervals** — don't call it on every record if processing is very fast (performance impact)
3. **Handle cancellation gracefully** — clean up any partial work, log the cancellation, and exit the loop
4. **Don't throw exceptions on cancellation** — cancellation is a normal exit path, not an error

Cancellation with Transaction Control

```
public void run()
{
    VendTable vendTable;

    while select vendTable
    {
        if (this.checkCancel())
        {
            info('Cancellation requested. Finishing current transaction...');
            break;
        }

        ttsbegin;
        try
        {
            this.processVendor(vendTable);
            ttscommit;
        }
        catch (Exception::Error)
        {
            ttsabort;
            warning(strFmt('Failed to process vendor %1.', vendTable.AccountNum));
        }
    }
}
```

Important: When `checkCancel()` returns `true` inside a `ttsbegin/ttscommit` block, the current transaction is still committed. The cancellation check happens after the transaction completes. This ensures data consistency — partial records are not left in an inconsistent state.

11.7 Retry Logic — `Global::retry()` and Custom Retry Counters

Retry logic allows a batch job to recover from transient failures (e.g., network timeouts, deadlocks, temporary service unavailability).

`Global::retry()` — Built-in Retry

`Global::retry()` is a simple retry mechanism that retries the current operation a specified number of times with a delay between retries.

```
public void processVendor(VendTable _vendTable)
{
    int retryCount = 0;
    int maxRetries = 3;
    boolean success = false;

    while (!success && retryCount < maxRetries)
    {
        try
        {
            // Attempt the operation
            this.callExternalService(_vendTable);
            success = true;
        }
```

```
        catch (Exception::Error)
        {
            retryCount++;
            if (retryCount < maxRetries)
            {
                // Wait before retrying (exponential backoff)
                Global::retry(retryCount * 5);  // Wait 5, 10, 15 seconds
                warning(strFmt('Retry %1 of %2 for vendor %3...',
                    retryCount, maxRetries, _vendTable.AccountNum));
            }
            else
            {
                error(strFmt('Failed to process vendor %1 after %2 retries.',
                    _vendTable.AccountNum, maxRetries));
            }
        }
    }
}
```

Custom Retry Counters

For more control over retry behavior, implement custom retry logic:

```
class VendReconciliationRetryHandler
{
    static int maxRetries = 3;
    static int retryDelaySeconds = 5;

    public static boolean executeWithRetry(Container _operation)
    {
        int attempt = 0;
        boolean success = false;

        while (!success && attempt < maxRetries)
        {
            attempt++;
            try
            {
                // Execute the operation
                _operation.call();
                success = true;
            }
            catch (Exception::Error)
            {
                if (attempt < maxRetries)
                {
                    // Exponential backoff: 5s, 10s, 20s
                    int delay = retryDelaySeconds * (2 ^ (attempt - 1));
                    Global::retry(delay);
                    info(strFmt('Attempt %1 failed. Retrying in %2 seconds...', attempt, delay));
                }
                else
                {
                    error(strFmt('Operation failed after %1 attempts.', maxRetries));
                }
            }
        }

        return success;
```

```
    }
}
```

When to Retry vs. When to Fail

Error Type	Action
Transient errors (network timeout, deadlock, temporary service unavailability)	Retry
Permanent errors (invalid data, missing records, permission denied)	Fail immediately — retry won't help
Resource errors (disk full, memory low)	Retry with backoff, but also alert the operator
Logic errors (bug in code, incorrect configuration)	Fail immediately — fix the code

Retry Best Practices

1. **Use exponential backoff** — wait longer between each retry attempt (5s, 10s, 20s, 40s...)
2. **Limit retry attempts** — 3-5 retries is typically sufficient; more retries waste resources
3. **Log each retry attempt** — include the attempt number, error message, and wait time
4. **Distinguish transient from permanent errors** — don't retry permanent errors
5. **Alert on final failure** — if all retries fail, send a notification to the operations team
6. **Make retry counts configurable** — use a parameter or configuration key so operators can adjust without code changes

---

11.8 Activity — Batch Reconciliation Job for 100K+ Records

> **Activity**: Create a batch reconciliation job for 100K+ vendor records with the following requirements: 1. The job must process vendors in batches of 1,000 records at a time to avoid memory issues 2. The job must display progress (percent complete and records processed) 3. The job must support cancellation — the user should be able to stop the job at any time 4. The job must implement retry logic with exponential backoff for transient failures 5. The job must send an email notification upon completion (success or failure summary) 6. The job must log all errors to a custom error log table for later review 7. The job must support a "dry run" mode that simulates processing without writing to the database 8. The job must handle the case where the batch is cancelled mid-transaction (rollback the current batch, not the entire job) 9. The job must be configurable via a dialog with parameters: vendor group, date range, batch size, max retries, dry run mode 10. The job must be scheduled to run nightly via the batch framework
>
> **Activity Hints** (Multiple Valid Architectural Choices):
>
> - **Hint A — Retry/cancellation/notification patterns**: Option A1 — use `Global::retry()` with exponential backoff, `checkCancel()` inside the loop, and `PrintJobSettings` for email notification (recommended — follows Microsoft patterns). Option A2 — implement custom retry logic with a retry counter table (more control but more complex). Option A3 — use `RunBaseBatch` with built-in retry support (simplest but least configurable).
> - **Hint B — Batch processing strategy**: Option B1 — process in chunks of 1,000 records using `while select` with `next` and a counter (recommended — memory-efficient, allows cancellation between batches). Option B2 — use `insert_recordset`/`update_recordset` for set-

based processing (fastest but less granular control over cancellation and error handling). Option B3 — use a cursor-based approach (not recommended — cursors are slow and don't scale).

- **Hint C — Dry run mode**: Option C1 — use a boolean flag that skips `insert()`/`update()` calls and only logs what would have happened (recommended — simple and effective). Option C2 — use a separate staging table that captures all changes without writing to the target tables (more detailed but more complex). Option C3 — run the job in a test company (valid but requires a separate environment).
- **Hint D — Error logging**: Option D1 — write errors to a custom `BatchErrorLog` table with fields for batch ID, vendor, error message, timestamp (recommended — persistent, queryable). Option D2 — use the Infolog and rely on the batch job history (simpler but less structured). Option D3 — write errors to a file (valid for external systems but not queryable from D365 F&O).
- **Hint E — Scheduling**: Option E1 — use the LCS release pipeline to schedule the batch job (recommended — integrates with the standard deployment process). Option E2 — use the `BatchHeader` to schedule the job for a specific time (more direct control). Option C3 — use Azure DevOps cron trigger to submit the batch job (valid but requires CI/CD pipeline setup).

Expected Approach (Ideal — in detail)

Batch Error Log Table

```
table 50102 BatchErrorLog
{
    DataClassification = CustomerContent;

    fields
    {
        field(1; RecId; int64) { }
        field(2; BatchId; Guid) { }
        field(3; VendorAccount; AccountNum) { }
        field(4; ErrorMessage; Text[250]) { }
        field(5; ErrorDateTime; DateTime) { }
        field(6; RetryCount; int) { }
        field(7; Resolved; boolean) { }
    }

    keys
    {
        key(PK; RecId) { Clustered = true; }
        key(IX_BatchId; BatchId) { }
    }
}
```

Complete Batch Job Implementation

```
class VendReconciliationBatchJob extends RunBaseBatch
{
    VendGroup vendGroup;
    FromDate fromDate;
    ToDate toDate;
    int batchSize;
```

```
    int maxRetries;
    boolean dryRun;
    boolean sendEmail;

    // Progress tracking
    int totalProcessed = 0;
    int totalFailed = 0;
    int totalSkipped = 0;
    Guid batchId;

    // Dialog
    public Object dialog()
    {
        Dialog dialog = super::dialog();
        dialog.addFieldValue(typeid(VendGroup), vendGroup, 'Vendor Group');
        dialog.addFieldValue(typeid(FromDate), fromDate, 'From Date');
        dialog.addFieldValue(typeid(ToDate), toDate, 'To Date');
        dialog.addFieldValue(typeid(int), batchSize, 'Batch Size');
        dialog.addFieldValue(typeid(int), maxRetries, 'Max Retries');
        dialog.addFieldValue(typeid(boolean), dryRun, 'Dry Run Mode');
        dialog.addFieldValue(typeid(boolean), sendEmail, 'Send Email Notification');
        return dialog;
    }

    // Pack
    public container pack()
    {
        return [vendGroup, fromDate, toDate, batchSize, maxRetries, dryRun, sendEmail];
    }

    // Unpack
    public boolean unpack(container _packedClass)
    {
        vendGroup = _packedClass.packGet(1);
        fromDate = _packedClass.packGet(2);
        toDate = _packedClass.packGet(3);
        batchSize = _packedClass.packGet(4);
        maxRetries = _packedClass.packGet(5);
        dryRun = _packedClass.packGet(6);
        sendEmail = _packedClass.packGet(7);
        return true;
    }

    // Run on batch tier
    public RunOn runOn()
    {
        return RunOn::Batch;
    }

    // Business logic
    public void run()
    {
        VendTable vendTable;
        int batchCount = 0;
        int currentBatchSize = 0;

        batchId = Guid::newGuid();
        info(strFmt('Starting vendor reconciliation batch job. Batch ID: %1', batchId));

        while select vendTable
            where vendTable.CustGroup == vendGroup
        {
```

```
            // Check cancellation
            if (this.checkCancel())
            {
                info('Job was cancelled by the user.');
                break;
            }

            // Check batch size limit
            if (batchSize > 0 && currentBatchSize >= batchSize)
            {
                info(strFmt('Batch size limit of %1 reached. Pausing...', batchSize));
                // In a real implementation, you would commit the current batch
                // and resume in the next batch execution
                break;
            }

            // Process with retry logic
            if (this.processVendorWithRetry(vendTable))
            {
                totalProcessed++;
            }
            else
            {
                totalFailed++;
            }

            currentBatchSize++;
            totalProcessed++;

            // Update progress
            this.lastValue(strFmt('Processed %1 records, %2 failed...',
                totalProcessed, totalFailed));
        }

        // Summary
        info(strFmt('Reconciliation complete: %1 processed, %2 failed, %3 skipped.',
            totalProcessed, totalFailed, totalSkipped));

        // Send email notification
        if (sendEmail)
        {
            this.sendNotification(totalProcessed, totalFailed);
        }
    }

    private boolean processVendorWithRetry(VendTable _vendTable)
    {
        int retryCount = 0;
        boolean success = false;

        while (!success && retryCount <= maxRetries)
        {
            try
            {
                if (!dryRun)
                {
                    ttsbegin;
                    this.reconcileVendor(_vendTable);
                    ttscommit;
                }
                success = true;
            }
```

```
            catch (Exception::Error)
            {
                retryCount++;
                if (retryCount <= maxRetries)
                {
                    int delay = 5 * (2 ^ (retryCount - 1));  // Exponential backoff
                    Global::retry(delay);
                    warning(strFmt('Retry %1 of %2 for vendor %3...',
                        retryCount, maxRetries, _vendTable.AccountNum));
                }
                else
                {
                    // Log the error
                    this.logError(_vendTable, 'Max retries exceeded');
                    error(strFmt('Failed to reconcile vendor %1 after %2 retries.',
                        _vendTable.AccountNum, maxRetries));
                }
            }
        }

        return success;
    }

    private void reconcileVendor(VendTable _vendTable)
    {
        // Actual reconciliation logic
        info(strFmt('Reconciling vendor %1...', _vendTable.AccountNum));
    }

    private void logError(VendTable _vendTable, str _error)
    {
        BatchErrorLog errorLog;
        errorLog.initValue();
        errorLog.BatchId = batchId;
        errorLog.VendorAccount = _vendTable.AccountNum;
        errorLog.ErrorMessage = _error;
        errorLog.ErrorDateTime = datetimeUtil::getSystemDateTime();
        errorLog.RetryCount = maxRetries;
        errorLog.insert();
    }

    private void sendNotification(int _processed, int _failed)
    {
        PrintJobSettings printJobSettings = new PrintJobSettings();
        printJobSettings.parmPrintMedium(PrintMedium::Email);
        printJobSettings.parmEmailTo('compliance@acme.com');
        printJobSettings.parmEmailSubject(strFmt('Vendor Reconciliation: %1 processed, %2 failed',
            _processed, _failed));
        printJobSettings.parmEmailBody(strFmt(
            'The nightly vendor reconciliation job has completed.\n\n' +
            'Total processed: %1\n' +
            'Total failed: %2\n' +
            'Batch ID: %3',
            _processed, _failed, batchId));
    }

    // Batch metadata
    public BatchHeader batchInfo()
    {
        BatchHeader batchHeader = super::batchInfo();
        batchHeader.parmDescription('Vendor Reconciliation Batch Job');
        batchHeader.parmBatchGroup('BATCHGROUP1');
```

```
        batchHeader.parmExecutionStyle(BatchExecutionStyle::OnDemand);
        return batchHeader;
    }

    // Static entry point
    public static void main(Args _args)
    {
        VendReconciliationBatchJob job = new VendReconciliationBatchJob();
        if (_args && _args.record())
        {
            job.parmVendTable(_args.record());
        }
        if (job.prompt())
        {
            job.run();
        }
    }
}
```

Model Manifest Dependencies

```
<ModelManifest>
  <Name>AcmeBatchReconciliation</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
  <ConfigurationKey>AcmeBatchReconciliationEnabled</ConfigurationKey>
</ModelManifest>
```

Remaining Chapters — Detailed Outlines

Chapter 12 — Testing & QA

12.1 The `SysTest` Framework

The `SysTest` framework is D365 F&O's built-in unit testing framework. It enables developers to write automated tests that verify business logic, data integrity, and integration points.

Why Unit Testing Matters in D365 F&O

Benefit	Description
Regression prevention	Tests catch bugs introduced by new code before they reach production
Confidence in refactoring	You can safely modify code knowing tests will catch regressions
Documentation	Tests serve as living documentation of expected behavior
LCS quality metrics	Test coverage is visible in LCS for compliance and audit purposes
Faster debugging	Failed tests pinpoint the exact location and nature of bugs

Test Class Structure

Every test class in D365 F&O follows this structure:

```
class VendComplianceTest extends SysTestCase
{
    // Shared test data — created once for all test methods in this class
    public void setUp()
    {
        // Create test data that all test methods need
        // This runs once before any test method in the class
    }

    // Individual test method
    [SysTestMethodAttribute]
    public static void testComplianceCodeValidation()
    {
        // Arrange — set up test data
        // Act — execute the code under test
        // Assert — verify the expected outcome
    }

    // Cleanup — runs after each test method
    public void tearDown()
    {
        // Clean up test data
    }
}
```

Key Attributes

Attribute	Purpose	When to Use

| `[SysTestMethodAttribute]` | Marks a method as a test method | On every test method |

Note: Setup and cleanup are implemented by overriding the `setUp()` and `tearDown()` methods in a class extending `SysTestCase`, not via attributes. Alternatively, test methods can be prefixed with `test` instead of using the `[SysTestMethodAttribute]` attribute.

12.2 `assert` Methods

The `SysTest` framework provides a rich set of assertion methods for verifying test outcomes.

Core Assertion Methods

Method	Signature	Purpose	Example
`assertEquals`	`assertEquals(expected, actual, message)`	Verifies two values are equal	`assertEquals(5, result, 'Expected 5 records')`
`assertNotEqual`	`assertNotEqual(expected, actual, message)`	Verifies two values are not equal	`assertNotEqual(0, result, 'Result should not be zero')`
`assertGreater`	`assertGreater(actual, expected, message)`	Verifies actual > expected	`assertGreater(count, 0, 'Should have at least 1 record')`
`assertLess`	`assertLess(actual, expected, message)`	Verifies actual < expected	`assertLess(count, 100, 'Should have fewer than 100 records')`
`assertGreaterOrEqual`	`assertGreaterOrEqual(actual, expected, message)`	Verifies actual >= expected	`assertGreaterOrEqual(count, 1, 'Should have at least 1 record')`
`assertLessOrEqual`	`assertLessOrEqual(actual, expected, message)`	Verifies actual <= expected	`assertLessOrEqual(count, 10, 'Should have at most 10 records')`
`assertException`	`assertException(className::methodName, expectedException, message)`	Verifies a method throws an exception	`assertException(classStr(VendComplianceValidator), methodStr(VendComplianceValidator, validate), error::Error)`
`assertNull`	`assertNull(value, message)`	Verifies a value is null	`assertNull(result, 'Result should be null for invalid input')`
`assertNotNull`	`assertNotNull(value, message)`	Verifies a value is not null	`assertNotNull(result, 'Result should not be null')`
`assertTrue`	`assertTrue(condition, message)`	Verifies a boolean is true	`assertTrue(validationPassed, 'Validation should pass')`

Method	Signature	Purpose	Example
`assertFalse`	`assertFalse(condition, message)`	Verifies a boolean is false	`assertFalse(validationPassed, 'Validation should fail')`
`assertError`	`assertError(statement, message)`	Verifies a statement throws an error	`assertError(element.validateWrite(), 'Should fail for empty account')`
`assertWarn`	`assertWarn(statement, message)`	Verifies a statement produces a warning	`assertWarn(element.checkCredit(), 'Should warn for high credit')`
`assertInfo`	`assertInfo(statement, message)`	Verifies a statement produces an info message	`assertInfo(element.process(), 'Processing completed')`

Assertion Examples

```
[SysTestMethodAttribute]
public static void testComplianceCodeValidation()
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    boolean validationResult;

    // Arrange – create a test vendor
    vendTable.initValue();
    vendTable.AccountNum = 'TESTVENDOR01';
    vendTable.Name = 'Test Vendor';
    vendTable.CreditMax = 600000;
    vendTable.insert();

    // Act – validate the compliance code
    complianceCode = '';  // Empty compliance code
    validationResult = VendComplianceValidator::validate(vendTable, complianceCode);

    // Assert – validation should fail for high-credit vendor without compliance code
    assertFalse(validationResult, 'Validation should fail for vendor >$500K without compliance code');
}

[SysTestMethodAttribute]
public static void testComplianceCodeValidation_PassesForLowCredit()
{
    VendTable vendTable;
    ComplianceCode complianceCode;
    boolean validationResult;

    // Arrange – create a test vendor with low credit
    vendTable.initValue();
    vendTable.AccountNum = 'TESTVENDOR02';
    vendTable.Name = 'Test Vendor 2';
    vendTable.CreditMax = 100000;  // Below $500K threshold
    vendTable.insert();

    // Act – validate with empty compliance code
    complianceCode = '';
    validationResult = VendComplianceValidator::validate(vendTable, complianceCode);

    // Assert – validation should pass for low-credit vendor
```

```
    assertTrue(validationResult, 'Validation should pass for vendor <$500K without compliance code');
}

[SysTestMethodAttribute]
public static void testExceptionOnInvalidInput()
{
    // Assert — calling validate with null vendor should throw an exception
    assertException(
        classStr(VendComplianceValidator),
        methodStr(VendComplianceValidator, validate),
        error::Error,
        'Should throw error for null vendor');
}

[SysTestMethodAttribute]
public static void testRecordCount()
{
    int recordCount;

    // Act — count compliance records
    select count(RecId) from recordCount
        from APCustomsDeclaration;

    // Assert — should have at least 1 record from test setup
    assertGreater(recordCount, 0, 'Should have at least 1 compliance record');
}
```

12.3 Test Data Creation Patterns

Creating test data is one of the most important aspects of writing maintainable tests. The patterns below ensure tests are isolated, repeatable, and easy to understand.

Pattern 1: `construct()` Factory Method

The `construct()` factory method creates a test record with sensible defaults, which individual tests can then customize.

```
class TestVendTable
{
    public static VendTable construct()
    {
        VendTable vendTable;

        vendTable.initValue();
        vendTable.AccountNum = 'TESTVENDOR';
        vendTable.Name = 'Test Vendor';
        vendTable.CreditMax = 500000;
        vendTable.Currency = 'USD';
        vendTable.GroupName = 'VENDGROUP1';

        return vendTable;
    }
}
```

Usage in tests:

```
[SysTestMethodAttribute]
public static void testWithConstruct()
{
    VendTable vendTable = TestVendTable::construct();
    vendTable.insert();

    // Test logic using vendTable
    assertTrue(vendTable.AccountNum == 'TESTVENDOR', 'Account number should match');
}
```

Pattern 2: `newTest()` Factory Method

The `newTest()` method allows passing specific parameters to customize the test record.

```
class TestVendTable
{
    public static VendTable newTest(AccountNum _accountNum, Currency _currency, Real _creditMax)
    {
        VendTable vendTable;

        vendTable.initValue();
        vendTable.AccountNum = _accountNum;
        vendTable.Currency = _currency;
        vendTable.CreditMax = _creditMax;

        return vendTable;
    }
}
```

Usage in tests:

```
[SysTestMethodAttribute]
public static void testWithNewTest()
{
    VendTable vendTable = TestVendTable::newTest('VEND001', 'USD', 750000);
    vendTable.insert();

    // Test with high credit vendor
    assertFalse(VendComplianceValidator::validate(vendTable, ''), 'Should fail for high credit without
code');
}
```

Pattern 3: Shared Setup with `SysTestSetup`

When multiple test methods need the same base data, use `SysTestSetup` to create it once for the entire test class.

```
class VendComplianceTest extends SysTestCase
{
    // Shared test data — created once for all test methods
    public void setUp()
    {
        VendTable vendTable;

        // Create a high-credit vendor
        vendTable.initValue();
        vendTable.AccountNum = 'HIGHCREDIT01';
```

```
        vendTable.Name = 'High Credit Vendor';
        vendTable.CreditMax = 750000;
        vendTable.insert();

        // Create a low-credit vendor
        vendTable.initValue();
        vendTable.AccountNum = 'LOWCREDIT01';
        vendTable.Name = 'Low Credit Vendor';
        vendTable.CreditMax = 100000;
        vendTable.insert();
    }

    [SysTestMethodAttribute]
    public static void testHighCreditVendor()
    {
        VendTable vendTable = VendTable::find('HIGHCREDIT01');
        assertFalse(VendComplianceValidator::validate(vendTable, ''), 'High credit vendor needs compliance
code');
    }

    [SysTestMethodAttribute]
    public static void testLowCreditVendor()
    {
        VendTable vendTable = VendTable::find('LOWCREDIT01');
        assertTrue(VendComplianceValidator::validate(vendTable, ''), 'Low credit vendor does not need
compliance code');
    }

    public void tearDown()
    {
        // Delete test vendors
        VendTable vendTable;
        while select vendTable
            where vendTable.AccountNum like 'TEST%'
               || vendTable.AccountNum like 'HIGHCREDIT%'
               || vendTable.AccountNum like 'LOWCREDIT%'
        {
            vendTable.delete();
        }
    }
}
```

Pattern 4: Per-Method Setup (Isolated Tests)

For tests that need completely different data setups, create data within each test method. This provides maximum isolation but can be repetitive.

```
[SysTestMethodAttribute]
public static void testIsolatedSetup()
{
    VendTable vendTable;

    // Create data specific to this test
    vendTable.initValue();
    vendTable.AccountNum = 'ISOLATED01';
    vendTable.CreditMax = 600000;
    vendTable.insert();

    // Test logic
    assertFalse(VendComplianceValidator::validate(vendTable, ''), 'Should fail');
```

```
    // Clean up
    vendTable.delete();
}
```

Choosing Between Shared Setup and Per-Method Setup

Factor	Shared Setup (`SysTestSetup`)	Per-Method Setup
Speed	Faster — data created once	Slower — data created per test
Isolation	Lower — tests share data	Higher — each test is independent
Maintainability	Easier — one setup method	More repetitive — each test creates its own data
Test order dependency	Higher — tests may depend on shared state	Lower — each test is self-contained
Best for	Tests that read the same base data	Tests that modify or delete shared data

Best practice: Use shared setup for read-only test data. Use per-method setup for tests that modify or delete data.

12.4 Test Coverage Metrics in LCS

LCS provides built-in test coverage metrics that help you understand how much of your code is covered by automated tests.

Coverage Tab in LCS

The **Coverage** tab in LCS shows:

- **Line coverage**: Percentage of code lines executed during test runs
- **Method coverage**: Percentage of methods called during test runs
- **Class coverage**: Percentage of classes with at least one test method
- **Assembly coverage**: Percentage of assemblies covered by tests

Test Cases View

The **Test Cases** view in LCS provides:

- **Test case list**: All test methods organized by test class
- **Execution history**: When each test was last run and its result
- **Pass/Fail status**: Visual indicators of test outcomes
- **Duration**: How long each test takes to execute
- **Error details**: Full error messages and stack traces for failed tests

Interpreting Coverage Metrics

Coverage Level	Interpretation
> 80%	Excellent — most code paths are tested

Coverage Level	Interpretation
60–80%	Good — core logic is well covered
40–60%	Moderate — important paths may be untested
< 40%	Low — significant gaps in test coverage

Important: Coverage percentage alone is not a quality metric. A test that covers 80% of lines but doesn't test edge cases is less valuable than a test that covers 60% of lines with thorough edge case testing.

Best Practices for Test Coverage

1. **Test edge cases** — boundary values, null inputs, empty collections, maximum values
2. **Test failure paths** — not just the happy path, but also error conditions
3. **Test security** — verify that users without the right permissions cannot access data
4. **Test integration points** — verify that external calls and event handlers work correctly
5. **Run tests in CI/CD** — automate test execution as part of the build pipeline
6. **Review coverage reports** — identify untested code paths and add tests for critical paths

---

12.5 Integration Testing with LCS Test Case Management

LCS provides a **Test Case Management** feature that integrates with the `SysTest` framework for end-to-end integration testing.

Test Case Lifecycle

```
[1] Create test case in LCS
    |  - Define test steps, expected results, and test data
    |
[2] Associate test case with SysTest class
    |  - Map LCS test cases to SysTestMethod attributes
    |
[3] Execute tests in a test environment
    |  - Run tests manually or via CI/CD pipeline
    |
[4] Review results in LCS
    |  - Pass/Fail status, error details, screenshots
    |
[5] Promote to UAT and Production
    |  - Only promote when all test cases pass
```

Creating Test Cases in LCS

1. Navigate to **Test Management** in LCS
2. Create a **Test Plan** for the release
3. Add **Test Cases** to the plan:
4. **Test Case Name**: Descriptive name (e.g., "Validate compliance code for high-credit vendors")
5. **Test Steps**: Step-by-step instructions
6. **Expected Result**: What should happen
7. **Test Data**: Any data needed for the test
8. **Associated Test Method**: The `SysTestMethod` that implements this test
9. Assign test cases to test environments

Running Tests in LCS

Tests can be run in LCS through:

1. **Manual execution** — run tests from the LCS test case management UI
2. **CI/CD pipeline** — run tests automatically as part of the build pipeline
3. **Scheduled execution** — run tests on a schedule (e.g., nightly)

Test Execution in CI/CD Pipeline

```
# Azure DevOps pipeline step for running tests
- task: Dynamics365Test@1
  inputs:
    solution: '**/*.sln'
    testClasses: 'VendComplianceTest'
    configuration: 'Release'
    publishResults: true
```

When tests run in the CI/CD pipeline: - Test results are published to LCS - Failed tests block the deployment pipeline - Coverage metrics are updated in LCS - Test execution reports are available for audit

---

12.6 Activity — Write a Complete Test Class for Sales Order Discount Calculation

> **Activity**: Write a complete test class covering edge cases for sales order discount calculation. The discount calculation has the following rules: 1. Standard discount: 5% for orders over $10,000 2. VIP discount: 10% for VIP customers (CustGroup = 'VIP') 3. Volume discount: 15% for orders over $50,000 4. Combined discount: VIP + Volume discounts stack (VIP gets 10% + Volume gets 15% = 25% for orders over $50,000) 5. No discount for orders under $10,000 (unless VIP, which gets 10% regardless of order amount) 6. Maximum discount cap: 30% (no discount can exceed 30% of the order total)
>
> Write test methods covering: - Standard discount for orders over $10,000 - VIP discount for orders under $10,000 - VIP discount for orders over $50,000 (combined with volume) - Volume discount for non-VIP customers over $50,000 - No discount for orders under $10,000 for non-VIP customers - Maximum discount cap (30%) is enforced - Edge case: exactly $10,000 order (boundary value) - Edge case: exactly $50,000 order (boundary value) - Edge case: order with $0 total - Edge case: negative order total (error handling)
>
> **Activity Hints** (Multiple Valid Test Data Setup Approaches):
>
> - **Hint A — Shared setup vs. per-method setup**: Option A1 — use `SysTestSetup` to create shared customer records (VIP and non-VIP) and reuse them across tests (recommended — efficient, consistent). Option A2 — use per-method setup for complete test isolation (valid but slower). Option A3 — use a hybrid — shared setup for read-only data (customers), per-method setup for order-specific data (valid and practical).
> - **Hint B — Test data creation**: Option B1 — use a `TestCustTable` class with `construct()` and `newTest()` factory methods (recommended — clean, reusable). Option B2 — create test data inline in each test method (simple but repetitive). Option B3 — use a test data builder pattern with method chaining (most flexible but most complex).

> - **Hint C — Edge case testing**: Option C1 — test boundary values explicitly ($10,000, $50,000, $0, negative) (recommended — boundary value analysis is a standard testing technique). Option C2 — use parameterized tests with a data table (more elegant but requires more framework support). Option C3 — test only the happy path and rely on manual testing for edge cases (not recommended — automated tests should cover edge cases).
> - **Hint D — Assertion strategy**: Option D1 — use `assertEquals` for exact value comparisons and `assertTrue`/`assertFalse` for boolean conditions (recommended — clear and explicit). Option D2 — use `assertGreater`/`assertLess` for range-based assertions (valid for discount percentage checks). Option D3 — use `assertException` for error cases (necessary for negative order total).

Expected Approach (Ideal — in detail)

Test Data Setup with Shared `SysTestSetup`

```
class SalesOrderDiscountTest extends SysTestCase
{
    // Shared test data — created once for all test methods
    public void setUp()
    {
        CustTable custTable;

        // Create VIP customer
        custTable.initValue();
        custTable.AccountNum = 'VIP001';
        custTable.Name = 'VIP Customer';
        custTable.CustGroup = 'VIP';
        custTable.insert();

        // Create standard customer
        custTable.initValue();
        custTable.AccountNum = 'STD001';
        custTable.Name = 'Standard Customer';
        custTable.CustGroup = 'STD';
        custTable.insert();
    }

    [SysTestMethodAttribute]
    public static void testStandardDiscount_Over10K()
    {
        SalesTable salesTable;
        SalesLine salesLine;
        Real discountPercent;

        // Arrange — create a sales order over $10,000 for a standard customer
        salesTable = SalesTable::construct();
        salesTable.initValue();
        salesTable.CustomerAccount = 'STD001';
        salesTable.SalesDate = today();
        salesTable.insert();

        salesLine = SalesLine::construct();
        salesLine.initValue();
        salesLine.SalesTableId = salesTable.SalesId;
        salesLine.ItemId = 'ITEM001';
        salesLine.SalesQty = 100;
        salesLine.LineAmount = 15000;  // Over $10,000
```

```
        salesLine.insert();

        // Act – calculate discount
        discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

        // Assert – standard discount of 5%
        assertEquals(5, discountPercent, 'Standard discount should be 5% for orders over $10,000');

        // Cleanup
        salesLine.delete();
        salesTable.delete();
    }

    [SysTestMethodAttribute]
    public static void testVIPDiscount_Under10K()
    {
        SalesTable salesTable;
        SalesLine salesLine;
        Real discountPercent;

        // Arrange – create a sales order under $10,000 for a VIP customer
        salesTable = SalesTable::construct();
        salesTable.initValue();
        salesTable.CustomerAccount = 'VIP001';
        salesTable.SalesDate = today();
        salesTable.insert();

        salesLine = SalesLine::construct();
        salesLine.initValue();
        salesLine.SalesTableId = salesTable.SalesId;
        salesLine.ItemId = 'ITEM001';
        salesLine.SalesQty = 10;
        salesLine.LineAmount = 5000;  // Under $10,000
        salesLine.insert();

        // Act – calculate discount
        discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

        // Assert – VIP discount of 10% regardless of order amount
        assertEquals(10, discountPercent, 'VIP discount should be 10% even for orders under $10,000');

        // Cleanup
        salesLine.delete();
        salesTable.delete();
    }

    [SysTestMethodAttribute]
    public static void testVIPVolumeDiscount_Over50K()
    {
        SalesTable salesTable;
        SalesLine salesLine;
        Real discountPercent;

        // Arrange – create a sales order over $50,000 for a VIP customer
        salesTable = SalesTable::construct();
        salesTable.initValue();
        salesTable.CustomerAccount = 'VIP001';
        salesTable.SalesDate = today();
        salesTable.insert();

        salesLine = SalesLine::construct();
        salesLine.initValue();
```

```
        salesLine.SalesTableId = salesTable.SalesId;
        salesLine.ItemId = 'ITEM001';
        salesLine.SalesQty = 500;
        salesLine.LineAmount = 60000;  // Over $50,000
        salesLine.insert();

        // Act — calculate discount
        discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

        // Assert — combined VIP (10%) + Volume (15%) = 25%
        assertEquals(25, discountPercent, 'VIP + Volume discount should be 25% for orders over $50,000');

        // Cleanup
        salesLine.delete();
        salesTable.delete();
    }

    [SysTestMethodAttribute]
    public static void testVolumeDiscount_NonVIP_Over50K()
    {
        SalesTable salesTable;
        SalesLine salesLine;
        Real discountPercent;

        // Arrange — create a sales order over $50,000 for a standard customer
        salesTable = SalesTable::construct();
        salesTable.initValue();
        salesTable.CustomerAccount = 'STD001';
        salesTable.SalesDate = today();
        salesTable.insert();

        salesLine = SalesLine::construct();
        salesLine.initValue();
        salesLine.SalesTableId = salesTable.SalesId;
        salesLine.ItemId = 'ITEM001';
        salesLine.SalesQty = 500;
        salesLine.LineAmount = 60000;  // Over $50,000
        salesLine.insert();

        // Act — calculate discount
        discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

        // Assert — volume discount of 15% only (no VIP)
        assertEquals(15, discountPercent, 'Volume discount should be 15% for non-VIP orders over
$50,000');

        // Cleanup
        salesLine.delete();
        salesTable.delete();
    }

    [SysTestMethodAttribute]
    public static void testNoDiscount_Under10K_NonVIP()
    {
        SalesTable salesTable;
        SalesLine salesLine;
        Real discountPercent;

        // Arrange — create a sales order under $10,000 for a standard customer
        salesTable = SalesTable::construct();
        salesTable.initValue();
        salesTable.CustomerAccount = 'STD001';
```

```
        salesTable.SalesDate = today();
        salesTable.insert();

        salesLine = SalesLine::construct();
        salesLine.initValue();
        salesLine.SalesTableId = salesTable.SalesId;
        salesLine.ItemId = 'ITEM001';
        salesLine.SalesQty = 5;
        salesLine.LineAmount = 5000;  // Under $10,000
        salesLine.insert();

        // Act – calculate discount
        discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

        // Assert – no discount for non-VIP orders under $10,000
        assertEquals(0, discountPercent, 'No discount for non-VIP orders under $10,000');

        // Cleanup
        salesLine.delete();
        salesTable.delete();
    }

    [SysTestMethodAttribute]
    public static void testMaxDiscountCap_30Percent()
    {
        SalesTable salesTable;
        SalesLine salesLine;
        Real discountPercent;

        // Arrange – create a sales order that would exceed 30% discount
        // VIP (10%) + Volume (15%) + Special (10%) = 35% but capped at 30%
        salesTable = SalesTable::construct();
        salesTable.initValue();
        salesTable.CustomerAccount = 'VIP001';
        salesTable.SalesDate = today();
        salesTable.insert();

        salesLine = SalesLine::construct();
        salesLine.initValue();
        salesLine.SalesTableId = salesTable.SalesId;
        salesLine.ItemId = 'ITEM001';
        salesLine.SalesQty = 1000;
        salesLine.LineAmount = 100000;  // Over $50,000
        salesLine.insert();

        // Act – calculate discount
        discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

        // Assert – discount capped at 30%
        assertLessOrEqual(discountPercent, 30, 'Discount should not exceed 30%');
        assertEquals(30, discountPercent, 'Discount should be capped at 30%');

        // Cleanup
        salesLine.delete();
        salesTable.delete();
    }

    [SysTestMethodAttribute]
    public static void testBoundaryValue_Exactly10K()
    {
        SalesTable salesTable;
        SalesLine salesLine;
```

```
    Real discountPercent;

    // Arrange – create a sales order exactly at $10,000
    salesTable = SalesTable::construct();
    salesTable.initValue();
    salesTable.CustomerAccount = 'STD001';
    salesTable.SalesDate = today();
    salesTable.insert();

    salesLine = SalesLine::construct();
    salesLine.initValue();
    salesLine.SalesTableId = salesTable.SalesId;
    salesLine.ItemId = 'ITEM001';
    salesLine.SalesQty = 100;
    salesLine.LineAmount = 10000;  // Exactly $10,000
    salesLine.insert();

    // Act – calculate discount
    discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

    // Assert – standard discount of 5% applies at exactly $10,000 (boundary)
    assertEquals(5, discountPercent, 'Standard discount should apply at exactly $10,000');

    // Cleanup
    salesLine.delete();
    salesTable.delete();
}

[SysTestMethodAttribute]
public static void testBoundaryValue_Exactly50K()
{
    SalesTable salesTable;
    SalesLine salesLine;
    Real discountPercent;

    // Arrange – create a sales order exactly at $50,000
    salesTable = SalesTable::construct();
    salesTable.initValue();
    salesTable.CustomerAccount = 'STD001';
    salesTable.SalesDate = today();
    salesTable.insert();

    salesLine = SalesLine::construct();
    salesLine.initValue();
    salesLine.SalesTableId = salesTable.SalesId;
    salesLine.ItemId = 'ITEM001';
    salesLine.SalesQty = 500;
    salesLine.LineAmount = 50000;  // Exactly $50,000
    salesLine.insert();

    // Act – calculate discount
    discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

    // Assert – volume discount of 15% applies at exactly $50,000 (boundary)
    assertEquals(15, discountPercent, 'Volume discount should apply at exactly $50,000');

    // Cleanup
    salesLine.delete();
    salesTable.delete();
}

[SysTestMethodAttribute]
```

```
public static void testZeroOrderTotal()
{
    SalesTable salesTable;
    SalesLine salesLine;
    Real discountPercent;

    // Arrange – create a sales order with $0 total
    salesTable = SalesTable::construct();
    salesTable.initValue();
    salesTable.CustomerAccount = 'STD001';
    salesTable.SalesDate = today();
    salesTable.insert();

    salesLine = SalesLine::construct();
    salesLine.initValue();
    salesLine.SalesTableId = salesTable.SalesId;
    salesLine.ItemId = 'ITEM001';
    salesLine.SalesQty = 0;
    salesLine.LineAmount = 0;  // Zero total
    salesLine.insert();

    // Act – calculate discount
    discountPercent = SalesOrderDiscount::calculateDiscount(salesTable);

    // Assert – no discount for $0 order
    assertEquals(0, discountPercent, 'No discount for $0 order');

    // Cleanup
    salesLine.delete();
    salesTable.delete();
}

[SysTestMethodAttribute]
public static void testNegativeOrderTotal_ThrowsError()
{
    SalesTable salesTable;
    SalesLine salesLine;

    // Arrange – create a sales order with negative total
    salesTable = SalesTable::construct();
    salesTable.initValue();
    salesTable.CustomerAccount = 'STD001';
    salesTable.SalesDate = today();
    salesTable.insert();

    salesLine = SalesLine::construct();
    salesLine.initValue();
    salesLine.SalesTableId = salesTable.SalesId;
    salesLine.ItemId = 'ITEM001';
    salesLine.SalesQty = 1;
    salesLine.LineAmount = -5000;  // Negative total
    salesLine.insert();

    // Assert – should throw an error for negative order total
    assertException(
        classStr(SalesOrderDiscount),
        methodStr(SalesOrderDiscount, calculateDiscount),
        error::Error,
        'Should throw error for negative order total');

    // Cleanup
    salesLine.delete();
```

```
        salesTable.delete();
    }

    public void tearDown()
    {
        // Clean up all test data
        SalesLine salesLine;
        SalesTable salesTable;
        CustTable custTable;

        while select salesLine
            where salesLine.ItemId == 'ITEM001'
        {
            salesLine.delete();
        }

        while select salesTable
            where salesTable.CustomerAccount like 'VIP%'
               || salesTable.CustomerAccount like 'STD%'
        {
            salesTable.delete();
        }

        while select custTable
            where custTable.AccountNum like 'VIP%'
               || custTable.AccountNum like 'STD%'
        {
            custTable.delete();
        }
    }
}
```

Design Rationale

1. **Shared setup** creates VIP and standard customers once — all test methods reuse them
2. **Per-method cleanup** ensures each test leaves no residual data that could affect other tests
3. **Boundary value tests** ($10,000 exactly, $50,000 exactly) verify the discount thresholds work correctly at the boundaries
4. **Edge case tests** ($0 total, negative total) verify the system handles unusual inputs gracefully
5. **Combined discount test** (VIP + Volume) verifies that discounts stack correctly
6. **Maximum cap test** verifies that the 30% discount cap is enforced
7. **Error case test** uses `assertException` to verify that negative order totals are rejected

Model Manifest Dependencies

```
<ModelManifest>
  <Name>AcmeSalesOrderDiscountTests</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <References>
    <ModelReference><Name>ApplicationSuite</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
    <ModelReference><Name>ApplicationFoundation</Name><MinVersion>10.0.0.0</MinVersion></ModelReference>
  </References>
  <ConfigurationKey>AcmeSalesOrderDiscountTestsEnabled</ConfigurationKey>
</ModelManifest>
```

Remaining Chapters — Detailed Outlines

Chapter 13 — Deployment, DevOps & LCS

13.1 Lifecycle Services (LCS) — Deep Dive

Lifecycle Services (LCS) is the central hub for managing the D365 F&O application lifecycle. It's a cloud-based portal that orchestrates everything from environment provisioning to deployment and monitoring.

LCS Environment Tiers

Tier	Purpose	VM Count	Typical Use
Development	Active development and testing	1-2 AOS nodes	Developer work, feature development, unit testing
Test/QA	Integration and regression testing	2 AOS nodes (HA)	Automated testing, UAT, integration testing
Staging/Pre-Production	Final validation before production	2 AOS nodes (HA)	Performance testing, user acceptance, go-live validation
Production	Live, customer-facing environment	2+ AOS nodes (HA)	Day-to-day business operations
Demo	Demonstrations and training	1 AOS node	Sales demos, training sessions

LCS Project Structure

Every LCS project corresponds to a D365 F&O deployment. The project contains:

- **Environments**: All the VMs and services for the deployment
- **Deployment Pipelines**: The CI/CD pipeline configuration
- **Release Gates**: Approval checkpoints between environments
- **Build Artifacts**: NuGet packages and deployable packages
- **Monitoring**: SLA tracking, performance metrics, alerting
- **Application Lifecycle**: Version history, deployment history, incident tracking

Environment Provisioning

Environments are provisioned through LCS and deployed to your Azure subscription:

1. **Create the project** in LCS with your Azure subscription details
2. **Select the topology** (Demo, Dev/Test, HA Production)
3. **Choose VM sizes** for each node type (AOS, Batch, Report, etc.)
4. **Configure networking** (VNets, subnets, NSGs)
5. **LCS provisions the VMs** and installs the D365 F&O binaries
6. **Initial sync** pulls the base application code from Microsoft

Release Gates

Release gates control the flow of deployments between environments:

```
Dev → Test Gate → Staging Gate → Production Gate
  |         |           |              |
  v         v           v              v
 Deploy    Automated   Manual         Manual
 to Test   Test Run    Approval       Approval
```

Gate Types:

Gate Type	Description	When It Applies
Automated	Deployment proceeds without human intervention	Dev → Test (for build validation)
Manual Approval	A designated approver must approve before deployment proceeds	Test → Staging, Staging → Production
Scheduled	Deployment occurs at a predetermined time	Production deployments during maintenance windows

Deployment Pipelines in LCS

LCS integrates with Azure DevOps for deployment orchestration:

1. **Build Pipeline** (Azure DevOps): Compiles the model project, creates a deployable package
2. **Release Pipeline** (Azure DevOps): Deploys the package to LCS environments
3. **LCS Deployment Orchestration**: LCS manages the actual deployment to the AOS nodes
4. **Post-Deployment Validation**: LCS runs health checks after deployment

---

13.2 Azure DevOps / GitHub Integration

Build Pipeline (YAML)

The build pipeline compiles the X++ code, runs best practice checks, and creates a deployable package.

Complete Build Pipeline YAML:

```yaml
trigger:
  branches:
    include:
      - main
      - release/*

pool:
  vmImage: 'windows-latest'

variables:
  - group: 'd36fo-build-variables'
  - name: buildPlatform
    value: 'Any CPU'
  - name: buildConfiguration
    value: 'Release'

steps:
  # Step 1: Install NuGet packages
  - task: NuGetCommand@2
    displayName: 'Restore NuGet packages'
```

```
    inputs:
      command: 'restore'
      feedsToUse: 'config'
      nugetConfigPath: 'nuget.config'

  # Step 2: Update model versions
  - task: PowerShell@2
    displayName: 'Update model versions'
    inputs:
      targetType: 'filePath'
      filePath: '$(Build.SourcesDirectory)/build/UpdateModelVersions.ps1'
      arguments: '-version $(Build.BuildNumber)'

  # Step 3: Build the solution
  - task: MSBuild@1
    displayName: 'Build D365 F&O solution'
    inputs:
      solution: '**/*.sln'
      platform: '$(buildPlatform)'
      configuration: '$(buildConfiguration)'
      msbuildArguments: >

/p:BuildTasksDirectory="$(Pipeline.Workspace)\NuGets\Microsoft.Dynamics.AX.Platform.CompilerPackage\DevAlm
"
        /p:MetadataDirectory="$(Build.SourcesDirectory)\Metadata"

/p:FrameworkDirectory="$(Pipeline.Workspace)\NuGets\Microsoft.Dynamics.AX.Platform.CompilerPackage"
        /p:ReferenceFolder="$(Pipeline.Workspace)\NuGets"
        /p:OutputDirectory="$(Build.BinariesDirectory)"

  # Step 4: Install NuGet 3.3.0 (required for deployable package generation)
  - task: NuGetCommand@2
    displayName: 'Install NuGet 3.3.0'
    inputs:
      command: 'install'
      feedsToUse: 'config'
      nugetConfigPath: 'nuget.config'
      versioning: 'specific'
      version: '3.3.0'

  # Step 5: Create deployable package
  - task: PowerShell@2
    displayName: 'Create deployable package'
    inputs:
      targetType: 'filePath'
      filePath: '$(Build.SourcesDirectory)/build/CreateDeployablePackage.ps1'
      arguments: '-outputPath "$(Build.ArtifactStagingDirectory)"'

  # Step 6: Publish the deployable package
  - task: PublishBuildArtifacts@1
    displayName: 'Publish deployable package'
    inputs:
      PathtoPublish: '$(Build.ArtifactStagingDirectory)'
      ArtifactName: 'drop'
      publishLocation: 'Container'
```

Release Pipeline (YAML)

The release pipeline deploys the package to LCS environments:

```yaml
trigger: none

pr: none

pool:
  vmImage: 'windows-latest'

variables:
  - group: 'd36fo-release-variables'

stages:
  - stage: DeployToTest
    displayName: 'Deploy to Test'
    jobs:
      - deployment: DeployTest
        environment: 'Test'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: PowerShell@2
                  displayName: 'Deploy package to LCS'
                  inputs:
                    targetType: 'filePath'
                    filePath: '$(Pipeline.Workspace)/build/DeployToLCS.ps1'
                    arguments: >
                      -lcsProjectId "$(LcsProjectId)"
                      -environmentName "Test"
                      -packagePath "$(Pipeline.Workspace)/drop/*.zip"
                      -lcsToken "$(LcsAccessToken)"

  - stage: DeployToStaging
    displayName: 'Deploy to Staging'
    dependsOn: DeployToTest
    # Manual approval gate configured in Azure DevOps
    jobs:
      - deployment: DeployStaging
        environment: 'Staging'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: PowerShell@2
                  displayName: 'Deploy package to LCS'
                  inputs:
                    targetType: 'filePath'
                    filePath: '$(Pipeline.Workspace)/build/DeployToLCS.ps1'
                    arguments: >
                      -lcsProjectId "$(LcsProjectId)"
                      -environmentName "Staging"
                      -packagePath "$(Pipeline.Workspace)/drop/*.zip"
                      -lcsToken "$(LcsAccessToken)"

  - stage: DeployToProduction
    displayName: 'Deploy to Production'
    dependsOn: DeployToStaging
    # Manual approval gate configured in Azure DevOps
    jobs:
      - deployment: DeployProduction
        environment: 'Production'
        strategy:
          runOnce:
```

```
            deploy:
              steps:
                - task: PowerShell@2
                  displayName: 'Deploy package to LCS'
                  inputs:
                    targetType: 'filePath'
                    filePath: '$(Pipeline.Workspace)/build/DeployToLCS.ps1'
                    arguments: >
                      -lcsProjectId "$(LcsProjectId)"
                      -environmentName "Production"
                      -packagePath "$(Pipeline.Workspace)/drop/*.zip"
                      -lcsToken "$(LcsAccessToken)"
```

Agent Pools

Agent Pool	OS	Use Case
`windows-latest`	Windows Server 2022	Build agents (compilation, packaging)
`ubuntu-latest`	Ubuntu 20.04	Not used for D365 F&O builds (Windows required)
Self-hosted	Windows Server	On-premises builds with direct AOS access

Key Point: Hosted build automation supports compilation and packaging only — no X++ unit testing (SysTest), database sync, or AOS-dependent features. For those, you need a self-hosted agent or a dedicated test environment.

13.3 Model Management

Model Manifest (`ModelManifest.xml`)

The `ModelManifest.xml` file defines the model's identity, dependencies, and configuration:

```
<?xml version="1.0" encoding="utf-8"?>
<ModelManifest>
  <Name>AcmeOrderToCash</Name>
  <Version>1.0.0.0</Version>
  <Layer>CUS</Layer>
  <Description>Acme Order-to-Cash extension model</Description>
  <References>
    <ModelReference>
      <Name>ApplicationSuite</Name>
      <MinVersion>10.0.0.0</MinVersion>
    </ModelReference>
    <ModelReference>
      <Name>ApplicationFoundation</Name>
      <MinVersion>10.0.0.0</MinVersion>
    </ModelReference>
    <ModelReference>
      <Name>ApplicationPlatform</Name>
      <MinVersion>10.0.0.0</MinVersion>
    </ModelReference>
  </References>
  <ConfigurationKey>AcmeOrderToCashEnabled</ConfigurationKey>
  <GeneratedBy>
```

```
    <Tool>Visual Studio</Tool>
    <Version>17.8.0</Version>
  </GeneratedBy>
</ModelManifest>
```

Model Dependency Order

The model dependency order determines the **layer order** — which model's code takes precedence when there are conflicts:

```
Layer Order (highest → lowest):
  CUS (Customer) – Your customizations
  VAR (VAR/Partner) – Partner solutions
  ISV (ISV) – Independent software vendor solutions
  CUS (Custom) – Microsoft customizations (rarely used)
  ApplicationSuite – Application business logic
  ApplicationFoundation – Framework and shared services
  ApplicationPlatform – Core platform (interfaces, kernel)
```

Critical Rule: A model can only depend on models in lower layers. Circular dependencies are not allowed and will cause compilation errors.

Model Versioning

Models use **semantic versioning** (`Major.Minor.Patch.Build`):

Version Component	When to Increment	Example
Major	Breaking changes, major feature releases	`2.0.0.0`
Minor	New features, backward-compatible additions	`1.1.0.0`
Patch	Bug fixes, backward-compatible patches	`1.0.1.0`
Build	Internal build number, CI/CD increment	`1.0.0.1234`

`AxModelStore` — Model Management API

The `AxModelStore` class provides programmatic access to model management operations:

```
// Get the model store instance
AxModelStore modelStore = AxModelStore::getModelStore();

// Get a model by name
AxModel model = modelStore->getModel('AcmeOrderToCash');

// Get all models in the store
AxModelCollection models = modelStore->getModels();

// Check model dependencies
AxModelDependencyCollection dependencies = model->getDependencies();
for (int i = 0; i < dependencies->size(); i++)
{
    AxModelDependency dependency = dependencies->get(i);
```

```
    info(strFmt('Depends on: %1 (v%2)',
        dependency->getName(),
        dependency->getVersion()));
}
```

13.4 Configuration Keys

Configuration keys control which features and modules are enabled in the D365 F&O instance. They act as feature flags at the model level.

Standard Configuration Keys

Key	Purpose	Default
`#ISO`	International Organization for Standardization features	Enabled
`#USMF`	United States Mexico Federal (demo company)	Enabled
`#CER`	Corporate Enterprise features	Disabled
`#RTI`	Real-Time Integration features	Disabled
`#GR`	General Ledger features	Enabled
`#AP`	Accounts Payable features	Enabled
`#AR`	Accounts Receivable features	Enabled
`#IN`	Inventory Management features	Enabled
`#PM`	Project Management features	Disabled

Custom Configuration Keys

When you create custom features, you should define a custom Configuration Key:

Step 1: Create the Configuration Key in the AOT

1. In Visual Studio, right-click your model → **New** → **Configuration Key**
2. Set the **Name** (e.g., `AcmeOrderToCashEnabled`)
3. Set the **Label** (e.g., "Acme Order-to-Cash Module")
4. Set the **Configuration Key Group** (e.g., "Order Management")

Step 2: Reference the Configuration Key in Code

```
// Check if the feature is enabled before executing custom logic
if (isConfigurationKeySet(configurationKeyNum(AcmeOrderToCashEnabled)))
{
    // Custom Order-to-Cash logic here
    this.processOrderToCash();
}
else
{
    info('Acme Order-to-Cash module is not enabled.');
}
```

Step 3: Reference the Configuration Key in Table Properties

On any custom table, set the **Configuration Key** property to your custom key. This ensures the table is only created in databases where the feature is enabled.

Configuration Key Best Practices

1. **Group related features** under a single configuration key (e.g., `AcmeComplianceEnabled` for all compliance features)
2. **Use descriptive names** that clearly indicate what the key controls
3. **Document every key** in the model's README with its purpose and default state
4. **Test with keys both enabled and disabled** to ensure graceful degradation
5. **Never hard-code feature flags** — always use `isConfigurationKeySet()` for runtime checks

---

13.5 Hotfix Deployment Strategy

Hotfixes (CUs — Cumulative Updates) are Microsoft's mechanism for delivering fixes and updates to D365 F&O. Understanding how they are applied is critical for maintaining a stable production environment.

The `.axupdate` File

Hotfixes are distributed as `.axupdate` files (also called application update packages). Each `.axupdate` file contains:

- **Metadata**: The update version, target model versions, and dependencies
- **X++ Code**: The updated X++ source files
- **Metadata**: Updated table definitions, forms, reports, etc.
- **Database Scripts**: SQL scripts for schema changes

Application Order

Hotfixes must be applied in a specific order:

```
1. Platform Update (PU) — Updates the Application Platform layer
2. Application Update 1 (AU1) — Updates Application 1
3. Application Update 2 (AU2) — Updates Application 2
4. Application Suite Update (ASU) — Updates Application Suite
5. Custom Model Updates — Your custom models (if applicable)
```

Critical Rule: You must apply updates in dependency order. A Platform Update must be applied before any Application Update that depends on it.

Dependency Ordering

```
ApplicationPlatform (PU)
    +-- ApplicationFoundation (AU1)
            +-- ApplicationSuite (AU2)
                    +-- Custom Models (CUS)
```

Hotfix Deployment Process

1. **Download the `.axupdate` file** from LCS (Life Cycle Services)
2. **Validate the update** — check version compatibility and dependencies
3. **Backup the environment** — take a snapshot of the current state
4. **Apply the update** through LCS deployment orchestration
5. **Run database sync** — synchronize the database schema with the updated metadata
6. **Compile the model** — ensure all code compiles successfully
7. **Run post-deployment validation** — verify the update was applied correctly
8. **Monitor for issues** — check LCS monitoring dashboards and Infolog

Rollback Strategy

If a hotfix causes issues:

1. **Identify the issue** — check LCS monitoring, Infolog, and batch job history
2. **Revert the update** — LCS supports rollback to the previous package version
3. **Restore from backup** if the rollback fails
4. **Document the issue** — file a support ticket with Microsoft

---

13.6 CI/CD Pipeline Design — Complete Pattern

The Full Pipeline

```
[Source Control] → [Build] → [Test] → [Deploy to Dev] → [Deploy to Test] → [Deploy to Staging] → [Deploy
to Production]
     |                 |          |          |               |                |                |
  Git/TFS         Compile     SysTest     Deploy          Automated       Manual           Manual
                  + PB      (if             package         test run        approval         approval
                  check     supported)      deploy          + regression    gate             gate
```

Pipeline Stages in Detail

Stage 1: Build

Step	Action	Success Criteria
1.1	Restore NuGet packages	All 5 packages restored successfully
1.2	Update model versions	Version incremented to build number
1.3	Compile the solution	Zero compilation errors
1.4	Run best practice checks	No critical best practice violations
1.5	Create deployable package	`.zip` package created and published

Stage 2: Test

Step	Action	Success Criteria
2.1	Deploy package to test environment	Deployment successful
2.2	Run database sync	No sync errors

Step	Action	Success Criteria
2.3	Run SysTest unit tests	All tests pass (or acceptable failure rate)
2.4	Run regression test suite (RSAT)	No regressions detected
2.5	Validate custom functionality	Manual verification of key scenarios

Stage 3: Deploy to Staging

Step	Action	Success Criteria
3.1	Manual approval gate	Approved by release manager
3.2	Deploy package to staging	Deployment successful
3.3	Run performance tests	Response times within acceptable thresholds
3.4	Run user acceptance testing	UAT sign-off obtained
3.5	Validate security roles	Permission simulation passes for all roles

Stage 4: Deploy to Production

Step	Action	Success Criteria
4.1	Manual approval gate	Approved by release manager and business owner
4.2	Schedule deployment window	Outside business hours
4.3	Deploy package to production	Deployment successful
4.4	Run database sync	No sync errors
4.5	Run post-deployment validation	All health checks pass
4.6	Monitor for issues	No critical errors in Infolog or LCS

Rollback Strategy

Scenario	Rollback Action	Time to Rollback
Deployment fails at build stage	No rollback needed — pipeline stops	N/A
Deployment fails at test stage	Redeploy previous package to test	< 15 minutes
Deployment fails at staging	Redeploy previous package to staging	< 15 minutes
Deployment fails at production	Redeploy previous package to production	< 30 minutes
Issue discovered after production deployment	Rollback to previous package via LCS	< 1 hour

Activity — Design a Complete CI/CD Pipeline

> **Activity**: Design a complete CI/CD pipeline for a D365 F&O project that includes: 1. Build automation with Azure DevOps YAML pipelines 2. Test execution (SysTest unit tests + RSAT regression tests) 3. Configuration key documentation for all custom features 4. Release gates between environments 5. Rollback strategy for each environment
>
> **Requirements:** - The pipeline must support dev → test → staging → production flow - Each environment must have appropriate approval gates - The build pipeline must produce a deployable package - Test execution must include both automated and manual steps - Rollback must be documented for each environment
>
> **Activity Hints:** - **Hint A — Pipeline tool choice**: Option A1 — Azure DevOps with YAML pipelines (recommended — industry standard, version-controlled pipeline definition). Option A2 — GitHub Actions with workflow YAML (valid — simpler for smaller teams). Option A3 — Azure DevOps with classic designer pipelines (valid but less maintainable — no version control for pipeline definition). - **Hint B — Test execution strategy**: Option B1 — Run SysTest in the build pipeline (ideal but not supported by hosted agents — requires self-hosted agents). Option B2 — Run RSAT regression tests in the test environment deployment stage (recommended — works with hosted agents). Option B3 — Manual test execution with test plans in Azure DevOps Test Plans (valid for smaller teams without automated test infrastructure). - **Hint C — Rollback approach**: Option C1 — Redeploy the previous package via LCS (recommended — clean and reliable). Option C2 — Database restore from backup (nuclear option — use only if redeployment fails). Option C3 — Feature flags via ConfigurationKeys to disable problematic features without rollback (complementary approach — not a replacement for redeployment).

13.7 Model Deployment with `AxModelStore`

Deploying Models Programmatically

You can deploy models programmatically using the `AxModelStore` API:

```
// Deploy a model to the local AOS
AxModelStore modelStore = AxModelStore::getModelStore();

// Get the model to deploy
AxModel model = modelStore->getModel('AcmeOrderToCash');

// Check if the model is already deployed
if (model->getDeploymentState() == AxModelDeploymentState::Deployed)
{
    info('Model is already deployed.');
}
else
{
    // Deploy the model
    AxModelDeploymentResult result = model->deploy();

    if (result->getStatus() == AxModelDeploymentStatus::Success)
    {
        info('Model deployed successfully.');
    }
    else
    {
        error(strFmt('Model deployment failed: %1', result->getErrorMessage()));
```

```
    }
}
```

Synchronizing the Database After Deployment

After deploying a model, you must synchronize the database to create or update the corresponding database tables:

```
// Sync the database after model deployment
SysSetupStorage::syncDatabase();

// Or sync a specific table
SysTableSync::syncTable(tableNum(CustTable));
```

Checking Model Compilation Status

```
// Check if a model compiles successfully
AxModelBuildResult buildResult = model->build();

if (buildResult->getStatus() == AxModelBuildStatus::Success)
{
    info('Model compiled successfully.');
}
else
{
    // Get compilation errors
    AxModelBuildErrorCollection errors = buildResult->getErrors();
    for (int i = 0; i < errors->size(); i++)
    {
        AxModelBuildError error = errors->get(i);
        error(strFmt('Error in %1: %2', error->getSource(), error->getMessage()));
    }
}
```

13.8 Activity — Complete Deployment Strategy

> **Activity**: Design a complete deployment strategy for a D365 F&O project that includes: 1. LCS project setup with environment tiers 2. Azure DevOps build and release pipelines 3. Model versioning and dependency management 4. Configuration key documentation 5. Hotfix application process 6. Rollback procedures for each environment
>
> **Activity Hints:** - **Hint A — Environment topology**: Option A1 — Dev + Test + Staging + Production (recommended — follows Microsoft's recommended topology). Option A2 — Dev + Test + Production (simpler — valid for smaller deployments). Option A3 — Dev + Test + Staging + Pre-Production + Production (most granular — useful for large enterprise deployments with strict change management). - **Hint B — Deployment frequency**: Option B1 — Weekly deployments to Test, monthly to Production (recommended — balances velocity with stability). Option B2 — Daily deployments to Test, weekly to Production (faster cadence — requires robust automated testing). Option B3 — Monthly deployments to all environments (conservative — suitable for regulated industries). - **Hint C — Hotfix handling**: Option C1 — Apply hotfixes to Test first, then promote to Production after validation (recommended — follows the same path as custom code). Option C2 — Apply hotfixes directly to Production (Microsoft's recommended approach for critical security fixes

> — but skip validation). Option C3 — Apply hotfixes to Staging first, then Test, then Production (most conservative — useful for compliance-heavy environments).

Chapter 14 — Performance Tuning & Troubleshooting

14.1 SQL Query Optimization

The `exists join` vs `join` Decision

One of the most impactful performance optimizations in X++ is choosing the right join type:

```
// [ ] SLOW — join retrieves all columns from both tables
CustTable custTable;
VendTable vendTable;

while select custTable
    join vendTable
    where vendTable.AccountNum == custTable.AccountNum
{
    // Process each matching pair
}

// [x] FAST — exists join only checks for existence, doesn't retrieve data
CustTable custTable;
VendTable vendTable;

while select custTable
    exists join vendTable
    where vendTable.AccountNum == custTable.AccountNum
{
    // Process each customer that has a matching vendor
}
```

When to Use Each Join Type

Join Type	Use When	Performance
`join`	You need columns from both tables	Moderate — retrieves all joined data
`exists join`	You only need to check if related records exist	Fast — stops at first match
`noexists join`	You need records that have NO related records	Fast — inverse of exists join
`cross join`	You need every combination of rows	Very slow — avoid in production code
`outer join`	You need all records from one table even without matches	Moderate — retrieves unmatched rows

`RecordCount` Performance

The `recordCount()` method can be expensive on large tables:

```
// [ ] SLOW — recordCount() executes a full COUNT query
CustTable custTable;
int count = custTable.recordCount();  // Full table scan
```

```
// [x] FAST — use a select with countRecId()
CustTable custTable;
int count;

select countRecId(custTable);  // Uses SQL COUNT(RecId) — optimized
```

`countRecId` VS `recordCount`

Method	SQL Generated	Performance	Use When
`recordCount()`	`SELECT COUNT(*) FROM table`	Slow on large tables — full scan	Small tables only
`countRecId()`	`SELECT COUNT(RecId) FROM table`	Fast — uses clustered index	Large tables
`select countRecId(table)`	`SELECT COUNT(RecId) FROM table`	Fastest — optimized by SQL Server	Best practice for counting

Implicit Conversion Performance Trap

One of the most common performance issues is implicit data type conversion, which prevents SQL Server from using indexes:

```
// [ ] SLOW — str to int implicit conversion prevents index usage
CustTable custTable;

while select custTable
    where custTable.AccountNum == strFmt("%1", someIntValue)
// SQL Server must convert AccountNum (string) to int for each row — no index usage

// [x] FAST — explicit type matching allows index usage
CustTable custTable;
str accountNum = strFmt("%1", someIntValue);

while select custTable
    where custTable.AccountNum == accountNum;
// SQL Server uses the index on AccountNum directly
```

14.2 AOT Performance Anti-Patterns

Anti-Pattern 1: Heavy `init()` Method

The `init()` method runs every time a form is opened. Putting heavy queries or data retrieval in `init()` causes slow form loads:

```
// [ ] ANTI-PATTERN — Heavy query in init()
public void init()
{
    super();

    // This query runs EVERY time the form is opened
    // Even when the user is just viewing a single record
    CustTable custTable;
```

```
    while select custTable
        where custTable.CreditMax > 100000
    {
        // Processing logic that runs on every form open
        this.processCustomer(custTable);
    }
}

// [x] CORRECT — Move heavy processing to a button or event
public void clicked()
{
    super();
    // Heavy processing only runs when the user clicks the button
    this.processHighCreditCustomers();
}
```

Anti-Pattern 2: Complex Lookups in `active()`

The `active()` method runs every time a form record changes. Complex lookups in `active()` cause the form to feel sluggish:

```
// [ ] ANTI-PATTERN — Complex lookup in active()
public void active()
{
    super();

    // This lookup runs every time the user moves to a different record
    // and makes the form feel slow
    CustTable custTable = element.args().record();
    SysTableLookup sysTableLookup = SysTableLookup::newParameters(tableNum(CustTable), custTableControl);
    sysTableLookup.addLookupField(fieldNum(CustTable, AccountNum));
    sysTableLookup.addLookupField(fieldNum(CustTable, Name));
    sysTableLookup.addLookupField(fieldNum(CustTable, CreditMax));
    sysTableLookup.performFormLookup();
}

// [x] CORRECT — Use a simple lookup or set the lookup at design time
// The lookup should be set on the control's Lookup method, not in active()
```

Anti-Pattern 3: Recursive `super()` Calls

Calling `super()` in a method that also calls the same method on related records can create deep recursion:

```
// [ ] ANTI-PATTERN — Recursive super() calls
public void updateRelatedRecords()
{
    super();  // Calls base implementation

    // This calls updateRelatedRecords() on related records,
    // which in turn calls super() and processes THEIR related records
    // This can cause stack overflow on deep relationship chains
    CustTable relatedCust;
    while select relatedCust
        where relatedCust.ParentAccountNum == this.AccountNum
    {
        relatedCust.updateRelatedRecords();  // Recursive!
    }
}
```

```
// [x] CORRECT — Use a batch job or iterative approach
class UpdateRelatedRecordsBatch extends RunBaseBatch
{
    public void run()
    {
        // Process related records iteratively in a batch job
        // No recursion — processes one level at a time
    }
}
```

Anti-Pattern 4: `while select` with `firstonly` Mismatch

Using `while select` when you only need one record:

```
// [ ] SLOW — while select retrieves ALL matching records
CustTable custTable;
while select custTable
    where custTable.AccountNum == "CUST-001"
{
    // Only the first record is needed, but all are retrieved
    break;  // Breaking after first iteration is a workaround, not a fix
}

// [x] FAST — firstonly stops after the first match
CustTable custTable;
select firstonly custTable
    where custTable.AccountNum == "CUST-001";
```

14.3 Infolog Management Best Practices

The Infolog is the primary feedback mechanism for users. Managing it well is critical for a good user experience.

Infolog Message Types

Type	Method	User Impact	Use When
Info	`info()`	Green checkmark — informational	Process completed successfully
Warning	`warning()`	Yellow triangle — caution	Non-critical issue, operation can continue
Error	`error()`	Red X — blocks operation	Critical issue, operation must stop
CheckFailed	`checkFailed()`	Yellow triangle — validation failure	Field validation failed

Infolog Best Practices

1. **Be specific** — include the record ID and field name in error messages
2. **Use `checkFailed()` for field-level validation** — it highlights the specific field that failed
3. **Avoid `info()` in loops** — thousands of info messages will overwhelm the user
4. **Use `warning()` for non-critical issues** — don't block the user for minor issues

5. **Group related messages** — use a single `info()` with a summary rather than multiple messages

```
// [ ] BAD — too many messages in a loop
CustTable custTable;
while select custTable
    where custTable.CreditMax > 100000
{
    info(strFmt("Customer %1 has high credit limit", custTable.AccountNum));
    // Thousands of messages — user can't find the important ones
}

// [x] GOOD — summary message
CustTable custTable;
int highCreditCount;

while select custTable
    where custTable.CreditMax > 100000
{
    highCreditCount++;
}

info(strFmt("Found %1 customers with high credit limits.", highCreditCount));
```

14.4 Diagnosing Slow Forms

Using `SysPerformance`

The `SysPerformance` class provides built-in performance measurement:

```
// Measure the execution time of a code block
SysPerformance perf = new SysPerformance();
perf.startTimer();

// The code you want to measure
CustTable custTable;
while select custTable
    where custTable.CreditMax > 100000
{
    // Processing logic
}

perf.stopTimer();
info(strFmt('Execution time: %1 ms', perf.getElapsedTime()));
```

SQL Trace

Use SQL Server Profiler or Extended Events to trace the SQL queries generated by X++ code:

1. Open SQL Server Profiler
2. Connect to the D365 F&O SQL Server instance
3. Create a trace with the `SQL:BatchStarting` and `SQL:BatchCompleted` events
4. Filter by the AOS process ID or database name
5. Run the slow operation in F&O
6. Analyze the trace to identify slow queries

AOT Call Stack Profiling

The AOT provides a call stack view that shows the execution path of X++ code:

1. Set a breakpoint in Visual Studio
2. Run the operation that's slow
3. When the breakpoint hits, open the **Call Stack** window
4. Look for methods that are called repeatedly or take long to execute
5. Identify the bottleneck and optimize it

Common Slow Form Patterns

Pattern	Cause	Solution
Form loads slowly on open	Heavy `init()` method	Move processing to a button or event handler
Grid loads slowly	Missing index on sort column	Add index on the sort field
Lookup opens slowly	Complex query with joins	Simplify the lookup query, add indexes
Form is slow after navigation	Complex `active()` method	Simplify or move logic out of `active()`
Report takes >10 minutes	Large dataset in memory	Switch to TempDB pre-processing
Batch job runs slowly	Missing index on WHERE clause	Add index on filtered columns

---

14.5 Memory Management

Large Recordset `while select` Patterns

Processing large recordsets in memory can cause memory pressure and performance issues:

```
// [ ] SLOW — loads all records into memory at once
CustTable custTable;
int totalCredit = 0;

while select custTable
    where custTable.CreditMax > 0
{
    totalCredit += custTable.CreditMax;  // All records held in memory
}

// [x] FAST — processes records one at a time with minimal memory
CustTable custTable;
int totalCredit = 0;

while select custTable
    where custTable.CreditMax > 0
{
    totalCredit += custTable.CreditMax;
    // Record is released after each iteration
}
```

`Global::objType` for Type Checking

When you need to check the type of a `Common` record, use `Global::objType()` instead of `is` or `as` casts:

```
// [ ] SLOW — type checking with as cast
Common record = ...;
if (record is CustTable)
{
    CustTable custTable = record as CustTable;
    // Process custTable
}

// [x] FAST — type checking with Global::objType
Common record = ...;
if (Global::objType(record) == Types::Class)
{
    // Use the record directly
}
```

`GC.Collect` Considerations

In rare cases where large objects are created and destroyed in a loop, forcing garbage collection can help:

```
// Use GC.Collect sparingly — it's a last resort
static void processLargeDataset()
{
    for (int i = 0; i < 10000; i++)
    {
        // Process each item
        // ...

        // Force GC every 1000 iterations to prevent memory buildup
        if (i % 1000 == 0)
        {
            System.GC::Collect();
        }
    }
}
```

Warning: `GC.Collect()` is expensive and should only be used as a last resort. The .NET garbage collector is designed to manage memory automatically. Use it only when you've identified a memory pressure issue through profiling.

14.6 Activity — Diagnose and Optimize a Slow Form

> **Activity**: A form called `CustomerCreditSummary` takes 45+ seconds to load when there are 50,000+ customer records. Diagnose the root cause and propose three different optimization strategies with comparisons.
>
> **Scenario Details:** - The form displays a grid of customers with their credit limits, outstanding balances, and compliance status - The form has a `CustTable` data source with several related data sources - The `init()` method runs a complex query that joins `CustTable`, `CustGroup`, `APComplianceLog`, and `VendTable` - The `active()` method runs a lookup to populate a compliance status field - The form has a custom `executeQuery()` method that adds ranges programmatically
>
> **Three Optimization Proposals:**

> **Proposal 1: Optimize the `init()` query** - Remove the complex join from `init()` and move it to a background thread - Add indexes on the join columns (`CustGroup`, `APComplianceLog`) - Use `firstonly` where only one record is needed - **Expected improvement**: 45s → 15s (67% faster) - **Trade-off**: Initial load is faster but the compliance status may not be immediately available
>
> **Proposal 2: Simplify the `active()` lookup** - Replace the `SysTableLookup` in `active()` with a simple field assignment - Cache the compliance status in a member variable - Use a delegate to update the status when the record changes - **Expected improvement**: 45s → 8s (82% faster) - **Trade-off**: The compliance status lookup is less flexible but much faster
>
> **Proposal 3: Use TempDB for the data source** - Pre-load the customer data into a TempDB table on form open - The grid reads from TempDB instead of the live tables - Use a background job to refresh the TempDB data periodically - **Expected improvement**: 45s → 3s (93% faster) - **Trade-off**: The data may be slightly stale between refreshes, but the form is much more responsive
>
> **Activity Hints:** - **Hint A — Root cause identification**: Option A1 — The `init()` query is the primary bottleneck (most likely — complex joins in `init()` are the #1 cause of slow form loads). Option A2 — The `active()` lookup is the primary bottleneck (possible — if the lookup runs on every record navigation). Option A3 — Missing indexes on join columns (possible — check the SQL execution plan). Option A4 — Multiple root causes combined (most realistic — in practice, slow forms usually have multiple contributing factors). - **Hint B — Optimization strategy**: Option B1 — Fix the `init()` query first (recommended — highest impact, lowest risk). Option B2 — Fix the `active()` lookup second (good — addresses the second most common cause). Option B3 — Use TempDB for the data source (advanced — best for very large datasets but adds complexity). - **Hint C — Validation approach**: Option C1 — Measure before and after with `SysPerformance` (recommended — data-driven). Option C2 — Ask users to subjectively rate the form speed (valid but less precise). Option C3 — Use SQL Server Profiler to trace the queries (most accurate — shows exactly what SQL is being generated).

Chapter 15 — Capstone: End-to-End Production Scenario

15.1 Business Case: Manufacturing Procurement-to-Pay Compliance

Scenario Overview

Acme Manufacturing needs a custom compliance tracking system for their procurement-to-pay process. The system ensures that all purchase orders meet regulatory and internal compliance requirements before payment is processed.

Requirements

1. **Custom Table**: `POComplianceHeader` and `POComplianceLine` to track compliance checks on purchase orders
2. **Form**: `POComplianceForm` for entering and reviewing compliance data
3. **Custom Lookup**: Lookup to supplier compliance records when entering PO lines
4. **SSRS Report**: `POComplianceReport` for compliance summary and audit trail
5. **Batch Job**: Weekly compliance audit that checks all open POs for compliance gaps
6. **Data Entity**: `POComplianceEntity` for pushing compliance data to an external audit system
7. **Security Model**: Roles and duties for compliance officers, AP clerks, and managers
8. **Chain of Command**: Override on PO workflow to add compliance check before approval
9. **Event Handler**: On `PurchTable` to trigger compliance check when a PO is created

Architecture Overview

```
[Purchase Order Created]
        |
        v
[Event Handler: PurchTable::created]
        |
        v
[Compliance Check: POComplianceService]
        |
        +-- Compliant → PO proceeds to approval workflow
        |
        +-- Non-Compliant → PO blocked, compliance form opened
                            |
                            v
                    [User resolves compliance issue]
                            |
                            v
                    [Compliance check re-run]
                            |
                            v
                    [PO proceeds to approval]

[Weekly Batch Job: POComplianceAudit]
        |
        v
[Scan all open POs for compliance gaps]
        |
        v
[Generate compliance report (SSRS)]
```

```
        |
        v
[Push data to external audit system (Data Entity)]
```

15.2 Decision Points

Decision 1: Table Design — Staging Table or Direct Insert?

Question: Should compliance data be written directly to the target table, or should it go through a staging table first?

Option	Pros	Cons	Recommendation
Direct Insert	Simpler code, fewer tables	No validation before commit, harder to audit	[ ] Not recommended
Staging Table	Validation before commit, audit trail, error handling	More complex, extra table	[x] Recommended
TempDB Table	Fast, no persistence	No audit trail, lost on restart	[ ] Not for compliance data

Rationale: Compliance data requires an audit trail and validation before commit. The staging pattern ensures that invalid data is caught before it reaches the target table, and errors are logged with context.

Decision 2: Form Pattern — ListPage + Detail or Single Form?

Question: Should the compliance form use a ListPage + Detail pattern or a single form?

Option	Pros	Cons	Recommendation
ListPage + Detail	Standard F&O pattern, scalable for large datasets	More complex to build	[x] Recommended
Single Form	Simpler to build, good for small datasets	Doesn't scale, non-standard pattern	[ ] Not for production
Task Page	Wizard-style, guided workflow	Not suitable for data review	[ ] Not for this scenario

Rationale: The ListPage + Detail pattern is the standard F&O pattern for data review and editing. It scales well for large datasets and provides a familiar user experience.

Decision 3: Report Design — SSRS or Power BI?

Question: Should the compliance report be an SSRS paginated report or a Power BI dashboard?

Option	Pros	Cons	Recommendation
SSRS Report	Pixel-perfect, printable, standard F&O pattern	Static, requires report designer	[x] Recommended for audit reports
Power BI Dashboard	Interactive, real-time, visual	Requires Power BI license, not printable	[x] Recommended for operational dashboards
Both	Covers both use cases	More development effort	[x] Best option if budget allows

Rationale: SSRS is the standard for printable audit reports. Power BI is better for interactive operational dashboards. Both serve different purposes and should be used together.

Decision 4: Batch Job — RunBaseBatch or SysOperation?

Question: Should the weekly compliance audit use RunBaseBatch or SysOperation?

Option	Pros	Cons	Recommendation
RunBaseBatch	Familiar pattern, dialog for parameters	Legacy framework, less integration	[x] Recommended for batch jobs with user input
SysOperation	Modern framework, Process Automation integration	More complex setup	[x] Recommended for scheduled/recurring operations
SysOperation wrapping RunBaseBatch	Best of both worlds	Most complex	[x] Recommended for production

Rationale: The SysOperation framework provides better integration with Process Automation and scheduled execution. Wrapping the RunBaseBatch with SysOperation gives the best of both worlds — user input via dialog and scheduled execution via the service controller.

Decision 5: Data Entity — Staging Pattern or Direct Write-Back?

Question: Should the data entity use the staging pattern for write-back, or direct write-back?

Option	Pros	Cons	Recommendation
Staging Pattern	Validation, error handling, audit trail	More complex, extra table	[x] Recommended for compliance data
Direct Write-Back	Simpler, fewer tables	No validation, no audit trail	[ ] Not for compliance data
Read-Only Entity	Simple, no write-back complexity	Cannot push data to external system	[ ] Not if write-back is required

Rationale: Compliance data pushed to an external audit system must be validated and auditable. The staging pattern ensures data integrity and provides an audit trail.

Decision 6: Security Model — Three Duties or Two?

Question: Should the security model use three duties (Viewer, Processor, Admin) or two (ReadWrite, Admin)?

Option	Pros	Cons	Recommendation
Three Duties	Clear separation of concerns, SOX-compliant	More complex to design and maintain	[x] Recommended for production
Two Duties	Simpler, easier to maintain	Less granular, may not satisfy SOX	[ ] Not for regulated environments
Four Duties	Maximum granularity	Overly complex for most scenarios	[ ] Unless compliance requirements demand it

Rationale: Three duties provide clear separation of concerns and satisfy SOX compliance requirements. The Viewer duty allows read-only access, the Processor duty allows data entry and editing, and the Admin duty

provides full control.

Decision 7: CoC vs. Event Handler for PO Workflow

Question: Should the compliance check on the PO workflow use Chain of Command or an Event Handler?

Option	Pros	Cons	Recommendation
Chain of Command	Wraps the standard method, standard logic still runs	Tightly coupled to the base method	[x] Recommended for workflow integration
Event Handler	Decoupled, multiple subscribers possible	No control over execution order	[ ] Not for workflow integration
Delegate	Lightweight, callback pattern	No flow control	[ ] Not for this scenario

Rationale: Chain of Command is the right choice for workflow integration because it wraps the standard method and ensures the standard approval logic still runs. The compliance check is additive — it adds a gate before the standard approval process.

15.3 Ideal Solution Walkthrough

Step 1: Create the Custom Tables

```
table 50100 POComplianceHeader
{
    DataClassification = CustomerContent;
    Storage = InMemory;

    fields
    {
        field(1; RecId; int64) { }
        field(2; POId; Code[20]) { }
        field(3; PurchTableRecId; int64) { }
        field(4; ComplianceStatus; Enum POComplianceStatus) { }
        field(5; CheckedBy; UserId) { }
        field(6; CheckedDate; UtcDateTime) { }
        field(7; Notes; Text[250]) { }
    }

    keys
    {
        key(PK; RecId) { Clustered = true; }
        key(IX_POId; POId) { }
    }
}

table 50101 POComplianceLine
{
    DataClassification = CustomerContent;
    Storage = InMemory;

    fields
    {
        field(1; RecId; int64) { }
        field(2; POComplianceHeaderRecId; int64) { }
        field(3; LineNumber; int) { }
```

```
        field(4; Requirement; Enum POComplianceRequirement) { }
        field(5; Status; Enum POComplianceLineStatus) { }
        field(6; CheckedBy; UserId) { }
        field(7; Notes; Text[250]) { }
    }

    keys
    {
        key(PK; RecId) { Clustered = true; }
        key(IX_HeaderLine; POComplianceHeaderRecId, LineNumber) { }
    }
}
```

Step 2: Create the Compliance Service Class

```
class POComplianceService
{
    /// <summary>
    /// Check compliance for a purchase order.
    /// Returns true if the PO passes all compliance checks.
    /// </summary>
    public static boolean checkCompliance(PurchTable _purchTable)
    {
        POComplianceHeader complianceHeader;
        boolean allCompliant = true;

        // Check each compliance requirement
        // Requirement 1: Vendor must be approved
        if (!_purchTable.VendTable.Approved)
        {
            allCompliant = false;
            // Log non-compliance
        }

        // Requirement 2: Credit limit must not be exceeded
        if (_purchTable.AmountCurCredit > _purchTable.VendTable.CreditMax)
        {
            allCompliant = false;
            // Log non-compliance
        }

        // Requirement 3: PO must have a valid cost center
        if (_purchTable.CostCenterId == '')
        {
            allCompliant = false;
            // Log non-compliance
        }

        // Save compliance results
        complianceHeader.clear();
        complianceHeader.POId = _purchTable.PurchId;
        complianceHeader.PurchTableRecId = _purchTable.RecId;
        complianceHeader.ComplianceStatus = allCompliant
            ? POComplianceStatus::Compliant
            : POComplianceStatus::NonCompliant;
        complianceHeader.CheckedBy = curUserId();
        complianceHeader.CheckedDate = DateTimeUtil::utcNow();
        complianceHeader.insert();

        return allCompliant;
```

```
    }
}
```

Step 3: Chain of Command Override on PO Workflow

```
[ExtensionOf(classStr(PurchTable))]
final class PurchTable_ComplianceCocExtension
{
    /// <summary>
    /// Wrap the approve method to add compliance check before approval.
    /// </summary>
    public void approve()
    {
        // BEFORE: Check compliance before allowing approval
        if (!POComplianceService::checkCompliance(this))
        {
            // Compliance check failed — block approval
            checkFailed('Purchase order does not meet compliance requirements. Review the compliance form
for details.');
            return;  // Stop the approval chain
        }

        // BASE: Call the standard approve method
        next approve();

        // AFTER: Compliance check passed, approval completed
        info(strFmt('Purchase order %1 approved with compliance check passed.', this.PurchId));
    }
}
```

Step 4: Event Handler on PurchTable::created

```
class PurchTableEventHandler
{
    [EventHandler(eventStr(PurchTable::inserted))]
    public static void onPurchTableInserted(Common _sender)
    {
        PurchTable purchTable = _sender as PurchTable;

        // Trigger compliance check when a new PO is created
        boolean isCompliant = POComplianceService::checkCompliance(purchTable);

        if (!isCompliant)
        {
            // PO is non-compliant — notify the compliance team
            info(strFmt('Purchase order %1 created but failed compliance check. Please review.',
purchTable.PurchId));
        }
    }
}
```

Step 5: Create the Batch Job for Weekly Compliance Audit

```
class POComplianceAuditBatch extends RunBaseBatch
{
    PurchGroupId _purchGroupId;
    FromDate _fromDate;
```

```
    ToDate _toDate;

    public Object dialog()
    {
        Dialog dlg = super::dialog();
        dlg.addFieldValue(enumStr(PurchGroupId), _purchGroupId, "Purchase Group");
        dlg.addFieldValue(exttypstr(FromDate), _fromDate, "From Date");
        dlg.addFieldValue(exttypstr(ToDate), _toDate, "To Date");
        return dlg;
    }

    public boolean getFromDialog()
    {
        boolean ret = super::getFromDialog();
        _purchGroupId = dialog().dialogField(fieldNum(POComplianceAuditBatch, _purchGroupId)).value();
        _fromDate = dialog().dialogField(fieldNum(POComplianceAuditBatch, _fromDate)).value();
        _toDate = dialog().dialogField(fieldNum(POComplianceAuditBatch, _toDate)).value();
        return ret;
    }

    public void run()
    {
        PurchTable purchTable;
        POComplianceHeader complianceHeader;
        int totalChecked = 0;
        int nonCompliantCount = 0;

        while select purchTable
            where purchTable.PurchDate >= _fromDate
            && purchTable.PurchDate <= _toDate
            && purchTable.PurchGroupId == _purchGroupId
        {
            totalChecked++;
            if (!POComplianceService::checkCompliance(purchTable))
            {
                nonCompliantCount++;
            }
        }

        info(strFmt('Compliance audit complete. Checked %1 POs, found %2 non-compliant.',
            totalChecked, nonCompliantCount));
    }

    public boolean canGoBatch()
    {
        return true;
    }
}
```

Step 6: Create the Data Entity for External Audit System

```
// The data entity pushes compliance data to an external audit system
// using the staging pattern for write-back

// Entity: POComplianceEntity
// - Root entity: POComplianceHeader
// - Child entity: POComplianceLine
// - Staging table: POComplianceStaging
// - Mapping: Source fields → Staging → Target (external system)

// The entity is published via OData/REST and can be consumed by:
```

```
// - Power Automate flows
// - External audit applications
// - Regulatory reporting systems
```

Step 7: Design the Security Model

Role	Duties	Access
`ComplianceOfficer`	Viewer, Processor	Full compliance data access, can approve compliance exceptions
`APClerk`	Viewer	Read-only access to compliance data
`InventoryManager`	Processor	Can create and edit compliance records for their POs
`SystemAdmin`	Admin	Full control including security role management

Step 8: Deploy and Monitor

1. Deploy the model package through the CI/CD pipeline
2. Run database sync and compile
3. Test with "Run as" feature using each security role
4. Run the weekly compliance audit batch job manually first
5. Schedule the batch job as a recurring job
6. Monitor via LCS dashboards and Infolog
7. Review the compliance report monthly

15.4 Capstone Activity — Full Specification

> **Activity**: Implement the complete procurement-to-pay compliance scenario described in this chapter. You must make the following decisions and document your rationale for each:
>
> 1. **Table Design**: Staging table or direct insert? Justify your choice based on audit requirements.
> 2. **Form Pattern**: ListPage + Detail or single form? Justify based on user experience and scalability.
> 3. **Report Design**: SSRS or Power BI? Justify based on the audience and use case.
> 4. **Batch Job Framework**: RunBaseBatch or SysOperation? Justify based on the execution pattern.
> 5. **Data Entity Pattern**: Staging or direct write-back? Justify based on data integrity requirements.
> 6. **Security Model**: Three duties or two? Justify based on the organization's compliance requirements.
> 7. **Extension Pattern**: CoC or Event Handler? Justify based on the coupling requirements.
>
> For each decision, provide: - Your chosen approach - The alternative approach you considered - Why you chose your approach - What trade-offs your approach involves - How you would validate your choice in a test environment

> **Ideal Solution Walkthrough**: The detailed walkthrough above (Steps 1-8) shows the recommended approach with full code examples. Every decision is justified with methodology rationale, trade-off analysis, and production hardening notes.
>
> **Production Hardening Checklist**: - [ ] All custom tables have Configuration Keys - [ ] All custom forms have appropriate security roles assigned - [ ] All batch jobs have error handling with try/catch - [ ] All data entities have staging tables with validation - [ ] All CoC overrides call `super()` appropriately - [ ] All event handlers return `EventHandlerResult::NoAction` when appropriate - [ ] All custom code has been tested with "Run as" feature - [ ] All custom code has SysTest unit tests - [ ] The model is versioned and documented - [ ] The deployment pipeline includes automated testing

---

> **Next Step**: All 15 chapters are now complete. The learning guide covers the full D365 F&O technical curriculum from foundations through capstone projects. Review the guide for consistency, then update the memory files to reflect completion.

> **Next Step**: All 15 chapters are now complete. The learning guide covers the full D365 F&O technical curriculum from foundations through capstone projects. Review the guide for consistency, then update the memory files to reflect completion.

---

name: d365-xpp-quick-reference description: X++ syntax cheat sheet covering data types, CRUD patterns, control flow, classes, exceptions, common mistakes, and naming conventions metadata: node_type: memory type: reference originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-08-10T17:26:07.024Z

X++ Quick Reference Card

> **D365 Finance & Operations — X++ Syntax at a Glance** Keep this open while coding. For full details, refer to the Desktop Guide (Chapters 1–2).

Data Types

True Primitive Types

Per Microsoft's official X++ primitive type list — these are the built-in scalar types:

Type	Example	Notes
`int`	`int i = 5;`	32-bit signed integer
`int64`	`int64 recId = 5637144576;`	64-bit — used for `RecId`
`real`	`real price = 19.99;`	X++'s `real` is internally represented with decimal-like precision (not standard IEEE-754 binary float), which is why it's safe to use for monetary values via EDTs like `AmountMST` — it does NOT have the rounding problems that IEEE-754 doubles have in languages like C#/Java
`str`	`str name = "Customer";`	Unicode string
`boolean`	`boolean flag = true;`	`true` / `false`
`date`	`date d = today();`	No time component
`utcdatetime`	`utcdatetime udt = DateTimeUtil::utcNow();`	UTC datetime
`timeOfDay`	`timeOfDay t = 120000;`	Seconds since midnight (int)
`enum`	`Status::New`	Named constant set
`guid`	`guid g = Guid::newGuid();`	Globally unique identifier
`AnyType`	`AnyType val = 42;`	Universal type — use with care

Composite Types

Type	Example	Notes
`container`	`container c = [1, "two", 3.0];`	Typed ordered list — one of several composite types; array is the other most commonly used one
`array`	`int a[3];`	Fixed-size array — another composite type

True Primitives vs. Other Constructs

True primitive types: `anytype`, `boolean`, `date`, `enum`, `guid`, `int`, `int64`, `real`, `str`, `timeOfDay`, `utcdatetime`

These are **not** primitive or composite data types — they are language constructs that serve different purposes:

Construct	Example	What It Actually Is
`record`	`CustTable custTable;`	A **table buffer declaration** — a variable that holds a row from a data entity (table), not a standalone data type
`class`	`MyService service;`	A **class instance reference** — declares a variable that will hold an object, not a scalar value
`void`	`public void doSomething()`	The **absence of a return type** — indicates a method returns nothing; it is not a value type

---

Variable Declaration

```
// Standard declaration
int     i = 0;
str     s = "Hello";
real    r = 123.45;
date    d = today();
boolean flag = true;

// Static constant — class level
static const Str AppName = "MyApp";

// Static variable — shared across instances
static int instanceCount = 0;

// Instance variable — each object has its own copy
CustTable _custTable;
boolean   _isValid;
```

Control Flow

If / Else If / Else

```
if (score >= 90)
{
    info("Grade: A");
}
else if (score >= 80)
{
    info("Grade: B");
}
else
{
```

```
    info("Grade: C");
}
```

Switch (int, str, enum only)

```
switch (status)
{
    case Status::New:
        info("New record");
        break;

    case Status::Approved:
        info("Already approved");
        break;

    default:
        info("Unknown status");
        break;
}
```

For Loop

```
for (int i = 1; i <= 10; i++)
{
    info(strFmt("Iteration %1", i));
}

// Iterate a container
container numbers = [10, 20, 30];
for (int i = 1; i <= conLen(numbers); i++)
{
    info(strFmt("Element %1 = %2", i, conPeek(numbers, i)));
}
```

While Select — The Idiomatic X++ Iteration

```
// Basic
CustTable custTable;
while select custTable
{
    info(custTable.AccountNum);
}

// With where clause
while select custTable
    where custTable.Currency == "USD"
{
    info(custTable.AccountNum);
}

// With ordering
while select custTable
    order by custTable.AccountNum
{
    info(custTable.AccountNum);
}
```

```
// Firstonly — stops after first match
while select firstonly custTable
    where custTable.AccountNum == "CUST-001"
{
    info(custTable.Name);
}
```

Do While

```
int i = 1;
do
{
    info(strFmt("Value: %1", i));
    i++;
}
while (i <= 5);
```

CRUD Operations

Select

```
CustTable custTable;

// Simple select
select custTable
    where custTable.AccountNum == "CUST-001";

// Firstonly — optimized
select firstonly custTable
    where custTable.AccountNum == "CUST-001";

// Specific fields only
select custTable.Name, custTable.Phone();

// Exists join — check for related records without retrieving them
boolean hasVendor = exists join vendTable
    where vendTable.AccountNum == custTable.AccountNum;
```

Insert

```
CustTable custTable;
custTable.clear();
custTable.AccountNum = "CUST-NEW-001";
custTable.Name = "New Customer";
custTable.insert();
```

Update — `update_recordset` (preferred)

```
CustTable custTable;
update_recordset custTable
    setting custTable.Phone = "+1-555-0200"
    where custTable.AccountNum == "CUST-NEW-001";
```

Delete — `delete_from`

```
CustTable custTable;
delete_from custTable
    where custTable.AccountNum == "CUST-NEW-001";
```

Transactions

```
ttsBegin;
try
{
    CustTable custTable;
    custTable.clear();
    custTable.AccountNum = "CUST-TRANS-001";
    custTable.Name = "Transactional Customer";
    custTable.insert();

    CustGroup custGroup;
    custGroup.clear();
    custGroup.GroupName = "TRANSACTIONAL";
    custGroup.insert();

    ttsCommit;
}
catch (Exception::Error)
{
    ttsAbort;
    error("Transaction failed — rolled back.");
}
```

Method Overriding & `super()`

```
// In a table — overriding validateWrite
public boolean validateWrite()
{
    boolean ret;

    ret = super();  // Always call super() first in table methods

    // Custom validation
    if (this.AccountNum == "")
    {
        ret = false;
        error("Account number cannot be empty.");
    }

    return ret;
}
```

Chain of Command (`next` keyword)

```
[ExtensionOf(tableStr(CustTable))]
final class CustTable_Extension
{
    public boolean validateWrite()
```

```
    {
        boolean ret;

        ret = next validateWrite();  // Call next handler in the chain

        // Custom validation
        if (this.AccountNum == "")
        {
            ret = false;
            error("Account number cannot be empty.");
        }

        return ret;
    }
}
```

Exception Handling

Exception Types

Type	When
`Exception::Error`	General runtime error
`Exception::Warning`	Non-fatal warning
`Exception::Info`	Informational message
`Exception::Broken`	Object in an invalid state (e.g., deleted record)
`Exception::Deadlock`	SQL deadlock — retry
`Exception::DuplicateKey`	Unique index violation

Try / Catch Pattern

```
try
{
    CustTable custTable;
    select firstonly custTable
        where custTable.AccountNum == "NON-EXISTENT";

    if (!custTable)
    {
        throw error("Customer not found.");
    }
}
catch (Exception::Error)
{
    error("An error occurred.");
}
catch (Exception::Warning)
{
    warning("A warning occurred.");
}
catch (Exception::Info)
{
```

```
    info("Informational message.");
}
```

Deadlock Retry Pattern

```
int retryCount = 0;
boolean success = false;

while (!success && retryCount < 3)
{
    ttsBegin;
    try
    {
        // ... your database operations ...
        ttsCommit;
        success = true;
    }
    catch (Exception::Deadlock)
    {
        ttsAbort;
        retryCount++;
        info(strFmt("Deadlock — retry %1 of 3", retryCount));
    }
    catch (Exception::Error)
    {
        ttsAbort;
        error("A non-deadlock error occurred.");
        break;
    }
}
```

Classes & Objects

Class Declaration

```
/// <summary>Summary of what this class does.</summary>
class MyService
{
    // Static constant
    static const Str DefaultPrefix = "SRV";

    // Instance variables
    CustTable _custTable;
    boolean   _isValid;

    // Constructor
    public MyService(CustTable _custTable)
    {
        this._custTable = _custTable;
        this._isValid = false;
    }

    // Public method
    public boolean validate()
    {
        // ... validation logic ...
        return this._isValid;
```

```
    }

    // Public static method
    public static boolean isAccountValid(str _accountNum)
    {
        return (_accountNum != '' && strLen(_accountNum) >= 3);
    }
}
```

Inheritance

```
// Base class
class BasePaymentService
{
    public void pay(CustTable _custTable, Amount _amount)
    {
        info(strFmt("Processing payment of %1 for %2", _amount, _custTable.AccountNum));
    }
}

// Derived class
class CreditCardPaymentService extends BasePaymentService
{
    public void pay(CustTable _custTable, Amount _amount)
    {
        this.validateCard();
        super::pay(_custTable, _amount);
        this.logTransaction();
    }
}
```

Abstract Classes & Interfaces

```
// Abstract class
abstract class AbstractReportGenerator
{
    public abstract void generate();

    public void logGeneration()
    {
        info("Report generation started.");
    }
}

// Interface
interface IExportable
{
    void exportToFile(str _filePath);
    str getExportFormat();
}

// Implementing both
class CsvReportGenerator extends AbstractReportGenerator implements IExportable
{
    public void generate()
    {
        this.logGeneration();
        // CSV logic
    }
```

```
    public void exportToFile(str _filePath)
    {
        // Write CSV
    }

    public str getExportFormat()
    {
        return "CSV";
    }
}
```

Common Patterns

Container Operations

```
container c = [1, "two", 3.0];
int       len = conLen(c);          // 3
int       first = conPeek(c, 1);    // 1
container c2 = conIns(c, 2, "inserted");  // [1, "inserted", "two", 3.0]
```

String Formatting

```
info(strFmt("Hello, %1. Your balance is %2.", name, balance));
info(strFmt("Date: %1", date2str(today(), 123, 2, 2, 2, 4, DateFlags::None)));
```

Date/Time Utilities

```
date     d = today();
date     d2 = str2date("2026-01-15", 123);
int      year = year(d);
int      month = monthOfYear(d);
utcdatetime udt = DateTimeUtil::utcNow();
str      formatted = DateTimeUtil::toStr(udt);
```

File I/O

```
TextIO io = new TextIO(@"C:\temp\output.txt", "w");
io.write("Hello, file!");
io.close();
```

Infolog Messages

```
info("Informational message");      // Blue
warning("Warning message");         // Yellow
error("Error message");             // Red
```

Key System Classes

Class	Purpose
`Global`	Static utility methods (`Global::info()`, `Global::warning()`, `Global::error()`)
`SysTableLookup`	Build lookups for form controls
`QueryRun`	Execute a `Query` at runtime — the preferred D365 F&O approach for dynamic queries
`SrsReportDataProvider`	Data provider for SSRS reports
`RunBase`	Framework for batchable processes
`RunBaseBatch`	Batch execution framework
`SysOperationServiceController`	Modern service operation framework — replaces RunBase for service-based integrations
`NumberSeq`	Number sequence generation (safe under concurrency)
`DictTable` / `DictField` / `DictEnum`	Runtime table/field/enum metadata reflection
`Exception`	Exception type enum (`Exception::Error`, `Exception::Broken`, `Exception::Deadlock`, etc.)

Common Mistakes to Avoid

Mistake	Fix
`select` inside a `while` loop (N+1 queries)	Use `exists join` or `while select`
Forgetting `super()` in overridden methods	Always call `super()` first in table methods
Using `update` instead of `update_recordset` in loops	Use `update_recordset` for bulk updates
Declaring variables inside `if` blocks and expecting them outside	Declare at method scope
Not using `ttsBegin`/`ttsCommit` for multi-table inserts	Wrap related inserts in a transaction
Ignoring deadlocks	Implement retry logic with `catch (Exception::Deadlock)`
Using primitive types instead of EDTs	Always use EDTs for consistency and validation
Manual `RecId` assignment	Let the system auto-generate `RecId`

Naming Conventions

Element	Convention	Example
Table	`CustTable`, `VendTable`	PascalCase + `Table` suffix
EDT	`CustAccount`, `VendAccount`	PascalCase, no suffix
Class	`CustValidationService`	PascalCase + `Service/Handler/Extension`
Table extension	`[ExtensionOf(tableStr(CustTable))]`	`final` class, `ExtensionOf` attribute

Element	Convention	Example
EDT extension	`[ExtendsWithEDT('AddressCountryRegionId')]`	Static method with `_Ext` suffix
Event handler	`CustTable_inserted`	`TableName_EventName`
Delegate	`modifyPrice`	Method name on the delegate publisher

Key Microsoft Learn URLs

- D365 F&O Developer Documentation
- X++ Language Reference
- Tables and Fields
- Extended Data Types
- Chain of Command
- Event Handlers
- Data Entities
- SSRS Reporting
- Security Model
- LCS & DevOps

---

name: d365-xpp-exercises description: 17 hands-on X++ exercises with worked solutions covering CRUD, CoC, delegates, event handlers, transactions, error handling, and more metadata: node_type: memory type: reference originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-08-04T20:33:24.598Z

X++ Practice Exercises with Solutions

> **Hands-on coding practice for D365 F&O developers.** Each exercise has a problem statement, hints, and a worked solution. Try to solve each one yourself before reading the solution.

Exercise 1 — Variable Declaration and Type Casting

Problem

Write a method that takes a `container` with three elements (an `int`, a `str`, and a `real`) and prints each element to the Infolog with its type name.

Hints

- Use `conLen()` to get the container length
- Use `conPeek()` to retrieve elements by position (1-based)
- Use `typeid()` to get the type name of a variable
- Use `info()` to write to the Infolog

Solution

```
static void exercise1_containerTypes(Args _args)
{
    container c = [42, "Hello", 3.14];
    int     i;
    str     s;
    real    r;

    // Element 1: int
    i = conPeek(c, 1);
    info(strFmt("Element 1: value=%1, type=%2", i, typeid(i)));

    // Element 2: str
    s = conPeek(c, 2);
    info(strFmt("Element 2: value=%1, type=%2", s, typeid(s)));

    // Element 3: real
    r = conPeek(c, 3);
    info(strFmt("Element 3: value=%1, type=%2", r, typeid(r)));
}
```

Expected Infolog output:

```
Element 1: value=42, type=int
Element 2: value=Hello, type=str
Element 3: value=3.14, type=real
```

Exercise 2 — While Select with Filtering

Problem

Write a job that selects all customers whose `Currency` is "USD" and whose `GroupName` is "CORP", and prints their `AccountNum` and `Name` to the Infolog.

Hints

- Use `while select` with a `where` clause combining two conditions with `&&`
- Remember that `CustTable` has fields `AccountNum`, `Name`, `Currency`, and `GroupName`

Solution

```
static void exercise2_usdCorpCustomers(Args _args)
{
    CustTable custTable;
    int count = 0;

    while select custTable
        where custTable.Currency == "USD"
           && custTable.GroupName == "CORP"
    {
        info(strFmt("Account: %1 – Name: %2", custTable.AccountNum, custTable.Name));
        count++;
    }

    info(strFmt("Total customers found: %1", count));
}
```

Exercise 3 — Update Recordset (Bulk Update)

Problem

Write a job that updates the `Currency` field to "EUR" for all customers whose `GroupName` is "TEMP". Use `update_recordset` — do not use a `while select` loop with `.update()`.

Hints

- `update_recordset` generates a single SQL UPDATE statement
- The `setting` clause specifies which field to update and to what value
- The `where` clause filters which records to update

Solution

```
static void exercise3_bulkCurrencyUpdate(Args _args)
{
    CustTable custTable;
    int updatedCount;

    // Count how many records will be affected
    select countRecId from custTable
        where custTable.GroupName == "TEMP";
    updatedCount = custTable.countRecId;

    // Perform the bulk update
    update_recordset custTable
        setting custTable.Currency = "EUR"
```

```
        where custTable.GroupName == "TEMP";

    info(strFmt("Updated %1 customer(s) to EUR", updatedCount));
}
```

Exercise 4 — Transaction with Error Handling

Problem

Write a job that inserts a new customer and a corresponding customer group record in a single transaction. If either insert fails, roll back the entire transaction and display an error message.

Hints

- Use `ttsBegin` / `ttsCommit` / `ttsAbort`
- Wrap the inserts in a `try`/`catch` block
- Call `ttsAbort` in the catch block

Solution

```
static void exercise4_transactionalInsert(Args _args)
{
    CustTable    custTable;
    CustGroup    custGroup;
    str          newAccountNum = "CUST-TRANS-001";

    ttsBegin;
    try
    {
        // Insert the customer group first (parent)
        custGroup.clear();
        custGroup.GroupName = "TRANSACTIONAL";
        custGroup.insert();

        // Insert the customer (child)
        custTable.clear();
        custTable.AccountNum = newAccountNum;
        custTable.Name = "Transactional Customer";
        custTable.GroupName = "TRANSACTIONAL";
        custTable.Currency = "USD";
        custTable.insert();

        ttsCommit;
        info("Transaction committed — both records inserted successfully.");
    }
    catch (Exception::Error)
    {
        ttsAbort;
        error("Transaction failed — both records rolled back.");
    }
}
```

Exercise 5 — Method Overriding with `super()`

Problem

Create a table extension on `CustTable` that overrides `validateWrite()`. The extension should: 1. Call `super()` first 2. Add a validation that the `AccountNum` field must start with "CUST-" 3. Return `false` and display an error if the validation fails

Hints

- Use `[ExtensionOf(tableStr(CustTable))]` attribute
- Use `next validateWrite()` to call the base method
- Use `strStartsWith()` to check the prefix

Solution

```
[ExtensionOf(tableStr(CustTable))]
final class CustTable_Extension
{
    public boolean validateWrite()
    {
        boolean ret;

        // Call the base class validation first
        ret = next validateWrite();

        // Custom validation: AccountNum must start with "CUST-"
        if (ret && !strStartsWith(this.AccountNum, "CUST-"))
        {
            ret = false;
            error(strFmt("Account number '%1' must start with 'CUST-'.", this.AccountNum));
        }

        return ret;
    }
}
```

Exercise 6 — Exception Handling with Deadlock Retry

Problem

Write a method that attempts to update a customer record, with deadlock retry logic. The method should retry up to 3 times on deadlock, and display a message for each retry attempt.

Hints

- Use a `while` loop with a `retryCount` counter
- Catch `Exception::Deadlock` specifically
- Call `ttsAbort` before retrying
- Use `info()` to log retry attempts

Solution

```
static void exercise6_deadlockRetry(str _accountNum)
{
```

```
    CustTable custTable;
    int       retryCount = 0;
    boolean   success = false;

    while (!success && retryCount < 3)
    {
        ttsBegin;
        try
        {
            select firstonly custTable
                where custTable.AccountNum == _accountNum;

            if (custTable)
            {
                custTable.Name = strFmt("Updated Name %1", retryCount + 1);
                custTable.update();

                ttsCommit;
                success = true;
                info(strFmt("Customer updated successfully on attempt %1.", retryCount + 1));
            }
            else
            {
                ttsCommit;
                info("Customer not found.");
                success = true;  // Not an error, just not found
            }
        }
        catch (Exception::Deadlock)
        {
            ttsAbort;
            retryCount++;
            info(strFmt("Deadlock detected — retry %1 of 3", retryCount));
        }
        catch (Exception::Error)
        {
            ttsAbort;
            error("A non-deadlock error occurred.");
            break;
        }
    }

    if (!success)
    {
        error("Failed to update customer after 3 attempts.");
    }
}
```

Exercise 7 — Class with Constructor and Validation Methods

Problem

Create a class called `VendPaymentValidator` that: 1. Has a constructor accepting a `VendTable` record 2. Has a `validate()` method that checks: - `AccountNum` is not empty - `Name` is not empty - The vendor's `Currency` exists in `CurrencyTable` 3. Has a static method `isCurrencyValid(str _currencyCode)` that returns a boolean 4. Returns `true` from `validate()` only if all checks pass

Hints

- Use private helper methods for each validation rule
- Use `select firstonly` against `CurrencyTable` for the currency check
- Use `this.` to access instance variables from instance methods

Solution

```
class VendPaymentValidator
{
    VendTable _vendTable;
    boolean   _isValid;

    public VendPaymentValidator(VendTable _vendTable)
    {
        this._vendTable = _vendTable;
        this._isValid = false;
    }

    public boolean validate()
    {
        if (this.validateAccountNum()
         && this.validateName()
         && this.validateCurrency())
        {
            this._isValid = true;
        }
        return this._isValid;
    }

    private boolean validateAccountNum()
    {
        if (this._vendTable.AccountNum == '')
        {
            error("Vendor account number cannot be empty.");
            return false;
        }
        return true;
    }

    private boolean validateName()
    {
        if (this._vendTable.Name == '')
        {
            error("Vendor name cannot be empty.");
            return false;
        }
        return true;
    }

    private boolean validateCurrency()
    {
        CurrencyTable currencyTable;
        boolean       found;

        select firstonly currencyTable
            where currencyTable.CurrencyCode == this._vendTable.Currency;

        found = currencyTable.RecId != 0;

        if (!found)
        {
            error(strFmt("Currency '%1' is not valid.", this._vendTable.Currency));
```

```
        }

        return found;
    }

    public static boolean isCurrencyValid(str _currencyCode)
    {
        CurrencyTable currencyTable;
        boolean       found;

        select firstonly currencyTable
            where currencyTable.CurrencyCode == _currencyCode;

        found = currencyTable.RecId != 0;
        return found;
    }
}
```

Usage:

```
VendTable vendTable = VendTable::find("VEND-001");
VendPaymentValidator validator = new VendPaymentValidator(vendTable);

if (validator.validate())
{
    info("Vendor is valid for payment processing.");
}
else
{
    warning("Vendor validation failed — check the Infolog.");
}
```

Exercise 8 — Inheritance and Polymorphism

Problem

Create a base class `BaseDiscountCalculator` with a method `calculateDiscount(Amount _amount)` that returns a 0% discount. Then create two derived classes: - `GoldDiscountCalculator` — returns 10% discount - `PlatinumDiscountCalculator` — returns 20% discount

Write a job that creates each calculator, calls `calculateDiscount()` with an amount of 1000, and prints the result.

Hints

- Use `extends` for inheritance
- Override the `calculateDiscount` method in each derived class
- Use `super::` if you want to call the base implementation

Solution

```
// Base class
class BaseDiscountCalculator
{
    public Amount calculateDiscount(Amount _amount)
```

```
    {
        return 0;  // No discount by default
    }
}

// Gold tier — 10% discount
class GoldDiscountCalculator extends BaseDiscountCalculator
{
    public Amount calculateDiscount(Amount _amount)
    {
        return _amount * 0.10;
    }
}

// Platinum tier — 20% discount
class PlatinumDiscountCalculator extends BaseDiscountCalculator
{
    public Amount calculateDiscount(Amount _amount)
    {
        return _amount * 0.20;
    }
}

// Job to test the calculators
static void exercise8_discountCalculators(Args _args)
{
    Amount testAmount = 1000;
    BaseDiscountCalculator calculator;

    // Gold calculator
    calculator = new GoldDiscountCalculator();
    info(strFmt("Gold discount on %1: %2", testAmount, calculator.calculateDiscount(testAmount)));

    // Platinum calculator
    calculator = new PlatinumDiscountCalculator();
    info(strFmt("Platinum discount on %1: %2", testAmount, calculator.calculateDiscount(testAmount)));
}
```

Expected Infolog output:

```
Gold discount on 1000: 100
Platinum discount on 1000: 200
```

Exercise 9 — Event Handler

Problem

Create an event handler that runs after a `CustTable` record is inserted. The handler should log the new customer's `AccountNum` and `Name` to a custom log table called `CustCreationLog`.

Hints

- Use the `[SubscribesTo(tableStr(CustTable), inserted)]` attribute to register the handler
- Create a custom table `CustCreationLog` with fields: `AccountNum` (string 20), `CustomerName` (string 60), `CreatedDateTime` (utcDateTime)

Solution

First, create the custom table `CustCreationLog`:

Field	Type
`AccountNum`	`string 20`
`CustomerName`	`string 60`
`CreatedDateTime`	`utcdatetime`

Then, the event handler class:

```
class CustCreationEventHandler
{
    /// <summary>Event handler — runs after a CustTable record is inserted.</summary>
    [SubscribesTo(tableStr(CustTable), inserted)]
    public static void onCustTableInserted(CustTable _custTable)
    {
        CustCreationLog log;

        log.clear();
        log.AccountNum     = _custTable.AccountNum;
        log.CustomerName   = _custTable.Name;
        log.CreatedDateTime = DateTimeUtil::utcNow();
        log.insert();
    }
}
```

Note: The event handler must be in a class that is part of a model that references the `ApplicationSuite` model. The `[SubscribesTo]` attribute registers the handler automatically at compile time.

Exercise 10 — Delegate

Problem

Create a delegate on a class called `PriceCalculator` that allows consumers to customize the discount percentage. The delegate should be called during discount calculation, and consumers should be able to subscribe their own logic.

Hints

- Declare a delegate using the `delegate` keyword
- Use `delegate` on a static method that accepts a `PriceCalculator` instance and the original price
- Consumers subscribe using the `+=` operator
- The publisher calls the delegate with `this.priceDelegate(this, originalPrice)`

Solution

```
// Publisher class
class PriceCalculator
{
    // Delegate declaration — accepts a PriceCalculator and an Amount, returns a modified Amount
```

```
    public delegate Amount discountDelegate(PriceCalculator _calculator, Amount _originalPrice);

    // The delegate instance — consumers subscribe to this
    public discountDelegate priceDelegate;

    // Base discount percentage
    private Amount _baseDiscountPct = 0;

    public Amount calculateDiscountedPrice(Amount _originalPrice)
    {
        Amount discountedPrice;
        Amount discountAmount;

        // Calculate base discount
        discountAmount = _originalPrice * (_baseDiscountPct / 100);

        // Allow consumers to modify the discount via the delegate
        if (this.priceDelegate)
        {
            discountAmount = this.priceDelegate(this, _originalPrice);
        }

        discountedPrice = _originalPrice - discountAmount;
        return discountedPrice;
    }
}

// Consumer — subscribes to the delegate
class CustomDiscountSubscriber
{
    public static Amount customDiscount(PriceCalculator _calculator, Amount _originalPrice)
    {
        // Apply a 15% discount for VIP customers
        return _originalPrice * 0.15;
    }
}

// Job to test the delegate
static void exercise10_delegate(Args _args)
{
    PriceCalculator calculator = new PriceCalculator();
    Amount originalPrice = 1000;
    Amount discountedPrice;

    // Subscribe the custom discount logic
    calculator.priceDelegate += delegate(Amount _price)
    {
        return _price * 0.15;  // 15% discount
    };

    discountedPrice = calculator.calculateDiscountedPrice(originalPrice);
    info(strFmt("Original: %1 — Discounted: %2", originalPrice, discountedPrice));
}
```

Expected Infolog output:

```
Original: 1000 — Discounted: 850
```

Exercise 11 — Chain of Command (CoC)

Problem

Create two extension classes on `CustTable` that both modify the `validateWrite()` method using Chain of Command. The first extension should check that the `AccountNum` starts with "CUST-". The second extension should check that the `Name` is not empty. Both should call `next` so they work together.

Hints

- Each extension uses `[ExtensionOf(tableStr(CustTable))]`
- Each calls `next validateWrite()` to pass control to the next handler
- The execution order depends on the model layer order (CUS runs after SYS)

Solution

Extension 1 — AccountNum validation:

```
[ExtensionOf(tableStr(CustTable))]
final class CustTable_AccountNumValidation
{
    public boolean validateWrite()
    {
        boolean ret;

        ret = next validateWrite();

        if (ret && !strStartsWith(this.AccountNum, "CUST-"))
        {
            ret = false;
            error("Account number must start with 'CUST-'.");
        }

        return ret;
    }
}
```

Extension 2 — Name validation:

```
[ExtensionOf(tableStr(CustTable))]
final class CustTable_NameValidation
{
    public boolean validateWrite()
    {
        boolean ret;

        ret = next validateWrite();

        if (ret && this.Name == '')
        {
            ret = false;
            error("Customer name cannot be empty.");
        }

        return ret;
    }
}
```

How CoC works here: When `validateWrite()` is called on `CustTable`, the AOS executes both extensions in layer order. Each extension calls `next validateWrite()` to pass control to the next handler. If any handler returns `false`, the chain stops and the record is not saved.

Exercise 12 — Data Entity Creation

Problem

Describe the steps to create a simple data entity that exposes a custom table `VendAPCustomsDecl` (with fields `DeclId`, `DeclDate`, `CustomsRef`) to an external system via OData. List the AOT nodes you need to create and configure.

Hints

- A data entity consists of a `DataEntity` node, a `Query` node, and optionally a staging table
- The entity exposes fields via `IsPublic`, `Public Entity Name`, and `Public Collection Name` properties, with staging fields stored in a separate auto-generated staging table
- The entity must be registered in the `Data Entity` node

Solution

1. **Create the base table** `VendAPCustomsDecl` with fields:
2. `DeclId` (string 35, EDT)
3. `DeclDate` (date)
4. `CustomsRef` (string 50)
5. **Create a Query** `VendAPCustomsDeclQuery`:
6. Add `VendAPCustomsDecl` as the data source
7. Add all fields to the query
8. **Create the Data Entity** `VendAPCustomsDeclEntity`:
    - **AOT Node**: `Data Entities\VendAPCustomsDeclEntity`
    - Set the `Name` property to `VendAPCustomsDeclEntity`
    - Set `IsPublic = Yes` and define `Public Entity Name` and `Public Collection Name` for OData exposure
    - Set the `Query` property to point to `VendAPCustomsDeclQuery`
    - Set the `StagingTable` property if you need intermediate storage
9. **Add a Data Entity Field** for each field you want to expose:
10. `VendAPCustomsDeclEntity\Fields\DeclId`
11. `VendAPCustomsDeclEntity\Fields\DeclDate`
12. `VendAPCustomsDeclEntity\Fields\CustomsRef`
13. **Synchronize the database** — right-click the project → **Deploy** to generate the SQL table.
14. **Register the entity** — the entity is automatically available via OData at: `https://<your-environment>/data/VendAPCustomsDeclEntity`

Exercise 13 — SSRS Report Data Provider

Problem

Write the data provider class for an SSRS report that lists all customers in a specific group along with their total outstanding balance. The report should accept a `CustGroup` parameter.

Hints

- The data provider class extends `SrsReportDataProvider`
- Use `SrsReportDataContract` to pass parameters
- The `processReport()` method contains the query logic
- Use a `QueryRun` with a `Sum` aggregate to calculate the outstanding balance per customer
- The `QueryBuildDataSource::addAggregate()` method adds a sum field to the query

Solution

```
// Data contract for the report parameter
[DataContractAttribute]
class VendCustomerBalanceDPContract extends SrsReportDataContract
{
    CustGroup _custGroup;

    [DataMemberAttribute('CustGroup')]
    public CustGroup parmCustGroup(CustGroup _custGroup = _custGroup)
    {
        return _custGroup;
    }
}

// Data provider class
class VendCustomerBalanceDP extends SrsReportDataProvider
{
    CustGroup _custGroup;

    public void processReport()
    {
        VendCustomerBalanceTmp tmpTable;
        CustTable              custTable;
        Amount                 totalBalance;

        // Clear any existing data
        tmpTable.deleteFrom();

        // Build a query that sums outstanding balances per customer
        Query query = new Query();
        QueryBuildDataSource qbdsCustTable = query.addDataSource(tableNum(CustTable));
        QueryBuildDataSource qbdsCustTrans = qbdsCustTable.addDataSource(tableNum(CustTrans));

        // Join CustTrans to CustTable on AccountNum
        qbdsCustTrans.addLink(fieldNum(CustTable, AccountNum), fieldNum(CustTrans, AccountNum));

        // Filter: unsettled transactions with invoice date on or before today
        qbdsCustTrans.addRange(fieldNum(CustTrans, InvoiceAccountingDate)).value(queryValue(today()));
        qbdsCustTrans.addRange(fieldNum(CustTrans, HasNotBeenSettled)).value(queryValue(true));

        // Add sum aggregate on AmountMST
        qbdsCustTrans.addAggregate(fieldNum(CustTrans, AmountMST));
```

```
        // Filter by customer group
        qbdsCustTable.addRange(fieldNum(CustTable, GroupName)).value(queryValue(_custGroup));

        // Add the fields we need to the query
        qbdsCustTable.addField(fieldNum(CustTable, AccountNum));
        qbdsCustTable.addField(fieldNum(CustTable, Name));
        qbdsCustTable.addField(fieldNum(CustTable, GroupName));

        QueryRun queryRun = new QueryRun(query);

        while (queryRun.next())
        {
            custTable = queryRun.get(tableNum(CustTable));
            totalBalance = queryRun.getSum(fieldNum(CustTrans, AmountMST));

            tmpTable.clear();
            tmpTable.AccountNum = custTable.AccountNum;
            tmpTable.CustomerName = custTable.Name;
            tmpTable.GroupName = custTable.GroupName;
            tmpTable.TotalBalance = totalBalance;
            tmpTable.insert();
        }
    }
}
```

Exercise 14 — Error Handling Best Practices

Problem

Identify the error in each of the following code snippets and rewrite it correctly:

Snippet A — Missing `super()` call

```
public boolean validateWrite()
{
    if (this.AccountNum == '')
    {
        error("Account number cannot be empty.");
        return false;
    }
    return true;
}
```

Snippet B — `select` inside a `while` loop (N+1 problem)

```
CustTable custTable;
while select custTable
{
    VendTable vendTable;
    select firstonly vendTable
        where vendTable.AccountNum == custTable.AccountNum;

    if (vendTable)
    {
        info(strFmt("%1 – %2", custTable.AccountNum, vendTable.Name));
```

```
    }
}
```

Snippet C — Missing transaction

```
CustTable custTable;
custTable.clear();
custTable.AccountNum = "CUST-NEW-001";
custTable.Name = "New Customer";
custTable.insert();

CustGroup custGroup;
custGroup.clear();
custGroup.GroupName = "NEWGROUP";
custGroup.insert();
```

Solution

Snippet A — Fixed:

```
public boolean validateWrite()
{
    boolean ret;

    ret = super();  // Always call super() first

    if (ret && this.AccountNum == '')
    {
        ret = false;
        error("Account number cannot be empty.");
    }

    return ret;
}
```

Snippet B — Fixed:

```
// Use exists join instead of a select inside a while loop
CustTable custTable;
VendTable vendTable;
boolean hasVendor;

while select custTable
{
    hasVendor = exists join vendTable
        where vendTable.AccountNum == custTable.AccountNum;

    if (hasVendor)
    {
        info(strFmt("%1 – has vendor", custTable.AccountNum));
    }
}
```

Snippet C — Fixed:

```
ttsBegin;
try
```

```
{
    CustTable custTable;
    custTable.clear();
    custTable.AccountNum = "CUST-NEW-001";
    custTable.Name = "New Customer";
    custTable.insert();

    CustGroup custGroup;
    custGroup.clear();
    custGroup.GroupName = "NEWGROUP";
    custGroup.insert();

    ttsCommit;
}
catch (Exception::Error)
{
    ttsAbort;
    error("Failed to insert customer and group — rolled back.");
}
```

Exercise 15 — Putting It All Together

Problem

Write a complete X++ job that: 1. Creates a new customer with `AccountNum = "CUST-EX-001"`, `Name = "Example Corp"`, `Currency = "USD"`, `GroupName = "CORP"` 2. Validates the customer using a `CustValidationService` class (from Chapter 2's activity) 3. If valid, inserts the customer and logs the success to the Infolog 4. If invalid, logs the validation errors and does not insert 5. Wraps everything in a transaction with deadlock retry logic

Solution

```
static void exercise15_completeJob(Args _args)
{
    CustTable               custTable;
    CustValidationService   validator;
    boolean                 isValid;
    int                     retryCount = 0;
    boolean                 success = false;

    // Step 1: Create the customer record (in memory, not yet inserted)
    custTable.clear();
    custTable.AccountNum = "CUST-EX-001";
    custTable.Name = "Example Corp";
    custTable.Currency = "USD";
    custTable.GroupName = "CORP";

    // Step 2: Validate using the CustValidationService
    validator = new CustValidationService(custTable);
    isValid = validator.validate();

    if (!isValid)
    {
        warning("Customer validation failed — check the Infolog for details.");
    }
    else
    {
        // Step 3: Insert the customer within a transaction with deadlock retry
```

```
        while (!success && retryCount < 3)
        {
            ttsBegin;
            try
            {
                custTable.insert();

                ttsCommit;
                info(strFmt("Customer %1 created successfully.", custTable.AccountNum));
                success = true;
            }
            catch (Exception::Deadlock)
            {
                ttsAbort;
                retryCount++;
                info(strFmt("Deadlock — retry %1 of 3", retryCount));
            }
            catch (Exception::Error)
            {
                ttsAbort;
                error("An unexpected error occurred.");
                break;
            }
        }

    }

    if (!success)
    {
        error("Failed to create customer after 3 attempts.");
    }
}
```

Exercise 16 — Number Sequence

Problem

Create a custom table `ComplianceLog` with a `ComplianceId` field that uses a number sequence for its value. Write a job that inserts a new `ComplianceLog` record and verifies the number sequence was consumed correctly.

Hints

- Create a `NumberSeq` reference on the table's `ComplianceId` field
- Use `NumberSeq::newGetNumFromId()` to get the next number in the sequence
- The number sequence must be configured in the `NumberSeq` AOT node under the appropriate module

Solution

1. **Create the table** `ComplianceLog` with fields:
2. `ComplianceId` (string 20, EDT `ComplianceId`)
3. `Description` (string 60)

4. `CreatedDateTime` (utcDateTime)

5. **Create a NumberSeq reference** on `ComplianceId`:

6. In the AOT, right-click the `ComplianceId` field → **New NumberSeq Reference**
7. Set the `ReferenceField` property to `ComplianceId`
8. **Create the number sequence** in the AOT:
9. `NumberSeq > NumberSeqReference > ComplianceLog`
10. Set `Scope` to `Table`
11. Set the `NumberSequence` property to point to a new `NumberSeq` object
12. **The job**:

```
static void exercise16_numberSequence(Args _args)
{
    ComplianceLog log;
    NumberSeq numberSeq;

    ttsBegin;
    try
    {
        log.clear();

        // Get the next number from the number sequence
        numberSeq = NumberSeq::newGetNumFromId(
            NumberSeqReference::findReference(fieldNum(ComplianceLog, ComplianceId)).NumberSequenceId());
        log.ComplianceId = numberSeq.num();

        log.Description = "Compliance check entry";
        log.CreatedDateTime = DateTimeUtil::utcNow();
        log.insert();

        ttsCommit;
        info(strFmt("Compliance log created with ID: %1", log.ComplianceId));
    }
    catch (Exception::Error)
    {
        ttsAbort;
        error("Failed to create compliance log entry.");
    }
}
```

Exercise 17 — SysOperation Service

Problem

Create a `SysOperation` service class that accepts a `CustGroup` parameter and returns the count of customers and their total outstanding balance for that group. Use a `SysOperationServiceController` to run the service.

Hints

- The service class extends `SysOperationServiceBase`
- Use a `DataContract` class to pass parameters to the service
- Override the `process()` method to contain the business logic
- Use `SysOperationServiceController::runService()` to execute the service

Solution

```
// Data contract for service parameters
[DataContractAttribute]
class CustBalanceServiceContract extends SrsReportDataContract
{
    CustGroup _custGroup;

    [DataMemberAttribute('CustGroup')]
    public CustGroup parmCustGroup(CustGroup _custGroup = _custGroup)
    {
        return _custGroup;
    }
}

// Service class
class CustBalanceService extends SysOperationServiceBase
{
    [DataMemberAttribute('CustGroup')]
    public CustGroup parmCustGroup(CustGroup _custGroup = _custGroup)
    {
        return _custGroup;
    }

    public void process()
    {
        CustTable custTable;
        CustTrans custTrans;
        Amount totalBalance;
        int customerCount;

        while select custTable
            where custTable.GroupName == this.parmCustGroup()
        {
            customerCount++;

            // Use exists join pattern to check for unsettled transactions
            // and accumulate the total balance
            while select custTrans
                where custTrans.AccountNum == custTable.AccountNum
                   && custTrans.HasNotBeenSettled
            {
                totalBalance += custTrans.AmountMST;
            }
        }

        info(strFmt("Customers in group %1: %2", this.parmCustGroup(), customerCount));
        info(strFmt("Total outstanding balance: %1", totalBalance));
    }
}

// Job to run the service
static void exercise17_sysOperation(Args _args)
{
    SysOperationServiceController controller = new SysOperationServiceController(
        classStr(CustBalanceService),
        "CustBalanceService",
        SysOperationServiceController::getDefaultConfiguration());

    controller.parmArgs(new CustBalanceServiceContract());
    controller.parmArgs().parmCustGroup("CORP");
```

```
    controller.startOperation();
}
```

Exercise Index

#	Topic	Key Concept
1	Containers & Type Casting	`conPeek`, `typeid()`
2	While Select with Filtering	`while select` with `where`
3	Bulk Update	`update_recordset`
4	Transactions	`ttsBegin/ttsCommit/ttsAbort`
5	Method Overriding	`super()` in table extensions
6	Deadlock Handling	Retry pattern with `catch (Exception::Deadlock)`
7	Class Design	Constructor, private helpers, static methods
8	Inheritance & Polymorphism	`extends`, method overriding
9	Event Handlers	`[SubscribesTo]` attribute
10	Delegates	`delegate` keyword, `+=` subscription
11	Chain of Command	`next` keyword, multiple extensions
12	Data Entities	`DataEntity`, `Query`, OData exposure
13	SSRS Data Provider	`SrsReportDataProvider`, `processReport()`
14	Error Handling Best Practices	`super()`, `exists join`, transactions
15	Putting It All Together	Complete end-to-end job
16	Number Sequence	`NumberSeq`, `NumberSeqReference`
17	SysOperation Service	`SysOperationServiceBase`, `SysOperationServiceController`

#	Topic	Key Concept
1	Containers & Type Casting	`conPeek`, `typeid()`
2	While Select with Filtering	`while select` with `where`
3	Bulk Update	`update_recordset`
4	Transactions	`ttsBegin/ttsCommit/ttsAbort`
5	Method Overriding	`super()` in table extensions
6	Deadlock Handling	Retry pattern with `catch (Exception::Deadlock)`
7	Class Design	Constructor, private helpers, static methods

#	Topic	Key Concept
8	Inheritance & Polymorphism	`extends`, method overriding
9	Event Handlers	`[SubscribesTo]` attribute
10	Delegates	`delegate` keyword, `+=` subscription
11	Chain of Command	`next` keyword, multiple extensions
12	Data Entities	`DataEntity`, `Query`, OData exposure
13	SSRS Data Provider	`SrsReportDataProvider`, `processReport()`
14	Error Handling Best Practices	`super()`, `exists join`, transactions
15	Putting It All Together	Complete end-to-end job

---

name: d365-guide-cross-reference description: Maps main guide phases to Desktop guide chapters, topic lookup table, and recommended reading order metadata: node_type: memory type: reference originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-08-10T17:48:59.571Z

D365 F&O Guides — Cross-Reference Map

> **How to navigate both guides.** The main guide (`d365_fao_technical_concepts_guide.md`) is organized into 12 phases. The Desktop guide (`d365-learning-guide.md`) is organized into 15 chapters. This map shows where each concept lives in both documents.

Phase → Chapter Mapping

Main Guide Phase	Title	Desktop Guide Chapters	Key Topics Covered
Phase 1	Foundations (Months 1–2)	**Ch 1** + **Ch 2**	Platform architecture, AOT, model layers, X++ data types, variables, control flow, CRUD, VS environment
Phase 2	Development Deep Dive (Months 2–4)	**Ch 2** + **Ch 3** + **Ch 4** + **Ch 5** + **Ch 6**	Table design, EDTs, classes, AOT navigation, views/queries, form controls, business logic classes
Phase 3	Architecture & Infrastructure (Months 3–5)	**Ch 1**	Server architecture, cloud, metadata subsystem, identity/authentication
Phase 4	Extensibility Patterns (Months 4–6)	**Ch 7**	Chain of Command, method wrapping, event handlers, delegates, table/form extensions
Phase 5	Business Logic & Backend (Months 5–7)	**Ch 6** + **Ch 11**	RunBase, SysOperation, batch processing, job framework
Phase 6	Integration (Months 6–8)	**Ch 9**	Data entities, integration patterns, REST APIs, OData
Phase 7	Reporting & Analytics (Months 7–9)	**Ch 10**	SSRS reports, legacy AX reports, analytical reporting, BI
Phase 8	Testing & Debugging (Months 7–10)	**Ch 12**	Unit testing (SysTest), ATL, RSAT, debugging
Phase 9	DevOps & CI/CD (Months 8–10)	**Ch 13**	Build automation, LCS, Azure DevOps, ALM
Phase 10	Security & Compliance (Months 9–11)	**Ch 8**	Security model, roles/duties/privileges, XDS, SoD
Phase 11	Performance & Problem Solving (Months 10–12)	**Ch 14**	SQL optimization, caching, indexing, bottleneck identification
Phase 12	Solution Crafting & Capstone (Months 11–12)	**Ch 15**	End-to-end production scenario, design patterns, solution crafting

---

Topic → Location Lookup

Use this table to find where a specific topic is covered across both guides.

Topic	Main Guide	Desktop Guide
Platform architecture & four pillars	Phase 1, §1.1	Ch 1, §1.1
AOT (Application Object Tree)	Phase 2, §2.4	Ch 1, §1.2
Model layers (SYS → ISV → VAR → CUS → USR)	Phase 1, §1.1	Ch 1, §1.3
X++ data types & variables	Phase 1, §1.3	Ch 2, §2.1–2.2
Control flow (if, switch, for, while)	Phase 1, §1.3.1	Ch 2, §2.3
CRUD operations (select, insert, update, delete)	Phase 1, §1.3.2	Ch 2, §2.4
`super()` and method overriding	Phase 1, §1.3.3	Ch 2, §2.5
Exception handling	Phase 1, §1.3.4	Ch 2, §2.6
Class structure & constructors	Phase 2, §2.3.1	Ch 2, §2.7.1
Inheritance & polymorphism	Phase 2, §2.3.2	Ch 2, §2.7.2
Abstract classes & interfaces	Phase 2, §2.3.3	Ch 2, §2.7.3
Table design & properties	Phase 2, §2.1	Ch 3, §3.1
Extended Data Types (EDTs)	Phase 2, §2.2	Ch 3, §3.2
Field groups & indexes	Phase 2, §2.1	Ch 3, §3.3–3.4
Form architecture & controls	Phase 2, §2.6	Ch 4, §4.1
Views & queries	Phase 2, §2.5	Ch 5
Business logic classes	Phase 2, §2.3	Ch 6, §6.1
Chain of Command (CoC)	Phase 4, §4.2	Ch 7, §7.1
Event Handlers	Phase 4, §4.4	Ch 7, §7.2
Delegates	Phase 4, §4.5	Ch 7, §7.3
Table extensions	Phase 4, §4.6	Ch 3, §3.4
Form extensions	Phase 4, §4.7	Ch 4
Security model (roles, duties, privileges)	Phase 10, §10.1	Ch 8, §8.1
Extensible Data Security (XDS)	Phase 10, §10.4	Ch 8, §8.4
Data entities	Phase 6 (not in main guide detail)	Ch 9, §9.1
REST APIs / OData	Phase 6 (not in main guide detail)	Ch 9, §9.3
SSRS reports	Phase 7, §7.1	Ch 10, §10.1
Analytical reporting	Phase 7, §7.4	Ch 10, §10.4
Unit testing (SysTest)	Phase 8, §8.1	Ch 12, §12.1

Topic	Main Guide	Desktop Guide
Acceptance Test Library (ATL)	Phase 8, §8.2	Ch 12, §12.2
RSAT	Phase 8, §8.3	Ch 12, §12.3
Debugging	Phase 8, §8.4	Ch 12, §12.4
Azure DevOps build pipeline	Phase 9, §9.1	Ch 13, §13.1
LCS environments & release pipeline	Phase 9, §9.2	Ch 13, §13.2
SQL optimization	Phase 11, §11.1	Ch 14, §14.1
Caching strategies	Phase 11, §11.2	Ch 14, §14.2
Table indexing	Phase 11, §11.3	Ch 14, §14.3
End-to-end capstone	Phase 12, §12.3	Ch 15

Guide Comparison

Aspect	**Main Guide (`d365_fao_technical_concepts_guide.md`)**	**Desktop Guide (`d365-learning-guide.md`)**
Structure	12 phases organized by learning month	15 chapters organized by topic area
Depth	Conceptual overviews with key code snippets	Deep dives with full code examples, tables, and activities
Best for	Quick reference, understanding what to learn next	In-depth study, hands-on practice, exam prep
Length	~1,784 lines (~197 KB)	~10,000+ lines (~400 KB)
Activities	No — reference only	Yes — each chapter has an activity with hints and ideal solution
Approach	Linear progression (Phase 1 → 12)	Topic-focused (Ch 1 covers landscape, Ch 2 covers X++, etc.)

Recommended Reading Order

For a new intern starting the journey:

1. **Ch 1** → Understand the platform landscape
2. **Ch 2** → Learn X++ fundamentals and class design
3. **Main Guide Phase 1** → Reinforce foundations with the conceptual overview
4. **Ch 3** → Master tables and EDTs
5. **Main Guide Phase 2** → Deepen development knowledge
6. **Ch 4** → Build forms
7. **Ch 5** → Learn views and cross-table queries
8. **Ch 6** → Write business logic classes
9. **Ch 7** → Master extensibility patterns (CoC, Event Handlers, Delegates)
10. **Ch 8** → Understand security
11. **Ch 9** → Build integrations with data entities

12. **Ch 10** → Create reports
13. **Ch 11** → Write batch jobs
14. **Ch 12** → Write tests
15. **Ch 13** → Set up DevOps pipelines
16. **Ch 14** → Optimize performance
17. **Ch 15** → Capstone project

---

name: d365-glossary description: Comprehensive glossary of D365 F&O terms with one-sentence definitions and security hierarchy reference metadata: node_type: memory type: reference originSessionId: 369838ab-9513-4fe8-9b6d-09784c222811 modified: 2026-08-10T17:48:07.602Z

D365 Finance & Operations — Glossary

> **Key terms and concepts in Dynamics 365 Finance & Operations development.** Each entry includes a one-sentence definition and, where helpful, a cross-reference to the relevant chapter in the Desktop Guide.

A

Term	Definition
AOS (Application Object Server)	The server-side runtime engine that executes X++ IL code, manages transactions, serves metadata, and handles database connections.
Alternate Index	A non-clustered, unique constraint on a table that enforces uniqueness for a specific field or set of fields.
Application Platform	The lowest model layer in the D365 F&O stack — provides interfaces with the kernel and base infrastructure (AIF, Batch, RunBase, DictXX objects).
Application Foundation	The framework layer shared across all applications — includes dimension framework, GAB, number sequences, security, workflow, and BI.
Application Suite	The application-specific business logic layer — covers SCM, HCM, Professional Services, Retail, etc.
AOT (Application Object Tree)	The hierarchical, metadata-driven repository that contains every object in D365 F&O — tables, classes, forms, views, security roles, reports, and more.
Azure Active Directory (Azure AD)	The identity and authentication service that every D365 F&O user authenticates through; role-based access flows through the D365 security model.
Azure DevOps	The CI/CD platform used to build, test, and deploy D365 F&O solutions — compiles X++, produces `.axmodel` artifacts, and deploys to environments.

B

Term	Definition
Batch Processing	Running long-running or scheduled jobs in the background using the `RunBaseBatch` framework, which queues work for the Batch server.
Batch Header	The `BatchHeader` class that manages batch job metadata — job name, recurrence schedule, execution priority, and target server.
BI (Business Intelligence)	The analytics and reporting capabilities in D365 F&O — includes SSRS paginated reports, analytical reporting (Power BI), and the Financial Reporting module.

Term	Definition
CacheLookup	A table property that controls how SQL Server caches lookups of that table — values: `None`, `NotInTTS`, `Found`, `FoundAndEmpty`, `EntireTable`.
Chain of Command (CoC)	A pattern for resolving method conflicts between a base class and its extensions — the `next` keyword calls the next handler in the execution chain.
Configuration Key	A boolean property set on AOT objects that a system administrator enables/disables via the License Configuration form to control which features/modules are visible.
Container	A typed ordered list in X++ (similar to an array) that can hold elements of different types — accessed with `conPeek()` and `conLen()`.
Cross-Reference DB	A SQL Server Express LocalDB installed with Visual Studio that tracks object dependencies — enables "Find References" and "Depends on" queries.

D

Term	Definition
Data Entity	A D365 F&O object that exposes table data to external systems via OData/REST APIs — consists of a `DataEntity` node, a `Query`, and optionally a staging table.
Data Project	A Visual Studio project that groups data entities and their dependencies for deployment — used to build and export data integration packages.
Data Area	The top-level organizational unit in D365 F&O — also called a Legal Entity; represents a distinct legal entity for financial reporting and regulatory compliance.
Delegate	A type-safe function pointer in X++ that allows consumers to inject custom logic into a publisher class — subscribed to using the `+=` operator.
Deployment	The process of pushing compiled `.axmodel` artifacts from a build environment to a target environment (Dev → Test → Production).
Development Lifecycle	The end-to-end workflow for D365 F&O development: code → build → deploy → test → promote → production, managed through LCS and Azure DevOps.
DGML	Directed Graph Markup Language — a diagram format used in Visual Studio to visualize model dependency graphs in the AOT.

E

Term	Definition
EDT (Extended Data Type)	A type alias that wraps a base type (string, int, real) with additional properties like `StringSize`, validation rules, and field relations — ensures consistency across the data model.
EDT Extension	A mechanism to add new allowed values to an existing EDT (e.g., adding a new country code to `AddressCountryRegionId`).
E-Signature	The electronic signature framework in D365 F&O that enables digitally signed approvals and document workflows.
Element	A single object in the AOT — a table, class, form, view, EDT, enum, report, data entity, or any other metadata item.
Event Handler	A method that automatically executes in response to a specific system event. Table data events (e.g., `inserted`, `updated`, `deleted`) use `[DataEventHandler]`; custom delegate-based events use `[SubscribesTo]`.
Exception Types	The categories of errors in X++: `Error`, `Warning`, `Info`, `Broken`, `Deadlock`, and `DuplicateKey` — each handled in a separate `catch` block.
Extension	A customization approach that adds fields, methods, or event handlers to a base table or class without modifying the original — the preferred method over overlayering.
Extensible EDT	An EDT that allows consumers to add new values to base enums, enabling customization without modifying the base EDT definition.

F

Term	Definition
Field Group	A named set of fields defined at the table level — used for form auto-generation and report generation so that adding a field to the group automatically appears in all forms/reports using it.
Field Relation	An EDT property that links the EDT to another table (e.g., `CustAccount` EDT links to `CustTable`) — enables automatic lookup and validation in forms.
Financial Dimension	A category axis used in the chart of accounts to classify transactions — examples include Department, Cost Center, and Business Unit; the set of active financial dimensions defines the dimension framework.
Three-Tier Architecture	The architectural model for D365 F&O: Client (browser-based), Application (AOS), and Database tiers. Integration and Azure services are supporting components, not separate tiers.
Form Data Source	A node in a form that points to a table or query — controls which SQL query runs, which fields are available, and how joins work.
Form Link Type	Defines how parent and child data sources relate in a form — `Inner Join` (child only if parent exists), `Outer Join` (all

Term	Definition
	children regardless), `Exists Join` (parent only if children exist).

G

Term	Definition
Global Class	A class declared with the `Global` prefix — its methods can be called without instantiating the class (e.g., `Global::info()`).
Group Level Index	A multi-field index where the leading field determines selectivity — SQL can use the index for queries that filter on the leading field.

H

Term	Definition
Hash Index	An index type optimized for equality-only lookups (no range scans) — useful when queries always filter by exact match on a single field.
Hotfix	A targeted patch deployed to a production environment to fix a specific bug or issue — deployed through LCS release pipeline.

I

Term	Definition
IL (Intermediate Language)	The compiled output of X++ — X++ compiles to .NET CIL (Common Intermediate Language), the same IL used by C# and VB.NET.
Infolog	The message window in the D365 F&O client that displays informational messages (`info`), warnings (`warning`), and errors (`error`).
Integration Components	Supporting components that connect D365 F&O to external systems — includes Azure Service Bus, Azure Functions, Logic Apps, and OData/REST endpoints.
Inner Join	A form data source link type where child records only show when a matching parent record exists — the default link type.
Instance Variable	A variable declared within a class (not `static`) — each object instance has its own copy.

J

Term	Definition
Job	An ad-hoc X++ program that runs once — used for data migration, batch processing, testing, and one-time operations. Jobs are stored in the AOT under `Jobs`.

L

Term	Definition
LCS (Lifecycle Services)	Microsoft's cloud-based orchestration platform for D365 F&O — manages environments, build pipelines, release pipelines, monitoring, and hotfix deployment.
Layer	The upgrade boundary in D365 F&O — determines the order in which model changes are applied and which changes survive upgrades.
Layer Hierarchy	From lowest to highest: `SYS` (Microsoft-owned, cannot be modified) → `ISV` (independent software vendor) → `VAR` (value-added reseller) → `CUS` (customer-specific) → `USR` (user layer, highest — no key required to access, unlike `ISV`/`VAR`/`CUS`). Higher layers override lower layers. **Current guidance:** Microsoft now strongly favors the **extension model** (table extensions, class extensions, event handlers) over layering/overlayering for customizations. Overlayering on `CUS`/`USR` is considered a legacy mechanism mainly relevant when upgrading older on-prem / AX 2012-derived solutions; new development should use extensions to maintain forward-compatibility and reduce upgrade friction. (Models docs
Logic App	An Azure service for automating workflows — used in D365 F&O integration patterns to connect to external systems without writing custom code.

M

Term	Definition
Model	A deployable unit of metadata in D365 F&O — contains a collection of AOT objects (tables, classes, forms, etc.) and is defined by a `Model.xml` manifest file.
Model Store	The file system (Azure File Storage in cloud) where AOT metadata is stored as XML files — not a SQL database.
Model Manifest	The `Model.xml` file that defines a model's name, version, layer, dependencies, and configuration keys — the heart of a model project.
MICR	Magnetic Ink Character Recognition — the standard for printing bank cheque details; D365 F&O includes MICR check printing in the Accounts Payable module.

Term	Definition
Monster All-in-One Form	A form that tries to do everything — D365 F&O best practice is to use List Pages for browsing, Details Forms for editing, and Task Pages for wizard-style workflows.

N

Term	Definition
Number Sequence (`NumberSeq`)	A framework for generating sequential, unique identifiers safely under concurrent access — handles deduplication and gap-free numbering.
NuGet Packages	The NuGet packages required for a D365 F&O build, which vary by version — typically include X++ compiler targets, reference assemblies, and deployment tooling, restored automatically by the Dynamics365Build DevOps task.

O

Term	Definition
OData/REST API	The protocol used to expose D365 F&O data to external systems — data entities are automatically available via OData endpoints at `/data/<EntityName>`.
Outer Join	A form data source link type where ALL child records show even if no matching parent exists — used for "show me all children regardless of parent."
Overlayering	Modifying base Microsoft tables and classes directly — the deprecated approach that causes upgrade conflicts; replaced by the extension pattern.

P

Term	Definition
Package	The deployable unit in D365 F&O — a collection of models that are built and deployed together as a single `.axmodel` artifact.
Paginated Report	An SSRS report format that renders as a fixed-layout document (PDF, Excel, Word) — the standard report type in D365 F&O.
Presentation Tier	The web-based UI layer of D365 F&O — a single browser-based client (HTML5/CSS/JS) that runs in Edge, Chrome, and Safari; the AOS serves the web assets and the browser renders forms, menus, and navigation.
Primary Index	The clustered index on `RecId` that is auto-created for every table — should not be modified.

Term	Definition
Staging Table	An auto-generated table (e.g., `EntityNameStaging`) that holds intermediate state for data entities before data is pushed to the target system.
Public Entity Name / Public Collection Name	Properties on a data entity that control its OData exposure and endpoint names.

Q

Term	Definition
Query	An AOT node that defines data retrieval logic with ranges, joins, and sorting — used by views, forms, and reports.
Query Build Data Source	The programmatic API for building queries at runtime — `addRange()`, `addLink()`, and `addDataSource()` methods allow dynamic query construction.

R

Term	Definition
RecId	The auto-generated 64-bit record identifier (`RecId`) that is the primary key for every table in D365 F&O — should never be manually assigned.
Reference Group	An EDT property that declares which table the EDT references — enables `RefTableId` and `RefRecId` automatic fields on tables using that EDT.
Release Pipeline	The LCS pipeline that promotes builds from Test to Production environments — manages staging, validation, and deployment steps.
RunBase	The framework for creating batchable processes — a base class that provides the UI for parameter input and the infrastructure for background execution.
RunBaseBatch	The batch execution variant of RunBase — queues the process for background execution on the Batch server.

S

Term	Definition
Security Role	A collection of duties that defines what a user can do in D365 F&O — the hierarchy is: Permission → Privilege (composed of permissions) → Duty (composed of privileges) → Role (composed of duties) → User (assigned one or more roles).
SSRS (SQL Server Reporting Services)	The reporting engine used in D365 F&O for paginated reports — uses `SrsReportDataProvider` classes to supply data to report layouts.

Term	Definition
SrsReportDataProvider	The base class for SSRS report data providers — contains the `get()` method for standard providers and `processReport()` for `SrsReportDataProviderPreProcessTempDB` (TempDB pre-processing).
Static Variable	A variable declared with the `static` keyword — shared across all instances of the class, persists for the lifetime of the application.
Staging Table	An intermediate table used in data entities and integration patterns to accumulate, validate, and stage records before pushing them to the target system.
SysOperation	The modern framework for service operations with parameter classes — replaces the older RunBase pattern for service-based integrations.
SoD (Segregation of Duties)	A security principle that prevents a single user from having conflicting permissions — D365 F&O supports SoD validation through the security model and XDS policies.
SysTest	The built-in unit testing framework in D365 F&O — tests are classes that extend `SysTestCase` and are executed through the Test Runner.
System Class	A class in the `Sys**` namespace (e.g., `SysTableLookup`, `SrsReportDataProvider`) — provides framework-level functionality that developers use directly.

T

Term	Definition
Table Extension	The pattern for adding fields, indexes, relations, and methods to an existing Microsoft table without modifying the base table — the recommended approach over overlayering.
Transaction (`ttsBegin`/`ttsCommit`/`ttsAbort`)	A unit of work that ensures all database modifications succeed or all are rolled back — wraps multi-table inserts/updates in atomic operations.
TTS (Transaction Scope)	The transaction boundary created by `ttsBegin` — all SQL statements within the scope are committed together on `ttsCommit` or rolled back on `ttsAbort`.

U

Term	Definition
Update Action	A table property that defines what happens when a parent record is updated — options include `Cascade`, `Restricted`, and `None`.
UTC DateTime	A `utcdatetime` value that stores date and time in Coordinated Universal Time — used for audit fields and cross-timezone data.

V

Term	Definition
Validation	The process of checking data integrity before allowing a record to be saved — implemented via `validateWrite()` on tables and custom validation methods on classes.
Visual Studio (VS)	The IDE used for D365 F&O development — VS 2022 with the `Microsoft.Dynamics.FinOps.ToolsVS2022.vsix` extension provides the Modeling SDK, X++ editor, and debugging support.
View	A virtual table defined by a query that provides a read-only or updatable projection of data from multiple tables — can be extended to add fields to existing views.

W

Term	Definition
While Select	The standard X++ iteration pattern that combines a SQL `SELECT` with a `while` loop — generates efficient SQL and streams records one at a time from the database.
Workstation	A client machine (desktop, laptop, or tablet) accessing D365 F&O through a supported web browser (Edge, Chrome, Safari) — no separate client installation is required.
Workflow	The automated approval and business process engine in D365 F&O — workflows define routes, conditions, and actions for document approvals and task routing.

X

Term	Definition
X++	The primary development language for D365 F&O — object-oriented, application-aware, data-aware, compiles to .NET CIL, and runs on the AOS.
X++ IL	The compiled form of X++ code — X++ compiles to .NET CIL (Common Intermediate Language), the same IL used by C# and VB.NET.
XDS (Extensible Data Security)	A policy-based security framework that controls access to data at the row and field level — applies security rules dynamically based on context, roles, and query ranges.

Quick Reference: Security Hierarchy

```
Permission → Privilege (composed of permissions) → Duty (composed of privileges) → Role (composed of duties) → User (assigned one or more roles)
```

Level	What It Controls	Example
Permission	Access to a specific table/field + read/write/delete/create	`CustTable` Read, `CustTable` Write
Privilege	A collection of permissions	`CustTable Maintenance`
Duty	A collection of privileges	`Customer Maintenance`
Role	A collection of duties assigned to a user	`Accounts Receivable Manager`
User	Assigned one or more roles	`Ahmed` → `Accounts Receivable Manager`